transportation-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `transportation_ai999x.about()` for the same answer.

Every capability transportation-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import transportation_ai999x

result = transportation_ai999x.solve('transport_braking_distance', v='27.7778 m/s', a='3.4 m/s^2')
result.show()
# d = 113.471 m, t = 8.16994 s
```

By topic

topic	capabilities
Braking and stopping	Braking distance from speed and deceleration, Stopping sight distance (reaction plus braking, with grade), Speed estimate from skid-mark length, Distance to slow from one speed to another (friction and grade), Impact speed with an obstacle after reaction and braking, Theoretical stopping distance with aerodynamic and rolling resistance
Highway geometry	Maximum safe speed on a superelevated horizontal curve, Minimum radius of a horizontal curve, Side friction demanded on a superelevated curve, Superelevation required on a horizontal curve, Minimum length of a crest vertical curve for a sight distance, Sight distance available over a crest vertical curve, Horizontal sightline offset on a circular curve, Sight distance available on a curve with a roadside obstruction, Clothoid (spiral) transition curve coordinates, Curve radius or speed by the simplified e + f formula, Sag vertical curve length for headlight sight distance, Passing sight distance (four-component method)
Mode choice	Multinomial logit mode shares from utilities, Logit mode choice from attributes and coefficients, Direct and cross elasticities of logit choice probabilities, Nested logit choice probabilities, User benefit of a change from the logit logsum
Queueing	Single-server random queue (M/M/1), e.g. a toll booth, Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza, Single server with constant service time (M/D/1), Blocking probability with no waiting room (Erlang B), e.g. parking spaces, First-come-first-served queue from arrival and service times
Railway operations	Running time between stations (accelerate, cruise, brake), Minimum headway of trains at a station stop (moving block), Train braking distance with Davis resistance and gradient, Train acceleration time and distance (tractive effort curve, Davis resistance)
Route planning	Fastest route on a road network (link lengths and speeds), Zone-to-zone travel time matrix (skim) of a road network
Traffic flow	Greenshields model: capacity, optimum speed and density, Greenshields model: densities and speeds at a given flow, Fit the Greenshields model to speed-density observations, Fit the Greenberg (logarithmic) model to speed-density observations, Fit the Underwood (exponential) model to speed-density observations, Greenberg model: capacity and density at capacity, Underwood model: capacity and speed at capacity, Shockwave speed between two traffic states, Shockwave speed in a Greenshields traffic stream, Queue and

topic	capabilities
	delay at a temporary bottleneck (incident, lane closure), Queue from arrival counts and capacity per interval, Probability of n vehicle arrivals in an interval (Poisson), Probability of a headway longer than h (negative exponential), Travel-time reliability indices (buffer, planning time, travel time index)
Traffic signals	Yellow change interval (kinematic, ITE), Queue at a signal by shockwave analysis, Queue and uniform delay at a signal (deterministic D/D/1), Green splits by equal degree of saturation (flow-ratio method), Optimum cycle length by Webster's formula
Traffic studies	Time-mean and space-mean speed of spot speeds, 85th, 50th and 15th percentile spot speeds
Travel demand	All-or-nothing traffic assignment, Doubly constrained gravity model of trip distribution, Growth-factor (Furness) balancing of a trip matrix, Trip generation by multiple linear regression
Unsignalized intersections	Capacity of a minor stream by gap acceptance
Vehicle dynamics	Longitudinal weight transfer under braking and ideal brake proportioning, Lateral load transfer in a steady turn, Rollover threshold: static stability factor and rollover speed on a curve, Understeer gradient and characteristic or critical speed (bicycle model), Steady-state cornering: yaw rate, lateral acceleration, radius, sideslip, Steer angle needed for a turn of given radius (bicycle model), Yaw natural frequency and damping of the bicycle model, Quarter-car model: body and wheel-hop natural frequencies, Quarter-car model: road-to-body transmissibility at a frequency
Vehicle performance	Distance and time to accelerate (acceleration lane length), Overtaking manoeuvre: time and distance with constant acceleration, Coast-down time and distance (rolling and aerodynamic resistance), Acceleration time and distance under constant engine power, Maximum tractive force and acceleration limited by tyre friction, Minimum safe following gap when the leader brakes, Vehicle resistance forces and power at a speed, Top speed of a road vehicle

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above

field	meaning
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
transport_braking_distance	Braking distance from speed and deceleration	v, a	d, t
transport_stopping_sight_distance	Stopping sight distance (reaction plus braking, with grade)	v, t_r, f, a, G	SSD, d_r, d_b
transport_skid_speed	Speed estimate from skid-mark length	d, f, G	v, v_kmh
transport_speed_reduction_distance	Distance to slow from one speed to another (friction and grade)	v1, v2, f, G	d, t
transport_impact_speed	Impact speed with an obstacle after reaction and braking	v, D, t_r, f, G	v_impact, v_impact_kmh, d_stop
transport_acceleration_distance	Distance and time to accelerate (acceleration lane length)	v1, v2, a	d, t
transport_yellow_change_interval	Yellow change interval (kinematic, ITE)	v, t_r, a, G	Y, x_c
transport_overtaking_manoeuvre	Overtaking manoeuvre: time and distance with constant acceleration	v, a, s	t, d, v_f
transport_braking_with_air_resistance	Theoretical stopping distance with aerodynamic and rolling resistance	m, v1, mu, C_D, A, rho, f_r, G, eta, gamma, v2	S, t

id	operation	inputs	outputs
transport_coast_down	Coast-down time and distance (rolling and aerodynamic resistance)	m, v1, v2, C_D, A, rho, f_r, G, gamma	t, d
transport_acceleration_under_power	Acceleration time and distance under constant engine power	m, P, v1, v2, C_D, A, rho, f_r, G, eta, gamma	t, d
transport_traction_limited_acceleration	Maximum tractive force and acceleration limited by tyre friction	m, L, l_f, h, mu, f_r, drive	F_max, a_max, W_f, W_r
transport_braking_weight_transfer	Longitudinal weight transfer under braking and ideal brake proportioning	m, a, h, L, l_f	W_f, W_r, dW, front_share
transport_lateral_load_transfer	Lateral load transfer in a steady turn	m, a_y, h, t_w	W_out, W_in, dW
transport_rollover_threshold	Rollover threshold: static stability factor and rollover speed on a curve	t_w, h, R, e	SSF, v_roll, v_roll_kmh
transport_curve_safe_speed	Maximum safe speed on a superelevated horizontal curve	R, e, f	v, v_kmh
transport_minimum_curve_radius	Minimum radius of a horizontal curve	v, e, f	R_min
transport_side_friction_demand	Side friction demanded on a superelevated curve	v, R, e	f
transport_superelevation_required	Superelevation required on a horizontal curve	v, R, f	e, e_percent
transport_crest_curve_length	Minimum length of a crest vertical curve for a sight distance	g1, g2, S, h1, h2	L, case
transport_crest_curve_sight_distance	Sight distance available over a crest vertical curve	g1, g2, L, h1, h2	S, case
transport_horizontal_sight_offset	Horizontal sightline offset on a circular curve	R, S	M
transport_sight_distance_from_offset	Sight distance available on a curve with a roadside obstruction	R, M	S
transport_clothoid_coordinates	Clothoid (spiral) transition curve coordinates	R, L_s, s	x, y, theta
transport_greenshields_capacity	Greenshields model: capacity, optimum speed and density	vf, kj	q_max, k_m, v_m
transport_greenshields_speed_at_flow	Greenshields model: densities and speeds at a given flow	vf, kj, q	k_1, v_1, k_2, v_2
transport_greenshields_fit	Fit the Greenshields model to speed-density observations	speeds, densities	vf, kj, q_max, r2

id	operation	inputs	outputs
transport_greenberg_fit	Fit the Greenberg (logarithmic) model to speed-density observations	speeds, densities	v0, kj, q_max, r2
transport_underwood_fit	Fit the Underwood (exponential) model to speed-density observations	speeds, densities	vf, k0, q_max, r2
transport_greenberg_capacity	Greenberg model: capacity and density at capacity	v0, kj	q_max, k_m
transport_underwood_capacity	Underwood model: capacity and speed at capacity	vf, k0	q_max, v_m, k_m
transport_shockwave_speed	Shockwave speed between two traffic states	q1, k1, q2, k2	u, u_ms, direction
transport_greenshields_shockwave	Shockwave speed in a Greenshields traffic stream	vf, kj, k1, k2	u, u_ms
transport_signal_queue_shockwave	Queue at a signal by shockwave analysis	q_a, k_a, k_j, q_s, k_s, r	L_max, t_max, w_queue, w_discharge
transport_signal_queue_uniform_delay	Queue and uniform delay at a signal (deterministic D/D/1)	lam, s, r, g	Q_max, t_clear, D_total, d_uniform, P_stopped
transport_bottleneck_queue	Queue and delay at a temporary bottleneck (incident, lane closure)	lam, mu, mu_r, t_r	Q_max, T, T_min, D_total
transport_interval_queue	Queue from arrival counts and capacity per interval	arrivals, capacity	queue, Q_max, served
transport_signal_green_splits	Green splits by equal degree of saturation (flow-ratio method)	C, L, y	g, X_c
transport_poisson_arrivals	Probability of n vehicle arrivals in an interval (Poisson)	q, t, n	P_n, P_le, P_ge
transport_headway_probability	Probability of a headway longer than h (negative exponential)	q, h, tau	P_ge, P_lt
transport_gap_acceptance_capacity	Capacity of a minor stream by gap acceptance	q, t_c, t_f, model	c
transport_time_and_space_mean_speed	Time-mean and space-mean speed of spot speeds	speeds	TMS, SMS, s
transport_spot_speed_percentiles	85th, 50th and 15th percentile spot speeds	speeds	p85, p50, p15
transport_safe_following_gap	Minimum safe following gap when the leader brakes	v, t_r, a_l, a_f	gap

id	operation	inputs	outputs
transport_mm1_queue	Single-server random queue (M/M/1), e.g. a toll booth	lam, mu	rho, L, Lq, W, Wq, P0
transport_mmc_queue	Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza	lam, mu, c	P_wait, Lq, Wq, W, L
transport_md1_queue	Single server with constant service time (M/D/1)	lam, mu	Lq, Wq, W, L
transport_erlang_b_blocking	Blocking probability with no waiting room (Erlang B), e.g. parking spaces	lam, mu, c	P_block, carried, lost_per_h
transport_fifo_queue_simulation	First-come-first-served queue from arrival and service times	arrivals, service, c	waits, mean_wait, max_wait, mean_time_in_system, last_departure, max_queue
transport_train_running_time	Running time between stations (accelerate, cruise, brake)	D, v_max, a, b	T, v_peak, profile
transport_rail_station_headway	Minimum headway of trains at a station stop (moving block)	v, a, b, L_t, t_d, s_m	h, trains_per_h
transport_train_braking_distance	Train braking distance with Davis resistance and gradient	m, v1, F_b, A, B, C, G, gamma, v2	S, t
transport_train_acceleration	Train acceleration time and distance (tractive effort curve, Davis resistance)	m, v1, v2, F_max, P, A, B, C, G, gamma	t, d
transport_fastest_route	Fastest route on a road network (link lengths and speeds)	roads, speeds, origin, destination, directed	route, time, distance
transport_travel_time_skim	Zone-to-zone travel time matrix (skim) of a road network	roads, speeds, directed	places, times
transport_all_or_nothing_assignment	All-or-nothing traffic assignment	network, od	flows, VHT
transport_gravity_model	Doubly constrained gravity model of trip distribution	O, D, costs, beta, deterrence	trips
transport_furness_balancing	Growth-factor (Furness) balancing of a trip matrix	base, O, D	trips
transport_logit_mode_shares	Multinomial logit mode shares from utilities	V	P, logsum
transport_logit_from_attributes	Logit mode choice from attributes and coefficients	X, beta, asc	V, P
transport_logit_elasticities	Direct and cross elasticities of logit choice probabilities	V, x, beta	P, direct, cross
transport_nested_logit	Nested logit choice probabilities	V, nest, lam	P, P_nest

id	operation	inputs	outputs
transport_logsum_user_benefit	User benefit of a change from the logit logsum	V0, V1, beta_cost	dCS, logsum0, logsum1
transport_trip_generation_regression	Trip generation by multiple linear regression	X, y, x_new	coef, r2, y_new
transport_understeer_gradient	Understeer gradient and characteristic or critical speed (bicycle model)	m, L, l_f, C_f, C_r	K, K_deg_per_g, behaviour, v_c, v_c_kmh
transport_steady_state_cornering	Steady-state cornering: yaw rate, lateral acceleration, radius, sideslip	m, L, l_f, C_f, C_r, u, delta	r, a_y, R, beta
transport_steer_angle_for_radius	Steer angle needed for a turn of given radius (bicycle model)	m, L, l_f, C_f, C_r, u, R	delta
transport_yaw_natural_frequency	Yaw natural frequency and damping of the bicycle model	m, Iz, L, l_f, C_f, C_r, u	wn, zeta, f_n
transport_quarter_car_frequencies	Quarter-car model: body and wheel-hop natural frequencies	m_s, m_u, k_s, k_t	f_body, f_wheel
transport_quarter_car_transmissibility	Quarter-car model: road-to-body transmissibility at a frequency	m_s, m_u, k_s, c_s, k_t, f	T_body, T_wheel
transport_webster_optimum_cycle	Optimum cycle length by Webster's formula	L, y	C0, Y, C_min, green_total
transport_simplified_curve_formula	Curve radius or speed by the simplified e + f formula	e, f, v, R	R, v, v_kmh
transport_sag_curve_headlight	Sag vertical curve length for headlight sight distance	g1, g2, S	L, A, K, case
transport_vehicle_resistance	Vehicle resistance forces and power at a speed	m, v, C_D, A, rho, f_r, G	F_roll, F_aero, F_grade, F_total, P, P_kW
transport_top_speed	Top speed of a road vehicle	m, P, C_D, A, rho, f_r, G, eta	v_max, v_max_kmh
transport_passing_sight_distance	Passing sight distance (four-component method)	v, m_diff, a, t1, t2, d3	PSD, d1, d2, d3, d4
transport_travel_time_reliability	Travel-time reliability indices (buffer, planning time, travel time index)	times, t_free	mean, t95, buffer_index, planning_time_index, travel_time_index

Braking distance from speed and deceleration, `transport_braking_distance`

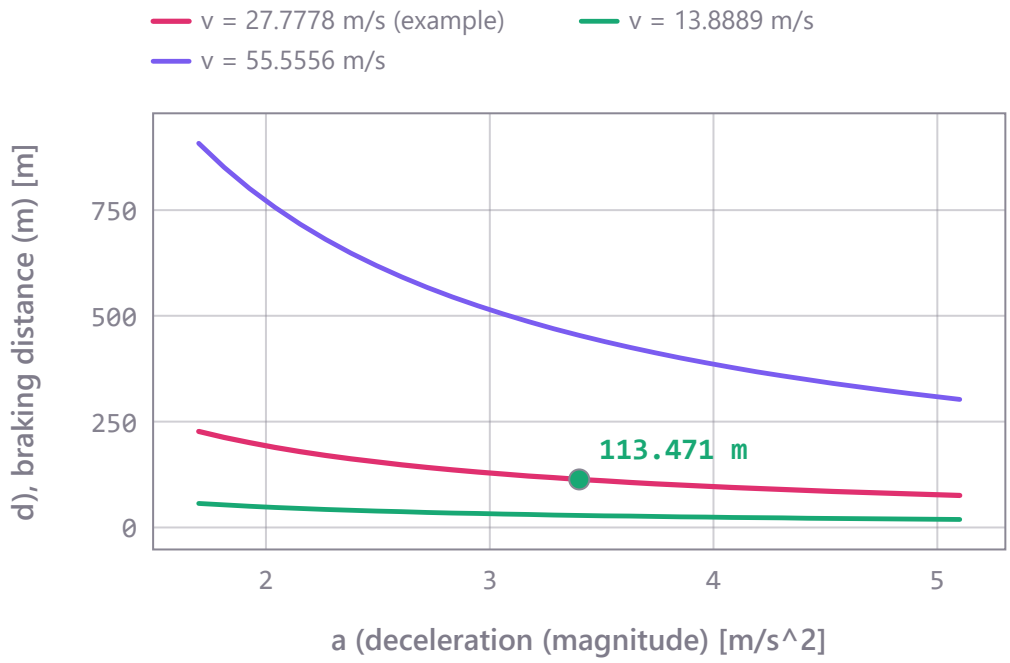
Distance and time for a vehicle to stop from speed v at a constant deceleration a (no reaction time).

$$d = \frac{v^2}{2a}, \quad t = \frac{v}{a}$$

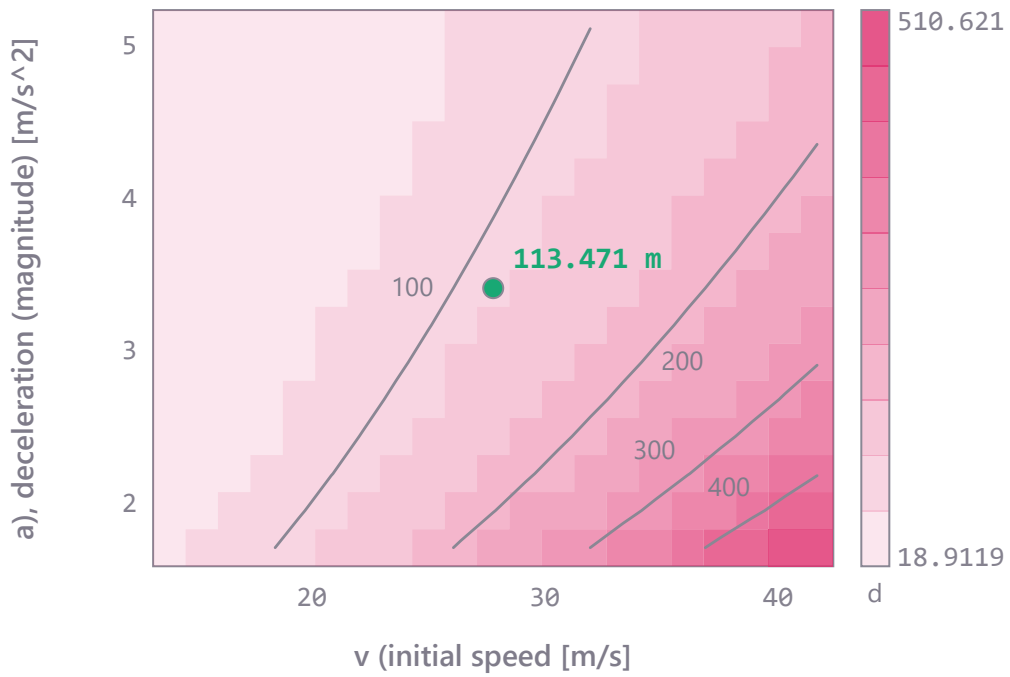
input	meaning	condition
`v`	initial speed	
`a`	deceleration (magnitude)	

output	meaning
`d`	braking distance (m)
`t`	braking time (s)

- **Method:** $d = v^2/(2a)$, $t = v/a$
- **Assumes:** $v > 0$; $a > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d (braking distance (m)) against a (deceleration (magnitude)) from 1.7 m/s^2 to 5.1 m/s^2. One curve for each value of v (initial speed): 27.7778 m/s, 13.8889 m/s, 55.5556 m/s. 31 points per curve computed by transportation-ai999x; marked: the worked example, a = 3.4 m/s^2, d = 113.471 m.



d (braking distance (m)) over v (initial speed) from 13.8889 m/s to 41.6667 m/s and a (deceleration (magnitude)) from 1.7 m/s^2 to 5.1 m/s^2: each of the 21 × 15 cells is one run of transportation-ai999x, shaded by its value (light 18.9119 m to dark 510.621 m); iso-lines at d = 100, 200, 300, 400 m. Marked: the worked example, v = 27.7778 m/s, a = 3.4 m/s^2, d = 113.471 m.

```
transportation_ai999x.solve('transport_braking_distance', v=27.77777777777778, a=3.4)
# -> d = 113.471 m, t = 8.16993 s   (SOLVED)
```

```
transportation_ai999x.solve('transport_braking_distance', v=13.88888888888889, a=7.0)
# -> d = 13.7787 m, t = 1.98413 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Braking distance with speed = 100 km/h and deceleration = 3.4 m/s^2.	SOLVED	d = 113.471 m, t = 8.16993 s
Braking distance with speed = 50 km/h and deceleration = 7 m/s^2.	SOLVED	d = 13.7787 m, t = 1.98413 s
Braking distance with speed = 100 km/h.	NEEDS_INPUT	transport_braking_distance needs a (deceleration (magnitude)). Example: solve('transport_braking_distance', v='27.7778 m
Braking distance with speed = 100 km/h and deceleration = 3.4 m/s^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Braking distance with speed = 100 km/h and deceleration = -3 m/s^2.	INVALID_INPUT	Braking distance from speed and deceleration: a must be positive.

Stopping sight distance (reaction plus braking, with grade), `transport_stopping_sight_distance`

Stopping sight distance = perception-reaction distance + braking distance on a grade, with either the longitudinal friction coefficient f (braking deceleration g (f + G)) or a deceleration rate a (AASHTO: a + g G,

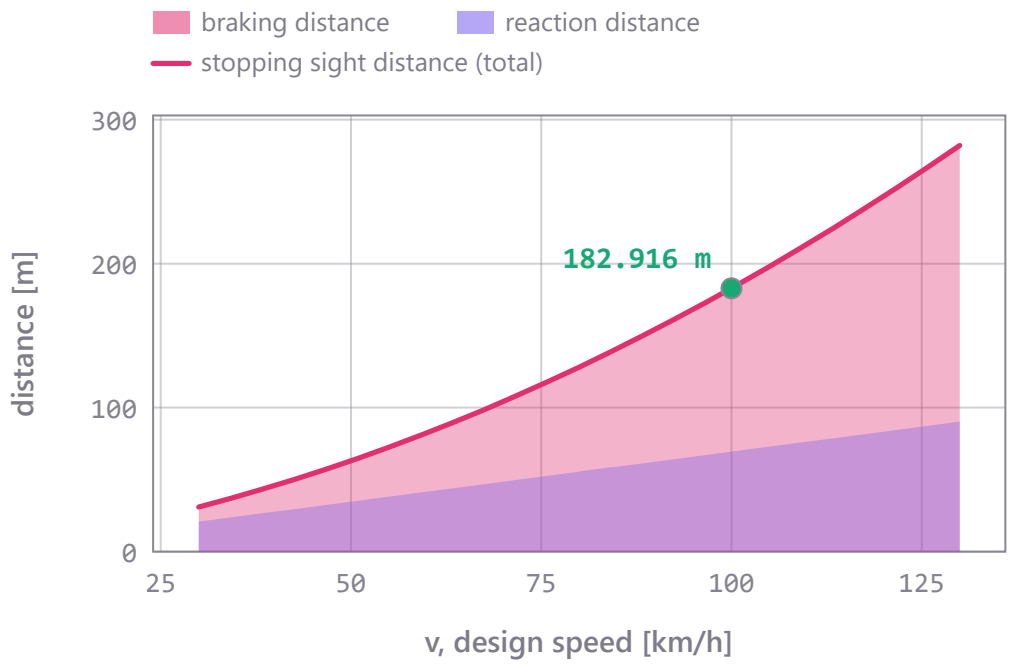
design value 3.4 m/s^2).

$$SSD = v\,t_r + \frac{v^2}{2g(f+G)}$$

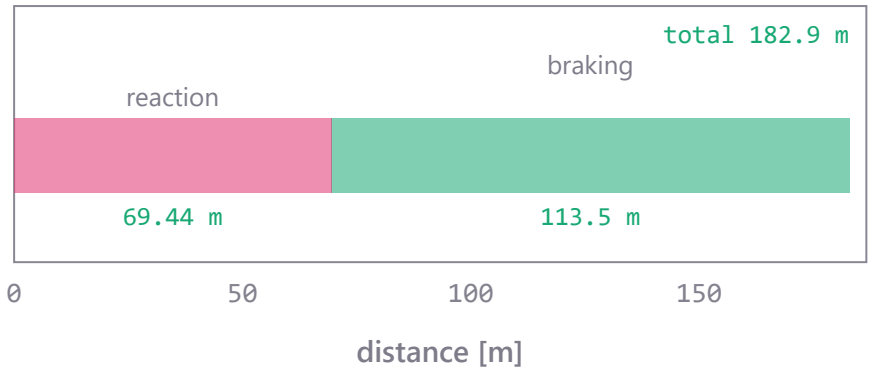
input	meaning	condition
`v`	design speed	
`t_r`	perception-reaction time	
`f`	longitudinal friction coefficient	
`a`	deceleration rate	
`G`	grade, decimal (positive uphill)	

output	meaning
`SSD`	stopping sight distance (m)
`d_r`	perception-reaction distance (m)
`d_b`	braking distance (m)

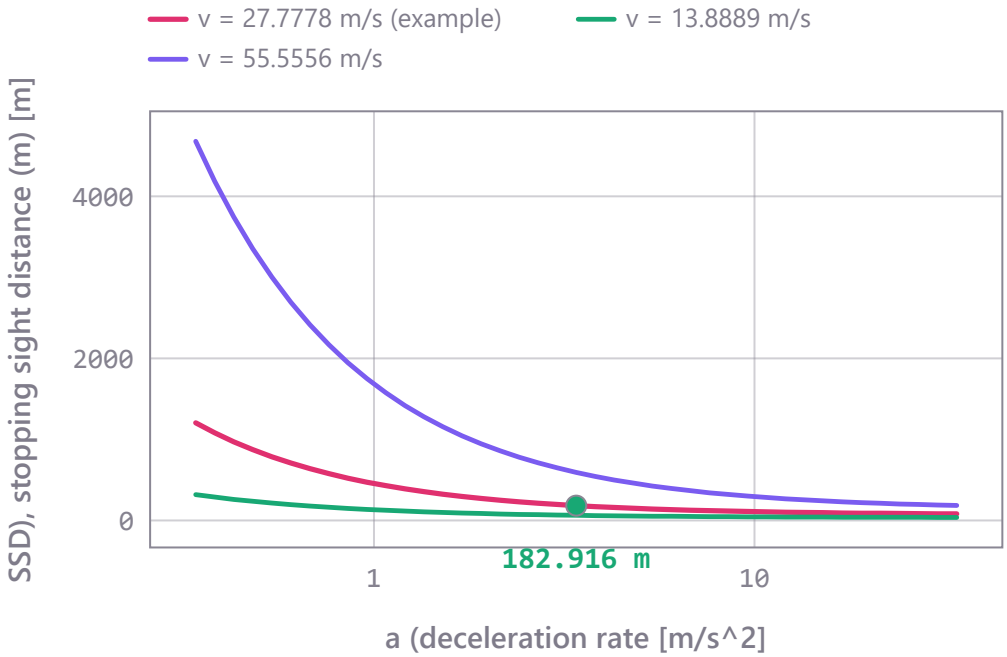
- **Method:** SSD = v t_r + v^2 / (2 g (f + G)), or with a deceleration rate v t_r + v^2 / (2 (a + g G))
- **Assumes:** $v > 0$; $t_r \geq 0$; exactly one of f, a; $f + G > 0$ (or a + g G > 0)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Stopping sight distance against design speed (reaction time 2.5 s, deceleration 3.4 m/s², level road), 25 speeds computed by the package: the lower band is the distance travelled during the reaction time, the upper band the braking distance. Marked: v = 100 km/h, SSD = 182.916 m (reaction 69.4444 m + braking 113.471 m).



Stopping sight distance at v = 100 km/h (reaction time 2.5 s, deceleration 3.4 m/s²): the reaction and braking distances as returned, to scale. Marked: 69.4444 m + 113.471 m = SSD 182.916 m.



SSD (stopping sight distance (m)) against a (deceleration rate) from 0.34 m/s^2 to 34 m/s^2 on a logarithmic axis. One curve for each value of v (design speed): 27.7778 m/s, 13.8889 m/s, 55.5556 m/s. Other inputs as in the worked example: t_r = 2.5 s. 41 points per curve computed by transportation-ai999x; marked: the worked example, a = 3.4 m/s^2, SSD = 182.916 m.

```
transportation_ai999x.solve('transport_stopping_sight_distance', v=27.77777777777778, t_r=2.5,
a=3.4)
# -> SSD = 182.916 m, d_r = 69.4444 m, d_b = 113.471 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_stopping_sight_distance', v=22.22222222222222, t_r=2.5,
f=0.35, G=-0.03)
# -> SSD = 134.21 m, d_r = 55.5556 m, d_b = 78.6549 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Stopping sight distance with speed = 100 km/h, reaction time = 2.5 s and deceleration = 3.4 m/s^2.	SOLVED	SSD = 182.916 m, d_r = 69.4444 m, d_b = 113.471 m
Stopping sight distance with speed = 80 km/h, reaction time = 2.5 s,	SOLVED	SSD = 134.21 m, d_r = 55.5556 m, d_b = 78.6549 m

question	outcome	result
friction = 0.35 and grade = -3 %.		
Stopping sight distance with speed = 100 km/h and deceleration = 3.4 m/s^2.	NEEDS_INPUT	transport_stopping_sight_distance needs t_r (perception-reaction time). Example: solve('transport_stopping_sight_distanc
Stopping sight distance with speed = 100 km/h, reaction time = 2.5 s and deceleration = 3.4 m/s^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Stopping sight distance with speed = 100 km/h, reaction time = 2.5 s, friction = 0.3 and grade = -40 %.	INVALID_INPUT	Stopping sight distance (reaction plus braking, with grade): the grade is too steep: the vehicle cannot stop (f + G must

Speed estimate from skid-mark length, `transport_skid_speed`

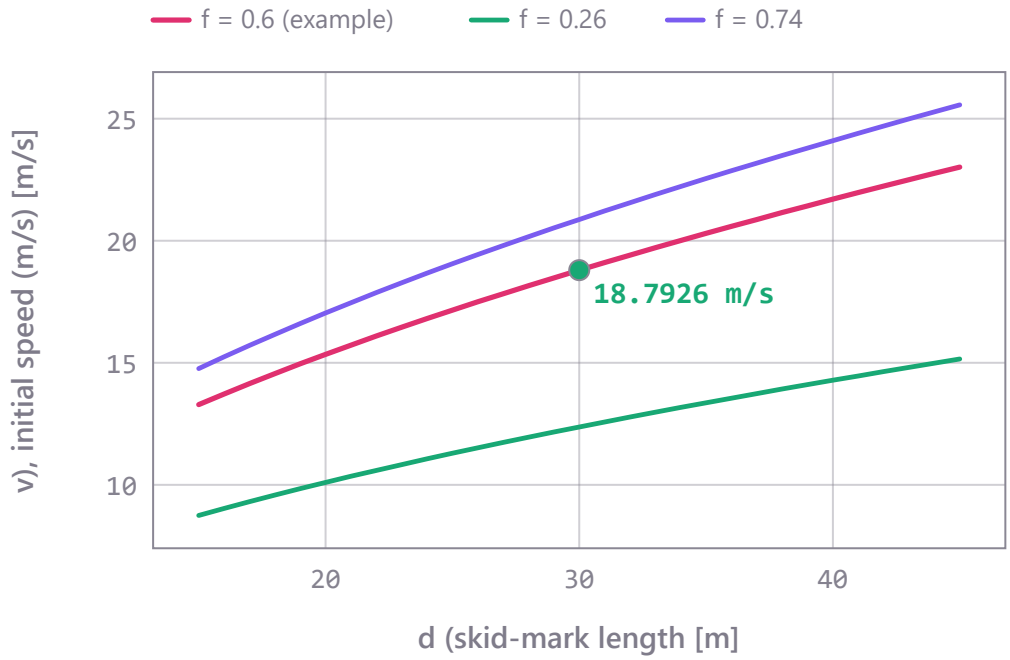
Initial speed of a vehicle that skidded to rest over a length d with friction coefficient f on a grade G.

$$v = \sqrt{2g(f+G)\,d}$$

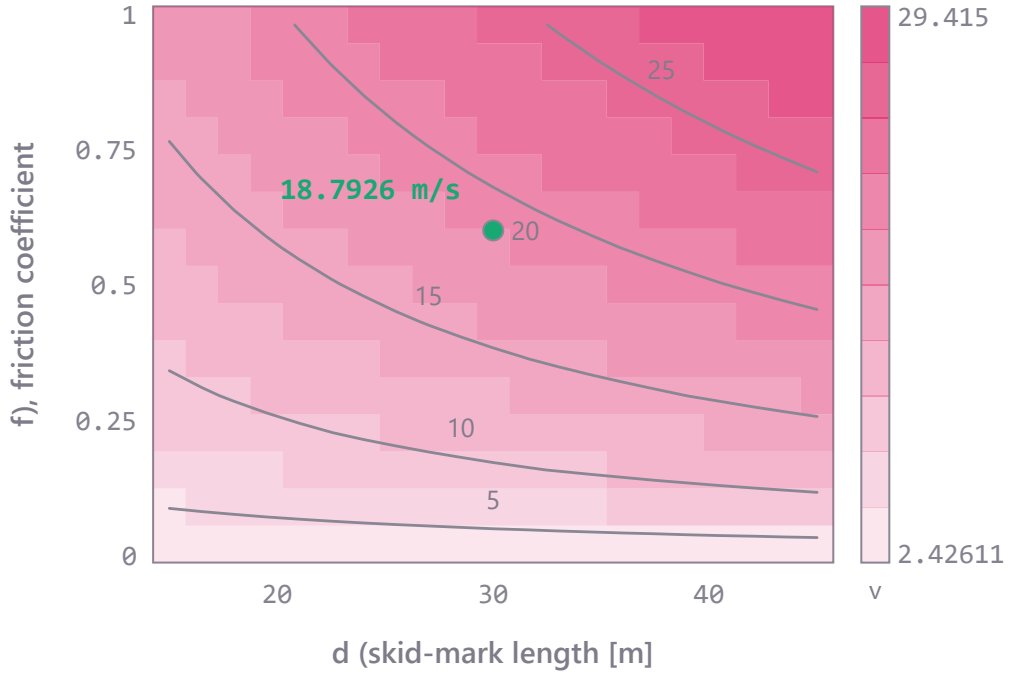
input	meaning	condition
`d`	skid-mark length	
`f`	friction coefficient	
`G`	grade, decimal (positive uphill)	

output	meaning
`v`	initial speed (m/s)
`v_kmh`	initial speed (km/h)

- **Method:** $v = sqrt$(2 g (f + G) d)
- **Assumes:** $d > 0$; $f > 0$; $f + G > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v (initial speed (m/s)) against d (skid-mark length) from 15 m to 45 m. One curve for each value of f (friction coefficient): 0.6, 0.26, 0.74. 31 points per curve computed by transportation-ai999x; marked: the worked example, d = 30 m, v = 18.7926 m/s.



v (initial speed (m/s)) over d (skid-mark length) from 15 m to 45 m and f (friction coefficient) from 0.02 to 0.98: each of the 21 × 15 cells is one run of transportation-ai999x, shaded by its value (light 2.42611 m/s to dark 29.415 m/s); iso-lines at v = 5, 10, 15, 20, 25 m/s. Marked: the worked example, d = 30 m, f = 0.6, v = 18.7926 m/s.

```
transportation_ai999x.solve('transport_skid_speed', d=30.0, f=0.6)
# -> v = 18.7926 m/s, v_kmh = 67.6532 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_skid_speed', d=45.0, f=0.5, G=-0.02)
# -> v = 20.5862 m/s, v_kmh = 74.1104 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Skid mark speed with skid length = 30 m and friction = 0.6.	SOLVED	v = 18.7926 m/s, v_kmh = 67.6532 km/h
Skid mark speed with skid length = 45 m, friction = 0.5 and grade = -2 %.	SOLVED	v = 20.5862 m/s, v_kmh = 74.1104 km/h
Skid mark speed with skid length = 30 m.	NEEDS_INPUT	transport_skid_speed needs f (friction coefficient). Example: solve('transport_skid_speed', d='30 m', f='0.6 1').
Skid mark speed with skid length = 30 m and friction = 0.6 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Skid mark speed with skid length = 30 m and friction = -0.6.	INVALID_INPUT	Speed estimate from skid-mark length: f must be positive.

Distance to slow from one speed to another (friction and grade),

`transport_speed_reduction_distance`

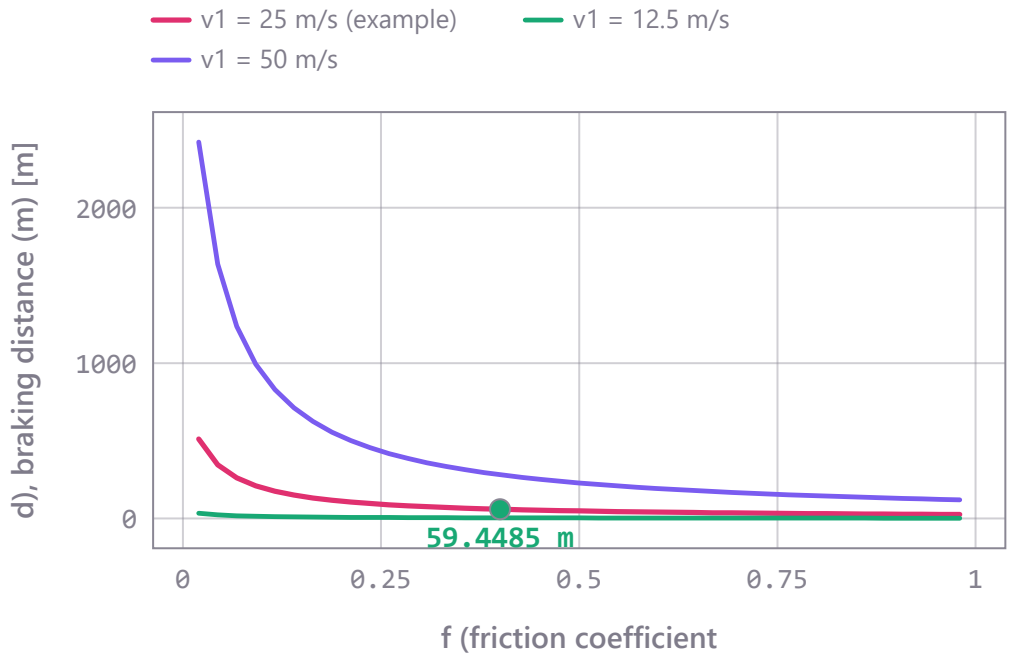
Braking distance and time to slow from v1 to v2 with friction coefficient f on a grade G (deceleration g (f + G)).

$$d = \frac{v_1^2 - v_2^2}{2g(f + G)}$$

input	meaning	condition
`v1`	initial speed	
`v2`	final speed	
`f`	friction coefficient	
`G`	grade, decimal (positive uphill)	

output	meaning
`d`	braking distance (m)
`t`	braking time (s)

- **Method:** d = (v1^2 - v2^2) / (2 g (f + G))
- **Assumes:** $v_1 > v_2 \geq 0$; $f + G > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d (braking distance (m)) against f (friction coefficient) from 0.02 to 0.98. One curve for each value of v1 (initial speed): 25 m/s, 12.5 m/s, 50 m/s. Other inputs as in the worked example: v2 = 11.11111111111111 m/s, G = 0.03. 41 points per curve computed by transportation-ai999x; marked: the worked example, f = 0.4, d = 59.4485 m.

```
transportation_ai999x.solve('transport_speed_reduction_distance', v1=25.0, v2=11.11111111111111,
f=0.4, G=0.03)
# -> d = 59.4485 m, t = 3.29253 s   (SOLVED)
```

```
transportation_ai999x.solve('transport_speed_reduction_distance', v1=33.333333333333336,
v2=16.666666666666668, f=0.3, G=-0.04)
# -> d = 163.36 m, t = 6.53441 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Speed reduction braking distance with initial speed = 90 km/h, final speed = 40 km/h, friction = 0.4 and grade = 3 %.	SOLVED	d = 59.4485 m, t = 3.29253 s
Speed reduction braking distance with initial speed = 120 km/h, final speed = 60 km/h, friction = 0.3 and grade = -4 %.	SOLVED	d = 163.36 m, t = 6.53441 s
Speed reduction braking distance with initial speed = 90 km/h, final speed = 40 km/h and grade = 3 %.	NEEDS_INPUT	transport_speed_reduction_distance needs f (friction coefficient). Example: solve('transport_speed_reduction_distance',
Speed reduction braking distance with initial speed = 90 km/h, final speed = 40 km/h, friction = 0.4 and grade = 3 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Speed reduction braking distance with initial speed = 40 km/h, final speed = 90 km/h and friction = 0.4.	INVALID_INPUT	Distance to slow from one speed to another (friction and grade): the final speed must be below the initial speed.

Impact speed with an obstacle after reaction and braking, `transport_impact_speed`

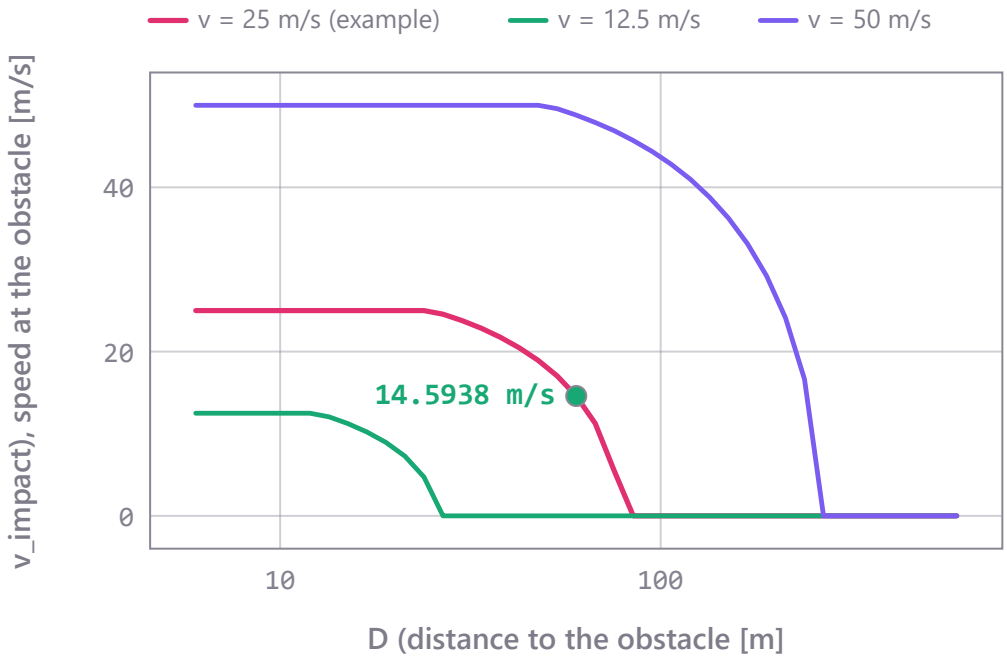
A driver at speed v sees an obstacle at distance D, reacts after t_r and brakes with friction f on grade G: the speed at the obstacle (0 if the vehicle stops in time) and the full stopping distance.

$$v_i = \sqrt{v^2 - 2g(f+G)(D - vt_r)}$$

input	meaning	condition
`v`	initial speed	
`D`	distance to the obstacle	
`t_r`	perception-reaction time	
`f`	friction coefficient	
`G`	grade, decimal (positive uphill)	

output	meaning
`v_impact`	speed at the obstacle (m/s), 0 if the vehicle stops before it
`v_impact_kmh`	speed at the obstacle (km/h)
`d_stop`	stopping distance from the moment the obstacle is seen (m)

- **Method:** $v_i = sqrt(v^2 - 2$ g (f + G) (D - v t_r)) when the obstacle lies within the braking phase
- **Assumes:** $v > 0$; $D > 0$; $t_r \geq 0$; $f + G > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v_impact (speed at the obstacle (m/s), 0 if the vehicle stops before it) against D (distance to the obstacle) from 6 m to 600 m on a logarithmic axis. One curve for each value of v (initial speed): 25 m/s, 12.5 m/s, 50 m/s. Other inputs as in the worked example: t_r = 1.0 s, f = 0.6. 41 points per curve computed by transportation-ai999x; marked: the worked example, D = 60 m, v_impact = 14.5938 m/s.

```
transportation_ai999x.solve('transport_impact_speed', v=25.0, D=60.0, t_r=1.0, f=0.6)
# -> v_impact = 14.5938 m/s, v_impact_kmh = 52.5378 km/h, d_stop = 78.0921 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_impact_speed', v=16.666666666666668, D=30.0, t_r=1.5, f=0.7)
# -> v_impact = 14.4606 m/s, v_impact_kmh = 52.058 km/h, d_stop = 45.2256 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Impact speed with speed = 90 km/h, distance = 60 m, reaction time = 1 s and friction = 0.6.	SOLVED	v_impact = 14.5938 m/s, v_impact_kmh = 52.5378 km/h, d_stop = 78.0921 m
Impact speed with speed = 60 km/h, distance = 30 m, reaction time = 1.5 s and friction = 0.7.	SOLVED	v_impact = 14.4606 m/s, v_impact_kmh = 52.058 km/h, d_stop = 45.2256 m
Impact speed with speed = 90 km/h, distance = 60 m and reaction time = 1 s.	NEEDS_INPUT	transport_impact_speed needs f (friction coefficient). Example: solve('transport_impact_speed', v='25 m/s', D='60 m', t_
Impact speed with speed = 90 km/h, distance = 60 m, reaction time = 1 s and friction = 0.6 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Impact speed with speed = 90 km/h, distance = -60 m, reaction time = 1 s and friction = 0.6.	INVALID_INPUT	Impact speed with an obstacle after reaction and braking: D must be positive.

Distance and time to accelerate (acceleration lane length), `transport_acceleration_distance`

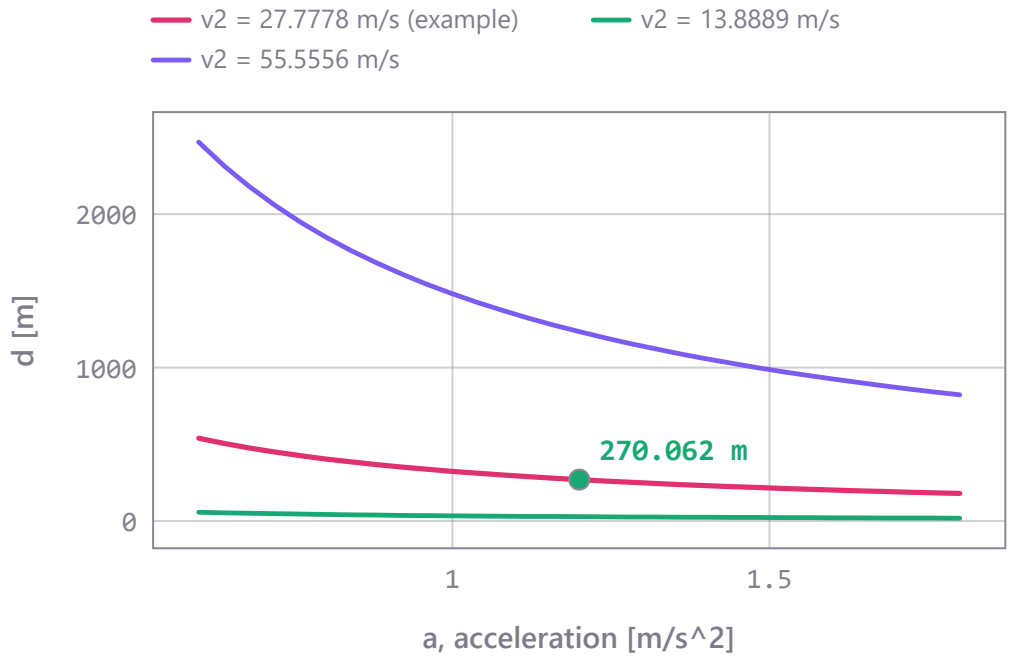
Distance and time for a vehicle to accelerate from v1 to v2 at a constant acceleration a (for example the length of an acceleration lane).

$$d = \frac{v_2^2 - v_1^2}{2a}$$

input	meaning	condition
`v1`	initial speed	
`v2`	final speed	
`a`	acceleration	

output	meaning
`d`	distance travelled while accelerating (m)
`t`	time taken (s)

- **Method:** $d = (v_2^2 - v_1^2)/(2a)$, $t = (v_2 - v_1)/a$
- **Assumes:** $v_2 > v_1 \geq 0$; $a > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



d (distance travelled while accelerating (m)) against a (acceleration) from 0.6 m/s^2 to 1.8 m/s^2. One curve for each value of v2 (final speed): 27.7778 m/s, 13.8889 m/s, 55.5556 m/s. Other inputs as in the worked example: v1 = 11.11111111111111 m/s. 31 points per curve computed by transportation-ai999x; marked: the worked example, a = 1.2 m/s^2, d = 270.062 m.

```
transportation_ai999x.solve('transport_acceleration_distance', v1=11.11111111111111,
v2=27.77777777777778, a=1.2)
# -> d = 270.062 m, t = 13.8889 s   (SOLVED)
```

```
transportation_ai999x.solve('transport_acceleration_distance', v1=0.0, v2=13.88888888888889, a=2.5)
# -> d = 38.5802 m, t = 5.55556 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Acceleration distance with initial speed = 40 km/h, final speed = 100 km/h and acceleration = 1.2 m/s^2.	SOLVED	d = 270.062 m, t = 13.8889 s
Acceleration distance with initial speed = 0 km/h, final speed = 50 km/h and acceleration = 2.5 m/s^2.	SOLVED	d = 38.5802 m, t = 5.55556 s
Acceleration distance with initial speed = 40 km/h and final speed = 100 km/h.	NEEDS_INPUT	transport_acceleration_distance needs a (acceleration). Example: solve('transport_acceleration_distance', v1='11.1111 m/
Acceleration distance with initial speed = 40 km/h, final speed = 100 km/h and acceleration = 1.2 m/s^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Acceleration distance with initial speed = 100 km/h, final speed = 40 km/h and acceleration = 1.2 m/s^2.	INVALID_INPUT	Distance and time to accelerate (acceleration lane length): the final speed must exceed the initial speed.

Yellow change interval (kinematic, ITE), `transport_yellow_change_interval`

Minimum yellow change interval $Y = t_r + v/(2a + 2gG)$ so that a driver who is just too close to stop can clear at the approach speed, and the critical stopping distance from the stop line.

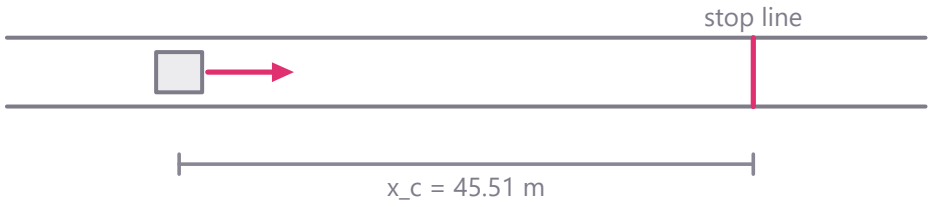
$$Y = t_r + \frac{v}{2a + 2gG}$$

input	meaning	condition
`v`	approach speed	
`t_r`	perception-reaction time	
`a`	comfortable deceleration rate	
`G`	grade, decimal (positive uphill)	

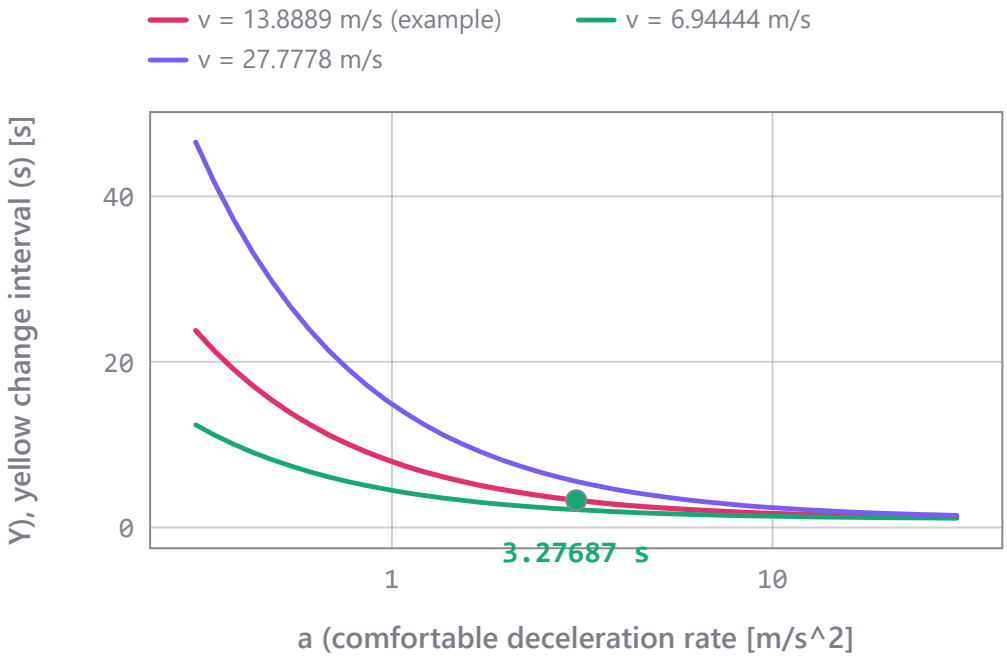
output	meaning
`Y`	yellow change interval (s)
`x_c`	critical stopping distance (m)

- **Method:** Y = t_r + v / (2a + 2 g G); $x_c = v$ t_r + v^2 / (2 (a + g G))
- **Assumes:** $v > 0$; a + g $G > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
yellow interval Y = 3.277 s
```



A vehicle approaching a signal at v = 50 km/h (reaction time 1.0 s, deceleration 3.05 m/s^2): from the returned critical distance x_c = 45.512 m it can just stop; the returned yellow change interval is Y = 3.27687 s. Distances to scale.



Y (yellow change interval (s)) against a (comfortable deceleration rate) from 0.305 m/s^2 to 30.5 m/s^2 on a logarithmic axis. One curve for each value of v (approach speed): 13.8889 m/s, 6.94444 m/s, 27.7778 m/s. Other inputs as in the worked example: t_r = 1.0 s. 41 points per curve computed by transportation-ai999x; marked: the worked example, a = 3.05 m/s^2, Y = 3.27687 s.

```
transportation_ai999x.solve('transport_yellow_change_interval', v=13.88888888888889, t_r=1.0,
a=3.05)
# -> Y = 3.27687 s, x_c = 45.512 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_yellow_change_interval', v=16.666666666666668, t_r=1.0,
a=3.05, G=-0.02)
# -> Y = 3.92008 s, x_c = 65.3347 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Yellow change interval with speed = 50 km/h, reaction time = 1 s and deceleration = 3.05 m/s^2.	SOLVED	Y = 3.27687 s, x_c = 45.512 m
Yellow change interval with speed = 60 km/h, reaction time = 1 s, deceleration = 3.05 m/s^2 and grade = -2 %.	SOLVED	Y = 3.92008 s, x_c = 65.3347 m
Yellow change interval with speed = 50 km/h and reaction time = 1 s.	NEEDS_INPUT	transport_yellow_change_interval needs a (comfortable deceleration rate). Example: solve('transport_yellow_change_interv
Yellow change interval with speed = 50 km/h, reaction time = 1 s and deceleration = 3.05 m/s^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Yellow change interval with speed = 50 km/h, reaction time = 1 s and deceleration = -3.05 m/s^2.	INVALID_INPUT	Yellow change interval (kinematic, ITE): a must be positive.

Overtaking manoeuvre: time and distance with constant acceleration,

`transport_overtaking_manoeuvre`

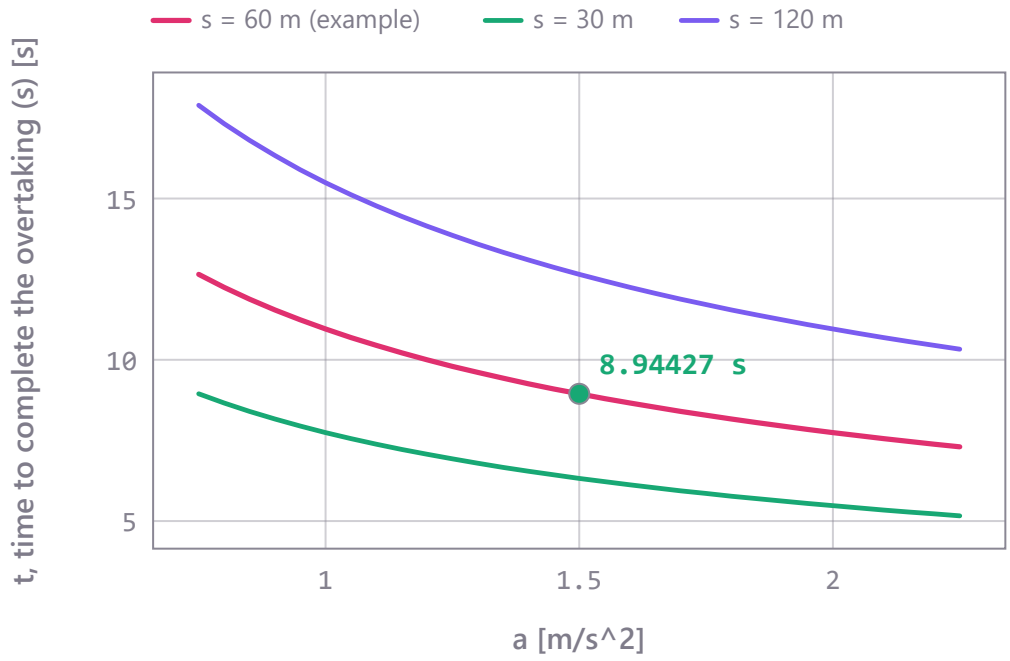
A vehicle that starts at the speed v of the vehicle it passes accelerates at a until it has gained the relative distance s (initial gap + both lengths + final gap): the time, its distance travelled and its final speed.

$$t = \sqrt{\frac{2s}{a}}, \quad d = vt + \tfrac{1}{2}at^2$$

input	meaning	condition
`v`	speed of the overtaken vehicle (and initial speed of the passing vehicle)	
`a`	acceleration of the passing vehicle	
`s`	relative distance to gain	

output	meaning
`t`	time to complete the overtaking (s)
`d`	distance travelled by the passing vehicle (m)
`v_f`	final speed of the passing vehicle (m/s)

- **Method:** $t = sqrt$(2 s / a), $d = v$ t + a $t^2/2$
- **Assumes:** $v \geq 0$; $a > 0$; $s > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (time to complete the overtaking (s)) against a (acceleration of the passing vehicle) from 0.75 m/s^2 to 2.25 m/s^2. One curve for each value of s (relative distance to gain): 60 m, 30 m, 120 m. Other inputs as in the worked example: v = 19.444444444444446 m/s. 31 points per curve computed by transportation-ai999x; marked: the worked example, a = 1.5 m/s^2, t = 8.94427 s.

```
transportation_ai999x.solve('transport_overtaking_manoeuvre', v=19.444444444444446, a=1.5, s=60.0)
# -> t = 8.94427 s, d = 233.916 m, v_f = 32.8609 m/s   (SOLVED)
```

```
transportation_ai999x.solve('transport_overtaking_manoeuvre', v=13.88888888888889, a=2.0, s=40.0)
# -> t = 6.32456 s, d = 127.841 m, v_f = 26.538 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Overtaking manoeuvre with speed = 70 km/h, acceleration = 1.5 m/s^2 and relative distance = 60 m.	SOLVED	t = 8.94427 s, d = 233.916 m, v_f = 32.8609 m/s
Overtaking manoeuvre with speed = 50 km/h, acceleration = 2 m/s^2 and relative distance = 40 m.	SOLVED	t = 6.32456 s, d = 127.841 m, v_f = 26.538 m/s
Overtaking manoeuvre with speed = 70 km/h and acceleration = 1.5 m/s^2.	NEEDS_INPUT	transport_overtaking_manoeuvre needs s (relative distance to gain). Example: solve('transport_overtaking_manoeuvre', v='
Overtaking manoeuvre with speed = 70 km/h, acceleration = 1.5 m/s^2 and relative distance = 60 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Overtaking manoeuvre with speed = 70 km/h, acceleration = -1.5 m/s^2 and relative distance = 60 m.	INVALID_INPUT	Overtaking manoeuvre: time and distance with constant acceleration: a must be positive.

Theoretical stopping distance with aerodynamic and rolling resistance,

`transport_braking_with_air_resistance`

Stopping (or slowing) distance and time of a road vehicle braking with tyre-road friction mu and braking efficiency eta, helped by aerodynamic drag (rho C_D A v^2 / 2), rolling resistance f_r and the grade; gamma is the mass factor for rotating parts (1.04 for braking).

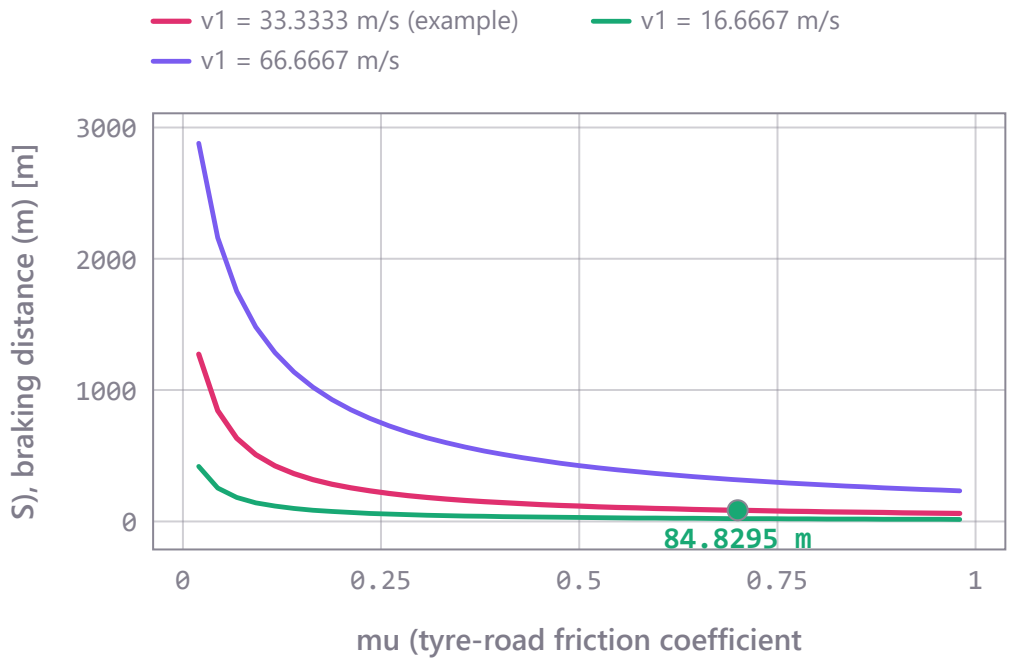
$$S = \frac{\gamma m}{2K} \ln \frac{F_0 + K v_1^2}{F_0 + K v_2^2}$$

input	meaning	condition
`m`	vehicle mass	
`v1`	initial speed	
`mu`	tyre-road friction coefficient	
`C_D`	drag coefficient	
`A`	frontal area	
`rho`	air density (default 1.2 kg/m^3)	
`f_r`	rolling resistance coefficient (default 0.01)	
`G`	grade, decimal (positive uphill)	
`eta`	braking efficiency (default 1)	
`gamma`	mass factor for braking (default 1.04)	

input	meaning	condition
v2	final speed (default 0)	

output	meaning
S	braking distance (m)
t	braking time (s)

- **Method:** $S = \gamma\, m/(2K)$ ln((F0 + K v_1^2)/($F_0 + K$ v2^2)), $K = \rho$ C_D $A/2$, $F_0 = W$ ((eta mu + f_r) cos(theta) + sin(theta))
- **Assumes:** m, v1, mu, C_D, $A > 0$; $0 < \eta \leq 1$; $v_2 < v_1$; a positive constant retarding force
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S (braking distance (m)) against mu (tyre-road friction coefficient) from 0.02 to 0.98. One curve for each value of v1 (initial speed): 33.3333 m/s, 16.6667 m/s, 66.6667 m/s. Other inputs as in the worked example: m = 1500.0 kg, C_D = 0.35, A = 2.2 m^2, f_r = 0.012, eta = 0.95. 41 points per curve computed by transportation-ai999x; marked: the worked example, mu = 0.7, S = 84.8295 m.

```
transportation_ai999x.solve('transport_braking_with_air_resistance', m=1500.0,
v1=33.333333333333336, mu=0.7, C_D=0.35, A=2.2, f_r=0.012, eta=0.95)
# -> S = 84.8295 m, t = 5.13282 s   (SOLVED)
```

```
transportation_ai999x.solve('transport_braking_with_air_resistance', m=2000.0,
v1=27.77777777777778, mu=0.5, C_D=0.4, A=2.6, G=-0.03, v2=8.333333333333334)
# -> S = 75.4846 m, t = 4.19776 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Theoretical stopping distance with mass = 1500 kg, initial speed = 120 km/h, friction = 0.7, drag coefficient = 0.35, frontal area = 2.2 m^2, rolling	SOLVED	S = 84.8295 m, t = 5.13282 s

question	outcome	result
resistance = 0.012 and braking efficiency = 0.95.		
Theoretical stopping distance with mass = 2000 kg, initial speed = 100 km/h, friction = 0.5, drag coefficient = 0.4, frontal area = 2.6 m^2, grade = -3 % and final speed = 30 km/h.	SOLVED	S = 75.4846 m, t = 4.19776 s
Theoretical stopping distance with mass = 1500 kg, initial speed = 120 km/h, friction = 0.7, drag coefficient = 0.35, rolling resistance = 0.012 and braking efficiency = 0.95.	NEEDS_INPUT	transport_braking_with_air_resistance needs A (frontal area). Example: solve('transport_braking_with_air_resistance', m=
Theoretical stopping distance with mass = 1500 kg, initial speed = 120 km/h, friction = 0.7, drag coefficient = 0.35, frontal area = 2.2 m^2, rolling resistance = 0.012 and braking efficiency = 0.95 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Theoretical stopping distance with mass = 1500 kg, initial speed = 120 km/h, friction = 0.7, drag coefficient = 0.35, frontal area = 2.2 m^2 and braking efficiency = 1.5.	INVALID_INPUT	Theoretical stopping distance with aerodynamic and rolling resistance: the braking efficiency must not exceed 1.

Coast-down time and distance (rolling and aerodynamic resistance), `transport_coast_down`

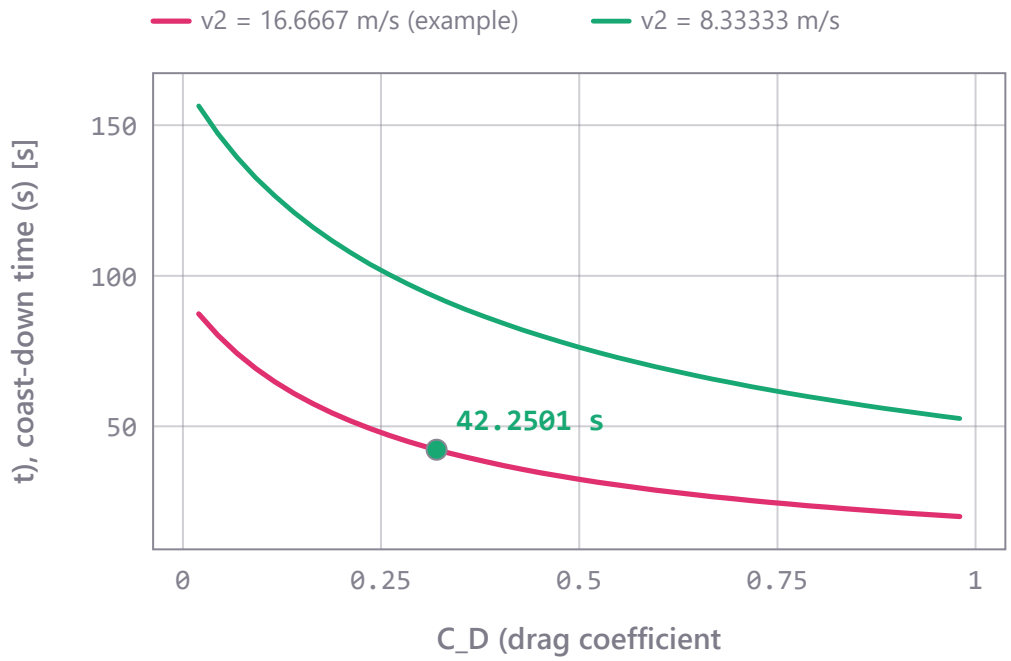
Time and distance for a vehicle coasting in neutral to slow from v1 to v2 under rolling resistance f_r, the grade and aerodynamic drag rho C_D A $v^2/2$.

$$t = \frac{\gamma m}{\sqrt{K F_0}} \left[\arctan\left(v_1 \sqrt{K/F_0} \right) - \arctan\left(v_2 \sqrt{K/F_0} \right) \right]$$

input	meaning	condition
`m`	vehicle mass	
`v1`	initial speed	
`v2`	final speed	
`C_D`	drag coefficient	
`A`	frontal area	
`rho`	air density (default 1.2 kg/m^3)	
`f_r`	rolling resistance coefficient (default 0.012)	
`G`	grade, decimal (positive uphill)	
`gamma`	mass factor (default 1)	

output	meaning
`t`	coast-down time (s)
`d`	coast-down distance (m)

- **Method:** $t = \gamma$ m / sqrt(K F0) (atan(v1 sqrt(K/F0)) - atan(v2 sqrt(K/F0))); $d = \gamma\, m/(2K)$ ln((F0 + K v_1^2)/($F_0 + K$ v2^2))
- **Assumes:** $v_2 < v_1$; $F_0 = W$ (f_r cos(theta) + sin(theta)) > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (coast-down time (s)) against C_D (drag coefficient) from 0.02 to 0.98. One curve for each value of v2 (final speed): 16.6667 m/s, 8.33333 m/s. Other inputs as in the worked example: m = 1400.0 kg, v1 = 27.77777777777778 m/s, A = 2.2 m^2. 41 points per curve computed by transportation-ai999x; marked: the worked example, C_D = 0.32, t = 42.2501 s.

```
transportation_ai999x.solve('transport_coast_down', m=1400.0, v1=27.77777777777778,
v2=16.666666666666668, C_D=0.32, A=2.2)
# -> t = 42.2501 s, d = 917.248 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_coast_down', m=1800.0, v1=22.22222222222222,
v2=5.555555555555555, C_D=0.38, A=2.5, f_r=0.015, G=0.01)
# -> t = 54.106 s, d = 717.141 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Coast down time with mass = 1400 kg, initial speed = 100 km/h, final speed = 60 km/h, drag coefficient = 0.32 and frontal area = 2.2 m^2.	SOLVED	t = 42.2501 s, d = 917.248 m
Coast down time with mass = 1800 kg, initial speed = 80 km/h, final speed = 20 km/h, drag coefficient = 0.38, frontal area = 2.5 m^2, rolling resistance = 0.015 and grade = 1 %.	SOLVED	t = 54.106 s, d = 717.141 m
Coast down time with mass = 1400 kg, initial speed = 100 km/h, final speed = 60 km/h and drag coefficient = 0.32.	NEEDS_INPUT	transport_coast_down needs A (frontal area). Example: solve('transport_coast_down', m='1400 kg', v1='27.7778 m/s', v2='1

question	outcome	result
Coast down time with mass = 1400 kg, initial speed = 100 km/h, final speed = 60 km/h, drag coefficient = 0.32 and frontal area = 2.2 m^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Coast down time with mass = 1400 kg, initial speed = 60 km/h, final speed = 100 km/h, drag coefficient = 0.32 and frontal area = 2.2 m^2.	INVALID_INPUT	Coast-down time and distance (rolling and aerodynamic resistance): the final speed must be below the initial speed.

Acceleration time and distance under constant engine power,

`transport_acceleration_under_power`

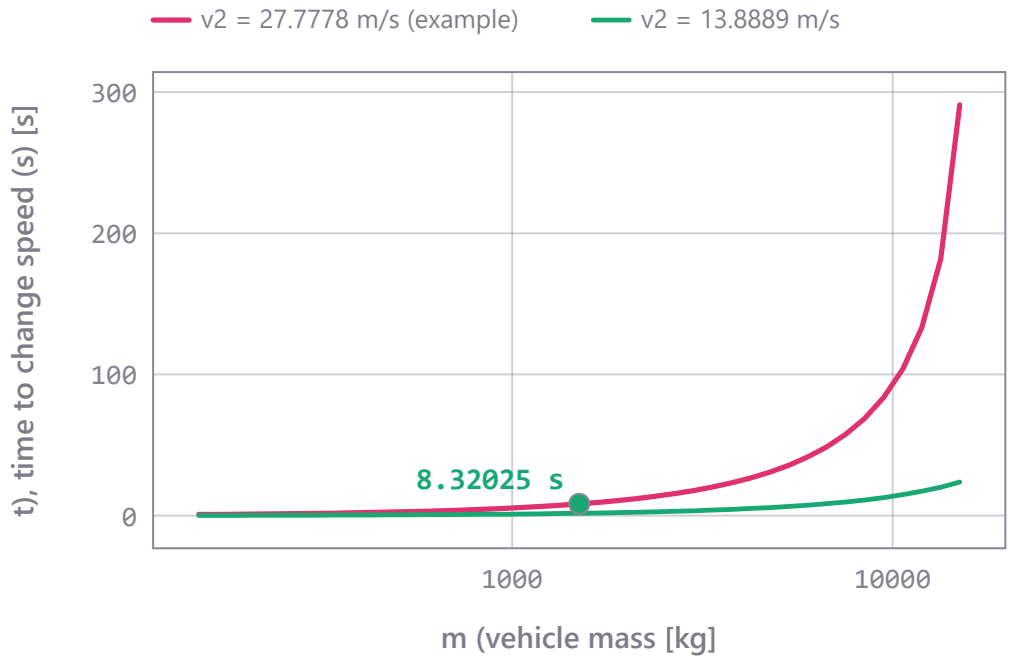
Time and distance for a vehicle of mass m to change speed from v1 to v2 with constant power P at the wheels (tractive force eta P / v) against rolling, aerodynamic and grade resistance. A truck slowing on an upgrade towards its crawl speed is covered too $(v_2 < v_1)$; both speeds must lie on the same side of the speed at which the power balances the resistance.

$$t = \int_{v_1}^{v_2} \frac{\gamma m \, dv}{\eta P / v - F_0 - K v^2}$$

input	meaning	condition
`m`	vehicle mass	
`P`	engine power	
`v1`	initial speed	
`v2`	final speed	
`C_D`	drag coefficient	
`A`	frontal area	
`rho`	air density (default 1.2 kg/m^3)	
`f_r`	rolling resistance coefficient (default 0.015)	
`G`	grade, decimal (positive uphill)	
`eta`	drivetrain efficiency (default 1)	
`gamma`	mass factor for rotating parts (default 1)	

output	meaning
`t`	time to change speed (s)
`d`	distance travelled (m)

- **Method:** t = integral of gamma m dv / (eta $P/v - F_0 - K$ v^2), d = integral of gamma m v dv / (...) (numerical quadrature)
- **Assumes:** v1, $v_2 > 0$ on the same side of the equilibrium speed; $\eta \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (time to change speed (s)) against m (vehicle mass) from 150 kg to 15000 kg on a logarithmic axis. One curve for each value of v2 (final speed): 27.7778 m/s, 13.8889 m/s. Other inputs as in the worked example: P = 75000.0 W, v1 = 5.555555555555555 m/s, C_D = 0.32, A = 2.2 m^2. 41 points per curve computed by transportation-ai999x; marked: the worked example, m = 1500 kg, t = 8.32025 s.

```
transportation_ai999x.solve('transport_acceleration_under_power', m=1500.0, P=75000.0,
v1=5.555555555555555, v2=27.77777777777778, C_D=0.32, A=2.2)
# -> t = 8.32025 s, d = 162.054 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_acceleration_under_power', m=25000.0, P=250000.0,
v1=22.22222222222222, v2=19.444444444444446, C_D=0.6, A=8.5, f_r=0.008, G=0.05)
# -> t = 20.2751 s, d = 419.709 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Acceleration under constant power with mass = 1500 kg, power = 75 kW, initial speed = 20 km/h, final speed = 100 km/h, drag coefficient = 0.32 and frontal area = 2.2 m^2.	SOLVED	t = 8.32025 s, d = 162.054 m
Acceleration under constant power with mass = 25000 kg, power = 250 kW, initial speed = 80 km/h, final speed = 70 km/h, drag coefficient = 0.6, frontal area = 8.5 m^2, rolling resistance = 0.008 and grade = 5 %.	SOLVED	t = 20.2751 s, d = 419.709 m
Acceleration under constant power with mass = 1500 kg, power = 75 kW, initial speed = 20 km/h, final speed = 100 km/h and drag coefficient = 0.32.	NEEDS_INPUT	transport_acceleration_under_power needs A (frontal area). Example: solve('transport_acceleration_under_power', m='1500
Acceleration under constant power with mass = 1500 kg, power = 75 kW, initial speed = 20 km/h, final speed = 100	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
km/h, drag coefficient = 0.32 and frontal area = 2.2 m^2 and colour = 3.		
Acceleration under constant power with mass = 1500 kg, power = 75 kW, initial speed = 20 km/h, final speed = 1000 km/h, drag coefficient = 0.32 and frontal area = 2.2 m^2.	INVALID_INPUT	Acceleration time and distance under constant engine power: the vehicle cannot go from v1 to v2: its equilibrium speed u

Maximum tractive force and acceleration limited by tyre friction,

`transport_traction_limited_acceleration`

The largest tractive force the road can transmit (rear-, front- or all-wheel drive), the resulting maximum acceleration and the dynamic axle loads, with weight transfer from a centre of gravity at height h and rolling resistance f_r (aerodynamic drag neglected, level road).

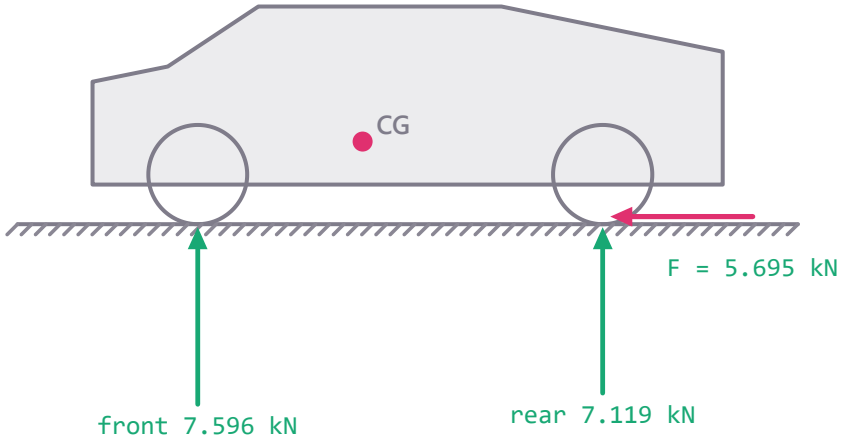
$$F_{max} = \frac{\mu W (l_f - f_{rl} h)/L}{1 - \mu h / L}$$

input	meaning	condition
`m`	vehicle mass	
`L`	wheelbase	
`l_f`	distance from the front axle to the centre of gravity	
`h`	height of the centre of gravity	
`mu`	tyre-road friction coefficient	
`f_r`	rolling resistance coefficient (default 0)	
`drive`	driven axles: rear, front or all	

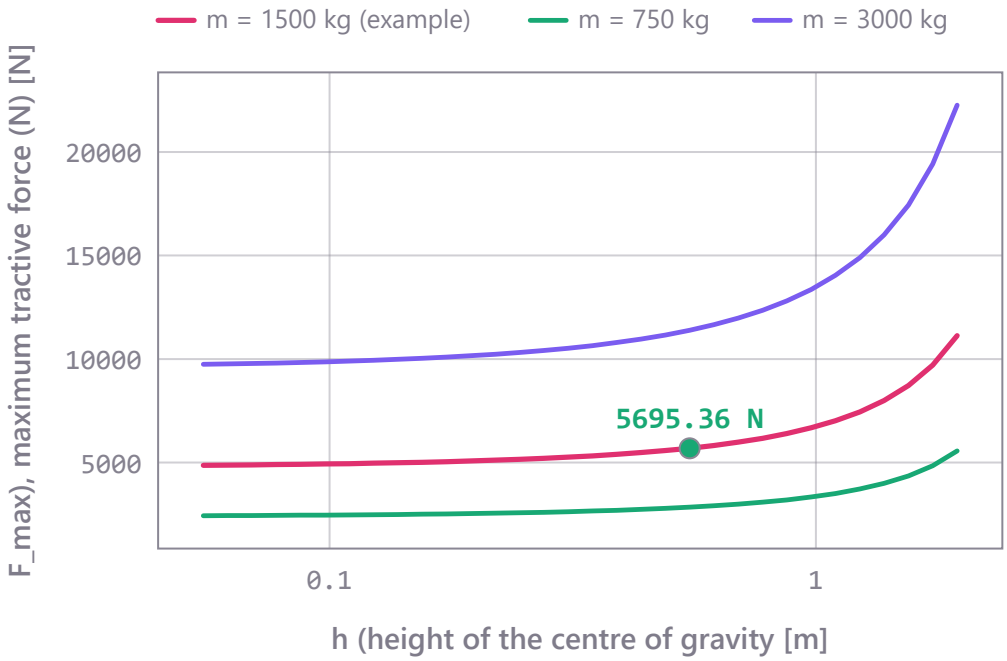
output	meaning
`F_max`	maximum tractive force (N)
`a_max`	maximum acceleration (m/s^2)
`W_f`	front axle load (N)
`W_r`	rear axle load (N)

- **Method:** RWD: $F = mu$ W (l_f - f_r $h)/L/(1 - mu$ h / L); FWD: $F = mu$ W (l_r + f_r $h)/L/(1 + mu$ h / L); AWD: $F = mu$ W
- **Assumes:** $0 < l_f < L$; mu $h < L$ for rear-wheel drive; both axles keep contact
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
rear-wheel drive, μ = 0.8: a_max = 3.679 m/s²
```



Side view of a rear-wheel-drive vehicle (m = 1500 kg, wheelbase 2.7 m, CG 1.1 m behind the front axle, 0.55 m high, μ = 0.8, rolling resistance 0.012) accelerating to the left at its traction limit. Marked: the returned largest tractive force 5.69536 kN at the driven wheels, acceleration 3.67918 m/s² and axle loads front 7.5958 kN, rear 7.1192 kN.



F_max (maximum tractive force (N)) against h (height of the centre of gravity) from 0.055 m to 5.5 m on a logarithmic axis. One curve for each value of m (vehicle mass): 1500 kg, 750 kg, 3000 kg. Other inputs as in the worked example: L = 2.7 m, l_f = 1.1 m, mu = 0.8, f_r = 0.012, drive = rear. 32 points per curve computed by transportation-ai999x; marked: the worked example, h = 0.55 m, F_max = 5695.36 N.

```
transportation_ai999x.solve('transport_traction_limited_acceleration', m=1500.0, L=2.7, l_f=1.1,
h=0.55, mu=0.8, f_r=0.012, drive='rear')
# -> F_max = 5695.36 N, a_max = 3.67918 m/s^2, W_f = 7595.8 N, W_r = 7119.2 N   (SOLVED)
```

```
transportation_ai999x.solve('transport_traction_limited_acceleration', m=1200.0, L=2.5, l_f=1.0,
h=0.5, mu=0.6, drive='front')
# -> F_max = 3783.86 N, a_max = 3.15321 m/s^2, W_f = 6306.43 N, W_r = 5465.57 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Traction limited acceleration with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.1 m, cg height = 0.55 m, friction = 0.8, rolling resistance = 0.012 and drive = rear.	SOLVED	F_max = 5695.36 N, a_max = 3.67918 m/s^2, W_f = 7595.8 N, W_r = 7119.2 N
Traction limited acceleration with mass = 1200 kg, wheelbase = 2.5 m, cg to front axle = 1 m, cg height = 0.5 m, friction = 0.6 and drive = front.	SOLVED	F_max = 3783.86 N, a_max = 3.15321 m/s^2, W_f = 6306.43 N, W_r = 5465.57 N
Traction limited acceleration with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.1 m, cg height = 0.55 m, rolling resistance = 0.012 and drive = rear.	NEEDS_INPUT	transport_traction_limited_acceleration needs mu (tyre-road friction coefficient). Example: solve('transport_traction_li
Traction limited acceleration with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.1 m, cg height = 0.55 m, friction = 0.8, rolling resistance = 0.012 and drive = rear and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Traction limited acceleration with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 3.1 m, cg height = 0.55 m, friction = 0.8 and drive = rear.	INVALID_INPUT	Maximum tractive force and acceleration limited by tyre friction: the distance from the front axle to the centre of grav

Longitudinal weight transfer under braking and ideal brake proportioning,

`transport_braking_weight_transfer`

Dynamic front and rear axle loads of a vehicle decelerating at a, the load transferred to the front axle, and the ideal front share of the braking force (both axles at the same friction use).

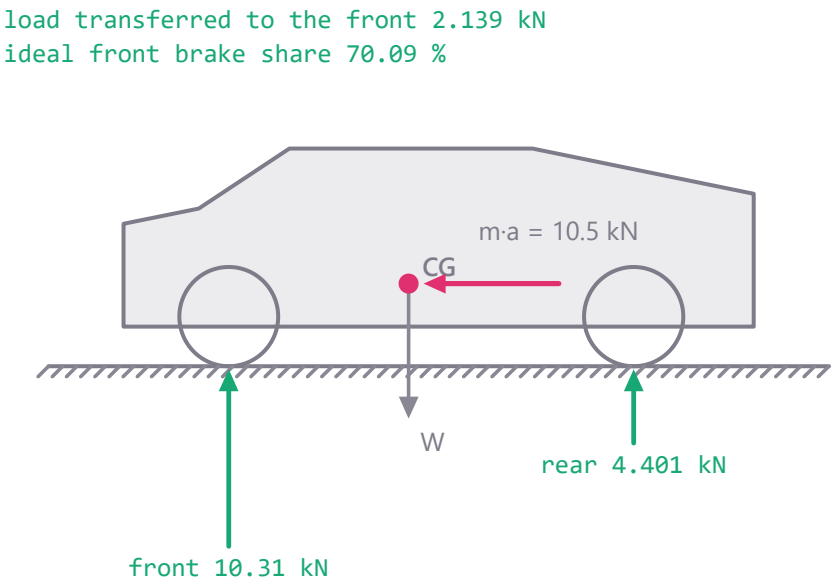
$$W_f = \frac{W l_r + m a h}{L}, \quad \Delta W = \frac{m a h}{L}$$

input	meaning	condition
`m`	vehicle mass	
`a`	deceleration	
`h`	height of the centre of gravity	
`L`	wheelbase	
`l_f`	distance from the front axle to the centre of gravity	

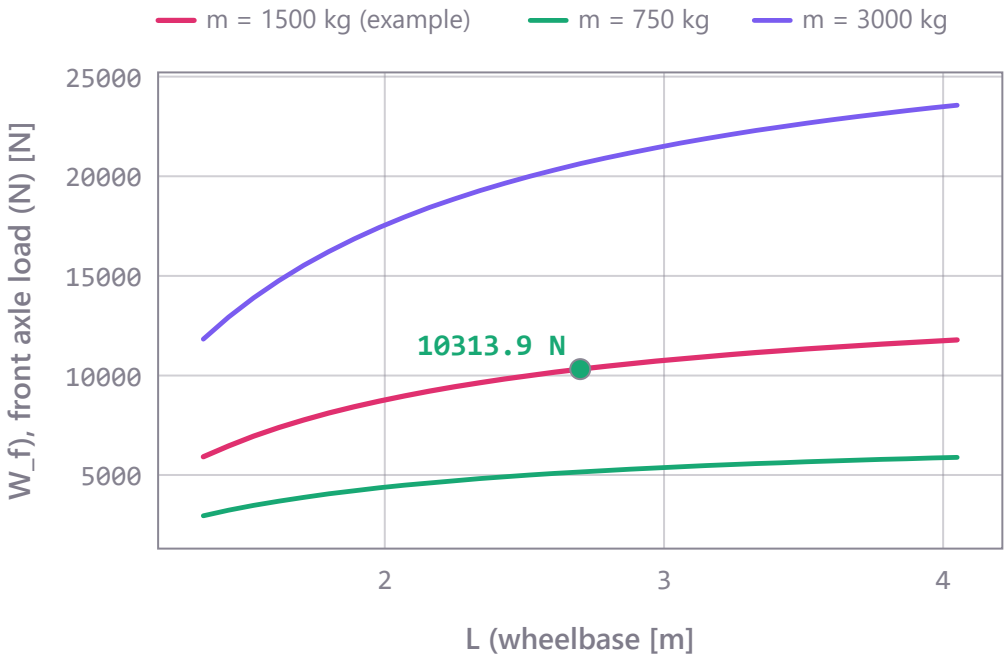
output	meaning
`W_f`	front axle load (N)
`W_r`	rear axle load (N)
`dW`	load transferred to the front axle (N)
`front_share`	ideal front brake share

- **Method:** W_f = (W l_r + m a h) / L, $dW = m$ a h / L, ideal front share = W_f / W
- **Assumes:** $a > 0$ (deceleration); $0 < l_f < L$; the rear axle keeps contact

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Side view of a vehicle (m = 1500 kg, wheelbase 2.7 m, CG 1.2 m behind the front axle and 0.55 m high, drawn to scale) braking at 7 m/s² towards the left. Marked: the returned dynamic axle loads front 10.3139 kN and rear 4.40111 kN (arrows to one scale), the load transferred 2.13889 kN and the ideal front brake share 0.70091; the inertia force m·a is drawn at the CG.



W_f (front axle load (N)) against L (wheelbase) from 1.35 m to 4.05 m. One curve for each value of m (vehicle mass): 1500 kg, 750 kg, 3000 kg. Other inputs as in the worked example: a = 7.0 m/s^2, h = 0.55 m, l_f = 1.2 m. 31 points per curve computed by transportation-ai999x; marked: the worked example, L = 2.7 m, W_f = 10313.9 N.

```
transportation_ai999x.solve('transport_braking_weight_transfer', m=1500.0, a=7.0, h=0.55, L=2.7,
l_f=1.2)
# -> W_f = 10313.9 N, W_r = 4401.11 N, dW = 2138.89 N, front_share = 0.70091   (SOLVED)
```

```
transportation_ai999x.solve('transport_braking_weight_transfer', m=2000.0, a=4.0, h=0.7, L=3.0,
l_f=1.4)
# -> W_f = 12330.7 N, W_r = 7289.33 N, dW = 1866.67 N, front_share = 0.628474   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Braking weight transfer with mass = 1500 kg, deceleration = 7 m/s^2, cg height = 0.55 m, wheelbase = 2.7 m and cg to front axle = 1.2 m.	SOLVED	W_f = 10313.9 N, W_r = 4401.11 N, dW = 2138.89 N, front_share = 0.70091
Braking weight transfer with mass = 2000 kg, deceleration = 4 m/s^2, cg height = 0.7 m, wheelbase = 3 m and cg to front axle = 1.4 m.	SOLVED	W_f = 12330.7 N, W_r = 7289.33 N, dW = 1866.67 N, front_share = 0.628474
Braking weight transfer with mass = 1500 kg, deceleration = 7 m/s^2, cg height = 0.55 m and wheelbase = 2.7 m.	NEEDS_INPUT	transport_braking_weight_transfer needs l_f (distance from the front axle to the centre of gravity). Example: solve('tra
Braking weight transfer with mass = 1500 kg, deceleration = 7 m/s^2, cg height = 0.55 m, wheelbase = 2.7 m and cg to front axle = 1.2 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Braking weight transfer with mass = 1500 kg, deceleration = 7 m/s^2, cg height = 0.55 m, wheelbase = 2.7 m and cg to front axle = 2.9 m.	INVALID_INPUT	Longitudinal weight transfer under braking and ideal brake proportioning: the distance from the front axle to the centre

Lateral load transfer in a steady turn, `transport_lateral_load_transfer`

Outer and inner wheel loads of a rigid vehicle (no suspension roll) cornering at lateral acceleration a_y, and the load transferred to the outer wheels.

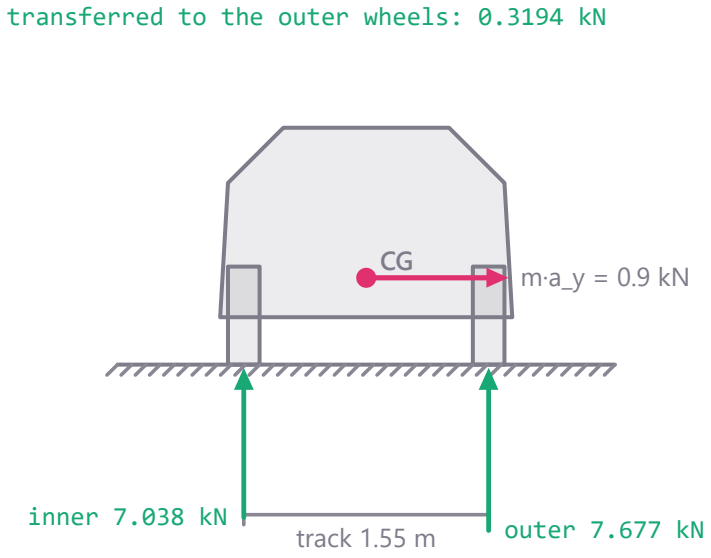
$$\Delta W = \frac{m a_y h}{t}$$

input	meaning	condition
`m`	vehicle mass	
`a_y`	lateral acceleration	
`h`	height of the centre of gravity	
`t_w`	track width	

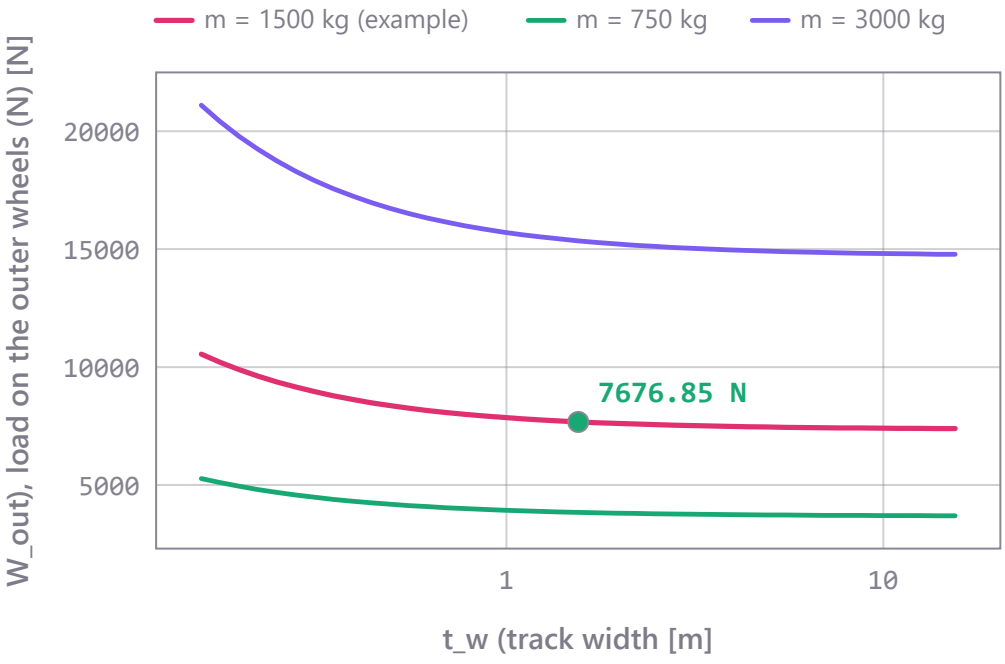
output	meaning
`W_out`	load on the outer wheels (N)
`W_in`	load on the inner wheels (N)
`dW`	load transferred to the outer wheels (N)

- **Method:** $dW = m$ a_y h / t, W_out = W/2 + dW, $W_in = W/2 - dW$
- **Assumes:** all inputs positive; the inner wheels keep contact

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Rear view of a vehicle (m = 1500 kg, track 1.55 m, CG 0.55 m high, drawn to scale) turning left at a lateral acceleration a_y = 0.6 m/s²: the outward inertia force at the CG and the returned wheel loads outer 7.67685 kN and inner 7.03815 kN (arrows to one scale); load transferred 0.319355 kN.



W_out (load on the outer wheels (N)) against t_w (track width) from 0.155 m to 15.5 m on a logarithmic axis. One curve for each value of m (vehicle mass): 1500 kg, 750 kg, 3000 kg. Other inputs as in the worked example: a_y = 0.6 m/s^2, h = 0.55 m. 41 points per curve computed by transportation-ai999x; marked: the worked example, t_w = 1.55 m, W_out = 7676.85 N.

```
transportation_ai999x.solve('transport_lateral_load_transfer', m=1500.0, a_y=0.6, h=0.55, t_w=1.55)
# -> W_out = 7676.85 N, W_in = 7038.15 N, dW = 319.355 N   (SOLVED)
```

```
transportation_ai999x.solve('transport_lateral_load_transfer', m=2200.0, a_y=4.0, h=0.8, t_w=1.6)
# -> W_out = 15191 N, W_in = 6391 N, dW = 4400 N   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Lateral load transfer with mass = 1500 kg, lateral acceleration = 0.6 m/s^2, cg height = 0.55 m and track width = 1.55 m.	SOLVED	W_out = 7676.85 N, W_in = 7038.15 N, dW = 319.355 N
Lateral load transfer with mass = 2200 kg, lateral acceleration = 4 m/s^2, cg height = 0.8 m and track width = 1.6 m.	SOLVED	W_out = 15191 N, W_in = 6391 N, dW = 4400 N
Lateral load transfer with mass = 1500 kg, lateral acceleration = 0.6 m/s^2 and cg height = 0.55 m.	NEEDS_INPUT	transport_lateral_load_transfer needs t_w (track width). Example: solve('transport_lateral_load_transfer', m='1500 kg',
Lateral load transfer with mass = 1500 kg, lateral acceleration = 0.6 m/s^2, cg height = 0.55 m and track width = 1.55 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Lateral load transfer with mass = 1500 kg, lateral acceleration = 0.6 m/s^2, cg height = -0.55 m and track width = 1.55 m.	INVALID_INPUT	Lateral load transfer in a steady turn: h must be positive.

Rollover threshold: static stability factor and rollover speed on a curve,

`transport_rollover_threshold`

Static stability factor SSF = t / (2h) of a rigid vehicle and the speed at which it would tip over on a curve of radius R with superelevation e (enough friction assumed for the tyres not to slide).

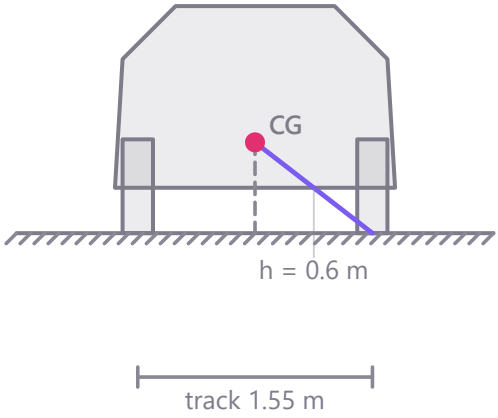
$$SSF = \frac{t}{2h}, \quad \frac{v^2}{gR} = \frac{SSF + e}{1 - SSF\,e}$$

input	meaning	condition
`t_w`	track width	
`h`	height of the centre of gravity	
`R`	curve radius	
`e`	superelevation rate, decimal	

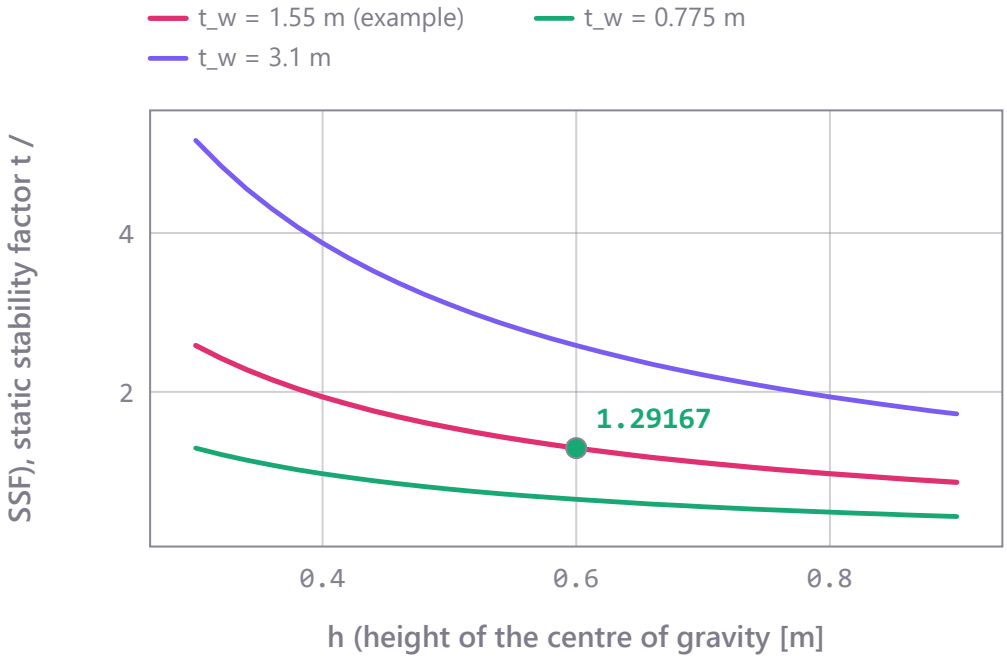
output	meaning
`SSF`	static stability factor $t/(2h)$ (rollover threshold in g)
`v_roll`	rollover speed on the curve (m/s)
`v_roll_kmh`	rollover speed (km/h)

- **Method:** SSF = t / (2h); v^2 / (g R) = (SSF + e) / (1 - SSF e)
- **Assumes:** SSF $e < 1$; tyre friction sufficient (sliding is not checked)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

```
static stability factor t/2h = 1.292
rollover speed on R = 100 m: 128.1 km/h
```



Rear view of a vehicle with track 1.55 m and CG height 0.6 m (to scale); the green line from the CG to the outer wheel's contact shows the tipping geometry. Marked: the returned static stability factor 1.29167 and the rollover speed 128.148 km/h on a curve of radius 100.0 m.



SSF (static stability factor t / (2h) (rollover threshold in g)) against h (height of the centre of gravity) from 0.3 m to 0.9 m. One curve for each value of t_w (track width): 1.55 m, 0.775 m, 3.1 m. Other inputs as in the worked example: R = 100.0 m. 31 points per curve computed by transportation-ai999x; marked: the worked example, h = 0.6 m, SSF = 1.29167.

```
transportation_ai999x.solve('transport_rollover_threshold', t_w=1.55, h=0.6, R=100.0)
# -> SSF = 1.29167, v_roll = 35.5967 m/s, v_roll_kmh = 128.148 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_rollover_threshold', t_w=1.8, h=1.9, R=200.0, e=0.06)
# -> SSF = 0.473684, v_roll = 32.8286 m/s, v_roll_kmh = 118.183 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rollover threshold with track width = 1.55 m, cg height = 0.6 m and radius = 100 m.	SOLVED	SSF = 1.29167, v_roll = 35.5967 m/s, v_roll_kmh = 128.148 km/h
Rollover threshold with track width = 1.8 m, cg height = 1.9 m, radius = 200 m and superelevation = 6 %.	SOLVED	SSF = 0.473684, v_roll = 32.8286 m/s, v_roll_kmh = 118.183 km/h
Rollover threshold with track width = 1.55 m and cg height = 0.6 m.	NEEDS_INPUT	transport_rollover_threshold needs R (curve radius). Example: solve('transport_rollover_threshold', t_w='1.55 m', h='0.6
Rollover threshold with track width = 1.55 m, cg height = 0.6 m and radius = 100 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Rollover threshold with track width = -1.55 m, cg height = 0.6 m and radius = 100 m.	INVALID_INPUT	Rollover threshold: static stability factor and rollover speed on a curve: t_w must be positive.

Maximum safe speed on a superelevated horizontal curve, `transport_curve_safe_speed`

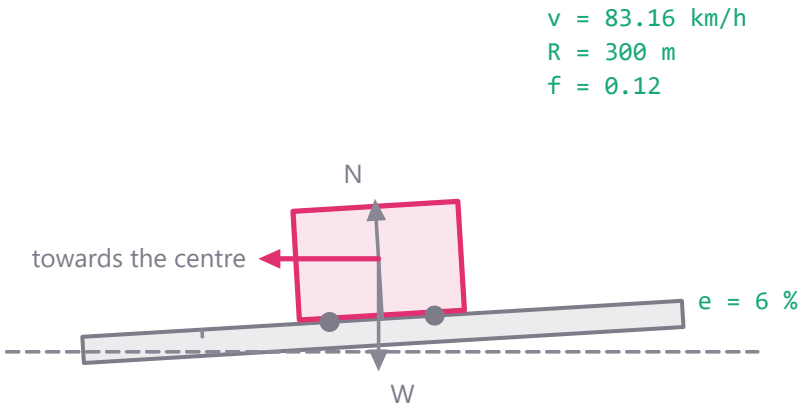
The highest speed at which a vehicle (point mass) can hold a curve of radius R with superelevation e and side friction factor f.

$$v = \sqrt{\frac{gR(e+f)}{1-ef}}$$

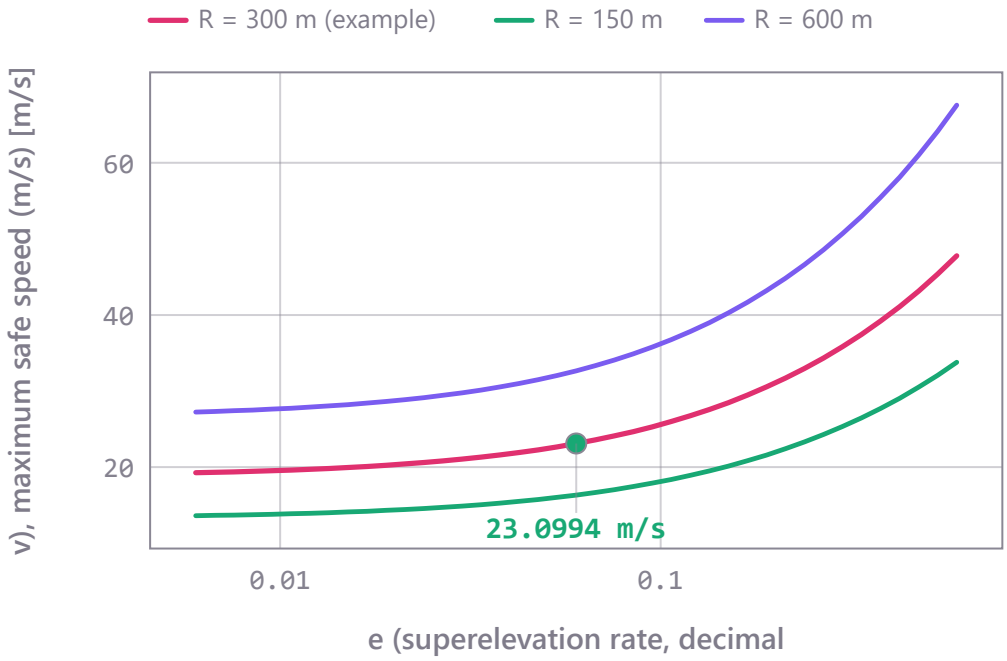
input	meaning	condition
`R`	curve radius	
`e`	superelevation rate, decimal	
`f`	side friction factor	

output	meaning
`v`	maximum safe speed (m/s)
`v_kmh`	maximum safe speed (km/h)

- **Method:** $v = sqrt$(g R (e + f) / (1 - e f)) (point-mass equilibrium)
- **Assumes:** $R > 0$; $f > 0$; e $f < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A banked carriageway with superelevation e = 0.06 (cross-slope to scale) on a radius R = 300.0 m with side friction f = 0.12: weight, normal reaction and the centripetal direction. Marked: the returned safe speed 83.1578 km/h (23.0994 m/s).



v (maximum safe speed (m/s)) against e (superelevation rate, decimal) from 0.006 to 0.6 on a logarithmic axis. One curve for each value of R (curve radius): 300 m, 150 m, 600 m. Other inputs as in the worked example: f = 0.12. 41 points per curve computed by transportation-ai999x; marked: the worked example, e = 0.06, v = 23.0994 m/s.

```
transportation_ai999x.solve('transport_curve_safe_speed', R=300.0, e=0.06, f=0.12)
# -> v = 23.0994 m/s, v_kmh = 83.1578 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_curve_safe_speed', R=150.0, e=0.08, f=0.15)
# -> v = 18.5083 m/s, v_kmh = 66.6297 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Maximum safe speed on a curve with radius = 300 m, superelevation = 6 %	SOLVED	v = 23.0994 m/s, v_kmh = 83.1578 km/h

question	outcome	result
and side friction = 0.12.		
Maximum safe speed on a curve with radius = 150 m, superelevation = 8 % and side friction = 0.15.	SOLVED	v = 18.5083 m/s, v_kmh = 66.6297 km/h
Maximum safe speed on a curve with radius = 300 m and superelevation = 6 %.	NEEDS_INPUT	transport_curve_safe_speed needs f (side friction factor). Example: solve('transport_curve_safe_speed', R='300 m', e='0.
Maximum safe speed on a curve with radius = 300 m, superelevation = 6 % and side friction = 0.12 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Maximum safe speed on a curve with radius = -300 m, superelevation = 6 % and side friction = 0.12.	INVALID_INPUT	Maximum safe speed on a superelevated horizontal curve: R must be positive.

Minimum radius of a horizontal curve, `transport_minimum_curve_radius`

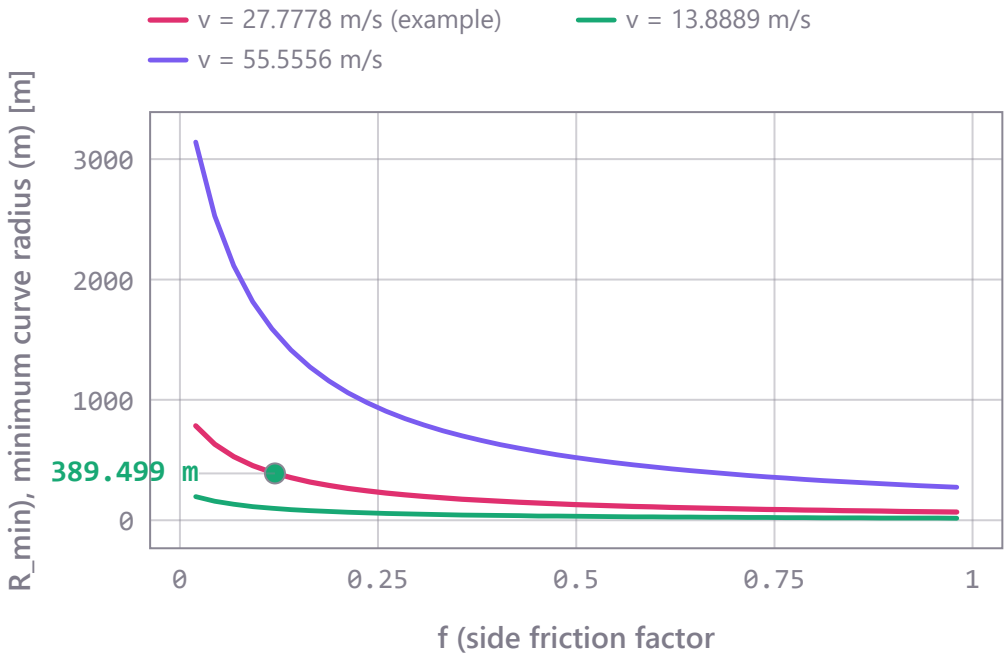
The smallest curve radius for a design speed v, maximum superelevation e and side friction factor f.

$$R_{min} = \frac{v^2(1 - ef)}{g(e + f)}$$

input	meaning	condition
`v`	design speed	
`e`	superelevation rate, decimal	
`f`	side friction factor	

output	meaning
`R_min`	minimum curve radius (m)

- **Method:** $R = v^2$ (1 - e f) / (g (e + f))
- **Assumes:** $v > 0$; $f > 0$; e $f < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R_min (minimum curve radius (m)) against f (side friction factor) from 0.02 to 0.98. One curve for each value of v (design speed): 27.7778 m/s, 13.8889 m/s, 55.5556 m/s. Other inputs as in the worked example: e = 0.08. 41 points per curve computed by transportation-ai999x; marked: the worked example, f = 0.12, R_min = 389.499 m.

```
transportation_ai999x.solve('transport_minimum_curve_radius', v=27.77777777777778, e=0.08, f=0.12)
# -> R_min = 389.499 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_minimum_curve_radius', v=16.666666666666668, e=0.06, f=0.17)
# -> R_min = 121.856 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Minimum curve radius with speed = 100 km/h, superelevation = 8 % and side friction = 0.12.	SOLVED	R_min = 389.499 m
Minimum curve radius with speed = 60 km/h, superelevation = 6 % and side friction = 0.17.	SOLVED	R_min = 121.856 m
Minimum curve radius with speed = 100 km/h and superelevation = 8 %.	NEEDS_INPUT	transport_minimum_curve_radius needs f (side friction factor). Example: solve('transport_minimum_curve_radius', v='27.77
Minimum curve radius with speed = 100 km/h, superelevation = 8 % and side friction = 0.12 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Minimum curve radius with speed = 100 km/h, superelevation = 8 % and side friction = -0.12.	INVALID_INPUT	Minimum radius of a horizontal curve: f must be positive.

Side friction demanded on a superelevated curve, `transport_side_friction_demand`

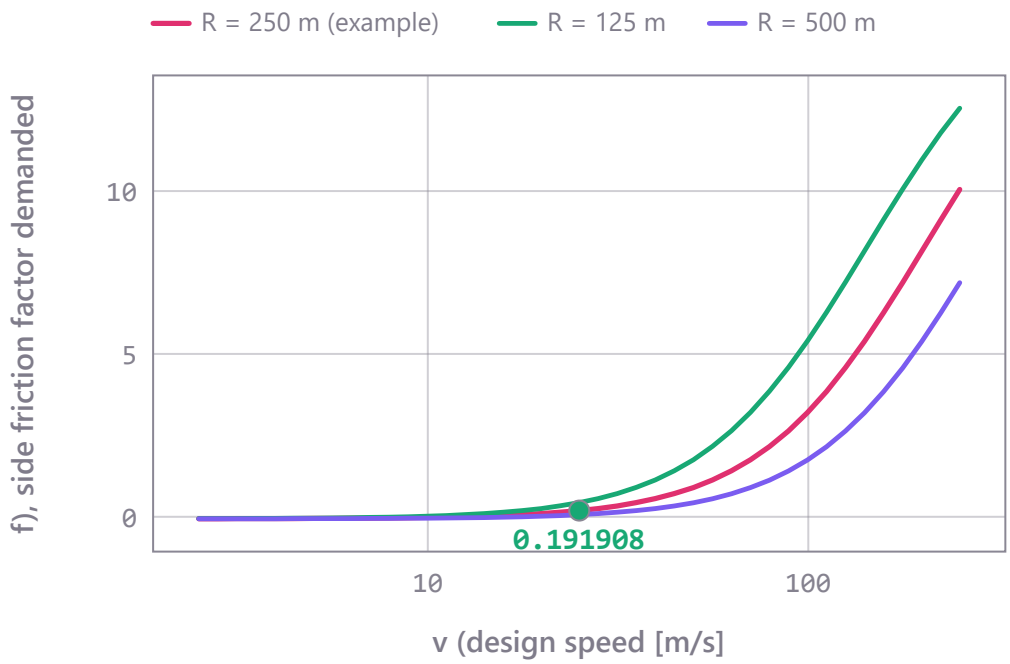
The side friction factor a vehicle needs at speed v on a curve of radius R with superelevation e (negative: the superelevation alone exceeds the demand and friction acts up the slope).

$$f = \frac{v^2/(gR) - e}{1 + e\,v^2/(gR)}$$

input	meaning	condition
`v`	design speed	
`R`	curve radius	
`e`	superelevation rate, decimal	

output	meaning
`f`	side friction factor demanded

- **Method:** f = (v^2/(gR) - e) / (1 + e $v^2/(gR)$)
- **Assumes:** $v > 0$; $R > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f (side friction factor demanded) against v (design speed) from 2.5 m/s to 250 m/s on a logarithmic axis. One curve for each value of R (curve radius): 250 m, 125 m, 500 m. Other inputs as in the worked example: e = 0.06. 41 points per curve computed by transportation-ai999x; marked: the worked example, v = 25 m/s, f = 0.191908.

```
transportation_ai999x.solve('transport_side_friction_demand', v=25.0, R=250.0, e=0.06)
# -> f = 0.191908 1   (SOLVED)
```

```
transportation_ai999x.solve('transport_side_friction_demand', v=13.88888888888889, R=80.0, e=0.04)
# -> f = 0.203793 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Side friction demand with speed = 90 km/h, radius = 250 m and superelevation = 6 %.	SOLVED	f = 0.191908 1
Side friction demand with speed = 50 km/h, radius = 80 m and superelevation = 4 %.	SOLVED	f = 0.203793 1
Side friction demand with speed = 90 km/h and radius = 250 m.	NEEDS_INPUT	transport_side_friction_demand needs e (superelevation rate, decimal). Example: solve('transport_side_friction_demand',
Side friction demand with speed = 90 km/h, radius = 250 m and superelevation = 6 % and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Side friction demand with speed = 90 km/h, radius = 0 m and superelevation = 6 %.	INVALID_INPUT	Side friction demanded on a superelevated curve: R must be positive.

Superelevation required on a horizontal curve, `transport_superelevation_required`

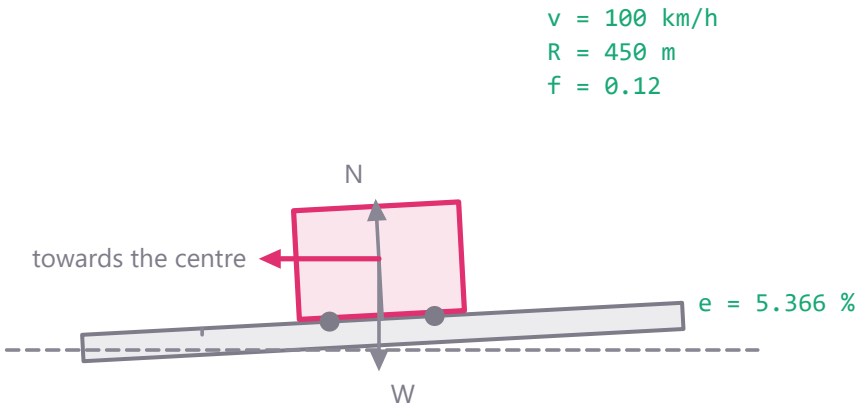
The superelevation rate needed at speed v on a curve of radius R when the side friction factor f is used.

$$e = \frac{v^2/(gR) - f}{1 + f\,v^2/(gR)}$$

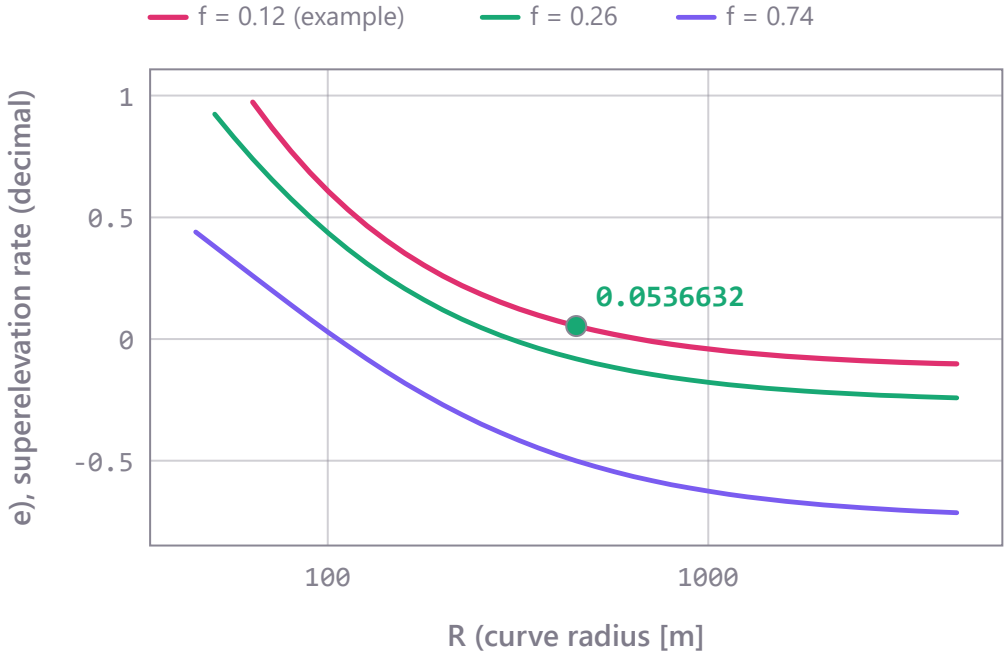
input	meaning	condition
`v`	design speed	
`R`	curve radius	
`f`	side friction factor	

output	meaning
`e`	superelevation rate (decimal)
`e_percent`	superelevation rate (%)

- **Method:** e = (v^2/(gR) - f) / (1 + f $v^2/(gR)$)
- **Assumes:** $v > 0$; $R > 0$; $f \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A banked carriageway (cross-slope to scale, exaggerated vehicle) for a speed v = 100 km/h on a radius R = 450.0 m with side friction f = 0.12: weight, normal reaction and the centripetal direction. Marked: the returned superelevation e = 5.36632 %.



e (superelevation rate (decimal)) against R (curve radius) from 45 m to 4500 m on a logarithmic axis. One curve for each value of f (side friction factor): 0.12, 0.26, 0.74. Other inputs as in the worked example: v = 27.77777777777778 m/s. 38 points per curve computed by transportation-ai999x; marked: the worked example, R = 450 m, e = 0.0536632.

```
transportation_ai999x.solve('transport_superelevation_required', v=27.77777777777778, R=450.0,
f=0.12)
# -> e = 0.0536632 1, e_percent = 5.36632   (SOLVED)
```

```
transportation_ai999x.solve('transport_superelevation_required', v=19.444444444444446, R=200.0,
f=0.14)
# -> e = 0.0513201 1, e_percent = 5.13201   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Superelevation required with speed = 100 km/h, radius = 450 m and side friction = 0.12.	SOLVED	e = 0.0536632 1, e_percent = 5.36632
Superelevation required with speed = 70 km/h, radius = 200 m and side friction = 0.14.	SOLVED	e = 0.0513201 1, e_percent = 5.13201
Superelevation required with speed = 100 km/h and radius = 450 m.	NEEDS_INPUT	transport_superelevation_required needs f (side friction factor). Example: solve('transport_superelevation_required', v=
Superelevation required with speed = 100 km/h, radius = 450 m and side friction = 0.12 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Superelevation required with speed = 100 km/h, radius = -450 m and side friction = 0.12.	INVALID_INPUT	Superelevation required on a horizontal curve: R must be positive.

Minimum length of a crest vertical curve for a sight distance, `transport_crest_curve_length`

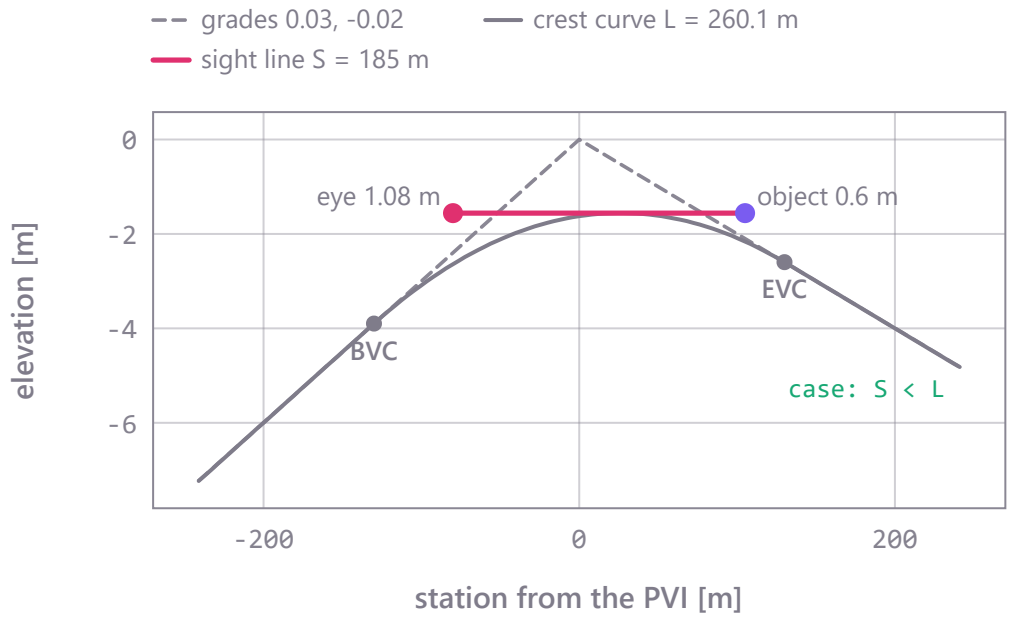
The shortest crest vertical curve between grades g1 and g2 that provides the sight distance S (AASHTO: eye height 1.08 m, object height 0.60 m unless stated), choosing the $S < L$ or $S > L$ form.

$$L = \frac{AS^2}{200\left(\sqrt{h_1} + \sqrt{h_2}\right)^2} \ (S < L), \qquad L = 2S - \frac{200\left(\sqrt{h_1} + \sqrt{h_2}\right)^2}{A} \ (S > L)$$

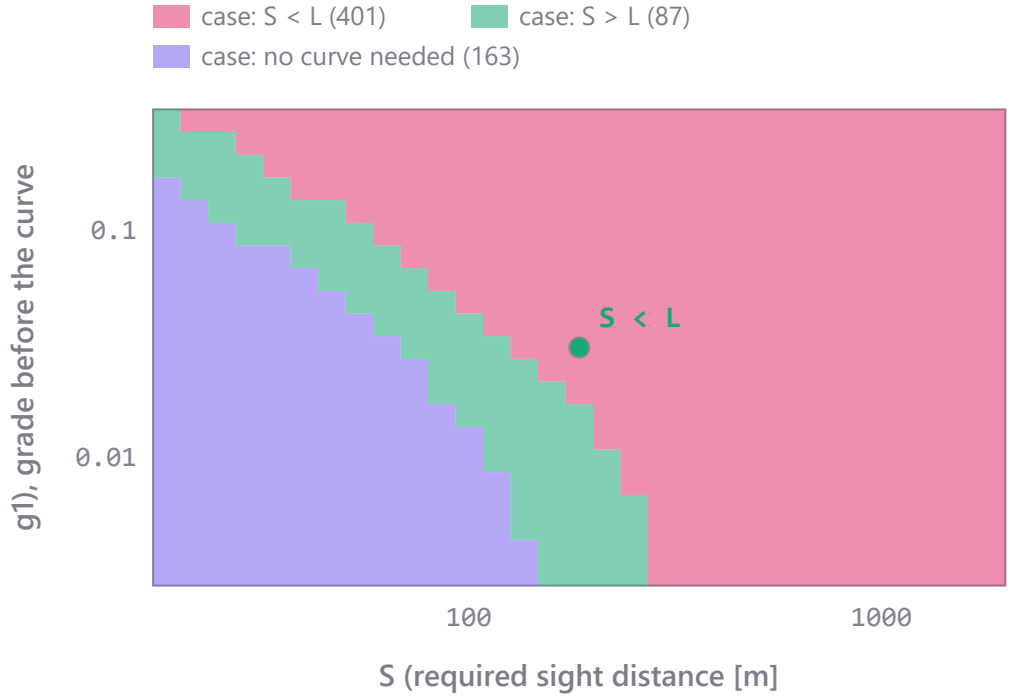
input	meaning	condition
`g1`	grade before the curve, decimal (positive uphill)	
`g2`	grade after the curve, decimal	
`S`	required sight distance	
`h1`	driver eye height (default 1.08 m)	
`h2`	object height (default 0.60 m)	

output	meaning
`L`	minimum curve length (m)
`case`	$S < L$, $S > L$ or no curve needed

- **Method:** $S < L$: $L = A$ S^2 / (200 (sqrt(h1) + sqrt(h2))^2); $S > L$: $L = 2S - 200$ (sqrt(h1) + sqrt(h2))^$2/A$ (A in %)
- **Assumes:** $g_1 > g_2$; $S > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The shortest crest vertical curve between grades g1 = 0.03 and g2 = -0.02 that gives a sight distance S = 185 m (eye 1.08 m, object 0.6 m): the returned length L = 260.071 m, case S < L. The road profile is drawn from the stated grades and the length L = 260.071 m (PVI at station 0); the sight line from the eye (1.08 m) to the object (0.6 m) touches the curve at its crest and is 185 m long, as returned. Heights are exaggerated by the axes.



S < L, S > L or no curve needed (case) over S (required sight distance) from 18.5 m to 1850 m and g1 (grade before the curve, decimal (positive uphill)) from 0.003 to 0.3 (logarithmic axes): each of the 31 × 21 cells is one run of transportation-ai999x, coloured by the verdict it returned, S < L: 401 cells, S > L: 87 cells, no curve needed: 163 cells. Other inputs as in the worked example: g2 = -0.02. Marked: the worked example, S = 185 m, g1 = 0.03 → S < L.

```
transportation_ai999x.solve('transport_crest_curve_length', g1=0.03, g2=-0.02, S=185.0)
# -> L = 260.071 m, case = S < L   (SOLVED)
```

```
transportation_ai999x.solve('transport_crest_curve_length', g1=0.01, g2=-0.015, S=200.0)
# -> L = 136.802 m, case = S > L   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Crest vertical curve length with g1 = 3 %, g2 = -2 % and sight distance = 185 m.	SOLVED	L = 260.071 m, case = S < L
Crest vertical curve length with g1 = 1 %, g2 = -1.5 % and sight distance = 200 m.	SOLVED	L = 136.802 m, case = S > L
Crest vertical curve length with g1 = 3 % and g2 = -2 %.	NEEDS_INPUT	transport_crest_curve_length needs S (required sight distance). Example: solve('transport_crest_curve_length', g1='0.03
Crest vertical curve length with g1 = 3 %, g2 = -2 % and sight distance = 185 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Crest vertical curve length with g1 = -2 %, g2 = 3 % and sight distance = 185 m.	INVALID_INPUT	Minimum length of a crest vertical curve for a sight distance: a crest curve needs g1 > g2.

Sight distance available over a crest vertical curve, `transport_crest_curve_sight_distance`

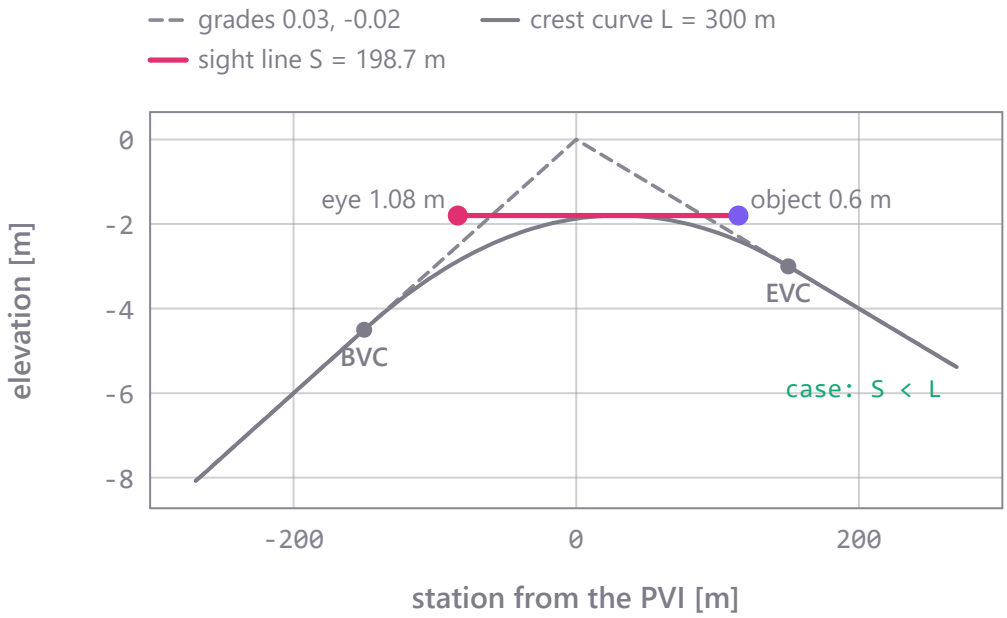
The sight distance a crest vertical curve of length L between grades g1 and g2 provides (eye height 1.08 m, object height 0.60 m unless stated).

$$S = \sqrt{\frac{200\left(\sqrt{h_1} + \sqrt{h_2}\right)^2 L}{A}}$$

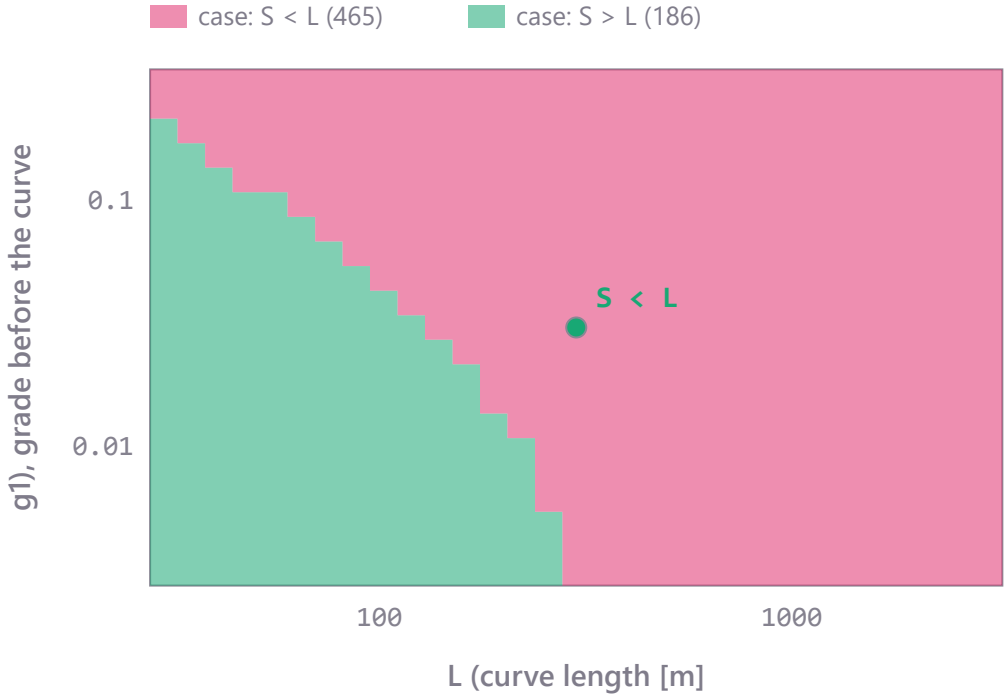
input	meaning	condition
`g1`	grade before the curve, decimal (positive uphill)	
`g2`	grade after the curve, decimal	
`L`	curve length	
`h1`	driver eye height (default 1.08 m)	
`h2`	object height (default 0.60 m)	

output	meaning
`S`	available sight distance (m)
`case`	$S < L$ or $S > L$

- **Method:** $S = sqrt$(200 (sqrt(h1) + sqrt(h2))^2 L / A) if that $is \leq L$, else $S = L/2 + 100$ (sqrt(h1) + sqrt(h2))^$2/A$
- **Assumes:** $g_1 > g_2$; $L \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A crest vertical curve between grades g1 = 0.03 and g2 = -0.02, length L = 300 m (eye height 1.08 m, object height 0.6 m). Marked: the returned sight distance S = 198.695 m, case S < L. The road profile is drawn from the stated grades and the length L = 300 m (PVI at station 0); the sight line from the eye (1.08 m) to the object (0.6 m) touches the curve at its crest and is 198.695 m long, as returned. Heights are exaggerated by the axes.



S < L or S > L (case) over L (curve length) from 30 m to 3000 m and g1 (grade before the curve, decimal (positive uphill)) from 0.003 to 0.3 (logarithmic axes): each of the 31 × 21 cells is one run of transportation-ai999x, coloured by the verdict it returned, S < L: 465 cells, S > L: 186 cells. Other inputs as in the worked example: g2 = -0.02. Marked: the worked example, L = 300 m, g1 = 0.03 → S < L.

```
transportation_ai999x.solve('transport_crest_curve_sight_distance', g1=0.03, g2=-0.02, L=300.0)
# -> S = 198.695 m, case = S < L   (SOLVED)
```

```
transportation_ai999x.solve('transport_crest_curve_sight_distance', g1=0.02, g2=-0.01, L=60.0)
# -> S = 139.666 m, case = S > L   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Crest vertical curve sight distance with g1 = 3 %, g2 = -2 % and curve length = 300 m.	SOLVED	S = 198.695 m, case = S < L
Crest vertical curve sight distance with g1 = 2 %, g2 = -1 % and curve length = 60 m.	SOLVED	S = 139.666 m, case = S > L
Crest vertical curve sight distance with g1 = 3 % and g2 = -2 %.	NEEDS_INPUT	transport_crest_curve_sight_distance needs L (curve length). Example: solve('transport_crest_curve_sight_distance', g1='
Crest vertical curve sight distance with g1 = 3 %, g2 = -2 % and curve length = 300 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Crest vertical curve sight distance with g1 = -2 %, g2 = 3 % and curve length = 300 m.	INVALID_INPUT	Sight distance available over a crest vertical curve: a crest curve needs g1 > g2.

Horizontal sightline offset on a circular curve, `transport_horizontal_sight_offset`

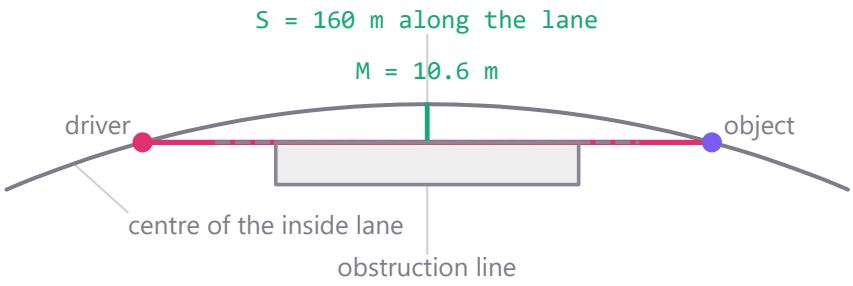
The clearance M needed from the centre of the inside lane to an obstruction (middle ordinate) so that a driver on a curve of radius R sees the sight distance S, both driver and object on the curve.

$$M = R\left(1 - \cos\frac{S}{2R}\right)$$

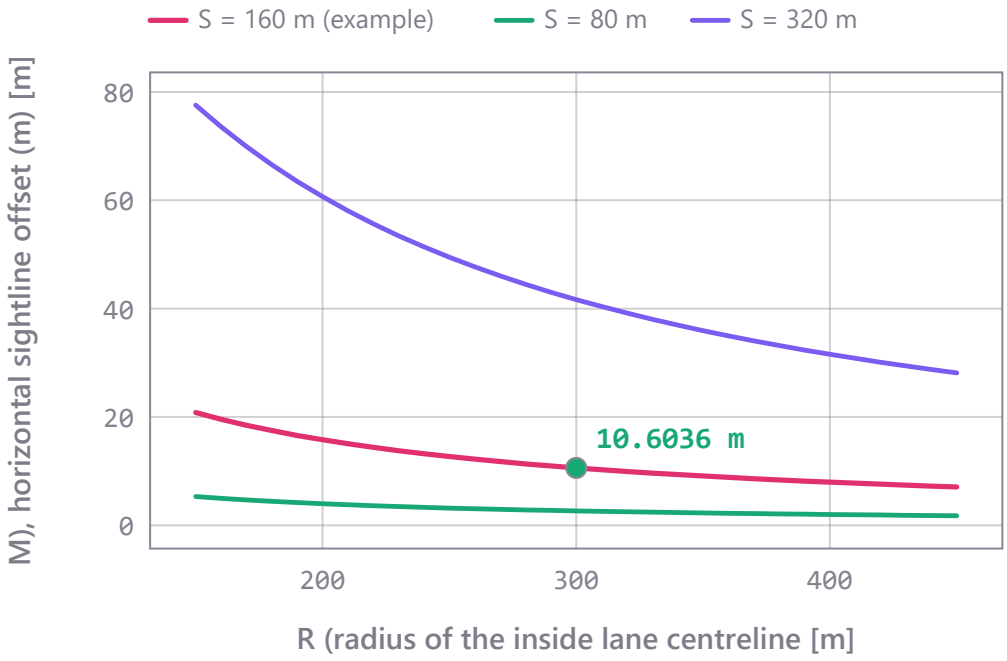
input	meaning	condition
`R`	radius of the inside lane centreline	
`S`	sight distance	

output	meaning
`M`	horizontal sightline offset (m)

- **Method:** $M = R$ (1 - cos($S/(2R)$))
- **Assumes:** $R > 0$; $0 < S < pi$ R
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Plan of the inside lane of a curve of radius R = 300 m with a sight distance S = 160 m measured along the lane: the sight line is the chord between driver and object, and the clear distance it needs to the obstruction is the returned middle ordinate M = 10.6036 m (geometry to scale).



M (horizontal sightline offset (m)) against R (radius of the inside lane centreline) from 150 m to 450 m. One curve for each value of S (sight distance): 160 m, 80 m, 320 m. 31 points per curve computed by transportation-ai999x; marked: the worked example, R = 300 m, M = 10.6036 m.

```
transportation_ai999x.solve('transport_horizontal_sight_offset', R=300.0, S=160.0)
# -> M = 10.6036 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_horizontal_sight_offset', R=150.0, S=110.0)
# -> M = 9.97087 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Horizontal sightline offset with radius = 300 m and sight distance = 160 m.	SOLVED	M = 10.6036 m
Horizontal sightline offset with radius = 150 m and sight distance = 110 m.	SOLVED	M = 9.97087 m
Horizontal sightline offset with radius = 300 m.	NEEDS_INPUT	transport_horizontal_sight_offset needs S (sight distance). Example: solve('transport_horizontal_sight_offset', R='300 m
Horizontal sightline offset with radius = 300 m and sight distance = 160 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Horizontal sightline offset with radius = 100 m and sight distance = 400 m.	INVALID_INPUT	Horizontal sightline offset on a circular curve: the sight distance must be shorter than half the circumference.

Sight distance available on a curve with a roadside obstruction,

`transport_sight_distance_from_offset`

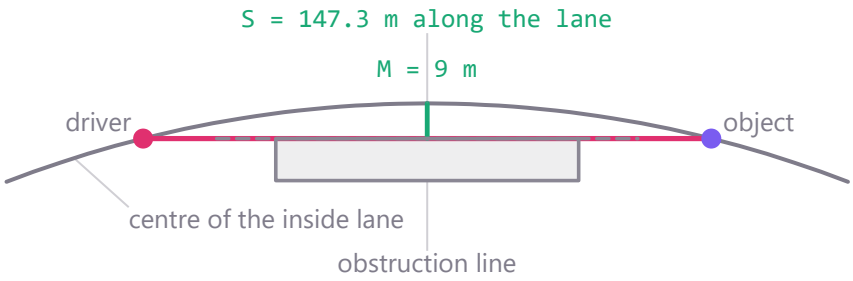
The sight distance available on a curve of radius R (inside lane centreline) when an obstruction stands at the offset M from that centreline.

$$S = 2R \arccos\left(1 - \frac{M}{R}\right)$$

input	meaning	condition
`R`	radius of the inside lane centreline	
`M`	offset to the obstruction	

output	meaning
`S`	available sight distance (m)

- **Method:** $S = 2R$ acos(1 - M / R)
- **Assumes:** $0 < M < R$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Plan of the inside lane of a curve of radius R = 300 m with an obstruction M = 9 m from the lane centre: the returned sight distance S = 147.339 m is the length along the lane whose chord just clears the obstruction (geometry to scale).



S (available sight distance (m)) against R (radius of the inside lane centreline) from 150 m to 450 m. One curve for each value of M (offset to the obstruction): 9 m, 4.5 m, 18 m. 31 points per curve computed by transportation-ai999x; marked: the worked example, R = 300 m, S = 147.339 m.

```
transportation_ai999x.solve('transport_sight_distance_from_offset', R=300.0, M=9.0)
# -> S = 147.339 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_sight_distance_from_offset', R=200.0, M=6.5)
# -> S = 102.259 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sight distance from offset with radius = 300 m and offset = 9 m.	SOLVED	S = 147.339 m
Sight distance from offset with radius = 200 m and offset = 6.5 m.	SOLVED	S = 102.259 m
Sight distance from offset with radius = 300 m.	NEEDS_INPUT	transport_sight_distance_from_offset needs M (offset to the obstruction). Example: solve('transport_sight_distance_from_
Sight distance from offset with radius = 300 m and offset = 9 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Sight distance from offset with radius = 100 m and offset = 150 m.	INVALID_INPUT	Sight distance available on a curve with a roadside obstruction: the offset must be less than the radius.

Clothoid (spiral) transition curve coordinates, `transport_clothoid_coordinates`

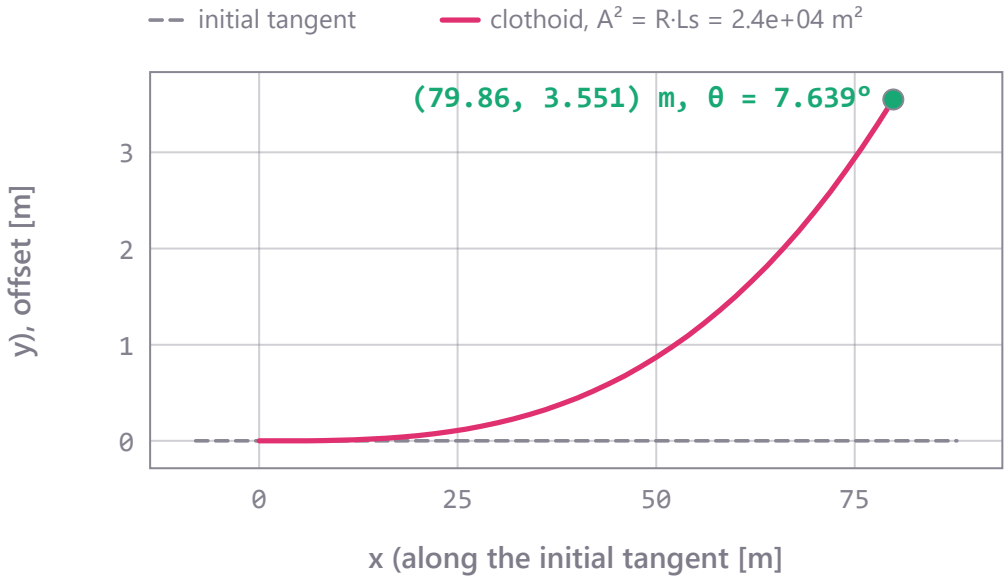
Coordinates along the tangent (x) and offset from it (y), and the tangent deflection angle, at distance s along a clothoid transition of length L_s that ends at a circular curve of radius R (A^2 = R L_s).

$$x = \int_0^s \cos\frac{u^2}{2A^2}\,du, \quad y = \int_0^s \sin\frac{u^2}{2A^2}\,du, \quad \theta = \frac{s^2}{2RL_s}$$

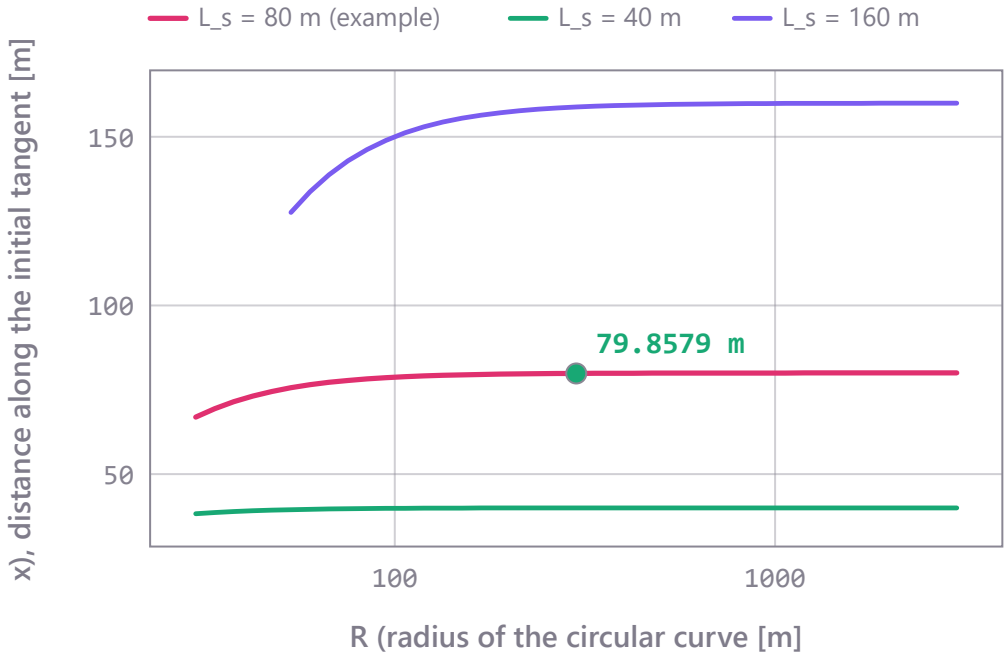
input	meaning	condition
`R`	radius of the circular curve	
`L_s`	spiral length	
`s`	distance along the spiral (default L_s)	

output	meaning
`x`	distance along the initial tangent (m)
`y`	offset from the tangent (m)
`theta`	tangent deflection angle (degrees)

- **Method:** $x = s\,(1 - \theta^2/10 + \theta^4/216 - \ldots)$, $y = s\,(\theta/3 - \theta^3/42 + \ldots)$, theta = s^2 / (2 R L_s)
- **Assumes:** $0 < s \le L_s$; tangent angle <= 90 degrees
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A clothoid transition of length Ls = 80 m into a circular curve of radius R = 300 m: 41 points (x, y) along it, each computed by the package (the offset axis is stretched). Marked: the returned end point x = 79.8579 m, y = 3.55104 m and tangent deflection θ = 7.63944° at s = 80 m.



x (distance along the initial tangent (m)) against R (radius of the circular curve) from 30 m to 3000 m on a logarithmic axis. One curve for each value of L_s (spiral length): 80 m, 40 m, 160 m. 41 points per curve computed by transportation-ai999x; marked: the worked example, R = 300 m, x = 79.8579 m.

```
transportation_ai999x.solve('transport_clothoid_coordinates', R=300.0, L_s=80.0)
# -> x = 79.8579 m, y = 3.55104 m, theta = 7.63944 deg   (SOLVED)
```

```
transportation_ai999x.solve('transport_clothoid_coordinates', R=500.0, L_s=120.0, s=60.0)
# -> x = 59.9946 m, y = 0.599961 m, theta = 1.71887 deg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Clothoid coordinates with radius = 300 m and spiral length = 80 m.	SOLVED	x = 79.8579 m, y = 3.55104 m, theta = 7.63944 deg
Clothoid coordinates with radius = 500 m, spiral length = 120 m and station = 60 m.	SOLVED	x = 59.9946 m, y = 0.599961 m, theta = 1.71887 deg
Clothoid coordinates with radius = 300 m.	NEEDS_INPUT	transport_clothoid_coordinates needs L_s (spiral length). Example: solve('transport_clothoid_coordinates', R='300 m', L_
Clothoid coordinates with radius = 300 m and spiral length = 80 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Clothoid coordinates with radius = 300 m, spiral length = 80 m and station = 120 m.	INVALID_INPUT	Clothoid (spiral) transition curve coordinates: the station must lie on the spiral (s <= L_s).

Greenshields model: capacity, optimum speed and density, `transport_greenshields_capacity`

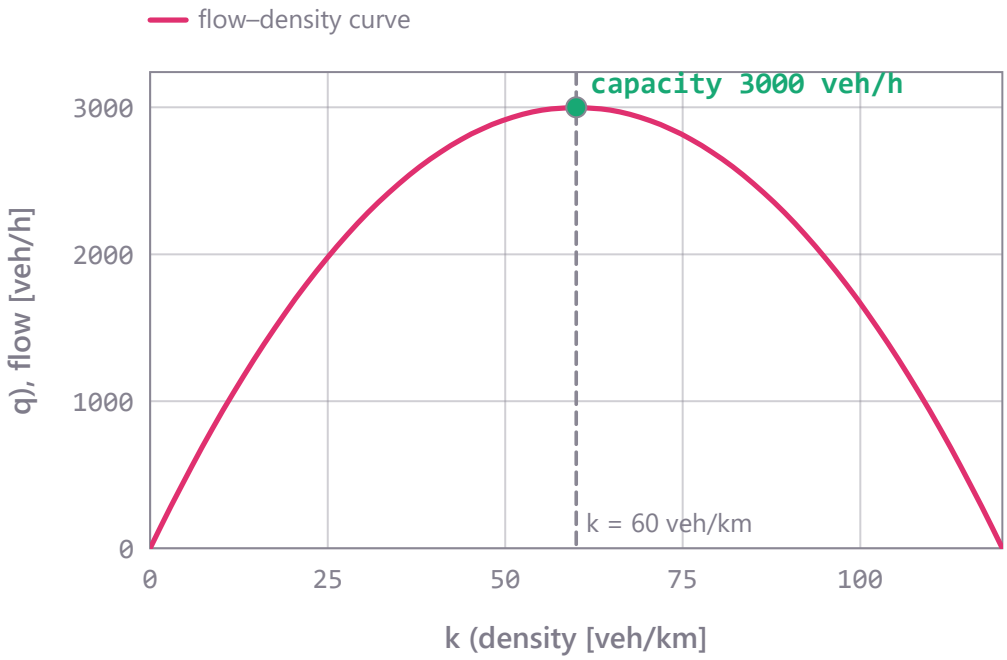
Capacity (maximum flow) of a traffic stream following Greenshields' linear speed-density model $v = vf\,(1 - k/kj)$, with the density and speed at which it occurs.

$$q_{max} = \frac{v_f k_j}{4}, \quad k_m = \frac{k_j}{2}, \quad v_m = \frac{v_f}{2}$$

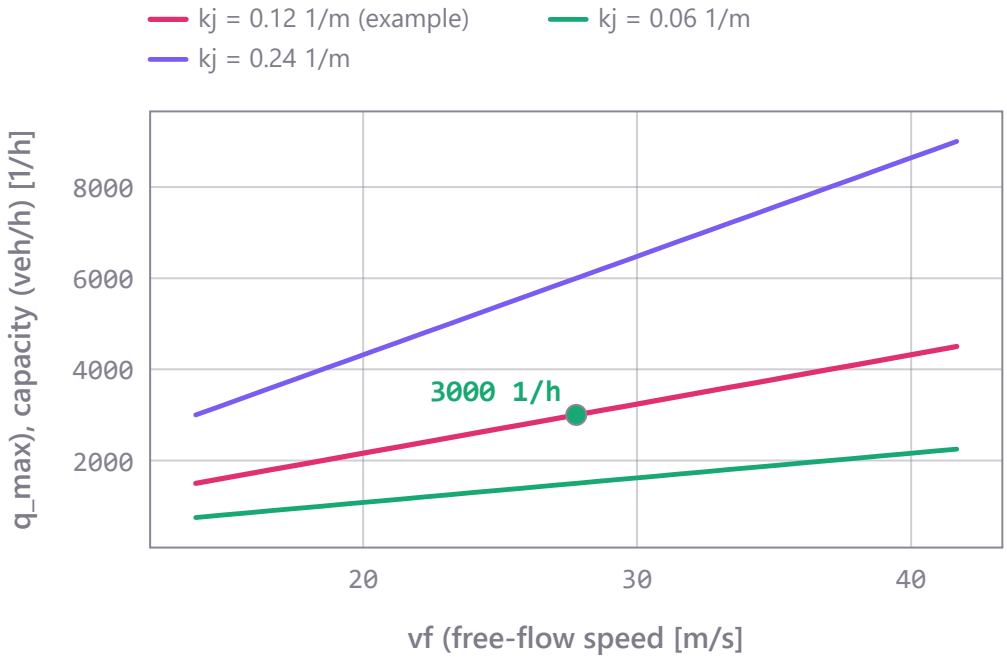
input	meaning	condition
`vf`	free-flow speed	
`kj`	jam density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	

output	meaning
`q_max`	capacity (veh/h)
`k_m`	density at capacity (veh/km)
`v_m`	speed at capacity (km/h)

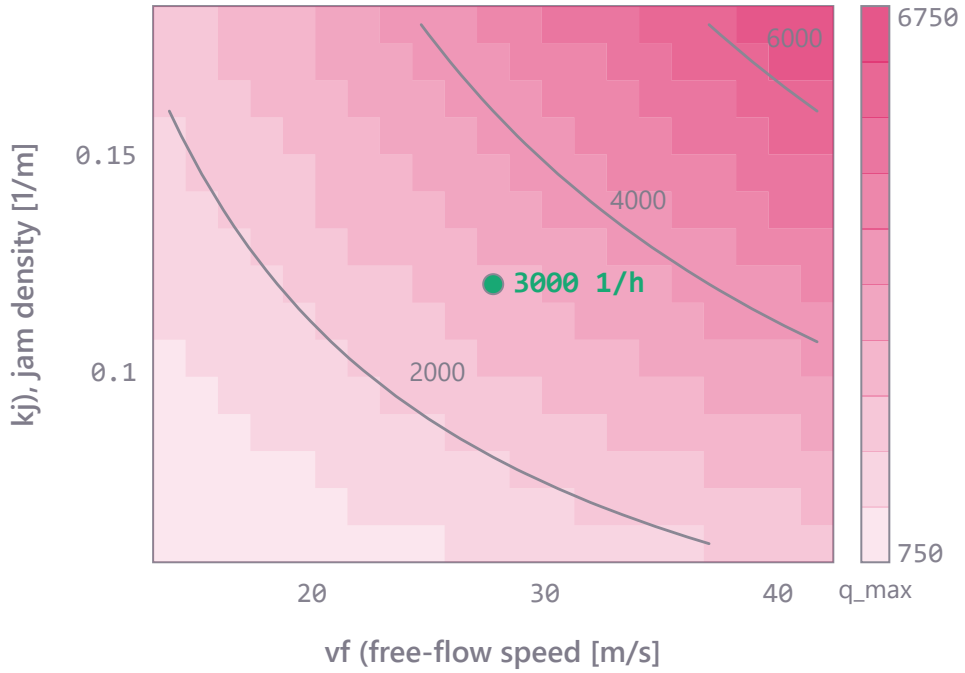
- **Method:** q_max = vf $kj/4$ at $k = kj/2$, $v = vf/2$
- **Assumes:** $vf > 0$; $kj > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Fundamental diagram for free-flow speed 100 km/h and jam density 120 veh/km: for each of 40 flows the package returns the two densities that carry it (uncongested left, congested right). Marked: the capacity it returned, q = 3000 veh/h at the optimum density k = 60 veh/km and speed 50 km/h.



q_max (capacity (veh/h)) against vf (free-flow speed) from 13.8889 m/s to 41.6667 m/s. One curve for each value of kj (jam density (veh/km in questions; veh/m in solve(), or '25 1/km')): 0.12 1/m, 0.06 1/m, 0.24 1/m. 31 points per curve computed by transportation-ai999x; marked: the worked example, vf = 27.7778 m/s, q_max = 3000 1/h.



q_max (capacity (veh/h)) over vf (free-flow speed) from 13.8889 m/s to 41.6667 m/s and kj (jam density (veh/km in questions; veh/m in solve(), or '25 1/km')) from 0.06 1/m to 0.18 1/m: each of the 21 × 15 cells is one run of transportation-ai999x, shaded by its value (light 750 1/h to dark 6750 1/h); iso-lines at q_max = 2000, 4000, 6000 1/h. Marked: the worked example, vf = 27.7778 m/s, kj = 0.12 1/m, q_max = 3000 1/h.

```
transportation_ai999x.solve('transport_greenshields_capacity', vf=27.77777777777778, kj=0.12)
# -> q_max = 3000 1/h, k_m = 60 1/km, v_m = 50 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_greenshields_capacity', vf=16.666666666666668, kj=0.15)
# -> q_max = 2250 1/h, k_m = 75 1/km, v_m = 30 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Greenshields capacity with free flow speed = 100 km/h and jam density = 120 veh/km.	SOLVED	q_max = 3000 1/h, k_m = 60 1/km, v_m = 50 km/h
Greenshields capacity with free flow speed = 60 km/h and jam density = 150 veh/km.	SOLVED	q_max = 2250 1/h, k_m = 75 1/km, v_m = 30 km/h
Greenshields capacity with free flow speed = 100 km/h.	NEEDS_INPUT	transport_greenshields_capacity needs kj (jam density (veh/km in questions; veh/m in solve(), or '25 1/km')). Example: s
Greenshields capacity with free flow speed = 100 km/h and jam density = 120 veh/km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Greenshields capacity with free flow speed = 100 km/h and jam density = -120 veh/km.	INVALID_INPUT	Greenshields model: capacity, optimum speed and density: kj must be positive.

Greenshields model: densities and speeds at a given flow, `transport_greenshields_speed_at_flow`

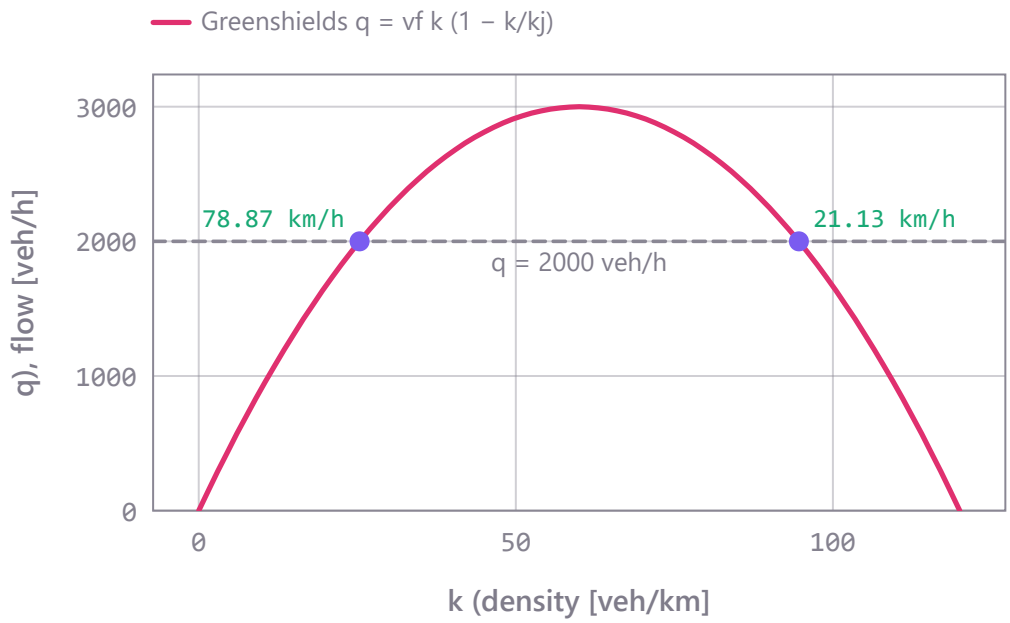
The two traffic states (uncongested and congested) of a Greenshields stream that carry the flow q: their densities and speeds.

$$k = \frac{k_j}{2}\left(1 \mp \sqrt{1 - \frac{4q}{v_f k_j}}\right)$$

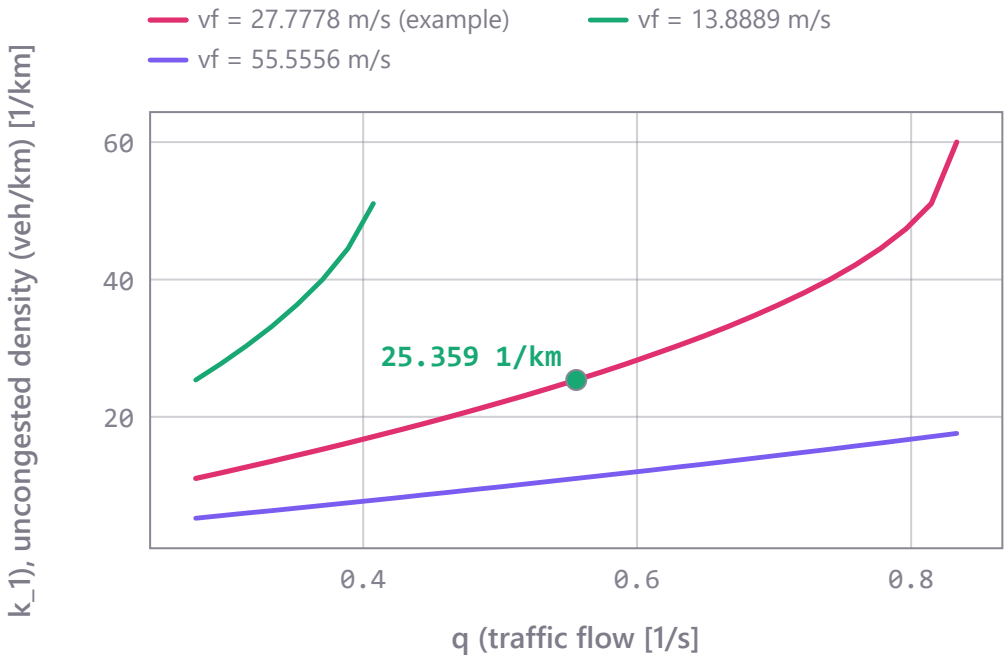
input	meaning	condition
`vf`	free-flow speed	
`kj`	jam density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`q`	traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	

output	meaning
`k_1`	uncongested density (veh/km)
`v_1`	uncongested speed (km/h)
`k_2`	congested density (veh/km)
`v_2`	congested speed (km/h)

- **Method:** $k = (kj/2)$(1 -/+ sqrt(1 - 4q/(vf kj))), $v = vf\ (1 - k/kj)$
- **Assumes:** $0 < q \leq vf\ kj/4$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Greenshields flow–density curve (vf = 100 km/h, kj = 120 veh/km, from these stated parameters) cut by the flow q = 2000 veh/h: the two states the package returned, k1 = 25.359 veh/km at 78.8675 km/h (uncongested) and k2 = 94.641 veh/km at 21.1325 km/h (congested).



k_1 (uncongested density (veh/km)) against q (traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.277778 1/s to 0.833333 1/s. One curve for each value of vf (free-flow speed): 27.7778 m/s, 13.8889 m/s, 55.5556 m/s. Other inputs as in the worked example: kj = 0.12 1/m. 31 points per curve computed by transportation-ai999x; marked: the worked example, q = 0.555556 1/s, k_1 = 25.359 1/km.

```
transportation_ai999x.solve('transport_greenshields_speed_at_flow', vf=27.77777777777778, kj=0.12,
q=0.5555555555555556)
# -> k_1 = 25.359 1/km, v_1 = 78.8675 km/h, k_2 = 94.641 1/km, v_2 = 21.1325 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_greenshields_speed_at_flow', vf=22.22222222222222, kj=0.14,
q=0.3333333333333333)
# -> k_1 = 17.085 1/km, v_1 = 70.2372 km/h, k_2 = 122.915 1/km, v_2 = 9.76284 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Greenshields speed at a flow with free flow speed = 100 km/h, jam density = 120 veh/km and flow = 2000 veh/h.	SOLVED	k_1 = 25.359 1/km, v_1 = 78.8675 km/h, k_2 = 94.641 1/km, v_2 = 21.1325 km/h
Greenshields speed at a flow with free flow speed = 80 km/h, jam density = 140 veh/km and flow = 1200 veh/h.	SOLVED	k_1 = 17.085 1/km, v_1 = 70.2372 km/h, k_2 = 122.915 1/km, v_2 = 9.76284 km/h
Greenshields speed at a flow with free flow speed = 100 km/h and jam density = 120 veh/km.	NEEDS_INPUT	transport_greenshields_speed_at_flow needs q (traffic flow (veh/h in questions; veh/s in solve(), or a string such as '6
Greenshields speed at a flow with free flow speed = 100 km/h, jam density = 120 veh/km and flow = 2000 veh/h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Greenshields speed at a flow with free flow speed = 100 km/h, jam density = 120 veh/km and flow = 4000 veh/h.	INVALID_INPUT	Greenshields model: densities and speeds at a given flow: the flow exceeds the capacity vf kj / 4 of this traffic stream

Fit the Greenshields model to speed-density observations, `transport_greenshields_fit`

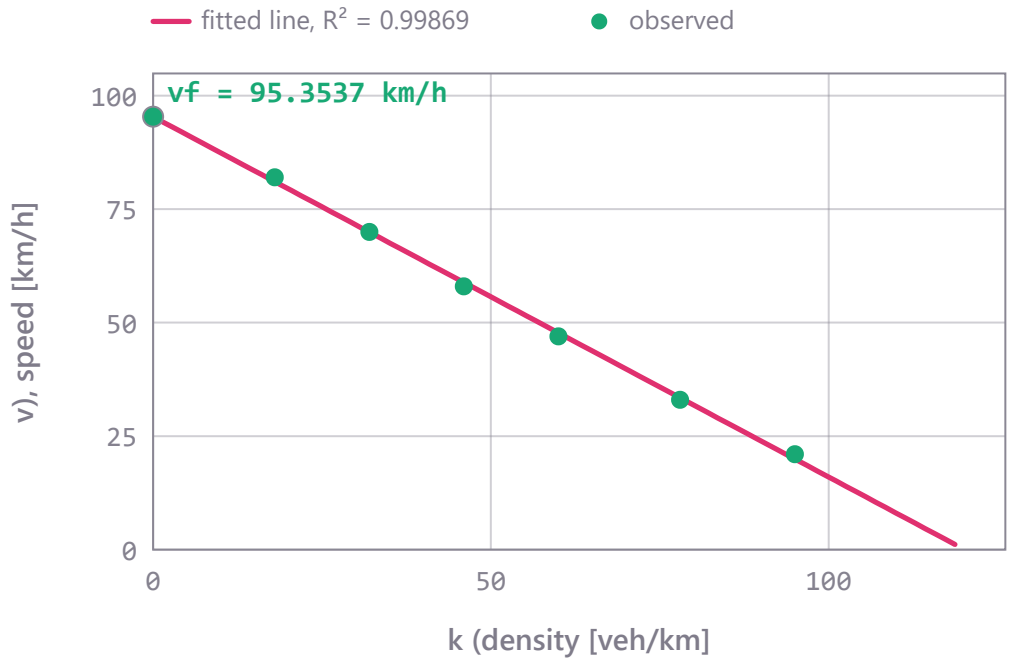
Least-squares fit of Greenshields' line $v = vf - (vf/kj)$ k to observed speeds (km/h) and densities (veh/km): the free-flow speed, jam density, capacity and the coefficient of determination of the line.

$$v = v_f - \frac{v_f}{k_j}k, \quad q_{max} = \frac{v_f k_j}{4}$$

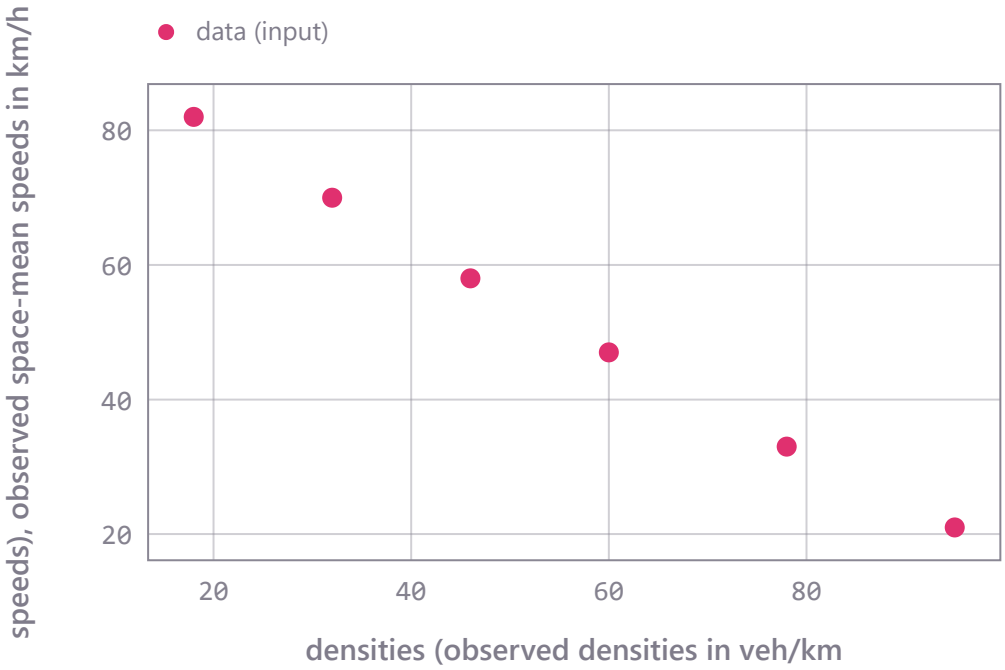
input	meaning	condition
`speeds`	observed space-mean speeds in km/h	
`densities`	observed densities in veh/km, in the same order	

output	meaning
`vf`	free-flow speed (km/h)
`kj`	jam density (veh/km)
`q_max`	capacity (veh/h)
`r2`	coefficient of determination of the fit

- **Method:** least squares of v on k: intercept vf, slope -vf/kj; q_max = vf $kj/4$
- **Assumes:** at least three observations; speeds fall with density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Six observed speed–density pairs and the linear speed–density model fitted to them by the package: free-flow speed vf = 95.3537 km/h, jam density kj = 120.141 veh/km, R^2 = 0.99869; the line is traced by the package's speeds at 80 flows up to the capacity 2863.96 veh/h. Marked: k = 0, v = vf.



The data speeds (observed space-mean speeds in km/h) against densities (observed densities in veh/km, in the same order) as given in the worked example (6 points). Also returned: vf = 95.3537 km/h, kj = 120.141 1/km, q_max = 2863.96 1/h, r2 = 0.99869.

```
transportation_ai999x.solve('transport_greenshields_fit', speeds=[82, 70, 58, 47, 33, 21],
densities=[18, 32, 46, 60, 78, 95])
# -> vf = 95.3537 km/h, kj = 120.141 1/km, q_max = 2863.96 1/h, r2 = 0.99869   (SOLVED)
```

```
transportation_ai999x.solve('transport_greenshields_fit', speeds=[61.5, 48.2, 39.0, 27.4, 15.8],
densities=[25, 42, 58, 80, 105])
# -> vf = 73.2006 km/h, kj = 130.338 1/km, q_max = 2385.2 1/h, r2 = 0.989429   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Greenshields fit with speeds = [82, 70, 58, 47, 33, 21] and densities = [18, 32, 46, 60, 78, 95].	SOLVED	vf = 95.3537 km/h, kj = 120.141 1/km, q_max = 2863.96 1/h, r2 = 0.99869
Greenshields fit with speeds = [61.5, 48.2, 39, 27.4, 15.8] and densities = [25, 42, 58, 80, 105].	SOLVED	vf = 73.2006 km/h, kj = 130.338 1/km, q_max = 2385.2 1/h, r2 = 0.989429
Greenshields fit with speeds = [82, 70, 58, 47, 33, 21].	NEEDS_INPUT	transport_greenshields_fit needs densities (observed densities in veh/km, in the same order). Example: solve('transport_
Greenshields fit with speeds = [82, 70, 58, 47, 33, 21] and densities = [18, 32, 46, 60, 78, 95] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Greenshields fit with speeds = [82, 70, 58] and densities = [18, 32].	INVALID_INPUT	densities: needs at least 3 entries, got 2.

Fit the Greenberg (logarithmic) model to speed-density observations,

`transport_greenberg_fit`

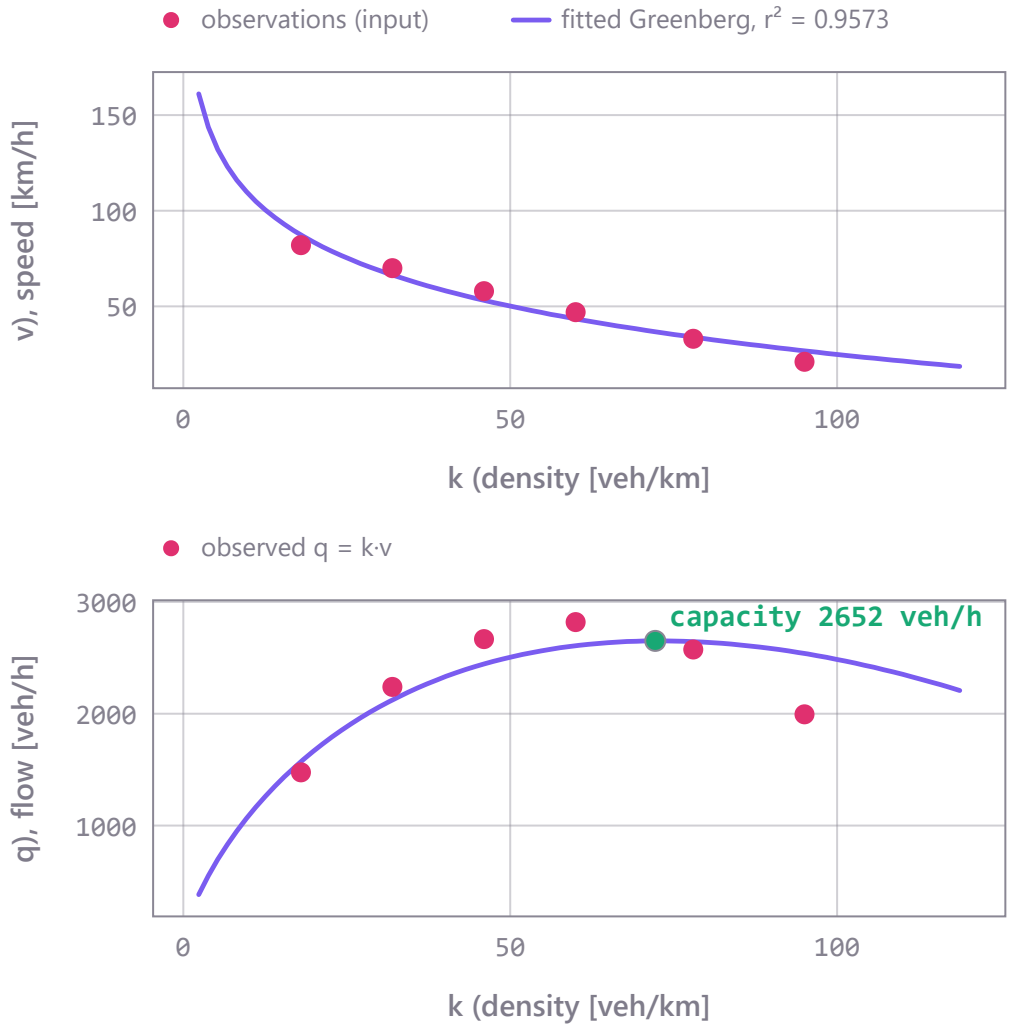
Least-squares fit of Greenberg's model $v = v_0$ ln(kj / k) (v against ln k) to observed speeds (km/h) and densities (veh/km): the optimum speed v0, jam density, capacity v0 kj / e and the coefficient of determination.

$$v = v_0 \ln \frac{k_j}{k}, \quad q_{max} = \frac{v_0 k_j}{e}$$

input	meaning	condition
`speeds`	observed space-mean speeds in km/h	
`densities`	observed densities in veh/km, in the same order	

output	meaning
`v0`	optimum speed v0 (km/h)
`kj`	jam density (veh/km)
`q_max`	capacity (veh/h)
`r2`	coefficient of determination (v on ln k)

- **Method:** least squares of v on ln k: slope -v0, intercept v0 ln kj; q_max = v0 kj / e
- **Assumes:** at least three observations; speeds fall with density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Speed–density observations (speeds [82, 70, 58, 47, 33, 21] km/h at densities [18, 32, 46, 60, 78, 95] veh/km) and the Greenberg model with the returned parameters v0 = 36.4384 km/h, kj = 197.836 veh/km (r^2 = 0.957329); below, the same as flow–density, observed flow k·v. Marked: the returned capacity 2651.98 veh/h (at the peak of the fitted curve).

```
transportation_ai999x.solve('transport_greenberg_fit', speeds=[82, 70, 58, 47, 33, 21], densities=
[18, 32, 46, 60, 78, 95])
# -> v0 = 36.4384 km/h, kj = 197.836 1/km, q_max = 2651.98 1/h, r2 = 0.957329   (SOLVED)
```

```
transportation_ai999x.solve('transport_greenberg_fit', speeds=[61.5, 48.2, 39.0, 27.4, 15.8],
densities=[25, 42, 58, 80, 105])
# -> v0 = 31.5044 km/h, kj = 186.575 1/km, q_max = 2162.38 1/h, r2 = 0.987736   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Greenberg fit with speeds = [82, 70, 58, 47, 33, 21] and densities = [18, 32, 46, 60, 78, 95].	SOLVED	v0 = 36.4384 km/h, kj = 197.836 1/km, q_max = 2651.98 1/h, r2 = 0.957329
Greenberg fit with speeds = [61.5, 48.2, 39, 27.4, 15.8] and densities = [25, 42, 58, 80, 105].	SOLVED	v0 = 31.5044 km/h, kj = 186.575 1/km, q_max = 2162.38 1/h, r2 = 0.987736
Greenberg fit with speeds = [82, 70, 58, 47, 33, 21].	NEEDS_INPUT	transport_greenberg_fit needs densities (observed densities in veh/km, in the same order). Example: solve('transport_gre

question	outcome	result
Greenberg fit with speeds = [82, 70, 58, 47, 33, 21] and densities = [18, 32, 46, 60, 78, 95] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Greenberg fit with speeds = [82, 70, 58] and densities = [18, 32].	INVALID_INPUT	densities: needs at least 3 entries, got 2.

Fit the Underwood (exponential) model to speed-density observations,

`transport_underwood_fit`

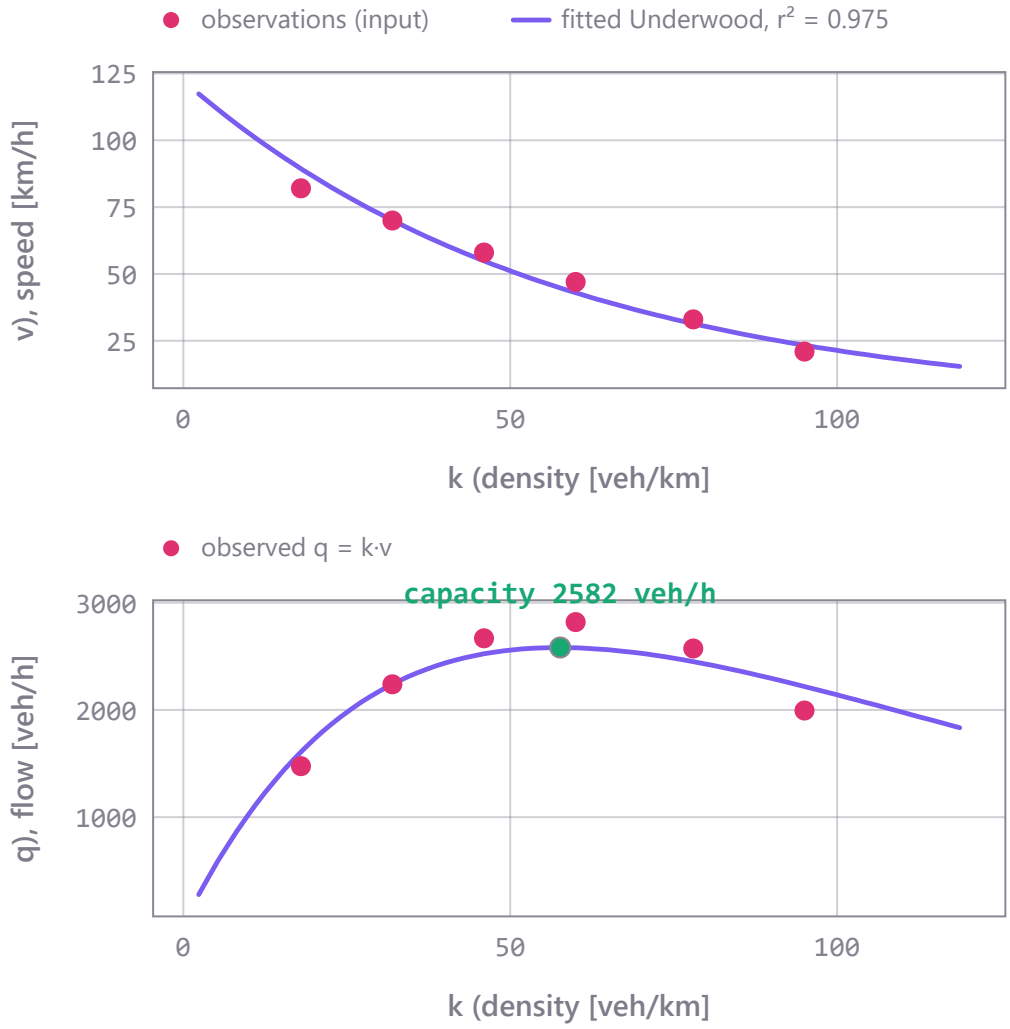
Least-squares fit of Underwood's model $v = vf$ exp(-k / k0) (ln v against k) to observed speeds (km/h) and densities (veh/km): the free-flow speed, the optimum density k0, capacity vf k_0/e and the coefficient of determination.

$$v = v_f\, e^{-k/k_0}, \quad q_{max} = \frac{v_f k_0}{e}$$

input	meaning	condition
`speeds`	observed space-mean speeds in km/h	
`densities`	observed densities in veh/km, in the same order	

output	meaning
`vf`	free-flow speed (km/h)
`k0`	optimum density (veh/km)
`q_max`	capacity (veh/h)
`r2`	coefficient of determination (ln v on k)

- **Method:** least squares of ln v on k: intercept ln vf, slope -1/k0; q_max = vf k_0/e
- **Assumes:** at least three observations; speeds fall with density
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Speed–density observations (speeds [82, 70, 58, 47, 33, 21] km/h at densities [18, 32, 46, 60, 78, 95] veh/km) and the Underwood model with the returned parameters vf = 122.302 km/h, k0 = 57.396 veh/km (r^2 = 0.974956); below, the same as flow–density, observed flow k·v. Marked: the returned capacity 2582.38 veh/h (at the peak of the fitted curve).

```
transportation_ai999x.solve('transport_underwood_fit', speeds=[82, 70, 58, 47, 33, 21], densities=
[18, 32, 46, 60, 78, 95])
# -> vf = 122.302 km/h, k0 = 57.396 1/km, q_max = 2582.38 1/h, r2 = 0.974956   (SOLVED)
```

```
transportation_ai999x.solve('transport_underwood_fit', speeds=[61.5, 48.2, 39.0, 27.4, 15.8],
densities=[25, 42, 58, 80, 105])
# -> vf = 97.9236 km/h, k0 = 59.7023 1/km, q_max = 2150.72 1/h, r2 = 0.98803   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Underwood fit with speeds = [82, 70, 58, 47, 33, 21] and densities = [18, 32, 46, 60, 78, 95].	SOLVED	vf = 122.302 km/h, k0 = 57.396 1/km, q_max = 2582.38 1/h, r2 = 0.974956
Underwood fit with speeds = [61.5, 48.2, 39, 27.4, 15.8] and densities = [25, 42, 58, 80, 105].	SOLVED	vf = 97.9236 km/h, k0 = 59.7023 1/km, q_max = 2150.72 1/h, r2 = 0.98803
Underwood fit with speeds = [82, 70, 58, 47, 33, 21].	NEEDS_INPUT	transport_underwood_fit needs densities (observed densities in veh/km, in the same order). Example: solve('transport_und

question	outcome	result
Underwood fit with speeds = [82, 70, 58, 47, 33, 21] and densities = [18, 32, 46, 60, 78, 95] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Underwood fit with speeds = [82, 70, 58] and densities = [18, 32].	INVALID_INPUT	densities: needs at least 3 entries, got 2.

Greenberg model: capacity and density at capacity, `transport_greenberg_capacity`

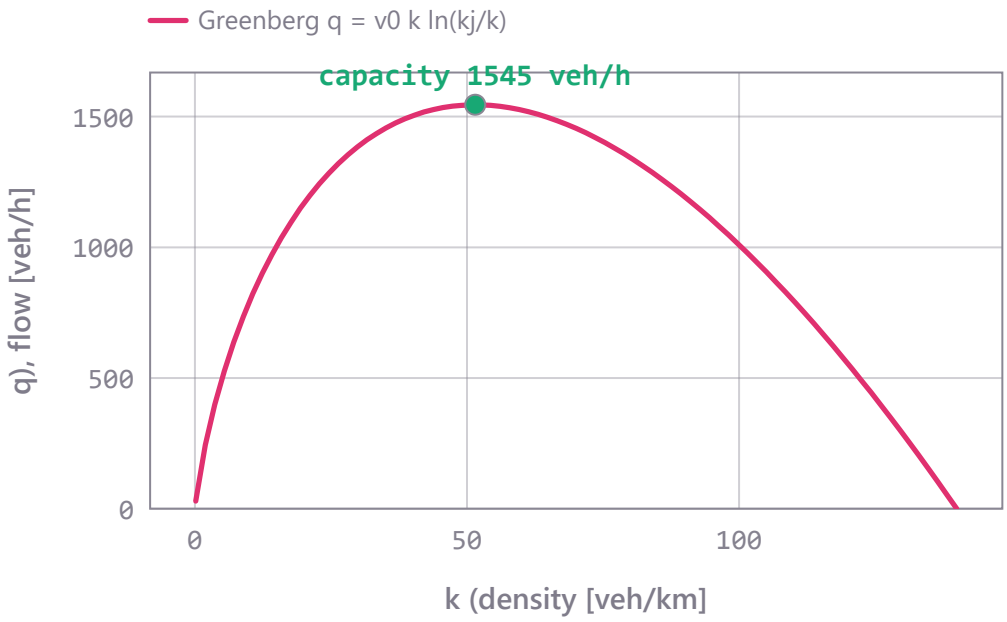
Capacity of a traffic stream following Greenberg's model $v = v_0$ ln(kj / k) and the density at which it occurs (the speed there is v0).

$$q_{max} = \frac{v_0 k_j}{e}$$

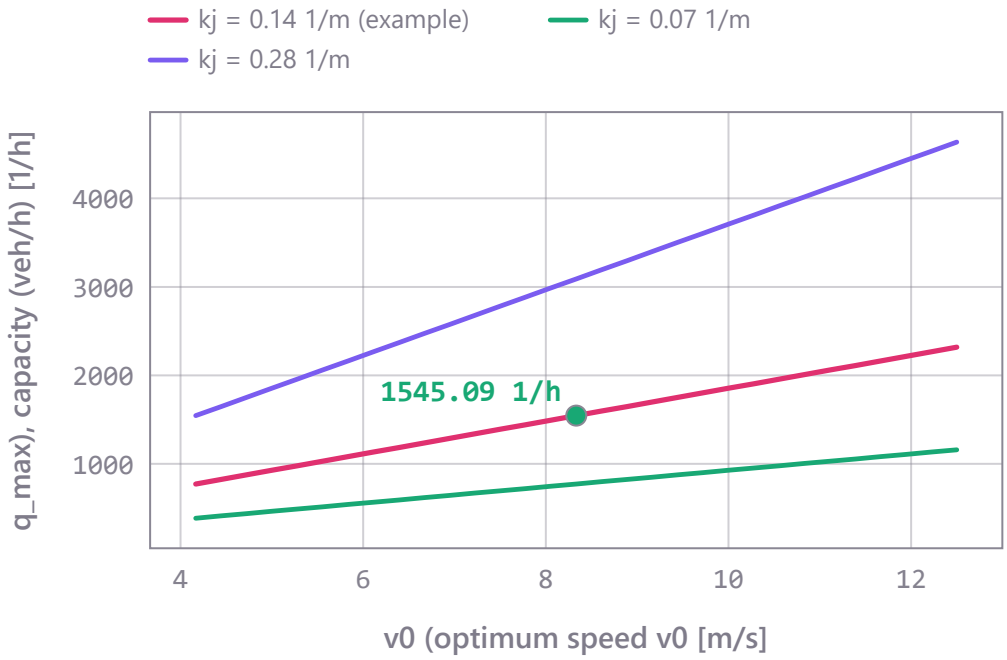
input	meaning	condition
`v0`	optimum speed v0	
`kj`	jam density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	

output	meaning
`q_max`	capacity (veh/h)
`k_m`	density at capacity (veh/km)

- **Method:** $q = v_0$ k ln(kj/k) is largest at $k = kj/e$: q_max = v0 kj / e
- **Assumes:** $v_0 > 0$; $kj > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Flow–density curve of Greenberg's model with optimum speed v0 = 30 km/h and jam density kj = 140 veh/km (the curve from these stated parameters). Marked: the returned capacity 1545.09 veh/h at density 51.5031 veh/km.



q_max (capacity (veh/h)) against v0 (optimum speed v0) from 4.16667 m/s to 12.5 m/s. One curve for each value of kj (jam density (veh/km in questions; veh/m in solve(), or '25 1/km')): 0.14 1/m, 0.07 1/m, 0.28 1/m. 31 points per curve computed by transportation-ai999x; marked: the worked example, v0 = 8.33333 m/s, q_max = 1545.09 1/h.

```
transportation_ai999x.solve('transport_greenberg_capacity', v0=8.333333333333334, kj=0.14)
# -> q_max = 1545.09 1/h, k_m = 51.5031 1/km   (SOLVED)
```

```
transportation_ai999x.solve('transport_greenberg_capacity', v0=6.944444444444445, kj=0.18)
# -> q_max = 1655.46 1/h, k_m = 66.2183 1/km   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Greenberg capacity with optimum speed = 30 km/h and jam density = 140 veh/km.	SOLVED	q_max = 1545.09 1/h, k_m = 51.5031 1/km
Greenberg capacity with optimum speed = 25 km/h and jam density = 180 veh/km.	SOLVED	q_max = 1655.46 1/h, k_m = 66.2183 1/km
Greenberg capacity with optimum speed = 30 km/h.	NEEDS_INPUT	transport_greenberg_capacity needs kj (jam density (veh/km in questions; veh/m in solve(), or '25 1/km')). Example: solv
Greenberg capacity with optimum speed = 30 km/h and jam density = 140 veh/km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Greenberg capacity with optimum speed = -30 km/h and jam density = 140 veh/km.	INVALID_INPUT	Greenberg model: capacity and density at capacity: v0 must be positive.

Underwood model: capacity and speed at capacity, `transport_underwood_capacity`

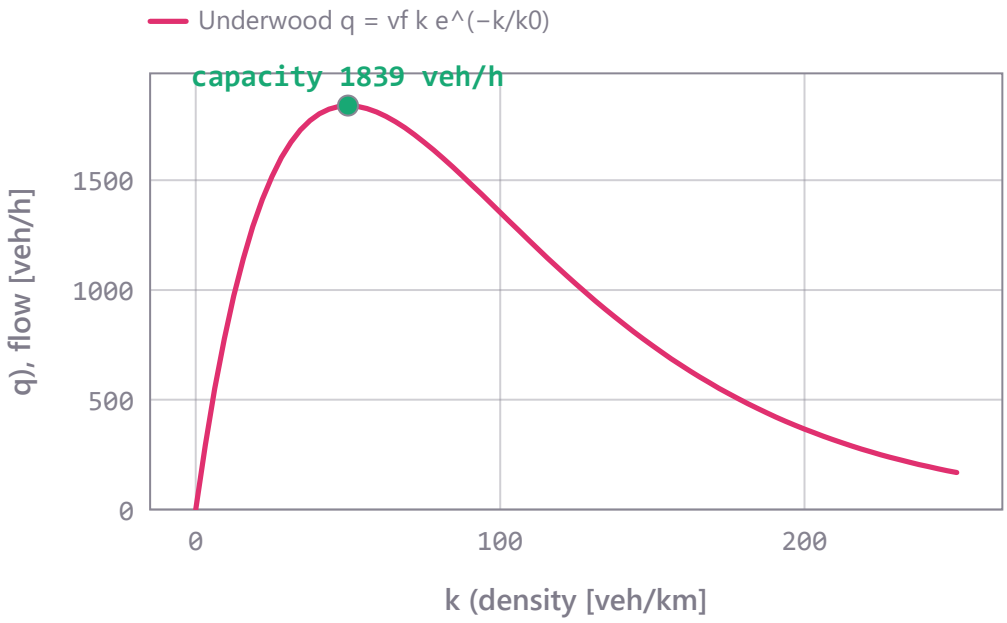
Capacity of a traffic stream following Underwood's model $v = vf$ exp(-k / k0), with the speed and density at capacity.

$$q_{max} = \frac{v_f k_0}{e}, \quad v_m = \frac{v_f}{e}$$

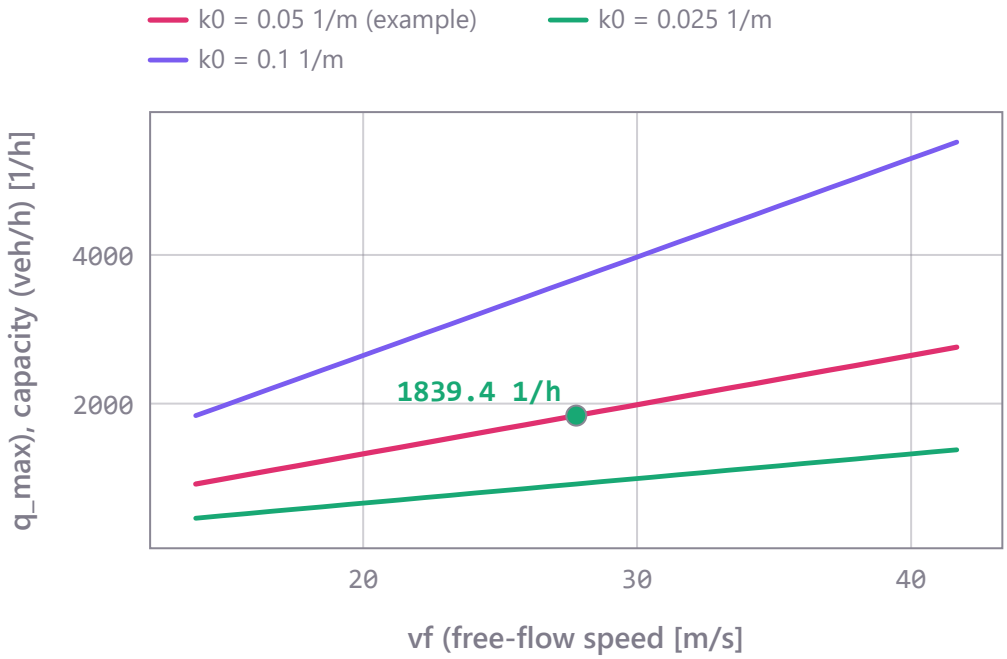
input	meaning	condition
`vf`	free-flow speed	
`k0`	optimum density k0 (veh/km in questions; veh/m in solve(), or '25 $1/km$')	

output	meaning
`q_max`	capacity (veh/h)
`v_m`	speed at capacity (km/h)
`k_m`	density at capacity (veh/km)

- **Method:** $q = vf$ k exp(-k/k0) is largest at $k = k_0$: q_max = vf k_0/e, $v = vf/e$
- **Assumes:** $vf > 0$; $k_0 > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Flow–density curve of Underwood's model with free-flow speed vf = 100 km/h and optimum density k0 = 50 veh/km (the curve from these stated parameters). Marked: the returned capacity 1839.4 veh/h at 50 veh/km, speed there 36.7879 km/h.



q_max (capacity (veh/h)) against vf (free-flow speed) from 13.8889 m/s to 41.6667 m/s. One curve for each value of k0 (optimum density k0 (veh/km in questions; veh/m in solve(), or '25 1/km')): 0.05 1/m, 0.025 1/m, 0.1 1/m. 31 points per curve computed by transportation-ai999x; marked: the worked example, vf = 27.7778 m/s, q_max = 1839.4 1/h.

```
transportation_ai999x.solve('transport_underwood_capacity', vf=27.77777777777778, k0=0.05)
# -> q_max = 1839.4 1/h, v_m = 36.7879 km/h, k_m = 50 1/km   (SOLVED)
```

```
transportation_ai999x.solve('transport_underwood_capacity', vf=22.22222222222222, k0=0.035)
# -> q_max = 1030.06 1/h, v_m = 29.4304 km/h, k_m = 35 1/km   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Underwood capacity with free flow speed = 100 km/h and optimum density = 50 veh/km.	SOLVED	q_max = 1839.4 1/h, v_m = 36.7879 km/h, k_m = 50 1/km
Underwood capacity with free flow speed = 80 km/h and optimum density = 35 veh/km.	SOLVED	q_max = 1030.06 1/h, v_m = 29.4304 km/h, k_m = 35 1/km
Underwood capacity with free flow speed = 100 km/h.	NEEDS_INPUT	transport_underwood_capacity needs k0 (optimum density k0 (veh/km in questions; veh/m in solve(), or '25 1/km')). Exampl
Underwood capacity with free flow speed = 100 km/h and optimum density = 50 veh/km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Underwood capacity with free flow speed = 100 km/h and optimum density = -50 veh/km.	INVALID_INPUT	Underwood model: capacity and speed at capacity: k0 must be positive.

Shockwave speed between two traffic states, `transport_shockwave_speed`

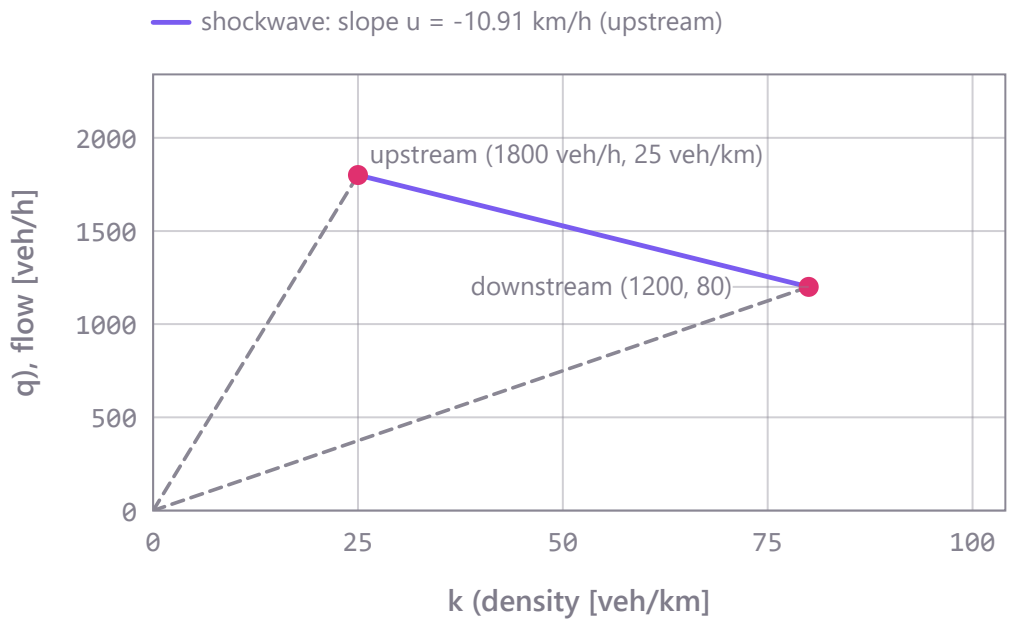
Speed of the boundary (shockwave) between an upstream traffic state (q1, k1) and a downstream state (q2, k2): $u = (q_2 - q_1)/(k_2 - k_1)$, negative when it moves upstream (for example the back of a queue).

$$u_w = \frac{q_2 - q_1}{k_2 - k_1}$$

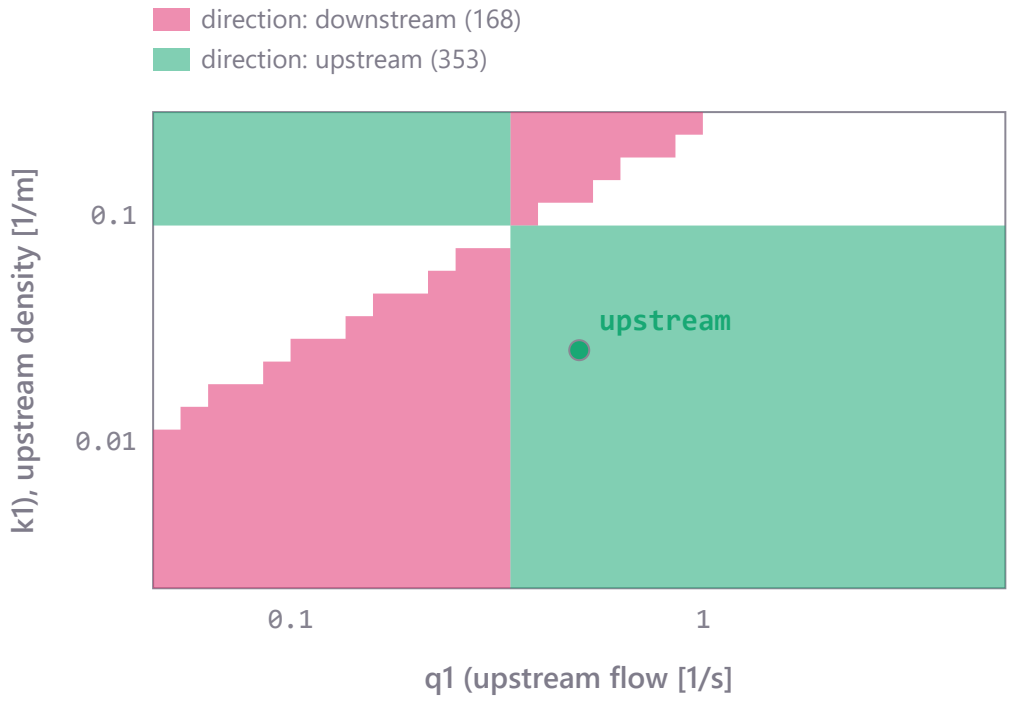
input	meaning	condition
`q1`	upstream flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`k1`	upstream density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`q2`	downstream flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`k2`	downstream density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	

output	meaning
`u`	shockwave speed (km/h), positive downstream
`u_ms`	shockwave speed (m/s)
`direction`	upstream, downstream or stationary

- **Method:** $u = (q_2 - q_1)/(k_2 - k_1)$
- **Assumes:** k1, $k_2 > 0$, q1, $q_2 \geq 0$; the denser state is the slower one
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two traffic states on flow–density axes: upstream q1 = 1800 veh/h at k1 = 25 veh/km and downstream q2 = 1200 veh/h at k2 = 80 veh/km (dashed rays: their speeds). The chord between them has the slope of the returned shockwave speed u = -10.9091 km/h (-3.0303 m/s), moving upstream.



upstream, downstream or stationary (direction) over q1 (upstream flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.05 1/s to 5 1/s and k1 (upstream density (veh/km in questions; veh/m in solve(), or '25 1/km')) from 0.0025 1/m to 0.25 1/m (logarithmic axes): each of the 31 × 21 cells is one run of transportation-ai999x, coloured by the verdict it returned, downstream: 168 cells, upstream: 353 cells; blank cells: inputs the capability refuses. Other inputs as in the worked example: q2 = 0.3333333333333333 1/s, k2 = 0.08 1/m. Marked: the worked example, q1 = 0.5 1/s, k1 = 0.025 1/m → upstream.

```
transportation_ai999x.solve('transport_shockwave_speed', q1=0.5, k1=0.025, q2=0.3333333333333333,
k2=0.08)
# -> u = -10.9091 km/h, u_ms = -3.0303 m/s, direction = upstream   (SOLVED)
```

```
transportation_ai999x.solve('transport_shockwave_speed', q1=0.0, k1=0.15, q2=0.5277777777777778,
k2=0.04)
# -> u = -17.2727 km/h, u_ms = -4.79798 m/s, direction = upstream   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Shockwave speed with upstream flow = 1800 veh/h, upstream density = 25 veh/km, downstream flow = 1200 veh/h and downstream density = 80 veh/km.	SOLVED	u = -10.9091 km/h, u_ms = -3.0303 m/s, direction = upstream
Shockwave speed with upstream flow = 0 veh/h, upstream density = 150 veh/km, downstream flow = 1900 veh/h and downstream density = 40 veh/km.	SOLVED	u = -17.2727 km/h, u_ms = -4.79798 m/s, direction = upstream
Shockwave speed with upstream flow = 1800 veh/h, upstream density = 25 veh/km and downstream flow = 1200 veh/h.	NEEDS_INPUT	transport_shockwave_speed needs k2 (downstream density (veh/km in questions; veh/m in solve(), or '25 1/km')). Example:
Shockwave speed with upstream flow = 1800 veh/h, upstream density = 25 veh/km, downstream flow = 1200 veh/h and downstream density = 80 veh/km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Shockwave speed with upstream flow = 3000 veh/h, upstream density = 60 veh/km, downstream flow = 500 veh/h and downstream density = 25 veh/km.	INVALID_INPUT	Shockwave speed between two traffic states: the states are inconsistent: a denser state must be slower.

Shockwave speed in a Greenshields traffic stream, `transport_greenshields_shockwave`

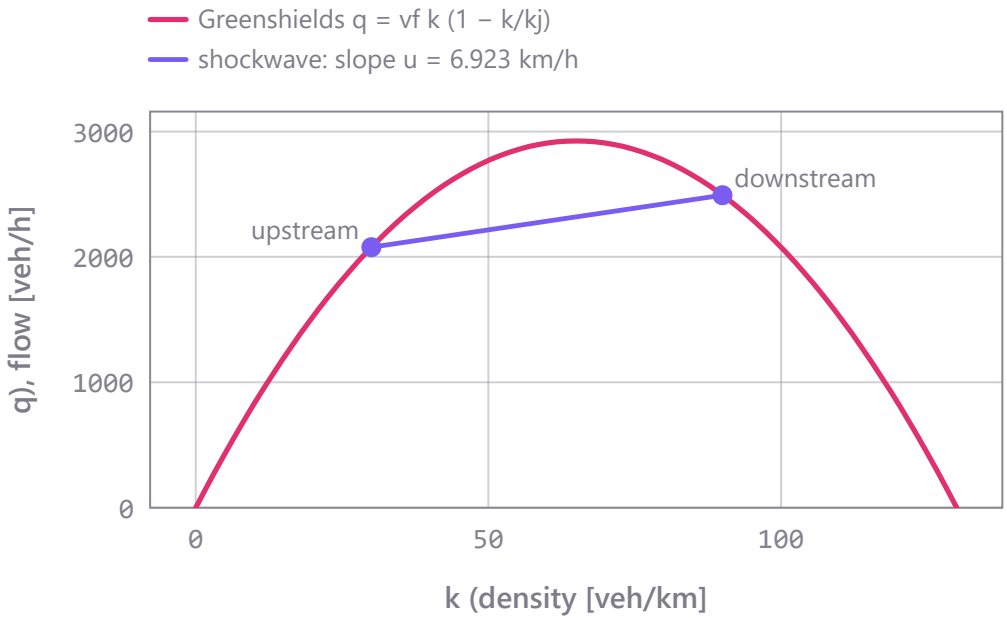
Speed of the shockwave between two states of a Greenshields stream with densities k1 (upstream) < k2 (downstream): $u = vf\,(1 - (k_1 + k_2)/kj)$.

$$u_w = v_f\left(1 - \frac{k_1 + k_2}{k_j}\right)$$

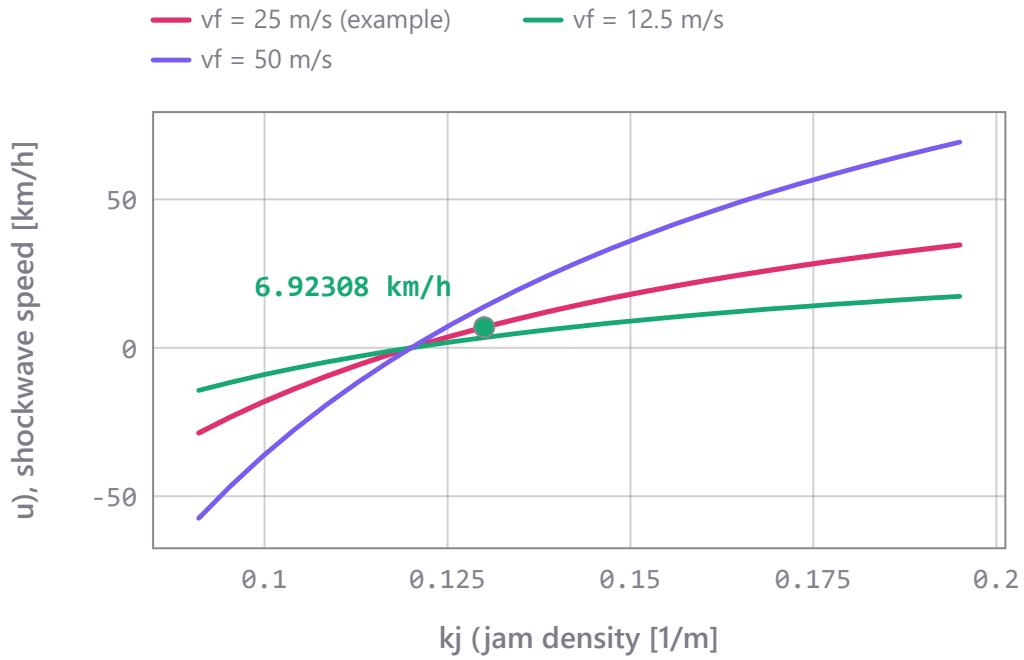
input	meaning	condition
`vf`	free-flow speed	
`kj`	jam density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`k1`	upstream density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`k2`	downstream density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	

output	meaning
`u`	shockwave speed (km/h), positive downstream
`u_ms`	shockwave speed (m/s)

- **Method:** $u = vf\,(1 - (k_1 + k_2)/kj)$
- **Assumes:** $0 < k_1 < k_2 < kj$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Greenshields flow–density curve (vf = 90 km/h, kj = 130 veh/km, from these stated parameters) with the upstream state k1 = 30 and the downstream state k2 = 90 veh/km: the chord joining them has the slope of the returned shockwave speed u = 6.92308 km/h (1.92308 m/s).



u (shockwave speed (km/h), positive downstream) against kj (jam density (veh/km in questions; veh/m in solve(), or '25 1/km')) from 0.065 1/m to 0.195 1/m. One curve for each value of vf (free-flow speed): 25 m/s, 12.5 m/s, 50 m/s. Other inputs as in the worked example: k1 = 0.03 1/m, k2 = 0.09 1/m. 25 points per curve computed by transportation-ai999x; marked: the worked example, kj = 0.13 1/m, u = 6.92308 km/h.

```
transportation_ai999x.solve('transport_greenshields_shockwave', vf=25.0, kj=0.13, k1=0.03, k2=0.09)
# -> u = 6.92308 km/h, u_ms = 1.92308 m/s   (SOLVED)
```

```
transportation_ai999x.solve('transport_greenshields_shockwave', vf=27.77777777777778, kj=0.12,
k1=0.02, k2=0.045)
# -> u = 45.8333 km/h, u_ms = 12.7315 m/s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Greenshields shockwave with free flow speed = 90 km/h, jam density = 130 veh/km, upstream density = 30 veh/km and downstream density = 90 veh/km.	SOLVED	u = 6.92308 km/h, u_ms = 1.92308 m/s
Greenshields shockwave with free flow speed = 100 km/h, jam density = 120 veh/km, upstream density = 20 veh/km and downstream density = 45 veh/km.	SOLVED	u = 45.8333 km/h, u_ms = 12.7315 m/s
Greenshields shockwave with free flow speed = 90 km/h, jam density = 130 veh/km and upstream density = 30 veh/km.	NEEDS_INPUT	transport_greenshields_shockwave needs k2 (downstream density (veh/km in questions; veh/m in solve(), or '25 1/km')). Ex
Greenshields shockwave with free flow speed = 90 km/h, jam density = 130 veh/km, upstream density = 30 veh/km and downstream density = 90 veh/km and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Greenshields shockwave with free flow speed = 90 km/h, jam density = 130	INVALID_INPUT	Shockwave speed in a Greenshields traffic stream: a Greenshields shock needs k1 < k2 < kj (a lighter

question	outcome	result
veh/km, upstream density = 100 veh/km and downstream density = 30 veh/km.		state running into

Queue at a signal by shockwave analysis, `transport_signal_queue_shockwave`

Kinematic-wave analysis of a signal: the queue grows upstream during the red at the shock speed between the arrival state (q_a, k_a) and jam density k_j, and from the start of green the discharge wave (saturation state q_s, k_s) chases it. The longest queue, the time it is reached (from the start of red) and both wave speeds.

$$t^* = \frac{w_2\, r}{w_2 - w_1}, \quad L_{max} = -w_1 t^*$$

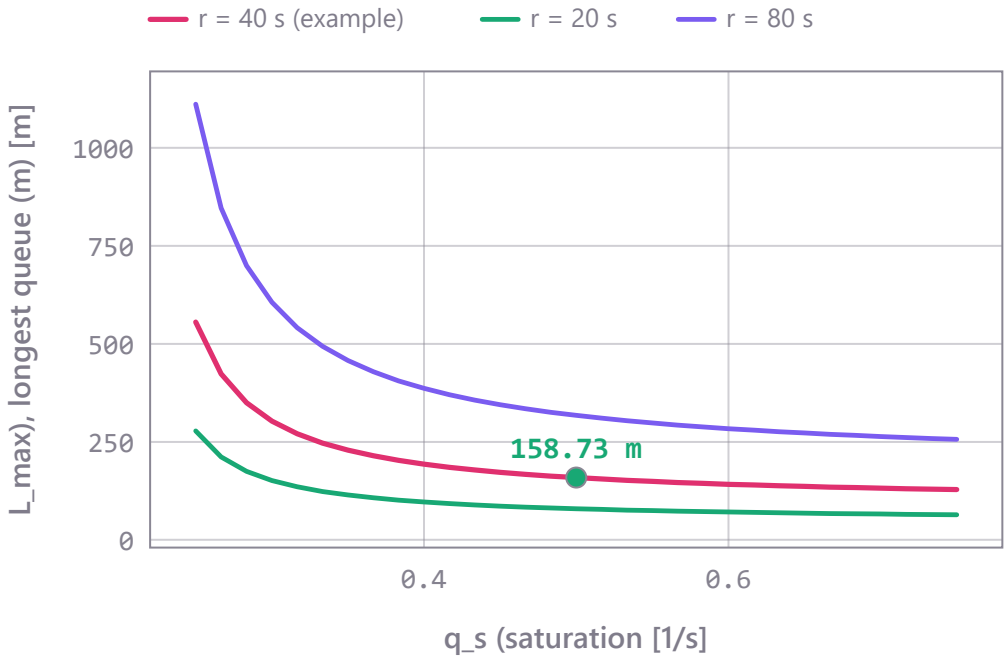
input	meaning	condition
`q_a`	arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`k_a`	arrival density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`k_j`	jam density (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`q_s`	saturation (discharge) flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`k_s`	density of the discharging stream (veh/km in questions; veh/m in solve(), or '25 $1/km$')	
`r`	red time	

output	meaning
`L_max`	longest queue (m)
`t_max`	time from the start of red to the longest queue (s)
`w_queue`	speed of the back of the queue (km/h)
`w_discharge`	speed of the discharge wave (km/h)

- **Method:** $w_1 = -q_a/(k_j - k_a)$, $w_2 = -q_s/(k_j - k_s)$; they meet at $t = w_2\, r/(w_2 - w_1)$; L_max = -w1 t
- **Assumes:** k_a, $k_s < k_j$; the discharge wave is faster than the queue growth
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Time–space diagram of the queue at a signal (arrivals 1000 veh/h at 20 veh/km, jam density 140 veh/km, discharge 1800 veh/h at 50 veh/km, red 40 s): the queue tail moves upstream at 8.33333 km/h, the discharge wave from the start of green at 20 km/h (both returned by the package); where they meet the queue is longest. Marked: t = 68.5714 s, maximum queue 158.73 m.



L_max (longest queue (m)) against q_s (saturation (discharge) flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.25 1/s to 0.75 1/s. One curve for each value of r (red time): 40 s, 20 s, 80 s. Other inputs as in the worked example: q_a = 0.2777777777777778 1/s, k_a = 0.02 1/m, k_j = 0.14 1/m, k_s = 0.05 1/m. 31 points per curve computed by transportation-ai999x; marked: the worked example, q_s = 0.5 1/s, L_max = 158.73 m.

```
transportation_ai999x.solve('transport_signal_queue_shockwave', q_a=0.2777777777777778, k_a=0.02,
k_j=0.14, q_s=0.5, k_s=0.05, r=40.0)
# -> L_max = 158.73 m, t_max = 68.5714 s, w_queue = -8.33333 km/h, w_discharge = -20 km/h
(SOLVED)
```

```
transportation_ai999x.solve('transport_signal_queue_shockwave', q_a=0.19444444444444445, k_a=0.015, k_j=0.15, q_s=0.4444444444444444, k_s=0.045, r=30.0)
# -> L_max = 65.4971 m, t_max = 45.4737 s, w_queue = -5.18519 km/h, w_discharge = -15.2381 km/h (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Queue at a signal by shockwave analysis with arrival flow = 1000 veh/h, arrival density = 20 veh/km, jam density = 140 veh/km, saturation flow = 1800 veh/h, discharge density = 50 veh/km and red time = 40 s.	SOLVED	L_max = 158.73 m, t_max = 68.5714 s, w_queue = -8.33333 km/h, w_discharge = -20 km/h
Queue at a signal by shockwave analysis with arrival flow = 700 veh/h, arrival density = 15 veh/km, jam density = 150 veh/km, saturation flow = 1600 veh/h, discharge density = 45 veh/km and red time = 30 s.	SOLVED	L_max = 65.4971 m, t_max = 45.4737 s, w_queue = -5.18519 km/h, w_discharge = -15.2381 km/h
Queue at a signal by shockwave analysis with arrival flow = 1000 veh/h, arrival density = 20 veh/km, jam density = 140 veh/km, saturation flow = 1800 veh/h and discharge density = 50 veh/km.	NEEDS_INPUT	transport_signal_queue_shockwave needs r (red time). Example: solve('transport_signal_queue_shockwave', q_a='0.277778 1/
Queue at a signal by shockwave analysis with arrival flow = 1000 veh/h, arrival density = 20 veh/km, jam density = 140 veh/km, saturation flow = 1800 veh/h, discharge density = 50 veh/km and red time = 40 s and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Queue at a signal by shockwave analysis with arrival flow = 1000 veh/h, arrival density = 160 veh/km, jam density = 140 veh/km, saturation flow = 1800 veh/h, discharge density = 50 veh/km and red time = 40 s.	INVALID_INPUT	Queue at a signal by shockwave analysis: the arrival and discharge densities must be below the jam density.

Queue and uniform delay at a signal (deterministic D/D/1),

`transport_signal_queue_uniform_delay`

Uniform arrivals at a fixed-time signal with saturation flow s, effective red r and effective green g (undersaturated): the largest queue, the time after the start of green until it clears, the total delay per cycle, the average (uniform) delay per vehicle and the share of vehicles that stop.

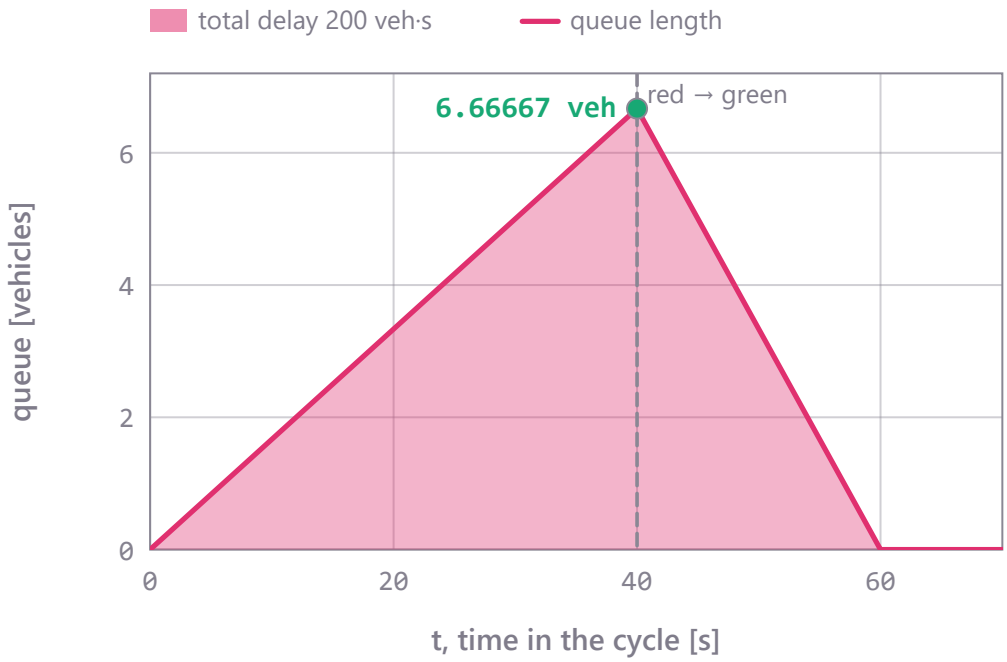
$$d = \frac{r^2}{2C\,(1-\lambda/s)} = \frac{C(1-g/C)^2}{2(1-\lambda/s)}$$

input	meaning	condition
`lam`	arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	

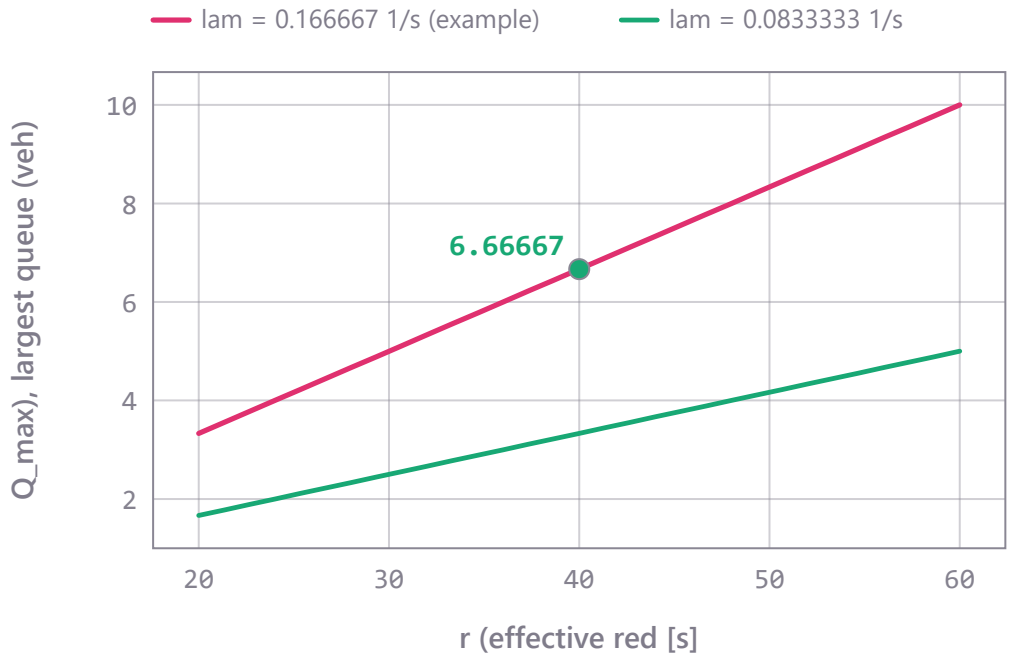
input	meaning	condition
`s`	saturation flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`r`	effective red	
`g`	effective green	

output	meaning
`Q_max`	largest queue (veh)
`t_clear`	time after the start of green to clear the queue (s)
`D_total`	total delay per cycle (veh s)
`d_uniform`	average delay per vehicle (s)
`P_stopped`	share of vehicles that stop

- **Method:** Q = lam r, t_c = lam r/(s - lam), D = lam r^2 / (2(1 - lam/s)), d = r^2 / (2C (1 - lam/s))
- **Assumes:** lam < s; the queue clears within the green (lam $C \leq s$ g)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Queue at a signal over one cycle (arrivals 600 veh/h, saturation flow 1800 veh/h, red 40 s, green 30 s): it grows during red to the maximum the package returned, 6.66667 vehicles, and clears 20 s into green; the shaded area is the total delay 200 veh·s (17.1429 s per vehicle, 85.7143 % stopped). Marked: t = 40 s, queue = 6.66667 vehicles.



Q_max (largest queue (veh)) against r (effective red) from 20 s to 60 s. One curve for each value of lam (arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')): 0.166667 1/s, 0.0833333 1/s. Other inputs as in the worked example: s = 0.5 1/s, g = 30.0 s. 31 points per curve computed by transportation-ai999x; marked: the worked example, r = 40 s, Q_max = 6.66667.

```
transportation_ai999x.solve('transport_signal_queue_uniform_delay', lam=0.16666666666666666, s=0.5,
r=40.0, g=30.0)
# -> Q_max = 6.66667, t_clear = 20 s, D_total = 200 s, d_uniform = 17.1429 s, P_stopped = 0.857143
(SOLVED)
```

```
transportation_ai999x.solve('transport_signal_queue_uniform_delay', lam=0.2222222222222222,
s=0.5277777777777778, r=35.0, g=45.0)
# -> Q_max = 7.77778, t_clear = 25.4545 s, D_total = 235.101 s, d_uniform = 13.2244 s, P_stopped =
0.755682   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Signal queue and uniform delay with arrival flow = 600 veh/h, saturation flow = 1800 veh/h, red = 40 s and green = 30 s.	SOLVED	Q_max = 6.66667, t_clear = 20 s, D_total = 200 s, d_uniform = 17.1429 s, P_stopped = 0.857143
Signal queue and uniform delay with arrival flow = 800 veh/h, saturation flow = 1900 veh/h, red = 35 s and green = 45 s.	SOLVED	Q_max = 7.77778, t_clear = 25.4545 s, D_total = 235.101 s, d_uniform = 13.2244 s, P_stopped = 0.755682
Signal queue and uniform delay with arrival flow = 600 veh/h, saturation flow = 1800 veh/h and red = 40 s.	NEEDS_INPUT	transport_signal_queue_uniform_delay needs g (effective green). Example: solve('transport_signal_queue_uniform_delay', l
Signal queue and uniform delay with arrival flow = 600 veh/h, saturation flow = 1800 veh/h, red	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
= 40 s and green = 30 s and colour = 3.		
Signal queue and uniform delay with arrival flow = 1200 veh/h, saturation flow = 1800 veh/h, red = 40 s and green = 30 s.	INVALID_INPUT	Queue and uniform delay at a signal (deterministic D/D/1): the queue does not clear within the green (oversaturated: lam

Queue and delay at a temporary bottleneck (incident, lane closure),

`transport_bottleneck_queue`

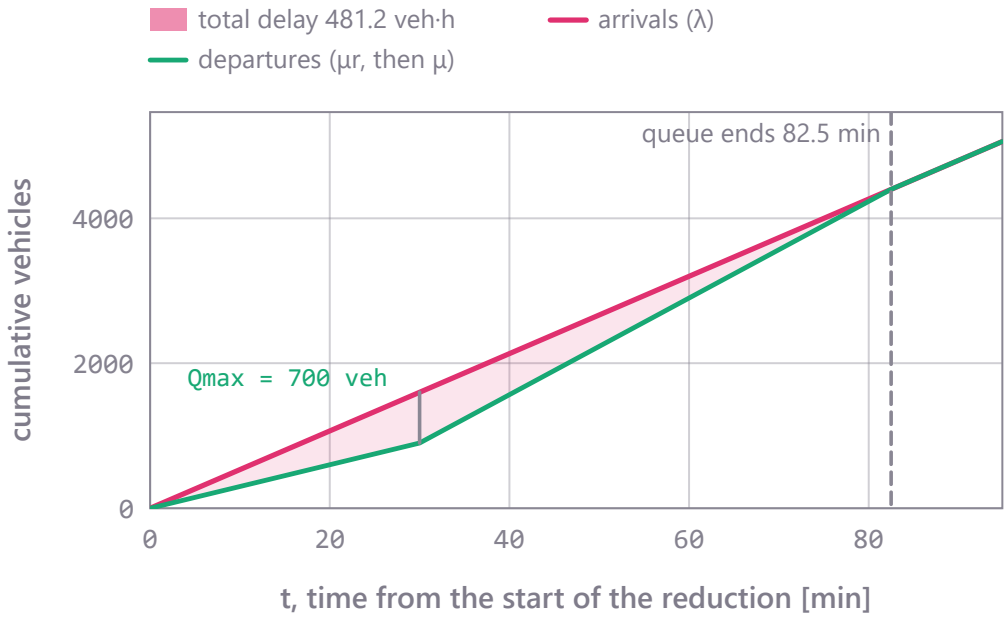
Deterministic queue when the capacity of a road falls from mu to mu_r for a time t_r (an incident or a work zone) while vehicles arrive at lam: the longest queue, how long the queue lasts and the total delay.

$$Q_{max} = (\lambda - \mu_r)t_r, \quad T = t_r + \frac{Q_{max}}{\mu - \lambda}, \quad D = \tfrac{1}{2} Q_{max} T$$

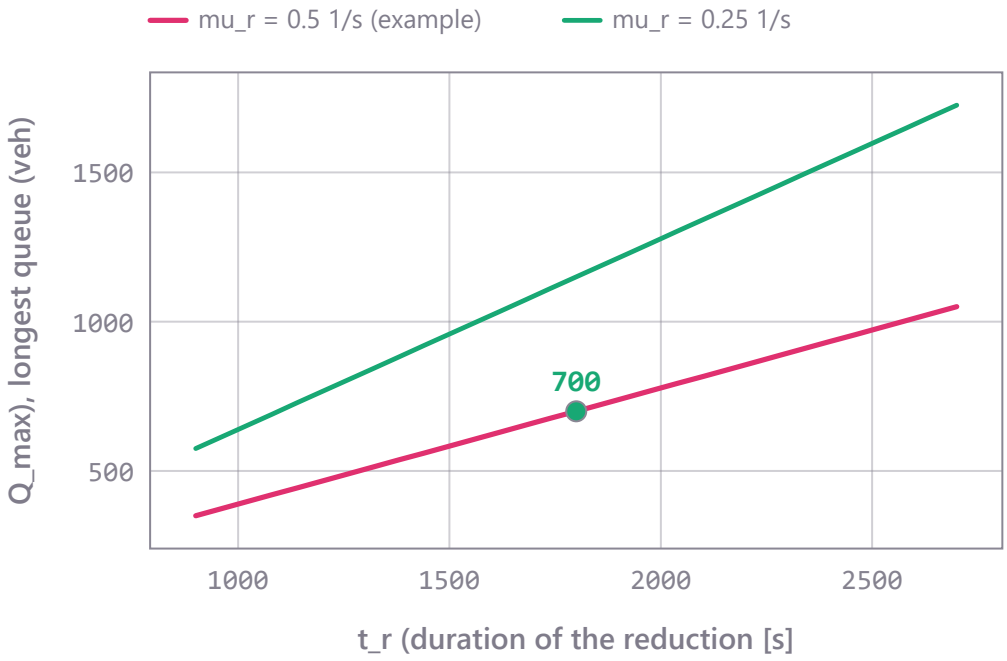
input	meaning	condition
`lam`	arrival flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`mu`	normal capacity (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`mu_r`	reduced capacity (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`t_r`	duration of the reduction	

output	meaning
`Q_max`	longest queue (veh)
`T`	duration of the queue from the start of the reduction (s)
`T_min`	duration of the queue (min)
`D_total`	total delay (veh h)

- **Method:** Q = (lam - mu_r) t_r, T = t_r + Q/(mu - lam), total delay = Q $T/2$
- **Assumes:** mu_r < lam < mu
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cumulative arrivals and departures at a bottleneck: arrivals at λ = 3200 veh/h, capacity cut from μ = 4000 to μr = 1800 veh/h for 30 min (the curves are drawn from these inputs). The vertical gap is the queue, the shaded area the delay. Marked: the returned longest queue 700 veh, queue duration 82.5 min and total delay 481.25 veh·h.



Q_max (longest queue (veh)) against t_r (duration of the reduction) from 900 s to 2700 s. One curve for each value of mu_r (reduced capacity (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')): 0.5 1/s, 0.25 1/s. Other inputs as in the worked example: lam = 0.8888888888888888 1/s, mu = 1.1111111111111112 1/s. 31 points per curve computed by transportation-ai999x; marked: the worked example, t_r = 1800 s, Q_max = 700.

```
transportation_ai999x.solve('transport_bottleneck_queue', lam=0.8888888888888888,
mu=1.1111111111111112, mu_r=0.5, t_r=1800.0)
# -> Q_max = 700, T = 4950 s, T_min = 82.5 min, D_total = 481.25 h   (SOLVED)
```

```
transportation_ai999x.solve('transport_bottleneck_queue', lam=0.4166666666666667,
mu=0.5555555555555556, mu_r=0.2777777777777778, t_r=1200.0)
# -> Q_max = 166.667, T = 2400 s, T_min = 40 min, D_total = 55.5556 h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bottleneck queue with arrival flow = 3200 veh/h, capacity = 4000 veh/h, reduced capacity = 1800 veh/h and duration = 30 min.	SOLVED	Q_max = 700, T = 4950 s, T_min = 82.5 min, D_total = 481.25 h
Bottleneck queue with arrival flow = 1500 veh/h, capacity = 2000 veh/h, reduced capacity = 1000 veh/h and duration = 20 min.	SOLVED	Q_max = 166.667, T = 2400 s, T_min = 40 min, D_total = 55.5556 h
Bottleneck queue with arrival flow = 3200 veh/h, capacity = 4000 veh/h and reduced capacity = 1800 veh/h.	NEEDS_INPUT	transport_bottleneck_queue needs t_r (duration of the reduction). Example: solve('transport_bottleneck_queue', lam='0.88
Bottleneck queue with arrival flow = 3200 veh/h, capacity = 4000 veh/h, reduced capacity = 1800 veh/h and duration = 30 min and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Bottleneck queue with arrival flow = 3200 veh/h, capacity = 3000 veh/h, reduced capacity = 1800 veh/h and duration = 30 min.	INVALID_INPUT	Queue and delay at a temporary bottleneck (incident, lane closure): need reduced capacity < arrival flow < full capacity

Queue from arrival counts and capacity per interval, `transport_interval_queue`

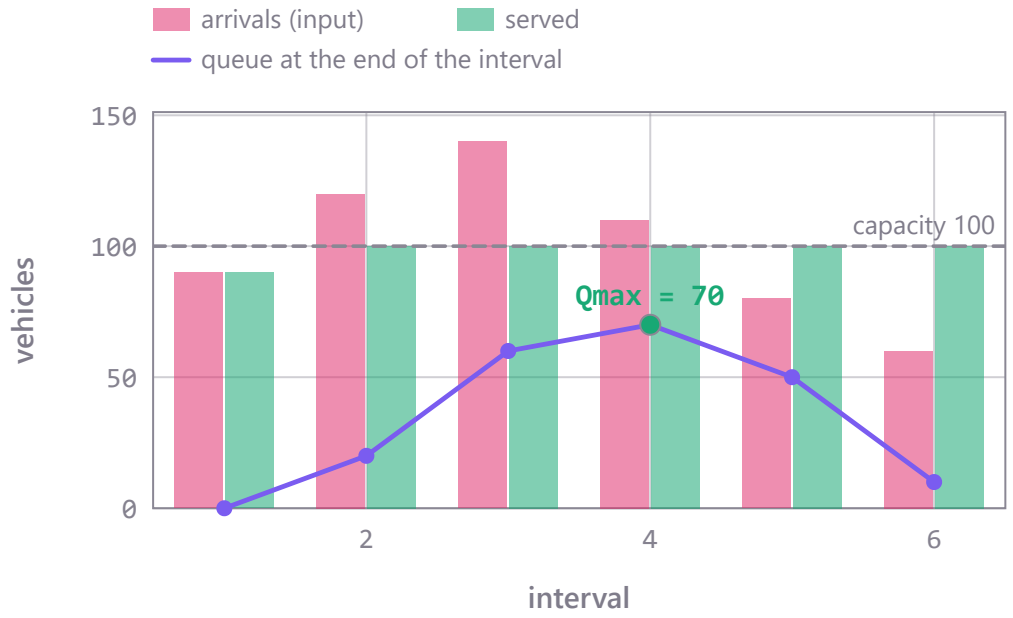
Vehicles queued at the end of each counting interval when the arrivals per interval are served up to the capacity per interval (deterministic queue, first in first out), the longest queue and the vehicles served per interval.

$$Q_k = \max\left(0,\ Q_{k-1} + a_k - c_k\right)$$

input	meaning	condition
`arrivals`	arrivals in each interval (veh)	
`capacity`	capacity of each interval (veh), or one value for all	

output	meaning
`queue`	queue at the end of each interval (veh)
`Q_max`	longest queue (veh)
`served`	vehicles served in each interval (veh)

- **Method:** Q_k = max(0, Q_{$k-1$} + a_k - c_k)
- **Assumes:** counts >= 0; capacity: one value or one per interval
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Arrivals [90, 120, 140, 110, 80, 60] per interval against a capacity of [100] per interval: the vehicles served and the queue left at the end of each interval as returned. Marked: the longest queue 70 vehicles.

```
transportation_ai999x.solve('transport_interval_queue', arrivals=[90, 120, 140, 110, 80, 60],
capacity=[100])
# -> queue = [0.0, 20.0, 60.0, 70.0, 50.0, 10.0], Q_max = 70, served = [90.0, 100.0, 100.0, 100.0,
100.0, 100.0]   (SOLVED)
```

```
transportation_ai999x.solve('transport_interval_queue', arrivals=[30, 55, 70, 40, 20], capacity=
[45, 45, 30, 45, 45])
# -> queue = [0.0, 10.0, 50.0, 45.0, 20.0], Q_max = 50, served = [30.0, 45.0, 30.0, 45.0, 45.0]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Interval queue with arrivals = [90, 120, 140, 110, 80, 60] and capacity = [100].	SOLVED	queue = [0.0, 20.0, 60.0, 70.0, 50.0, 10.0], Q_max = 70, served = [90.0, 100.0, 100.0, 100.0, 100.0, 100.0]
Interval queue with arrivals = [30, 55, 70, 40, 20] and capacity = [45, 45, 30, 45, 45].	SOLVED	queue = [0.0, 10.0, 50.0, 45.0, 20.0], Q_max = 50, served = [30.0, 45.0, 30.0, 45.0, 45.0]
Interval queue with arrivals = [90, 120, 140, 110, 80, 60].	NEEDS_INPUT	transport_interval_queue needs capacity (capacity of each interval (veh), or one value for all). Example: solve('transpo
Interval queue with arrivals = [90, 120, 140, 110, 80, 60] and capacity = [100] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Interval queue with arrivals = [90, 120, 140] and capacity = [100, 90].	INVALID_INPUT	Queue from arrival counts and capacity per interval: give one capacity for every interval (or a single capacity for all)

Green splits by equal degree of saturation (flow-ratio method), `transport_signal_green_splits`

Effective green time of each critical phase for a given cycle C and total lost time L, shared in proportion to the critical flow ratios $y_i = v_i/s_i$ (equal degree of saturation), and the critical degree of saturation X_c. Greens in

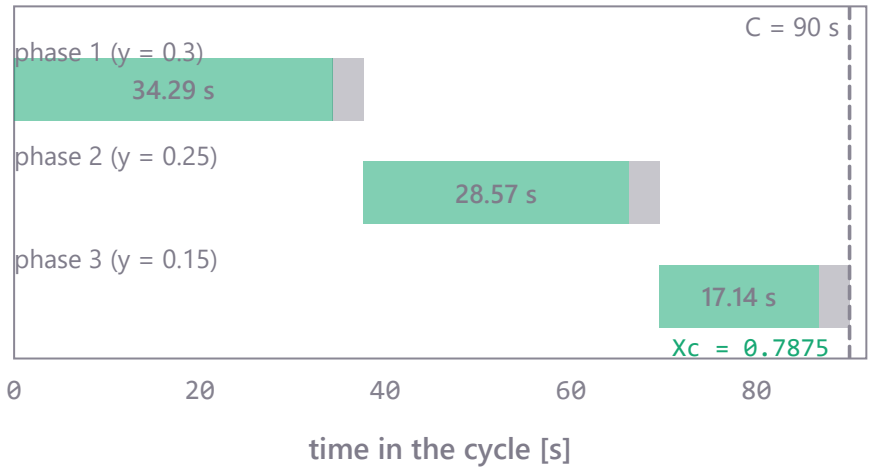
s.

$$g_i = \frac{y_i}{Y}(C - L), \quad X_c = \frac{Y\,C}{C - L}$$

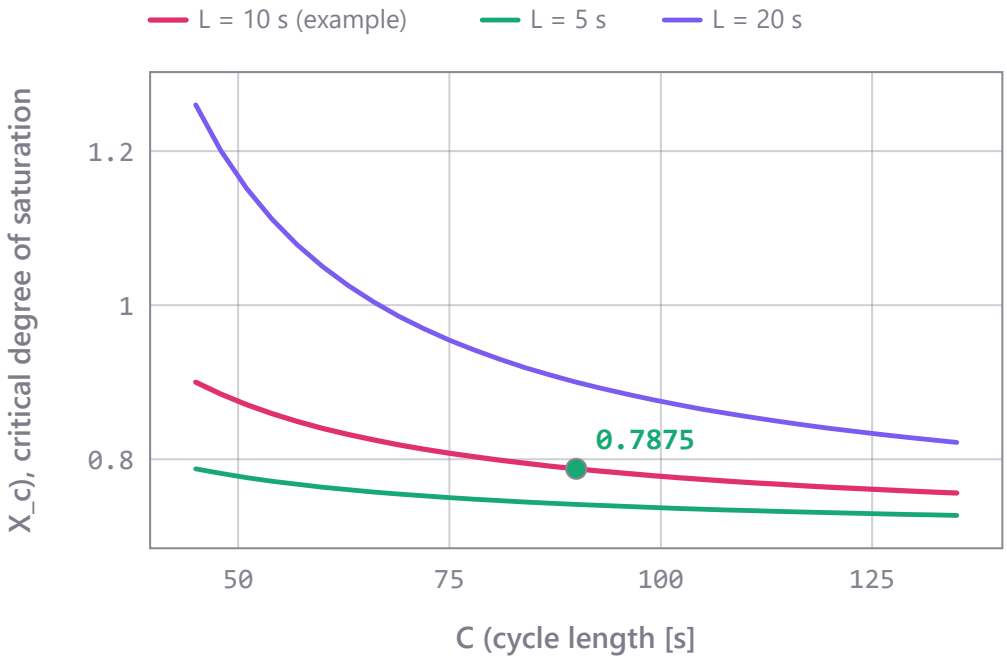
input	meaning	condition
C	cycle length	
L	total lost time per cycle	
y	critical flow ratio v/s of each phase	

output	meaning
g	effective green of each phase (s)
X_c	critical degree of saturation

- **Method:** $g_i = (y_i/Y)$(C - L), $X_c = Y$ C / (C - L), Y = sum y_i
- **Assumes:** $C > L$; sum of $y < 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Green splits of a 3-phase signal with cycle C = 90 s, total lost time L = 10 s and critical flow ratios [0.3, 0.25, 0.15]: the returned effective greens 34.2857, 28.5714, 17.1429 s in sequence (green), each followed by an equal share of the lost time (grey, an even split chosen for the drawing). Marked: critical volume-to-capacity ratio Xc = 0.7875.



X_c (critical degree of saturation) against C (cycle length) from 45 s to 135 s. One curve for each value of L (total lost time per cycle): 10 s, 5 s, 20 s. Other inputs as in the worked example: y = [0.3, 0.25, 0.15]. 31 points per curve computed by transportation-ai999x; marked: the worked example, C = 90 s, X_c = 0.7875.

```
transportation_ai999x.solve('transport_signal_green_splits', C=90.0, L=10.0, y=[0.3, 0.25, 0.15])
# -> g = [34.2857142857, 28.5714285714, 17.1428571429], X_c = 0.7875   (SOLVED)
```

```
transportation_ai999x.solve('transport_signal_green_splits', C=60.0, L=8.0, y=[0.4, 0.3])
# -> g = [29.7142857143, 22.2857142857], X_c = 0.807692   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Signal green splits with cycle = 90 s, lost time = 10 s and flow ratios = [0.3, 0.25, 0.15].	SOLVED	g = [34.2857142857, 28.5714285714, 17.1428571429], X_c = 0.7875
Signal green splits with cycle = 60 s, lost time = 8 s and flow ratios = [0.4, 0.3].	SOLVED	g = [29.7142857143, 22.2857142857], X_c = 0.807692
Signal green splits with cycle = 90 s and lost time = 10 s.	NEEDS_INPUT	transport_signal_green_splits needs y (critical flow ratio v/s of each phase). Example: solve('transport_signal_green_sp
Signal green splits with cycle = 90 s, lost time = 10 s and flow ratios = [0.3, 0.25, 0.15] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Signal green splits with cycle = 90 s, lost time = 10 s and flow ratios = [0.6, 0.5].	INVALID_INPUT	Green splits by equal degree of saturation (flow-ratio method): the critical flow ratios add up to 1 or more: no cycle c

Probability of n vehicle arrivals in an interval (Poisson), `transport_poisson_arrivals`

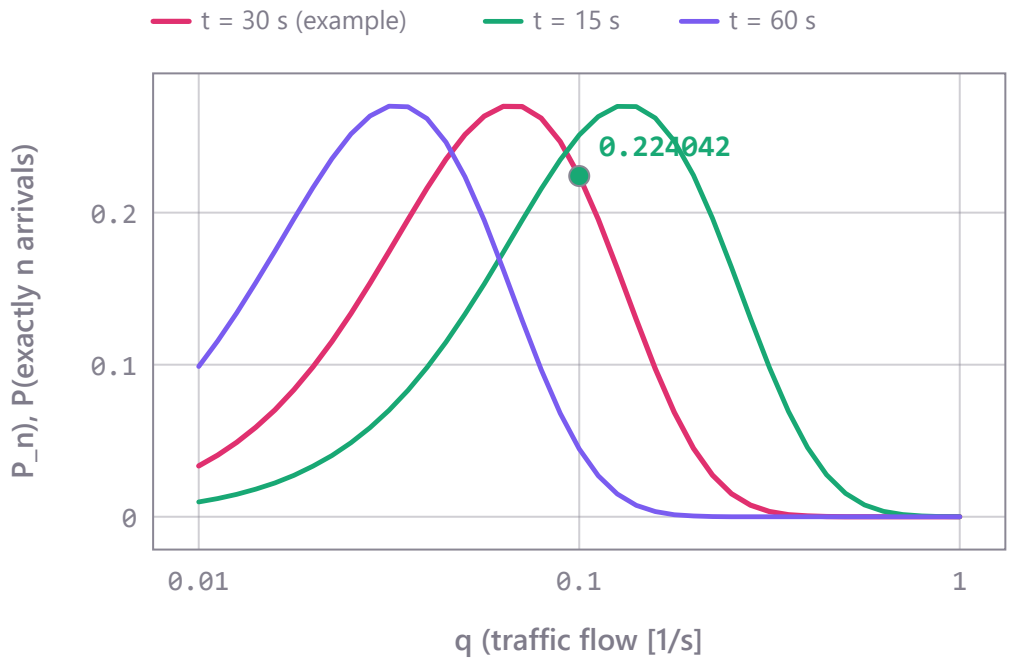
Probability that exactly n, at most n and at least n vehicles arrive in an interval t when arrivals are random (Poisson) at the flow q.

$$P(n) = \frac{(qt)^n e^{-qt}}{n!}$$

input	meaning	condition
`q`	traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`t`	interval length	
`n`	number of arrivals (whole number)	

output	meaning
`P_n`	P(exactly n arrivals)
`P_le`	P(at most n arrivals)
`P_ge`	P(at least n arrivals)

- **Method:** P(n) = (q t)^n e^(-q t) / n!
- **Assumes:** $q > 0$; $t > 0$; n a whole number >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P_n (P(exactly n arrivals)) against q (traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.01 1/s to 1 1/s on a logarithmic axis. One curve for each value of t (interval length): 30 s, 15 s, 60 s. Other inputs as in the worked example: n = 2. 41 points per curve computed by transportation-ai999x; marked: the worked example, q = 0.1 1/s, P_n = 0.224042.

```
transportation_ai999x.solve('transport_poisson_arrivals', q=0.1, t=30.0, n=2)
# -> P_n = 0.224042, P_le = 0.42319, P_ge = 0.800852   (SOLVED)
```

```
transportation_ai999x.solve('transport_poisson_arrivals', q=0.25, t=60.0, n=10)
# -> P_n = 0.0486108, P_le = 0.118464, P_ge = 0.930146   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Poisson arrivals with flow = 360 veh/h, interval = 30 s and arrivals = 2.	SOLVED	P_n = 0.224042, P_le = 0.42319, P_ge = 0.800852
Poisson arrivals with flow = 900 veh/h, interval = 60 s and arrivals = 10.	SOLVED	P_n = 0.0486108, P_le = 0.118464, P_ge = 0.930146
Poisson arrivals with flow = 360 veh/h and interval = 30 s.	NEEDS_INPUT	transport_poisson_arrivals needs n (number of arrivals (whole number)). Example: solve('transport_poisson_arrivals', q='
Poisson arrivals with flow = 360 veh/h, interval = 30 s and arrivals = 2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Poisson arrivals with flow = -360 veh/h, interval = 30 s and arrivals = 2.	INVALID_INPUT	Probability of n vehicle arrivals in an interval (Poisson): q must be positive.

Probability of a headway longer than h (negative exponential), `transport_headway_probability`

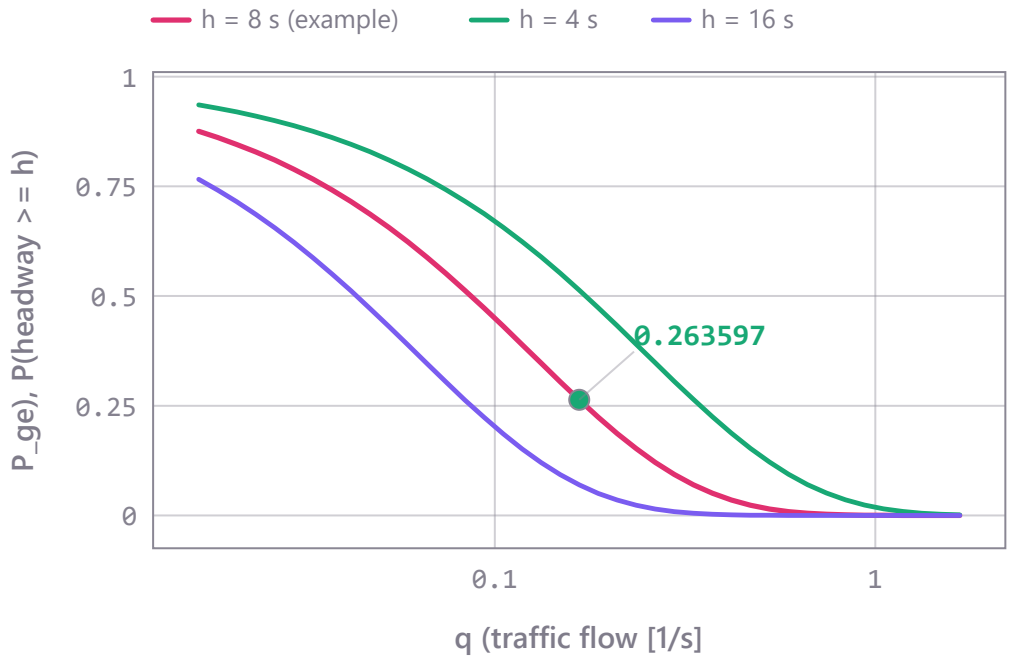
Probability that the time headway between random vehicles at flow q is at least h (and shorter than h), for the negative exponential distribution or the shifted one with a minimum headway tau.

$$P(H \geq h) = e^{-(h-\tau)/(\bar{h}-\tau)},\ \bar{h} = 1/q$$

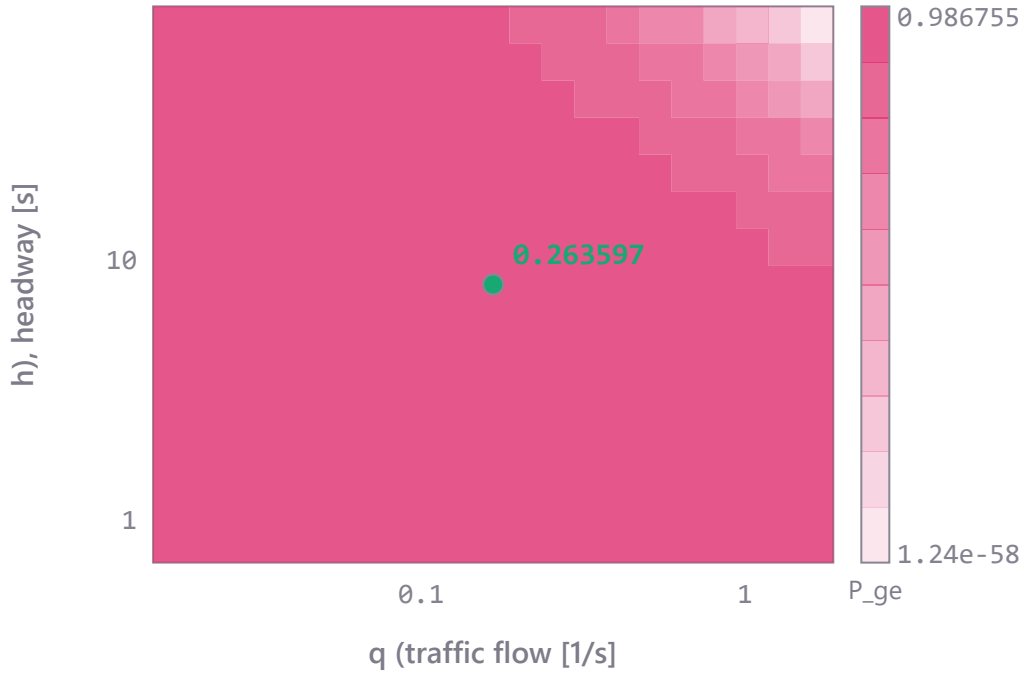
input	meaning	condition
`q`	traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`h`	headway	
`tau`	minimum headway (default 0)	

output	meaning
`P_ge`	P(headway >= h)
`P_lt`	P(headway < h)

- **Method:** P(H >= h) = exp(-$(h-\tau)/(1/q-\tau)$)
- **Assumes:** $q > 0$; $\tau < 1/q$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P_ge (P(headway >= h)) against q (traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0166667 1/s to 1.66667 1/s on a logarithmic axis. One curve for each value of h (headway): 8 s, 4 s, 16 s. 41 points per curve computed by transportation-ai999x; marked: the worked example, q = 0.166667 1/s, P_ge = 0.263597.



P_ge (P(headway >= h)) over q (traffic flow (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0166667 1/s to 1.66667 1/s and h (headway) from 0.8 s to 80 s (logarithmic axes): each of the 21 × 15 cells is one run of transportation-ai999x, shaded by its value (light 1.2418e-58 to dark 0.986755, logarithmic shading). Marked: the worked example, q = 0.166667 1/s, h = 8 s, P_ge = 0.263597.

```
transportation_ai999x.solve('transport_headway_probability', q=0.16666666666666666, h=8.0)
# -> P_ge = 0.263597, P_lt = 0.736403   (SOLVED)
```

```
transportation_ai999x.solve('transport_headway_probability', q=0.3333333333333333, h=4.0, tau=1.0)
# -> P_ge = 0.22313, P_lt = 0.77687   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Headway probability with flow = 600 veh/h and headway = 8 s.	SOLVED	P_ge = 0.263597, P_lt = 0.736403
Headway probability with flow = 1200 veh/h, headway = 4 s and minimum headway = 1 s.	SOLVED	P_ge = 0.22313, P_lt = 0.77687
Headway probability with flow = 600 veh/h.	NEEDS_INPUT	transport_headway_probability needs h (headway). Example: solve('transport_headway_probability', q='0.166667 1/s', h='8
Headway probability with flow = 600 veh/h and headway = 8 s and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Headway probability with flow = 1200 veh/h, headway = 4 s and minimum headway = 5 s.	INVALID_INPUT	Probability of a headway longer than h (negative exponential): the minimum headway must be shorter than the mean headway

Capacity of a minor stream by gap acceptance, `transport_gap_acceptance_capacity`

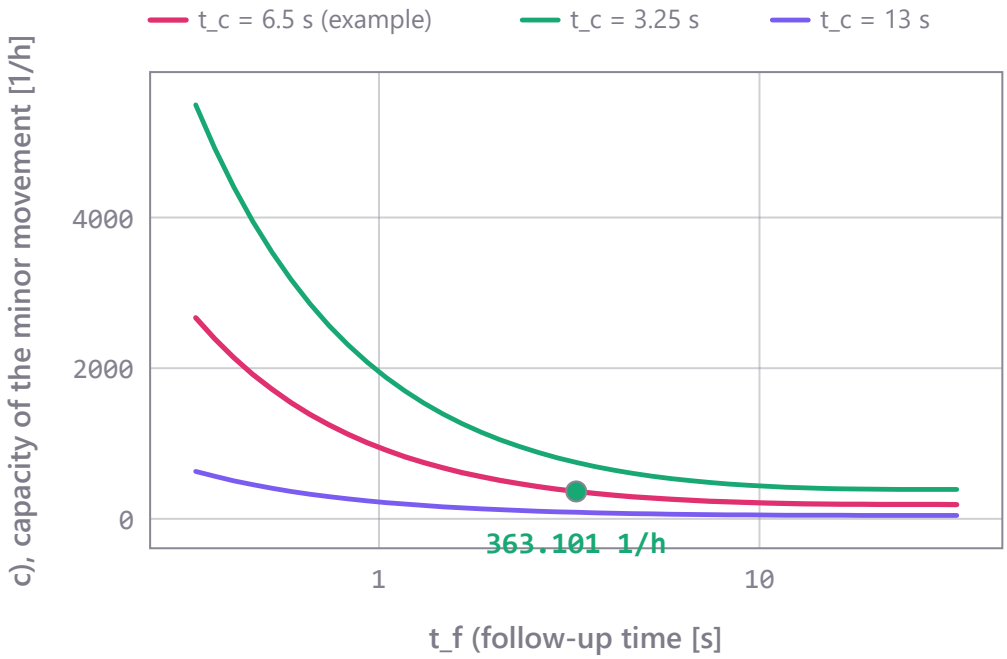
Capacity of a minor (stop- or yield-controlled) movement crossing a random major stream of flow q, with critical gap t_c and follow-up time t_f: Harders' formula (as in the HCM) or Siegloch's.

$$c = \frac{q\, e^{-q t_c}}{1 - e^{-q t_f}}$$

input	meaning	condition
`q`	major-stream flow (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`t_c`	critical gap	
`t_f`	follow-up time	
`model`	harders (HCM, default) or siegloch	

output	meaning
`c`	capacity of the minor movement (veh/h)

- **Method:** Harders: $c = q$ e^(-q t_c) / (1 - e^(-q t_f)); Siegloch: c = e^(-q $(t_c - t_f/2)$) / t_f
- **Assumes:** q, t_c, $t_f > 0$; Siegloch: $t_c > t_f/2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



c (capacity of the minor movement (veh/h)) against t_f (follow-up time) from 0.33 s to 33 s on a logarithmic axis. One curve for each value of t_c (critical gap): 6.5 s, 3.25 s, 13 s. Other inputs as in the worked example: q = 0.2222222222222222 1/s. 41 points per curve computed by transportation-ai999x; marked: the worked example, t_f = 3.3 s, c = 363.101 1/h.

```
transportation_ai999x.solve('transport_gap_acceptance_capacity', q=0.2222222222222222, t_c=6.5,
t_f=3.3)
# -> c = 363.101 1/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_gap_acceptance_capacity', q=0.16666666666666666, t_c=5.5,
t_f=3.0, model='siegloch')
# -> c = 616.101 1/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gap acceptance capacity with major flow = 800 veh/h, critical gap = 6.5 s and follow up time = 3.3 s.	SOLVED	c = 363.101 1/h
Gap acceptance capacity with major flow = 600 veh/h, critical gap = 5.5 s, follow up time = 3 s and model = siegloch.	SOLVED	c = 616.101 1/h
Gap acceptance capacity with major flow = 800 veh/h and critical gap = 6.5 s.	NEEDS_INPUT	transport_gap_acceptance_capacity needs t_f (follow-up time). Example: solve('transport_gap_acceptance_capacity', q='0.2
Gap acceptance capacity with major flow = 800 veh/h, critical gap = 6.5 s and follow up time = 3.3 s and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Gap acceptance capacity with major flow = 800 veh/h, critical gap = -6.5 s and follow up time = 3.3 s.	INVALID_INPUT	Capacity of a minor stream by gap acceptance: t_c must be positive.

Time-mean and space-mean speed of spot speeds, `transport_time_and_space_mean_speed`

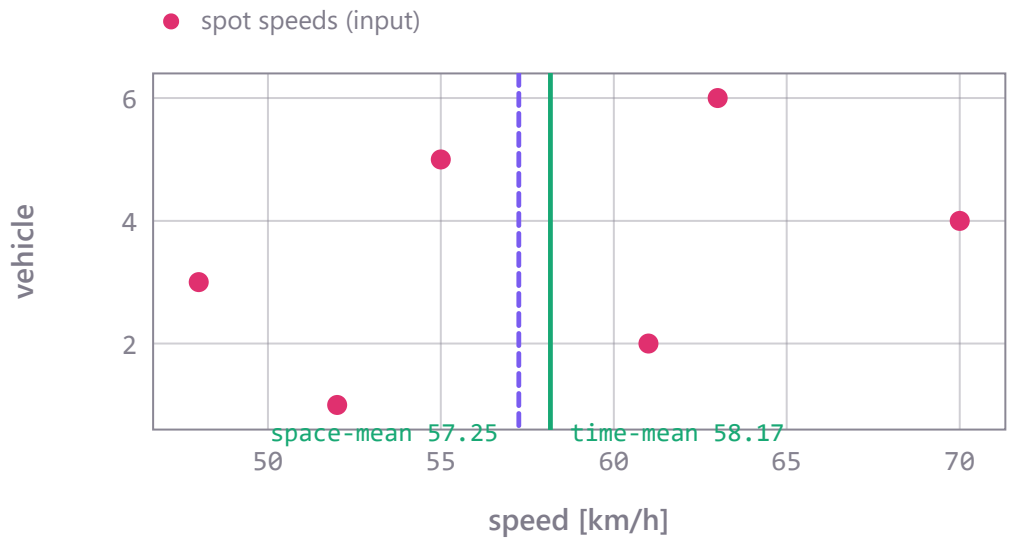
From spot speeds measured at a point: the time-mean speed (arithmetic mean), the space-mean speed (harmonic mean, the speed that matches travel times over a fixed length) and the sample standard deviation. Speeds in km/h.

$$\bar{v}_t = \frac{1}{n}\sum v_i, \quad \bar{v}_s = \frac{n}{\sum 1/v_i}$$

input	meaning	condition
`speeds`	spot speeds in km/h	

output	meaning
`TMS`	time-mean speed (km/h)
`SMS`	space-mean speed (km/h)
`s`	sample standard deviation (km/h)

- **Method:** TMS = sum v / n; SMS = n / sum(1/v); s = *sqrt*(sum (v - TMS)^2 / (n - 1))
- **Assumes:** at least two positive speeds
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Spot speeds of 6 vehicles ([52, 61, 48, 70, 55, 63] km/h). Marked: the returned time-mean speed 58.1667 km/h (arithmetic mean) and space-mean speed 57.2529 km/h (harmonic mean), standard deviation 8.03534 km/h.

```
transportation_ai999x.solve('transport_time_and_space_mean_speed', speeds=[52, 61, 48, 70, 55, 63])
# -> TMS = 58.1667 km/h, SMS = 57.2529 km/h, s = 8.03534 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_time_and_space_mean_speed', speeds=[88, 95, 102, 79, 110,
92, 85])
# -> TMS = 93 km/h, SMS = 92.0117 km/h, s = 10.4881 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Time mean and space mean speed with speeds = [52, 61, 48, 70, 55, 63].	SOLVED	TMS = 58.1667 km/h, SMS = 57.2529 km/h, s = 8.03534 km/h
Time mean and space mean speed with speeds = [88, 95, 102, 79, 110, 92, 85].	SOLVED	TMS = 93 km/h, SMS = 92.0117 km/h, s = 10.4881 km/h
Time mean and space mean speed with speeds = [52, 61, 48, 70, 55, 63] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Time mean and space mean speed with speeds = [52, -61, 48].	INVALID_INPUT	Time-mean and space-mean speed of spot speeds: every entry of speeds must be positive.

85th, 50th and 15th percentile spot speeds, `transport_spot_speed_percentiles`

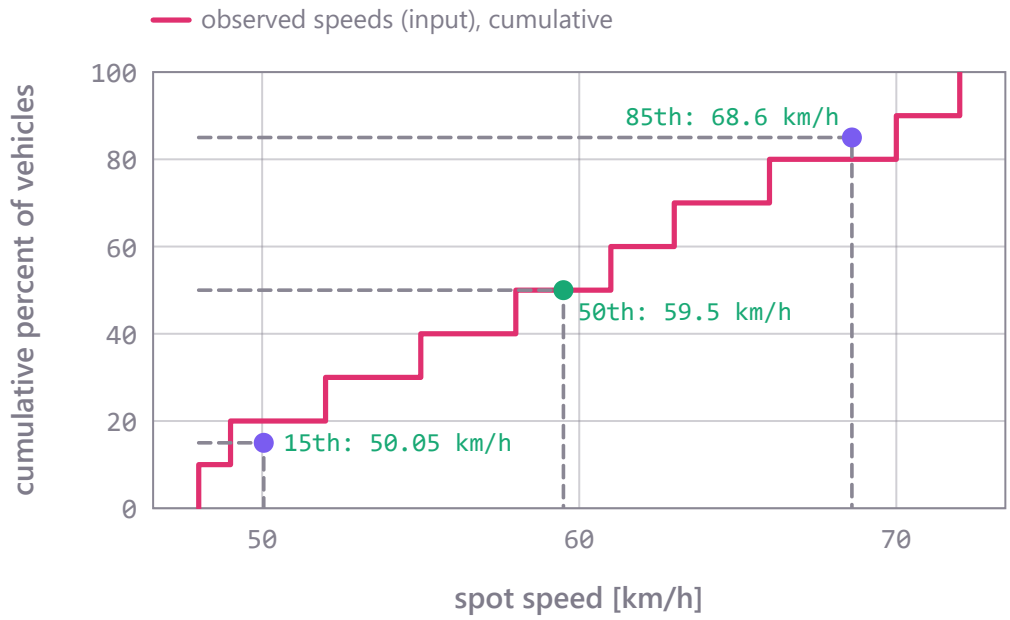
Percentile speeds of a spot-speed sample (85th percentile for speed limits and design, median, 15th) by linear interpolation between ranks (position $(n-1)\,p/100$, as Excel PERCENTILE.INC). Speeds in km/h.

$$v_p = v_{(i)} + f\,(v_{(i+1)} - v_{(i)}), \quad i + f = (n-1)\,p/100$$

input	meaning	condition
`speeds`	spot speeds in km/h	

output	meaning
`p85`	85th percentile speed (km/h)
`p50`	median speed (km/h)
`p15`	15th percentile speed (km/h)

- **Method:** sort; the p-th percentile at position $(n-1)\,p/100$ by linear interpolation
- **Assumes:** at least two positive speeds
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cumulative distribution of 10 spot speeds ([52, 61, 48, 70, 55, 63, 58, 66, 49, 72] km/h). Marked: the returned 15th, 50th and 85th percentile speeds 50.05, 59.5 and 68.6 km/h.

```
transportation_ai999x.solve('transport_spot_speed_percentiles', speeds=[52, 61, 48, 70, 55, 63, 58, 66, 49, 72])
# -> p85 = 68.6 km/h, p50 = 59.5 km/h, p15 = 50.05 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_spot_speed_percentiles', speeds=[88, 95, 102, 79, 110, 92, 85])
# -> p85 = 102.8 km/h, p50 = 92 km/h, p15 = 84.4 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Spot speed percentiles with speeds = [52, 61, 48, 70, 55, 63, 58, 66, 49, 72].	SOLVED	p85 = 68.6 km/h, p50 = 59.5 km/h, p15 = 50.05 km/h
Spot speed percentiles with speeds = [88, 95, 102, 79, 110, 92, 85].	SOLVED	p85 = 102.8 km/h, p50 = 92 km/h, p15 = 84.4 km/h
Spot speed percentiles with speeds = [52, 61, 48, 70, 55, 63, 58, 66, 49, 72] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Spot speed percentiles with speeds = [52, -61, 48].	INVALID_INPUT	85th, 50th and 15th percentile spot speeds: every entry of speeds must be positive.

Minimum safe following gap when the leader brakes, `transport_safe_following_gap`

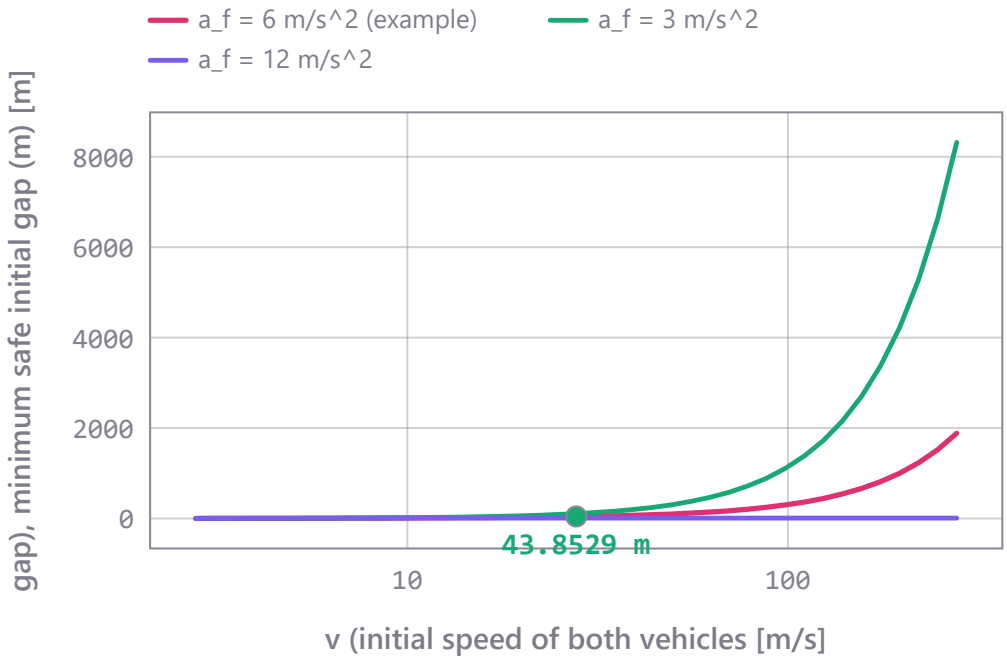
Smallest initial gap that avoids a rear-end collision when a leader brakes at a_l and the follower, at the same initial speed, reacts after t_r and brakes at a_f (the closest approach can happen while both still move).

$$g_{min} = \max_{t} \left[x_f(t) - x_l(t) \right]$$

input	meaning	condition
`v`	initial speed of both vehicles	
`t_r`	follower reaction time	
`a_l`	deceleration of the leader	
`a_f`	deceleration of the follower	

output	meaning
`gap`	minimum safe initial gap (m)

- **Method:** gap = max over time of (x_follower - x_leader): at the end, or when the speeds are equal
- **Assumes:** all inputs positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



gap (minimum safe initial gap (m)) against v (initial speed of both vehicles) from 2.77778 m/s to 277.778 m/s on a logarithmic axis. One curve for each value of a_f (deceleration of the follower): 6 m/s^2, 3 m/s^2, 12 m/s^2. Other inputs as in the worked example: t_r = 1.0 s, a_l = 8.0 m/s^2. 41 points per curve computed by transportation-ai999x; marked: the worked example, v = 27.7778 m/s, gap = 43.8529 m.

```
transportation_ai999x.solve('transport_safe_following_gap', v=27.77777777777778, t_r=1.0, a_l=8.0,
a_f=6.0)
# -> gap = 43.8529 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_safe_following_gap', v=22.22222222222222, t_r=1.2, a_l=5.0,
a_f=7.0)
# -> gap = 12.6 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Safe following gap with speed = 100 km/h, reaction time = 1 s, leader	SOLVED	gap = 43.8529 m

question	outcome	result
deceleration = 8 m/s^2 and follower deceleration = 6 m/s^2.		
Safe following gap with speed = 80 km/h, reaction time = 1.2 s, leader deceleration = 5 m/s^2 and follower deceleration = 7 m/s^2.	SOLVED	gap = 12.6 m
Safe following gap with speed = 100 km/h, reaction time = 1 s and leader deceleration = 8 m/s^2.	NEEDS_INPUT	transport_safe_following_gap needs a_f (deceleration of the follower). Example: solve('transport_safe_following_gap', v=
Safe following gap with speed = 100 km/h, reaction time = 1 s, leader deceleration = 8 m/s^2 and follower deceleration = 6 m/s^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Safe following gap with speed = 100 km/h, reaction time = 1 s, leader deceleration = -8 m/s^2 and follower deceleration = 6 m/s^2.	INVALID_INPUT	Minimum safe following gap when the leader brakes: a_l must be positive.

Single-server random queue (M/M/1), e.g. a toll booth, `transport_mm1_queue`

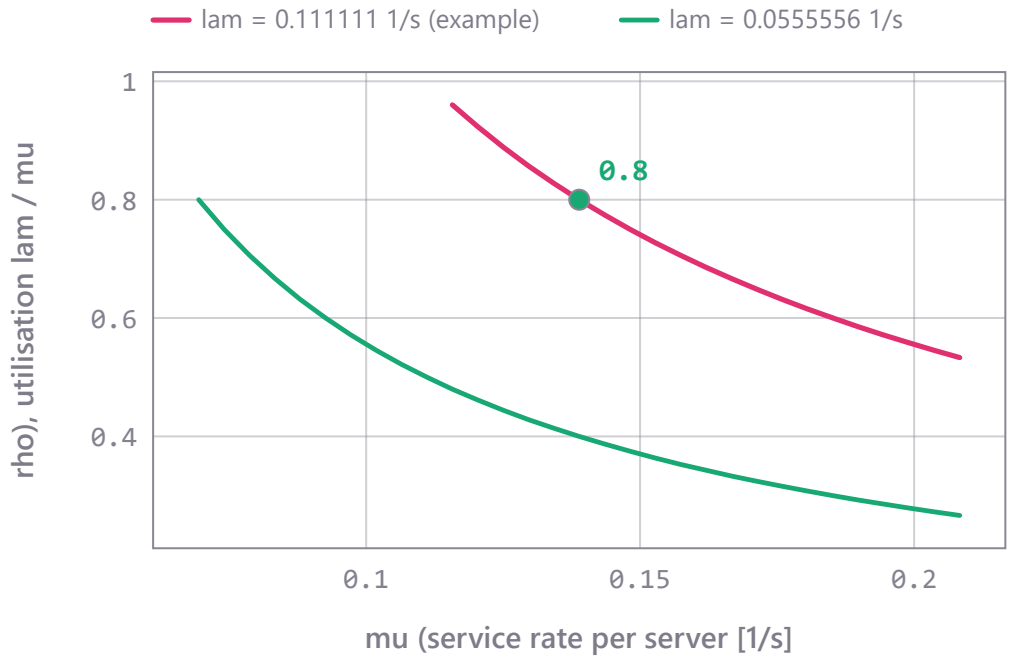
Poisson arrivals at lam and exponential service at mu at one server (a toll booth, a ramp meter): utilisation, mean number in the system and in the queue, mean time in the system and waiting, and the probability that the server is idle. Rates in veh/h.

$$L = \frac{\rho}{1-\rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\rho}{\mu - \lambda}$$

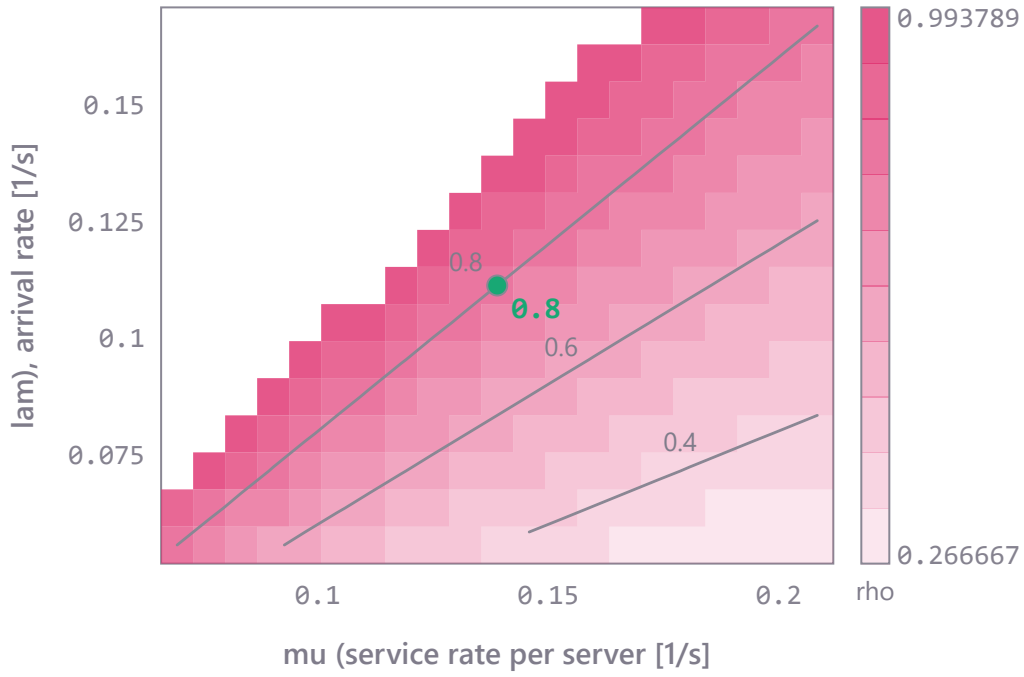
input	meaning	condition
`lam`	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`mu`	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	

output	meaning
`rho`	utilisation lam / mu
`L`	mean number in the system (veh)
`Lq`	mean number waiting (veh)
`W`	mean time in the system (s)
`Wq`	mean waiting time (s)
`P0`	probability the system is empty

- **Method:** rho = lam/mu, $L = \rho/(1-\rho)$, $Lq = \rho^2/(1-\rho)$, W = 1/(mu - lam), Wq = rho/(mu - lam)
- **Assumes:** lam < mu
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rho (utilisation lam / mu) against mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0694444 1/s to 0.208333 1/s. One curve for each value of lam (arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')): 0.111111 1/s, 0.0555556 1/s. 21 points per curve computed by transportation-ai999x; marked: the worked example, mu = 0.138889 1/s, rho = 0.8.



rho (utilisation lam / mu) over mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0694444 1/s to 0.208333 1/s and lam (arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0555556 1/s to 0.166667 1/s: each of the 21 × 15 cells is one run of transportation-ai999x, shaded by its value (light 0.266667 to dark 0.993789); iso-lines at rho = 0.4, 0.6, 0.8; blank cells: inputs the capability refuses. Marked: the worked example, mu = 0.138889 1/s, lam = 0.111111 1/s, rho = 0.8.

```
transportation_ai999x.solve('transport_mm1_queue', lam=0.1111111111111111, mu=0.1388888888888889)
# -> rho = 0.8, L = 4, Lq = 3.2, W = 36 s, Wq = 28.8 s, P0 = 0.2   (SOLVED)
```

```
transportation_ai999x.solve('transport_mm1_queue', lam=0.08333333333333333, mu=0.125)
# -> rho = 0.666667, L = 2, Lq = 1.33333, W = 24 s, Wq = 16 s, P0 = 0.333333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single server random queue with arrival rate = 400 veh/h and service rate = 500 veh/h.	SOLVED	rho = 0.8, L = 4, Lq = 3.2, W = 36 s, Wq = 28.8 s, P0 = 0.2
Single server random queue with arrival rate = 300 veh/h and service rate = 450 veh/h.	SOLVED	rho = 0.666667, L = 2, Lq = 1.33333, W = 24 s, Wq = 16 s, P0 = 0.333333
Single server random queue with arrival rate = 400 veh/h.	NEEDS_INPUT	transport_mm1_queue needs mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/
Single server random queue with arrival rate = 400 veh/h and service rate = 500 veh/h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Single server random queue with arrival rate = 600 veh/h and service rate = 500 veh/h.	INVALID_INPUT	Single-server random queue (M/M/1), e.g. a toll booth: the arrival rate must be below the service rate (rho < 1).

Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza, `transport_mmc_queue`

Poisson arrivals at lam served by c parallel exponential servers at mu each (toll lanes, loading bays, ticket windows): probability of waiting (Erlang C), mean queue, mean wait, mean time in the system and mean number in the system. Rates in veh/h.

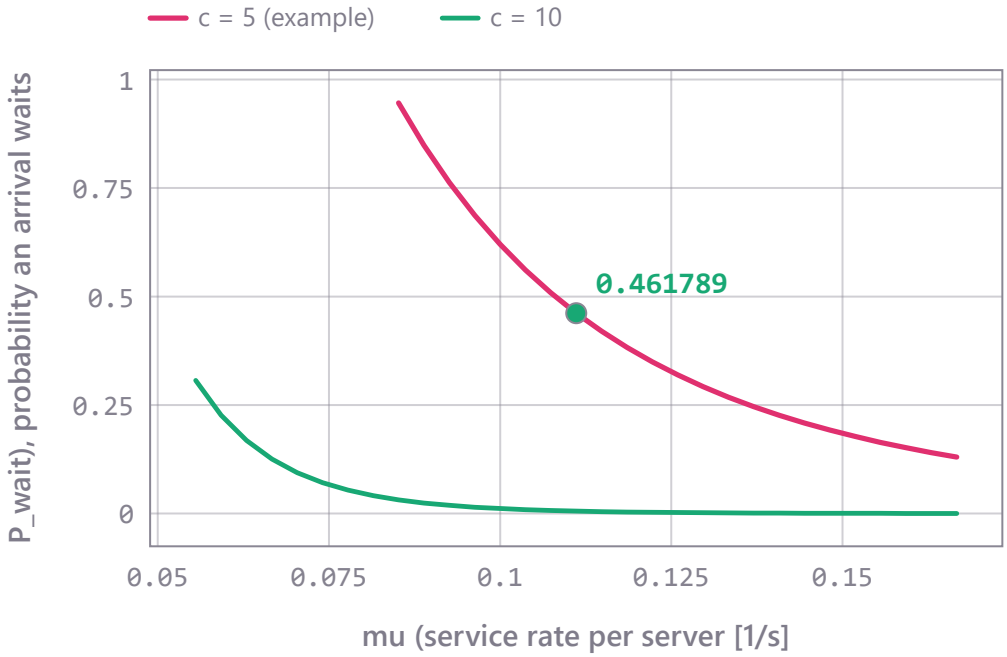
$$P_W = \frac{\frac{a^c}{c!}\frac{1}{1-\rho}}{\sum_{k=0}^{c-1}\frac{a^k}{k!} + \frac{a^c}{c!}\frac{1}{1-\rho}}, \quad L_q = P_W\frac{\rho}{1-\rho}$$

input	meaning	condition
`lam`	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`mu`	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`c`	number of servers (whole number)	

output	meaning
`P_wait`	probability an arrival waits (Erlang C)
`Lq`	mean number waiting (veh)
`Wq`	mean waiting time (s)
`W`	mean time in the system (s)
`L`	mean number in the system (veh)

- **Method:** Erlang C: P_wait = (a^c/c!)/(1 - rho) / (sum_{$k < c$} $a^k/k!$ + (a^c/c!)/$(1-\rho)$); Lq = P_wait $\rho/(1-\rho)$
- **Assumes:** lam < c mu; $1 \le c \le 200$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P_wait (probability an arrival waits (Erlang C)) against mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0555556 1/s to 0.166667 1/s. One curve for each value of c (number of servers (whole number)): 5, 10. Other inputs as in the worked example: lam = 0.4166666666666667 1/s. 23 points per curve computed by transportation-ai999x; marked: the worked example, mu = 0.111111 1/s, P_wait = 0.461789.

```
transportation_ai999x.solve('transport_mmc_queue', lam=0.4166666666666667, mu=0.1111111111111111,
c=5)
# -> P_wait = 0.461789, Lq = 1.38537, Wq = 3.32488 s, W = 12.3249 s, L = 5.13537   (SOLVED)
```

```
transportation_ai999x.solve('transport_mmc_queue', lam=0.25, mu=0.09722222222222222, c=3)
# -> P_wait = 0.742552, Lq = 4.45531, Wq = 17.8212 s, W = 28.107 s, L = 7.02674   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multi server random queue with arrival rate = 1500 veh/h, service rate = 400 veh/h and servers = 5.	SOLVED	P_wait = 0.461789, Lq = 1.38537, Wq = 3.32488 s, W = 12.3249 s, L = 5.13537
Multi server random queue with arrival rate = 900 veh/h, service rate = 350 veh/h and servers = 3.	SOLVED	P_wait = 0.742552, Lq = 4.45531, Wq = 17.8212 s, W = 28.107 s, L = 7.02674
Multi server random queue with arrival rate = 1500 veh/h and service rate = 400 veh/h.	NEEDS_INPUT	transport_mmc_queue needs c (number of servers (whole number)). Example: solve('transport_mmc_queue', lam='0.416667 1/s'
Multi server random queue with arrival rate = 1500 veh/h, service rate = 400 veh/h and servers = 5 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Multi server random queue with arrival rate = 2500 veh/h, service rate = 400 veh/h and servers = 5.	INVALID_INPUT	Multi-server random queue (M/M/c, Erlang C), e.g. a toll plaza: the arrival rate must be below c mu (rho < 1).

Single server with constant service time (M/D/1), `transport_md1_queue`

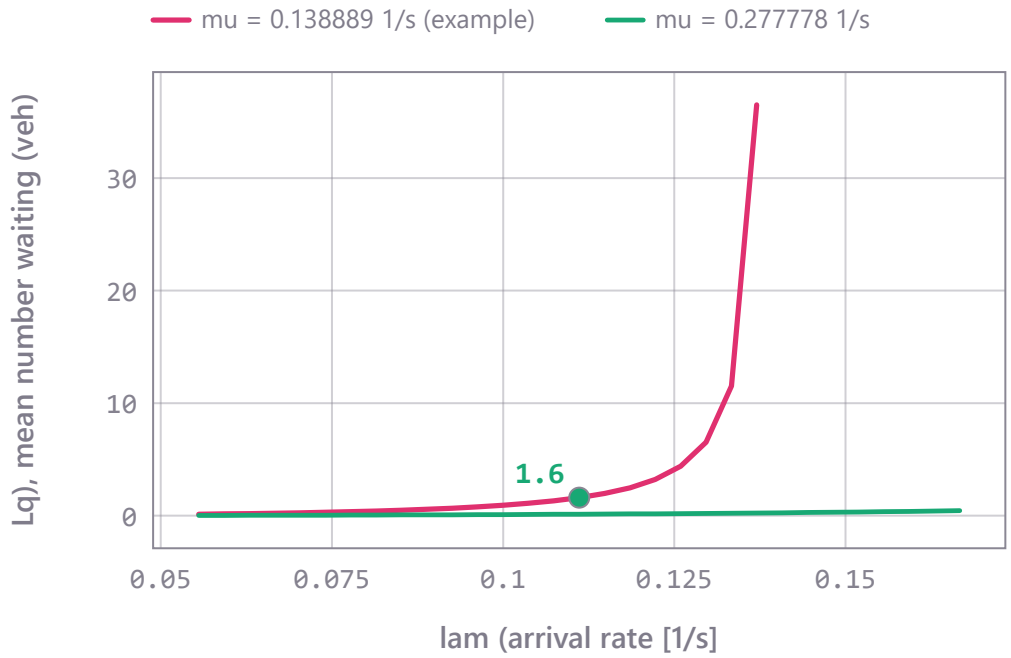
Poisson arrivals at lam and a constant service time $1/mu$ (an automated toll gate, a ramp meter): mean queue, mean wait, mean time in the system and mean number in the system. Rates in veh/h.

$$L_q = \frac{\rho^2}{2(1-\rho)}, \quad W_q = \frac{\rho}{2\mu(1-\rho)}$$

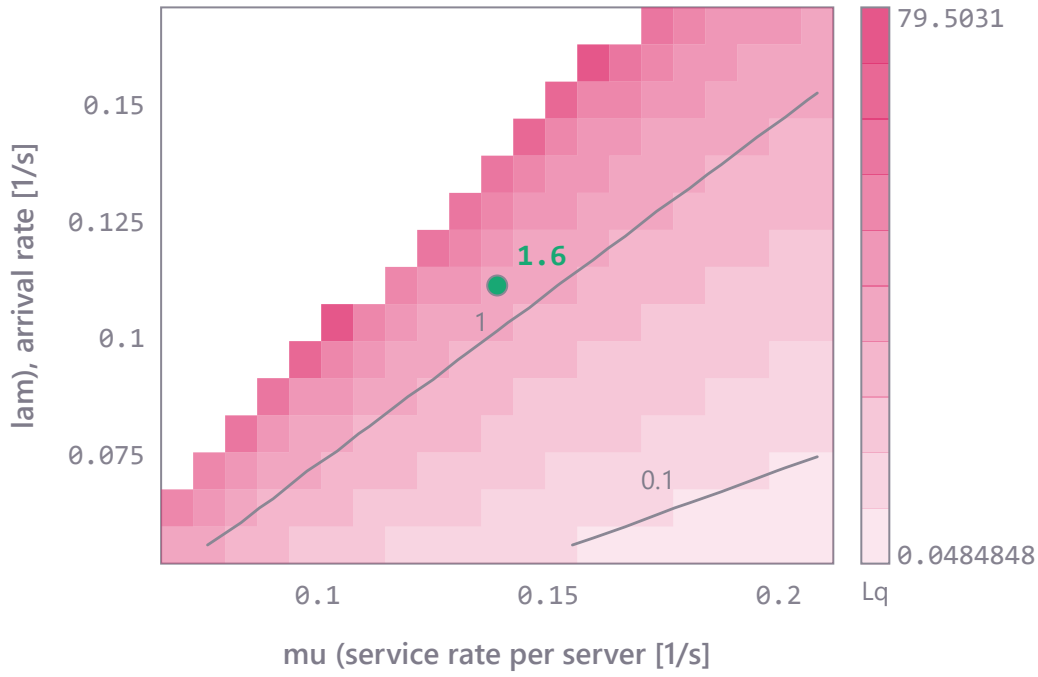
input	meaning	condition
`lam`	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`mu`	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	

output	meaning
`Lq`	mean number waiting (veh)
`Wq`	mean waiting time (s)
`W`	mean time in the system (s)
`L`	mean number in the system (veh)

- **Method:** Lq = rho^2 / (2(1 - rho)), Wq = rho / (2 mu $(1-\rho)$), W = (2 - rho)/(2 mu $(1-\rho)$)
- **Assumes:** lam < mu
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lq (mean number waiting (veh)) against lam (arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0555556 1/s to 0.166667 1/s. One curve for each value of mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')): 0.138889 1/s, 0.277778 1/s. 23 points per curve computed by transportation-ai999x; marked: the worked example, lam = 0.111111 1/s, Lq = 1.6.



Lq (mean number waiting (veh)) over mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0694444 1/s to 0.208333 1/s and lam (arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.0555556 1/s to 0.166667 1/s: each of the 21 × 15 cells is one run of transportation-ai999x, shaded by its value (light 0.0484848 to dark 79.5031, logarithmic shading); iso-lines at Lq = 0.1, 1; blank cells: inputs the capability refuses. Marked: the worked example, mu = 0.138889 1/s, lam = 0.111111 1/s, Lq = 1.6.

```
transportation_ai999x.solve('transport_md1_queue', lam=0.1111111111111111, mu=0.1388888888888889)
# -> Lq = 1.6, Wq = 14.4 s, W = 21.6 s, L = 2.4   (SOLVED)
```

```
transportation_ai999x.solve('transport_md1_queue', lam=0.3333333333333333, mu=0.5)
# -> Lq = 0.666667, Wq = 2 s, W = 4 s, L = 1.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Constant service time queue with arrival rate = 400 veh/h and service rate = 500 veh/h.	SOLVED	Lq = 1.6, Wq = 14.4 s, W = 21.6 s, L = 2.4
Constant service time queue with arrival rate = 1200 veh/h and service rate = 1800 veh/h.	SOLVED	Lq = 0.666667, Wq = 2 s, W = 4 s, L = 1.33333
Constant service time queue with arrival rate = 400 veh/h.	NEEDS_INPUT	transport_md1_queue needs mu (service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 1/
Constant service time queue with arrival rate = 400 veh/h and service rate = 500 veh/h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Constant service time queue with arrival rate = 600 veh/h and service rate = 500 veh/h.	INVALID_INPUT	Single server with constant service time (M/D/1): the arrival rate must be below the service rate (rho < 1).

Blocking probability with no waiting room (Erlang B), e.g. parking spaces,

`transport_erlang_b_blocking`

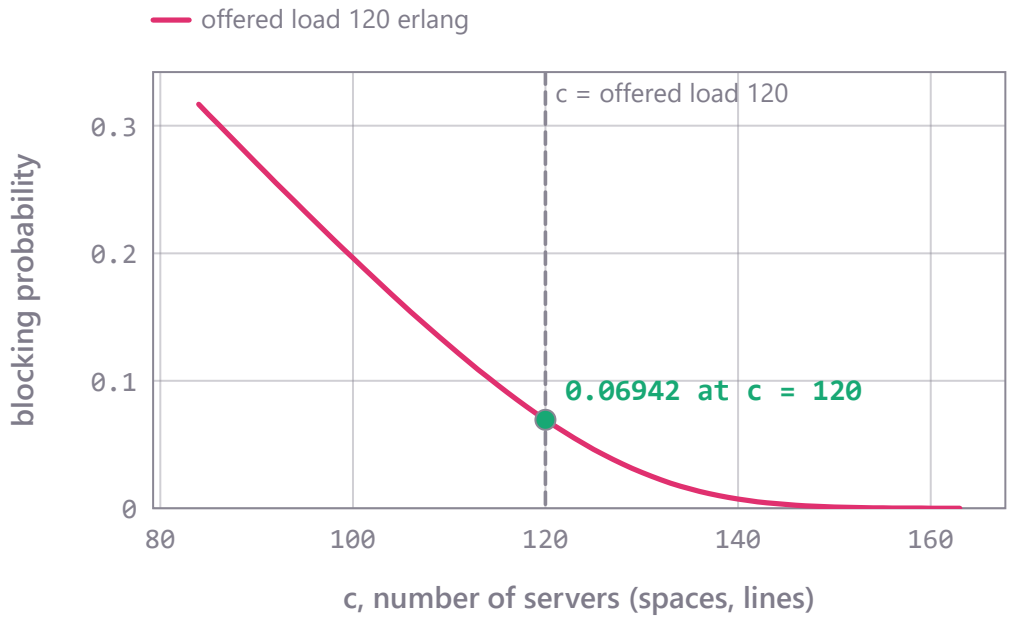
Arrivals at lam (veh/h) to c spaces or servers held for a mean time $1/mu$ with no waiting room (a car park, loading bays, taxi ranks): the probability an arrival is turned away (Erlang B), the mean number of occupied spaces and the vehicles lost per hour. Rates in veh/h.

$$B(c,a) = \frac{a^c/c!}{\sum_{k=0}^{c} a^k/k!}$$

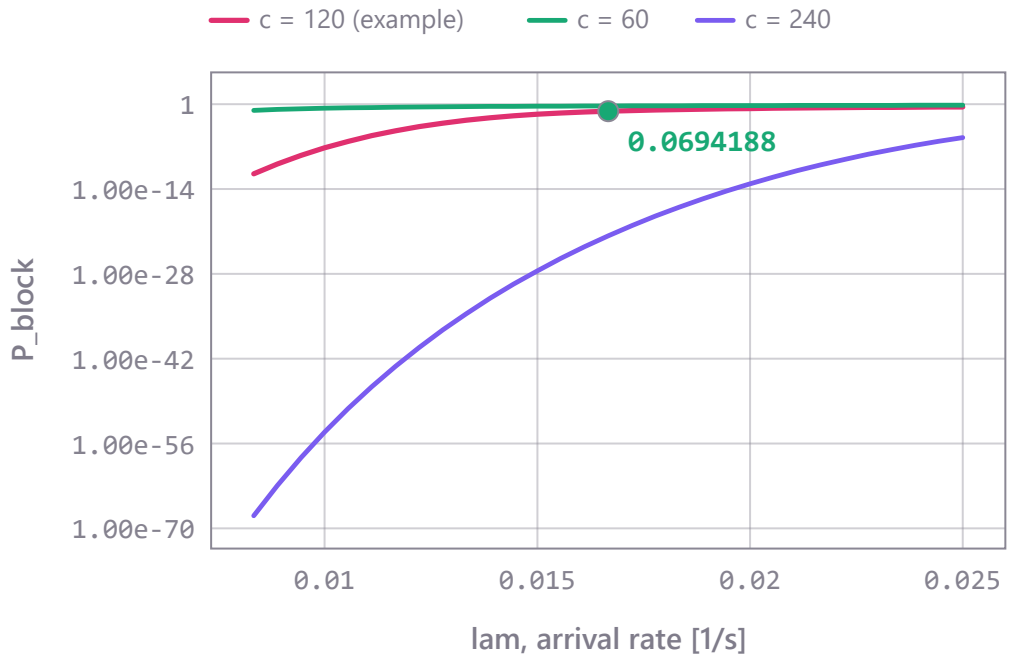
input	meaning	condition
`lam`	arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`mu`	service rate per server (veh/h in questions; veh/s in solve(), or a string such as '600 $1/h$')	
`c`	number of spaces or servers (whole number)	

output	meaning
`P_block`	probability an arrival is turned away
`carried`	mean number of occupied spaces
`lost_per_h`	vehicles turned away (veh/h)

- **Method:** Erlang B recursion B(k) = a B($k-1$)/(k + a B(k-1)), a = lam/mu
- **Assumes:** $1 \le c \le 1000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Erlang-B blocking probability against the number of servers for arrivals at λ = 0.0166667 /s and service rate μ = 0.000138889 /s (offered load λ/μ = 120 erlang): 80 server counts, each computed by the package. Marked: with c = 120 the returned blocking probability 0.0694188, carried load 111.67 erlang and 4.16513 arrivals lost per hour.



P_block (probability an arrival is turned away) against lam (arrival rate (veh/h in questions; veh/s in solve(), or a string such as '600 1/h')) from 0.00833333 1/s to 0.025 1/s, logarithmic output axis. One curve for each value of c (number of spaces or servers (whole number)): 120, 60, 240. Other inputs as in the worked example: mu = 0.0001388888888888889 1/s. 31 points per curve computed by transportation-ai999x; marked: the worked example, lam = 0.0166667 1/s, P_block = 0.0694188.

```
transportation_ai999x.solve('transport_erlang_b_blocking', lam=0.016666666666666666,
mu=0.0001388888888888889, c=120)
# -> P_block = 0.0694188, carried = 111.67, lost_per_h = 4.16513 1/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_erlang_b_blocking', lam=0.011111111111111112,
mu=0.0011111111111111111, c=12)
# -> P_block = 0.119739, carried = 8.80261, lost_per_h = 4.78957 1/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Erlang b blocking with arrival rate = 60 veh/h, service rate = 0.5 veh/h and spaces = 120.	SOLVED	P_block = 0.0694188, carried = 111.67, lost_per_h = 4.16513 1/h
Erlang b blocking with arrival rate = 40 veh/h, service rate = 4 veh/h and spaces = 12.	SOLVED	P_block = 0.119739, carried = 8.80261, lost_per_h = 4.78957 1/h
Erlang b blocking with arrival rate = 60 veh/h and service rate = 0.5 veh/h.	NEEDS_INPUT	transport_erlang_b_blocking needs c (number of spaces or servers (whole number)). Example: solve('transport_erlang_b_blo
Erlang b blocking with arrival rate = 60 veh/h, service rate = 0.5 veh/h and spaces = 120 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Erlang b blocking with arrival rate = 60 veh/h, service rate = -0.5 veh/h and spaces = 120.	INVALID_INPUT	Blocking probability with no waiting room (Erlang B), e.g. parking spaces: mu must be positive.

First-come-first-served queue from arrival and service times, `transport_fifo_queue_simulation`

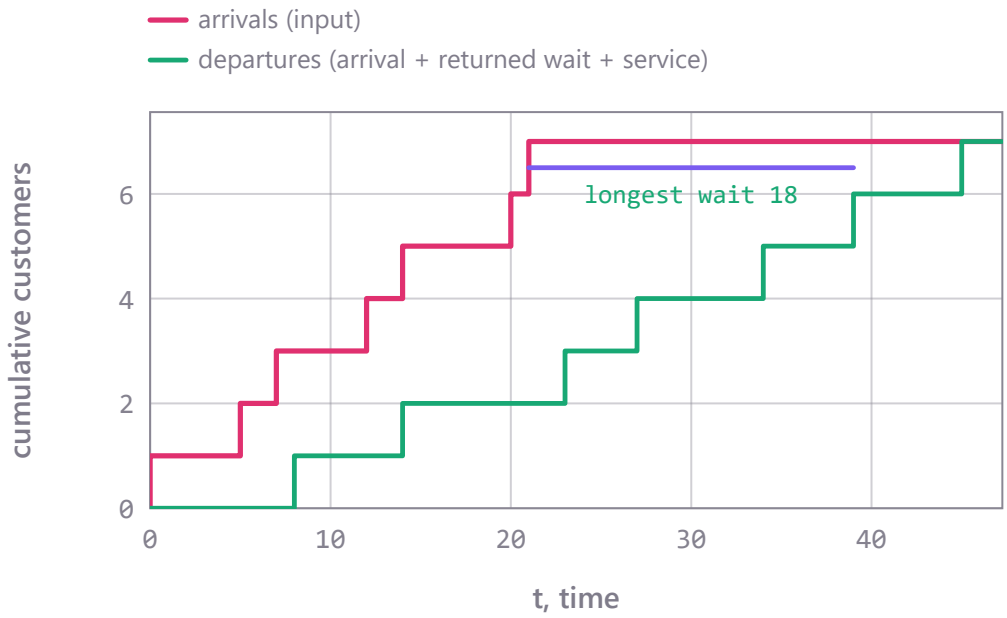
Waiting time of every vehicle (or passenger) at c identical servers in first-come-first-served order from given arrival times and service times (a toll plaza, a bus stop with c berths): mean and longest wait, mean time in the system, when the last one leaves and the longest waiting line. Times in s.

$$S_n = \max(A_n,\ \min_j F_j), \quad W_n = S_n - A_n$$

input	meaning	condition
`arrivals`	arrival times in s, increasing	
`service`	service times in s (one per arrival, or one for all)	
`c`	number of servers (default 1)	

output	meaning
`waits`	waiting time of each arrival (s)
`mean_wait`	mean waiting time (s)
`max_wait`	longest wait (s)
`mean_time_in_system`	mean time in the system (s)
`last_departure`	time the last one leaves (s)
`max_queue`	longest waiting line

- **Method:** each arrival starts at max(arrival, earliest free server) (Kiefer-Wolfowitz recursion)
- **Assumes:** arrival times increasing; service times > 0; $c \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A first-in-first-out queue: arrivals at [0, 5, 7, 12, 14, 20, 21] with service times [8, 6, 9, 4, 7, 5, 6]; departure times are arrival + the returned wait + service. Marked: the longest wait 18; also returned: mean wait 9.42857, mean time in the system 15.8571, last departure 45, longest queue 4.

```
transportation_ai999x.solve('transport_fifo_queue_simulation', arrivals=[0, 5, 7, 12, 14, 20, 21],
service=[8, 6, 9, 4, 7, 5, 6])
# -> waits = [0.0, 3.0, 7.0, 11.0, 13.0, 14.0, 18.0], mean_wait = 9.42857 s, max_wait = 18 s,
mean_time_in_system = 15.8571 s, last_departure = 45 s, max_queue = 4   (SOLVED)
```

```
transportation_ai999x.solve('transport_fifo_queue_simulation', arrivals=[0, 2, 3, 5, 9, 10, 11,
15], service=[12], c=2)
# -> waits = [0.0, 0.0, 9.0, 9.0, 15.0, 16.0, 25.0, 23.0], mean_wait = 12.125 s, max_wait = 25 s,
mean_time_in_system = 24.125 s, last_departure = 50 s, max_queue = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fifo queue simulation with arrivals = [0, 5, 7, 12, 14, 20, 21] and service = [8, 6, 9, 4, 7, 5, 6].	SOLVED	waits = [0.0, 3.0, 7.0, 11.0, 13.0, 14.0, 18.0], mean_wait = 9.42857 s, max_wait = 18 s, mean_time_in_system = 15.8571 s, last_departure = 45 s, max_queue = 4
Fifo queue simulation with arrivals = [0, 2, 3, 5, 9, 10, 11, 15], service = [12] and servers = 2.	SOLVED	waits = [0.0, 0.0, 9.0, 9.0, 15.0, 16.0, 25.0, 23.0], mean_wait = 12.125 s, max_wait = 25 s, mean_time_in_system = 24.125 s, last_departure = 50 s, max_queue = 5
Fifo queue simulation with arrivals = [0, 5, 7, 12, 14, 20, 21].	NEEDS_INPUT	transport_fifo_queue_simulation needs service (service times in s (one per arrival, or one for all)). Example: solve('tr
Fifo queue simulation with arrivals = [0, 5, 7, 12, 14, 20, 21] and service = [8, 6, 9, 4, 7, 5, 6] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Fifo queue simulation with arrivals = [5, 0, 7] and service = [8, 6, 9].	INVALID_INPUT	First-come-first-served queue from arrival and service times: arrival times must be in increasing order.

Running time between stations (accelerate, cruise, brake), `transport_train_running_time`

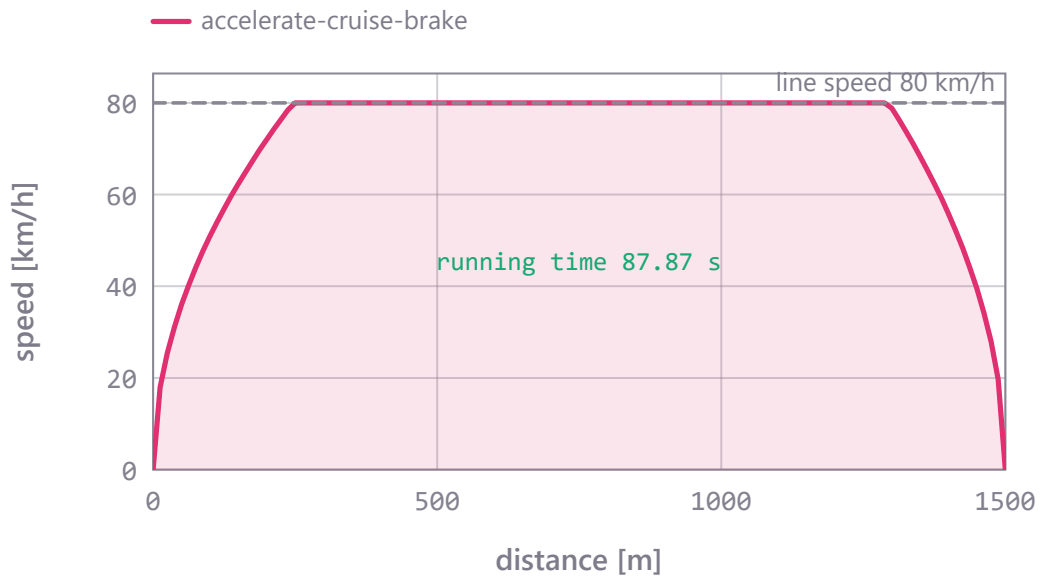
Minimum running time of a train (or bus) between two stops a distance D apart with acceleration a, braking rate b and line speed v_max; when the stops are too close to reach v_max the profile is accelerate-brake.

$$T = \frac{D}{v} + \frac{v}{2a} + \frac{v}{2b}$$

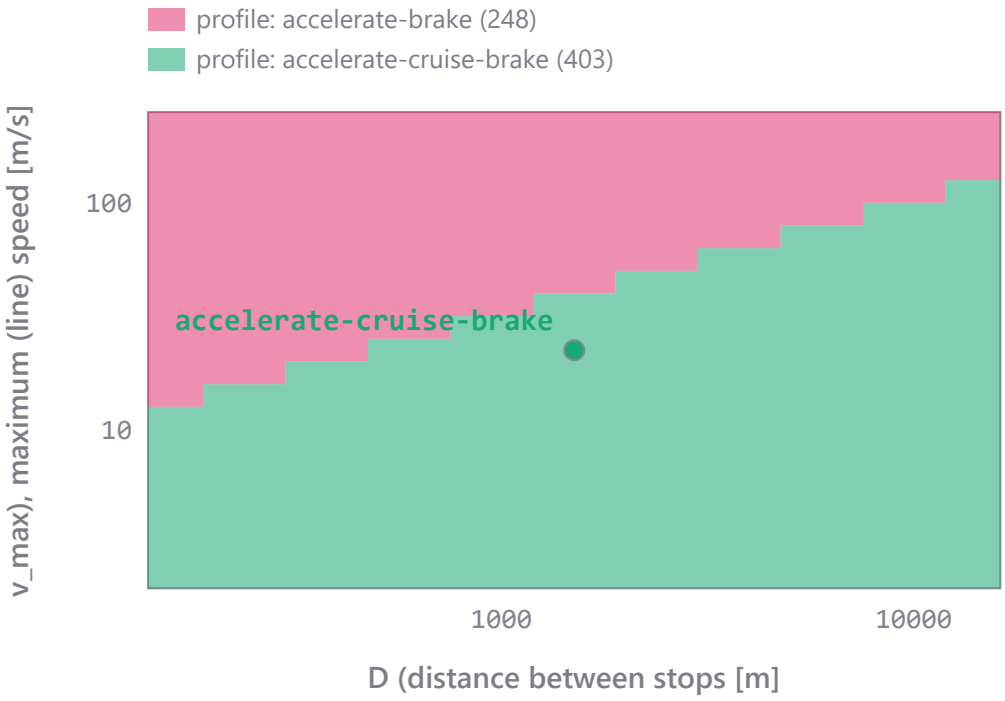
input	meaning	condition
`D`	distance between stops	
`v_max`	maximum (line) speed	
`a`	acceleration	
`b`	braking rate	

output	meaning
`T`	running time stop to stop (s)
`v_peak`	highest speed reached (m/s)
`profile`	accelerate-cruise-brake or accelerate-brake

- **Method:** if $v^2/(2a) + v^2/(2b) \le D$: $T = D/v + v/(2a) + v/(2b)$; else $v_p = sqrt$(2 D a b / (a + b)), $T = v_p/a + v_p/b$
- **Assumes:** all inputs positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Speed–distance profile between stations 1500 m apart (line speed 80 km/h, acceleration 1 m/s², braking 1.2 m/s²), drawn from these inputs with the returned peak speed 80 km/h; the running time it implies agrees with the returned T = 87.8704 s (accelerate-cruise-brake).



accelerate-cruise-brake or accelerate-brake (profile) over D (distance between stops) from 150 m to 15000 m and v_max (maximum (line) speed) from 2.22222 m/s to 222.222 m/s (logarithmic axes): each of the 31 × 21 cells is one run of transportation-ai999x, coloured by the verdict it returned, accelerate-brake: 248 cells, accelerate-cruise-brake: 403 cells. Other inputs as in the worked example: a = 1.0 m/s^2, b = 1.2 m/s^2. Marked: the worked example, D = 1500 m, v_max = 22.2222 m/s → accelerate-cruise-brake.

```
transportation_ai999x.solve('transport_train_running_time', D=1500.0, v_max=22.22222222222222,
a=1.0, b=1.2)
# -> T = 87.8704 s, v_peak = 22.2222 m/s, profile = accelerate-cruise-brake   (SOLVED)
```

```
transportation_ai999x.solve('transport_train_running_time', D=600.0, v_max=27.77777777777778,
a=0.8, b=1.0)
# -> T = 51.9615 s, v_peak = 23.094 m/s, profile = accelerate-brake   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Train running time with distance = 1.5 km, line speed = 80 km/h, acceleration = 1 m/s^2 and braking rate = 1.2 m/s^2.	SOLVED	T = 87.8704 s, v_peak = 22.2222 m/s, profile = accelerate-cruise-brake
Train running time with distance = 0.6 km, line speed = 100 km/h, acceleration = 0.8 m/s^2 and braking rate = 1 m/s^2.	SOLVED	T = 51.9615 s, v_peak = 23.094 m/s, profile = accelerate-brake
Train running time with distance = 1.5 km, line speed = 80 km/h and acceleration = 1 m/s^2.	NEEDS_INPUT	transport_train_running_time needs b (braking rate). Example: solve('transport_train_running_time', D='1500 m', v_max='2
Train running time with distance = 1.5 km, line speed = 80 km/h, acceleration = 1 m/s^2 and braking rate = 1.2 m/s^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Train running time with distance = -1.5 km, line speed = 80 km/h, acceleration = 1 m/s^2 and braking rate = 1.2 m/s^2.	INVALID_INPUT	Running time between stations (accelerate, cruise, brake): D must be positive.

Minimum headway of trains at a station stop (moving block), `transport_rail_station_headway`

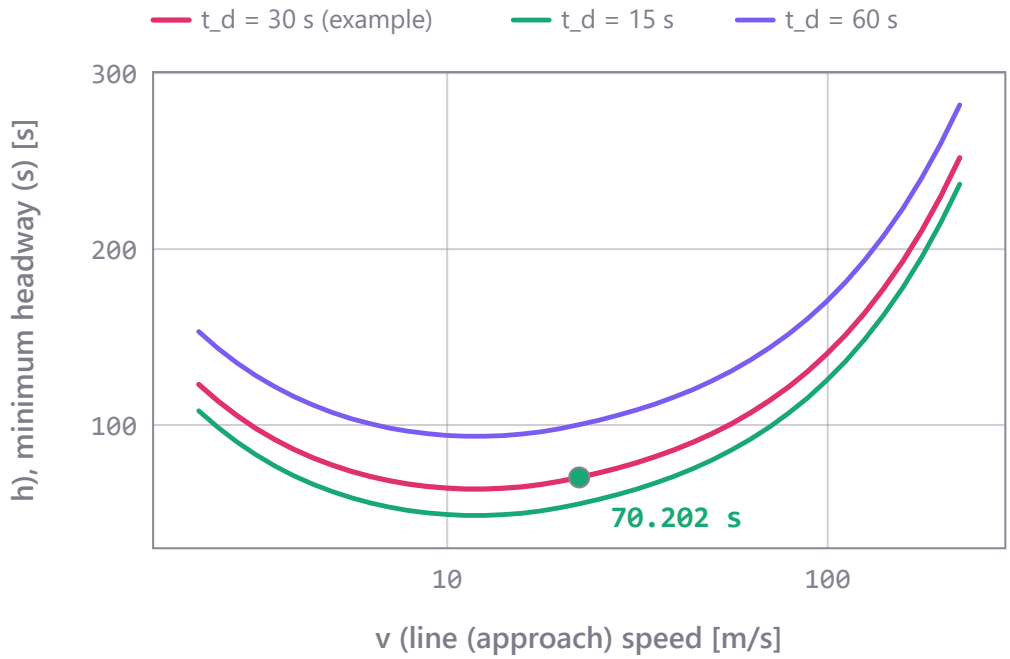
Shortest headway between trains serving a station under a brick-wall (moving-block) separation: the leader dwells t_d, accelerates at a until its rear clears the platform plus the safety margin s_m, while the follower approaches at the line speed v and brakes at b to stop at the platform; and the corresponding line capacity in trains per hour.

$$h = t_d + \sqrt{\frac{2(L_t + s_m)}{a}} + \frac{v}{b}$$

input	meaning	condition
`v`	line (approach) speed	
`a`	acceleration	
`b`	braking rate	
`L_t`	train length	
`t_d`	dwell time	
`s_m`	safety margin (default 0)	

output	meaning
`h`	minimum headway (s)
`trains_per_h`	line capacity (trains/h)

- **Method:** h = t_d + t_clear + v/b, t_clear = sqrt(2 (L + s)/a) (or $v/(2a) + (L + s)/v$ once the line speed is reached)
- **Assumes:** v, a, b, $L_t > 0$; t_d, $s_m \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



h (minimum headway (s)) against v (line (approach) speed) from 2.22222 m/s to 222.222 m/s on a logarithmic axis. One curve for each value of t_d (dwell time): 30 s, 15 s, 60 s. Other inputs as in the worked example: a = 1.0 m/s^2, b = 1.1 m/s^2, L_t = 150.0 m, s_m = 50.0 m. 41 points per curve computed by transportation-ai999x; marked: the worked example, v = 22.2222 m/s, h = 70.202 s.

```
transportation_ai999x.solve('transport_rail_station_headway', v=22.22222222222222, a=1.0, b=1.1,
L_t=150.0, t_d=30.0, s_m=50.0)
# -> h = 70.202 s, trains_per_h = 51.2806 1/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_rail_station_headway', v=16.666666666666668, a=1.2, b=1.0,
L_t=75.0, t_d=20.0)
# -> h = 47.847 s, trains_per_h = 75.2398 1/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Station headway with line speed = 80 km/h, acceleration = 1 m/s^2, braking rate = 1.1 m/s^2, train length = 150 m, dwell time = 30 s and safety margin = 50 m.	SOLVED	h = 70.202 s, trains_per_h = 51.2806 1/h
Station headway with line speed = 60 km/h, acceleration = 1.2 m/s^2, braking rate = 1 m/s^2, train length = 75 m and dwell time = 20 s.	SOLVED	h = 47.847 s, trains_per_h = 75.2398 1/h
Station headway with line speed = 80 km/h, acceleration = 1 m/s^2, braking rate = 1.1 m/s^2, train length = 150 m and safety margin = 50 m.	NEEDS_INPUT	transport_rail_station_headway needs t_d (dwell time). Example: solve('transport_rail_station_headway', v='22.2222 m/s',
Station headway with line speed = 80 km/h, acceleration = 1 m/s^2, braking rate = 1.1 m/s^2, train length = 150 m, dwell time = 30 s and safety margin = 50 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Station headway with line speed = 80 km/h, acceleration = -1 m/s^2, braking rate = 1.1 m/s^2, train length = 150 m and dwell time = 30 s.	INVALID_INPUT	Minimum headway of trains at a station stop (moving block): a must be positive.

Train braking distance with Davis resistance and gradient, `transport_train_braking_distance`

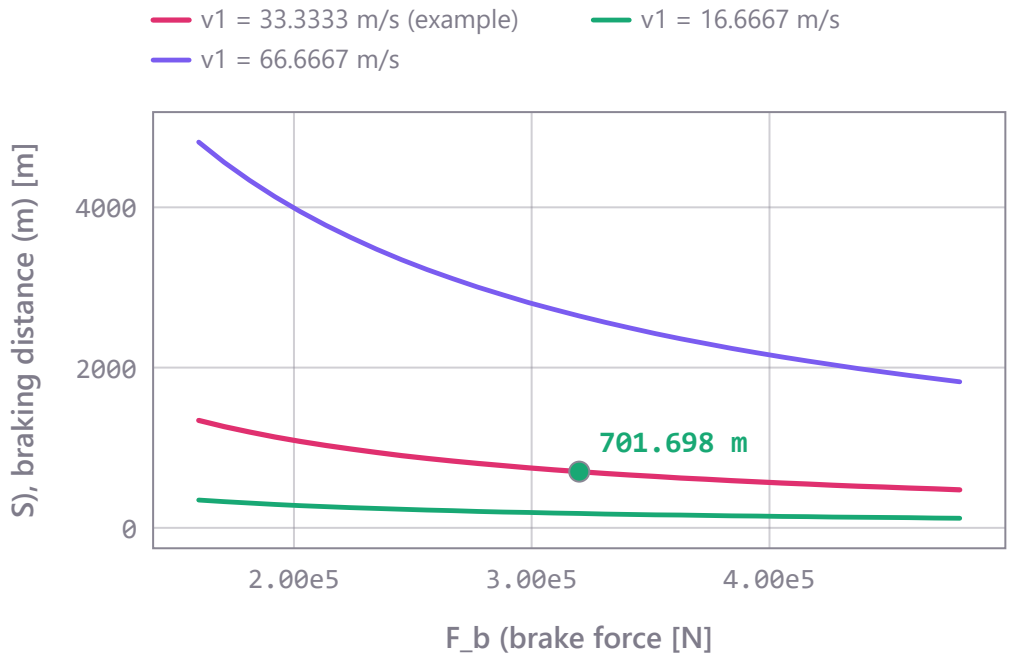
Braking distance and time of a train of mass m braking with a constant brake force F_b from v1 to v2, helped by the Davis train resistance $R = A + B$ v + C v^2 and the gradient (positive uphill), with a rotating-mass factor gamma.

$$S = \int_{v_2}^{v_1} \frac{\gamma m \, v \, dv}{F_b + A + Bv + Cv^2 + mg\sin\theta}$$

input	meaning	condition
`m`	train mass	
`v1`	initial speed	
`F_b`	brake force	
`A`	Davis coefficient A (constant resistance)	
`B`	Davis coefficient B (per speed)	
`C`	Davis coefficient C (per speed squared)	
`G`	grade, decimal (positive uphill)	
`gamma`	rotating-mass factor (default 1.06)	
`v2`	final speed (default 0)	

output	meaning
`S`	braking distance (m)
`t`	braking time (s)

- **Method:** S = integral from v2 to v1 of gamma m v dv / (F_b + A + B v + C $v^2 + m$ g sin(theta)) (quadrature)
- **Assumes:** F_b + R(v) + grade force > 0 over the speed range; $v_2 < v_1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



S (braking distance (m)) against F_b (brake force) from 160000 N to 480000 N. One curve for each value of v1 (initial speed): 33.3333 m/s, 16.6667 m/s, 66.6667 m/s. Other inputs as in the worked example: m = 400000.0 kg, A = 6000.0 N, B = 180.0 N*s/m, C = 10.368 N*s^2/m^2. 31 points per curve computed by transportation-ai999x; marked: the worked example, F_b = 320000 N, S = 701.698 m.

```
transportation_ai999x.solve('transport_train_braking_distance', m=400000.0, v1=33.333333333333336,
F_b=320000.0, A=6000.0, B=180.0, C=10.368)
# -> S = 701.698 m, t = 42.4729 s   (SOLVED)
```

```
transportation_ai999x.solve('transport_train_braking_distance', m=1200000.0, v1=25.0, F_b=600000.0,
A=14000.0, B=360.0, C=15.551999999999998, G=-0.008, v2=8.333333333333334)
# -> S = 664.542 m, t = 39.9643 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Train braking distance with mass = 400 t, initial speed = 120 km/h, brake force = 320 kN, davis a = 6 kN, davis b = 0.05 kN/(km/h) and davis c = 0.0008 kN/(km/h)^2.	SOLVED	S = 701.698 m, t = 42.4729 s
Train braking distance with mass = 1200 t, initial speed = 90 km/h, brake force = 600 kN, davis a = 14 kN, davis b = 0.1 kN/(km/h), davis c = 0.0012 kN/(km/h)^2, grade = -8 ‰ and final speed = 30 km/h.	SOLVED	S = 664.542 m, t = 39.9643 s
Train braking distance with mass = 400 t, initial speed = 120 km/h, brake force = 320 kN, davis a = 6 kN and davis b = 0.05 kN/(km/h).	NEEDS_INPUT	transport_train_braking_distance needs C (Davis coefficient C (per speed squared)). Example: solve('transport_train_brak
Train braking distance with mass = 400 t, initial speed = 120 km/h, brake force = 320 kN, davis a = 6 kN, davis b = 0.05 kN/(km/h) and davis c = 0.0008 kN/(km/h)^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Train braking distance with mass = 400 t, initial speed = 120 km/h, brake force = -320 kN, davis a = 6 kN, davis b = 0.05 kN/(km/h) and davis c = 0.0008 kN/(km/h)^2.	INVALID_INPUT	Train braking distance with Davis resistance and gradient: F_b must be positive.

Train acceleration time and distance (tractive effort curve, Davis resistance),

`transport_train_acceleration`

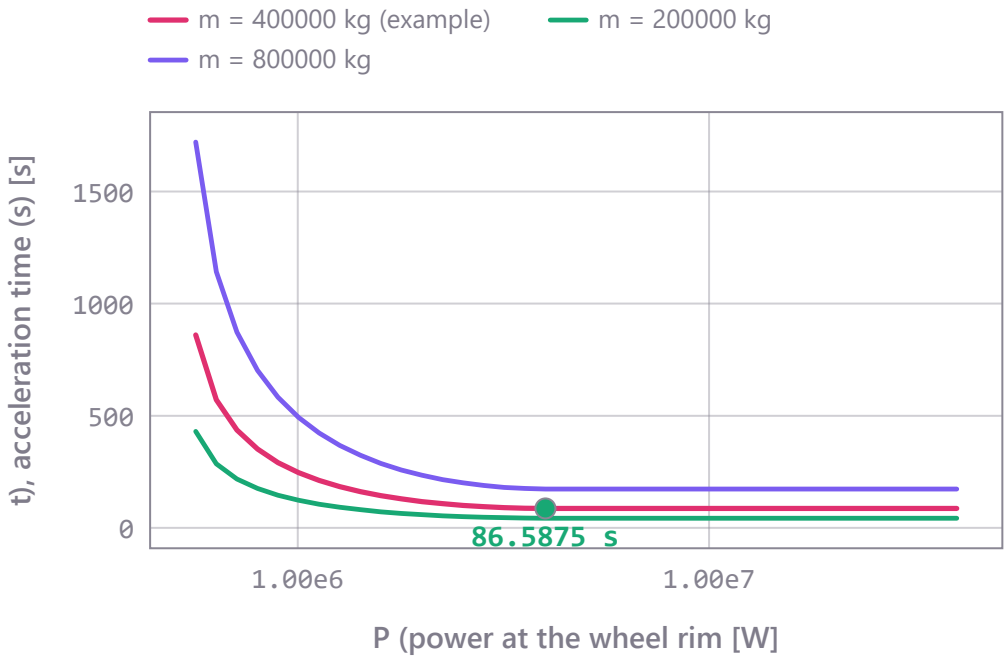
Time and distance for a train to accelerate from v1 to v2 with a tractive effort F_max up to the speed P / F_max and constant power P above it, against the Davis resistance A + B v + C v^2 and the gradient, with a rotating-mass factor gamma.

$$t = \int_{v_1}^{v_2} \frac{\gamma m \, dv}{\min(F_{max}, P/v) - (A + Bv + Cv^2) - mg\sin\theta}$$

input	meaning	condition
`m`	train mass	
`v1`	initial speed	
`v2`	final speed	
`F_max`	maximum (starting) tractive effort	
`P`	power at the wheel rim	
`A`	Davis coefficient A (constant resistance)	
`B`	Davis coefficient B (per speed)	
`C`	Davis coefficient C (per speed squared)	
`G`	grade, decimal (positive uphill)	
`gamma`	rotating-mass factor (default 1.08)	

output	meaning
`t`	acceleration time (s)
`d`	distance travelled (m)

- **Method:** t = integral of gamma m dv / (F(v) - R(v) - m g sin(theta)), F(v) = min(F_max, P/v) (quadrature)
- **Assumes:** tractive effort exceeds resistance up to v2; $v_2 > v_1 \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



t (acceleration time (s)) against P (power at the wheel rim) from 400000 W to 4.0000e+07 W on a logarithmic axis. One curve for each value of m (train mass): 400000 kg, 200000 kg, 800000 kg. Other inputs as in the worked example: v1 = 0.0 m/s, v2 = 27.77777777777778 m/s, F_max = 150000.0 N, A = 6000.0 N, B = 180.0 N*s/m, C = 10.368 N*s^2/m^2. 38 points per curve computed by transportation-ai999x; marked: the worked example, P = 4e+06 W, t = 86.5875 s.

```
transportation_ai999x.solve('transport_train_acceleration', m=400000.0, v1=0.0,
v2=27.77777777777778, F_max=150000.0, P=4000000.0, A=6000.0, B=180.0, C=10.368)
# -> t = 86.5875 s, d = 1222.69 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_train_acceleration', m=1500000.0, v1=5.555555555555555,
v2=22.22222222222222, F_max=400000.0, P=6000000.0, A=18000.0, B=432.0, C=15.551999999999998,
G=0.005)
# -> t = 105.136 s, d = 1555.69 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Train acceleration with mass = 400 t, initial speed = 0 km/h, final speed = 100 km/h, tractive effort = 150 kN, power = 4000 kW, davis a = 6 kN, davis b = 0.05 kN/(km/h) and davis c = 0.0008 kN/(km/h)^2.	SOLVED	t = 86.5875 s, d = 1222.69 m
Train acceleration with mass = 1500 t, initial speed = 20 km/h, final speed = 80 km/h, tractive effort = 400 kN, power = 6000 kW, davis a = 18 kN, davis b = 0.12 kN/(km/h), davis c = 0.0012 kN/(km/h)^2 and grade = 5 ‰.	SOLVED	t = 105.136 s, d = 1555.69 m
Train acceleration with mass = 400 t, initial speed = 0 km/h, final speed = 100 km/h, tractive effort = 150 kN, power = 4000 kW, davis a = 6 kN and davis b = 0.05 kN/(km/h).	NEEDS_INPUT	transport_train_acceleration needs C (Davis coefficient C (per speed squared)). Example: solve('transport_train_accelera
Train acceleration with mass = 400 t, initial speed = 0 km/h, final speed = 100 km/h,	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every

question	outcome	result
tractive effort = 150 kN, power = 4000 kW, davis a = 6 kN, davis b = 0.05 kN/(km/h) and davis c = 0.0008 kN/(km/h)^2 and colour = 3.		form: transportation_ai999x.templates()
Train acceleration with mass = 400 t, initial speed = 100 km/h, final speed = 0 km/h, tractive effort = 150 kN, power = 4000 kW, davis a = 6 kN, davis b = 0.05 kN/(km/h) and davis c = 0.0008 kN/(km/h)^2.	INVALID_INPUT	Train acceleration time and distance (tractive effort curve, Davis resistance): v2 must be positive.

Fastest route on a road network (link lengths and speeds), `transport_fastest_route`

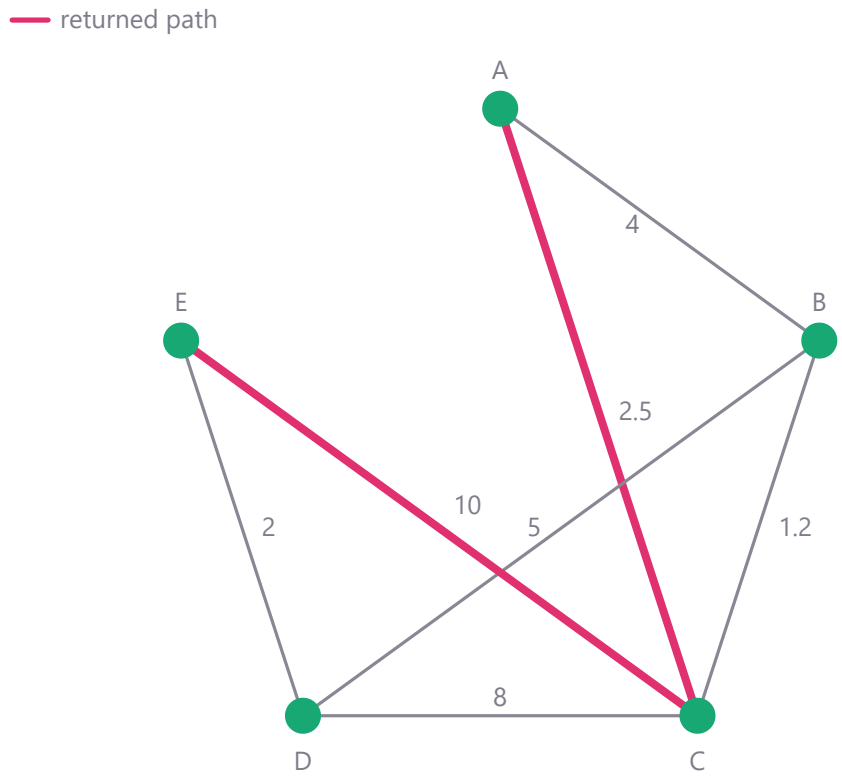
The quickest route between two places on a road network whose links are given by length (km) and speed (km/h): the route, its travel time in minutes and its length in km. Roads are two-way unless directed = yes.

$$t_{link} = \frac{60\,L}{v}, \quad \min \sum t_{link}$$

input	meaning	condition
`roads`	roads as 'A-B length' with lengths in km	
`speeds`	speed on each road in km/h, in the same order	
`origin`	starting place	
`destination`	destination	
`directed`	one-way links as written (default no)	

output	meaning
`route`	fastest route (places joined by '-')
`time`	travel time (min)
`distance`	route length (km)

- **Method:** Dijkstra's algorithm on link travel times 60 L / v (min)
- **Assumes:** one positive length and speed per road; the destination is reachable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network roads (roads as 'A-B length' with lengths in km) of the worked example (5 nodes, 7 edges, laid out on a circle); on each edge its stated weight; highlighted: the returned path A → C → E. Returned: time = 9.66667 min, distance = 12.5 km. Other inputs: speeds = [60, 50, 40, 80, 100, 90, 50], origin = A, destination = E.

```
transportation_ai999x.solve('transport_fastest_route', roads='A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2', speeds=[60, 50, 40, 80, 100, 90, 50], origin='A', destination='E')
# -> route = A-C-E, time = 9.66667 min, distance = 12.5 km   (SOLVED)
```

```
transportation_ai999x.solve('transport_fastest_route', roads='P-Q 3, Q-R 4, P-R 9, R-S 2, Q-S 7', speeds=[30, 60, 100, 50, 40], origin='P', destination='S', directed='yes')
# -> route = P-R-S, time = 7.8 min, distance = 11 km   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fastest route with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2], speeds = [60, 50, 40, 80, 100, 90, 50], origin = A and destination = E.	SOLVED	route = A-C-E, time = 9.66667 min, distance = 12.5 km
Fastest route with roads = [P-Q 3, Q-R 4, P-R 9, R-S 2, Q-S 7], speeds = [30, 60, 100, 50, 40], origin = P, destination = S and directed = yes.	SOLVED	route = P-R-S, time = 7.8 min, distance = 11 km
Fastest route with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2], speeds = [60, 50, 40, 80, 100, 90, 50] and origin = A.	NEEDS_INPUT	transport_fastest_route needs destination (destination). Example: solve('transport_fastest_route', roads='A-B 4, A-C 2.5
Fastest route with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2], speeds = [60, 50, 40, 80, 100, 90, 50], origin = A and destination = E and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Fastest route with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2], speeds = [60, 50, 40, 80, 100, 90, 50], origin = A and destination = Z.	INVALID_INPUT	Fastest route on a road network (link lengths and speeds): the origin and the destination must be places on the network.

Zone-to-zone travel time matrix (skim) of a road network, `transport_travel_time_skim`

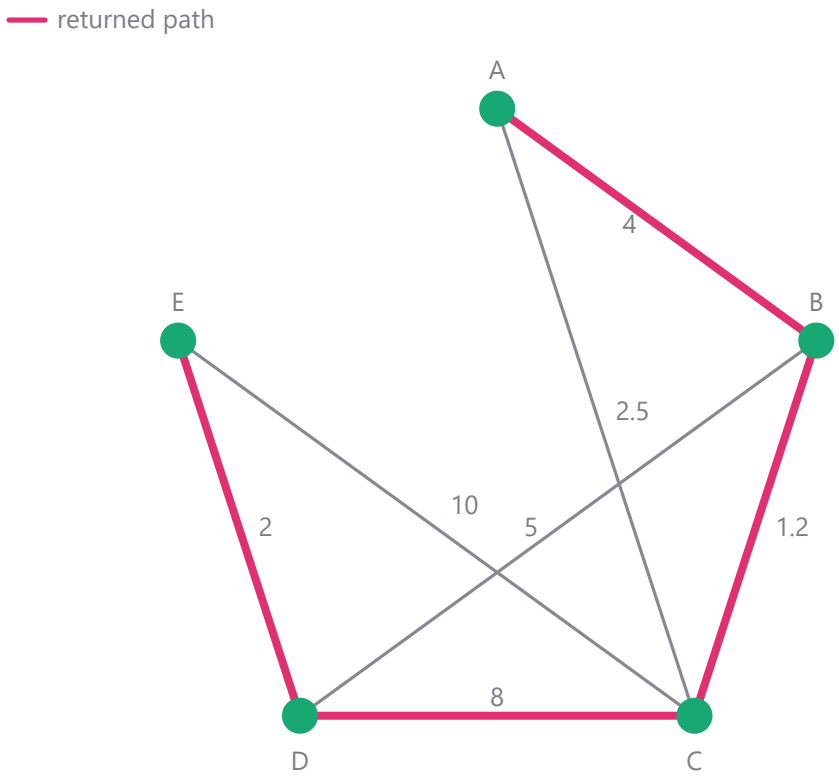
The fastest travel time in minutes between every pair of places of a road network given by link lengths (km) and speeds (km/h), as a matrix in alphabetical order of the places (the skim used for trip distribution).

$$t_{ij} = \min_{paths\ i \to j} \sum t_{link}$$

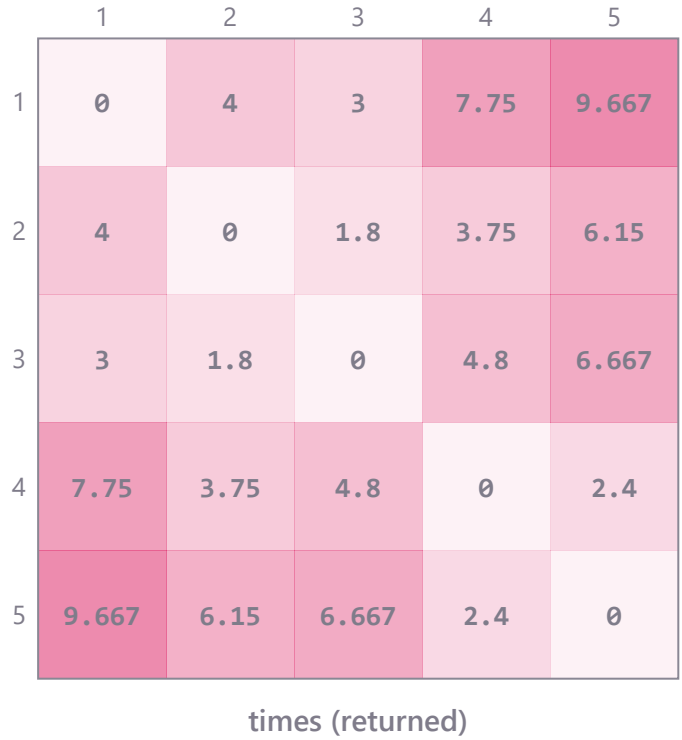
input	meaning	condition
`roads`	roads as 'A-B length' with lengths in km	
`speeds`	speed on each road in km/h, in the same order	
`directed`	one-way links as written (default no)	

output	meaning
`places`	places in the order of the matrix
`times`	travel time matrix (min)

- **Method:** Dijkstra from every place
- **Assumes:** every place reachable; at most 60 places
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The network roads (roads as 'A-B length' with lengths in km) of the worked example (5 nodes, 7 edges, laid out on a circle); on each edge its stated weight; highlighted: the returned path A → B → C → D → E. Other inputs: speeds = [60, 50, 40, 80, 100, 90, 50].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): times (returned). Other inputs: roads = A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10…, speeds = [60, 50, 40, 80, 100, 90, 50].

```
transportation_ai999x.solve('transport_travel_time_skim', roads='A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2', speeds=[60, 50, 40, 80, 100, 90, 50])
# -> places = A, B, C, D, E, times = [[0.0, 4.0, 3.0, 7.75, 9.66666666667], [4.0, 0.0, 1.8, 3.75, 6.15], [3.0, 1.8, 0.0, 4.8, 6.66666666667], [7.75, 3.75, 4.8, 0.0, 2.4], [9.66666666667, 6.15, 6.66666666667, 2.4, 0.0]]   (SOLVED)
```

```
transportation_ai999x.solve('transport_travel_time_skim', roads='P-Q 3, Q-R 4, P-R 9, R-S 2, Q-S 7', speeds=[30, 60, 100, 50, 40])
# -> places = P, Q, R, S, times = [[0.0, 6.0, 5.4, 7.8], [6.0, 0.0, 4.0, 6.4], [5.4, 4.0, 0.0, 2.4], [7.8, 6.4, 2.4, 0.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Travel time skim with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2] and speeds = [60, 50, 40, 80, 100, 90, 50].	SOLVED	places = A, B, C, D, E, times = [[0.0, 4.0, 3.0, 7.75, 9.66666666667], [4.0, 0.0, 1.8, 3.75, 6.15], [3.0, 1.8, 0.0, 4.8, 6.66666666667], [7.75, 3.75, 4.8, 0.0, 2.4], [9.66666666667, 6.15, 6.66666666667, 2.4, 0.0]]
Travel time skim with roads = [P-Q 3, Q-R 4, P-R 9, R-S 2, Q-S 7] and speeds = [30, 60, 100, 50, 40].	SOLVED	places = P, Q, R, S, times = [[0.0, 6.0, 5.4, 7.8], [6.0, 0.0, 4.0, 6.4], [5.4, 4.0, 0.0, 2.4], [7.8, 6.4, 2.4, 0.0]]
Travel time skim with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2].	NEEDS_INPUT	transport_travel_time_skim needs speeds (speed on each road in km/h, in the same order). Example: solve('transport_trave
Travel time skim with roads = [A-B 4, A-C 2.5, B-C 1.2, B-D 5, C-D 8, C-E 10, D-E 2] and speeds = [60, 50, 40, 80, 100, 90, 50] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Travel time skim with roads = [A-B 4, C-D 3] and speeds = [60, 50].	INVALID_INPUT	Zone-to-zone travel time matrix (skim) of a road network: every place must be reachable from every other.

All-or-nothing traffic assignment, `transport_all_or_nothing_assignment`

Loads every origin-destination demand onto its fastest path through a network of one-way links with travel times (min): the flow on each link (in the order given) and the total vehicle-hours travelled.

$$x_a = \sum_{rs} q_{rs}\, \delta_{a,p^*_{rs}}$$

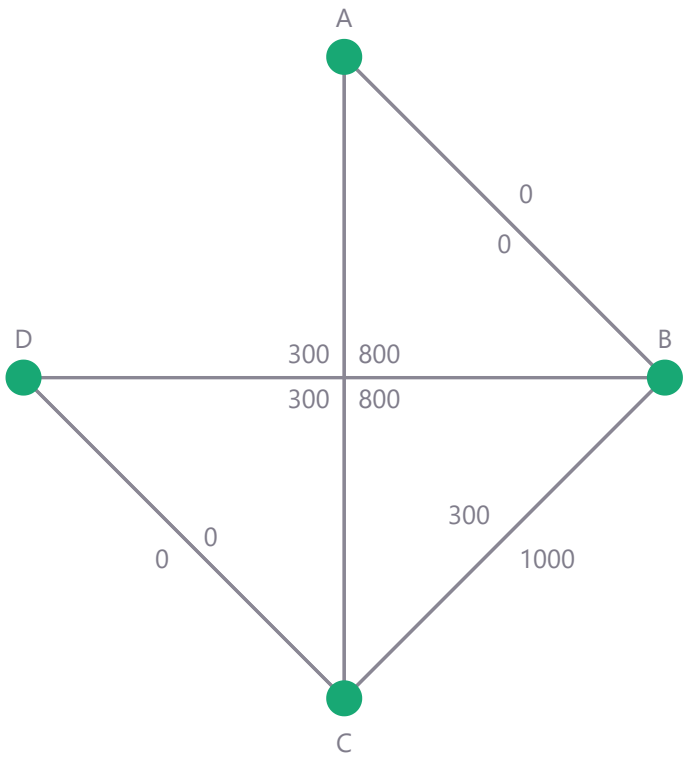
input	meaning	condition
`network`	one-way links as 'A-B time' with travel times in min	
`od`	trip demands as 'origin-destination trips' (veh)	

output	meaning
`flows`	flow on each link in the order given (veh)
`VHT`	vehicle-hours travelled (veh h)

- **Method:** for each trip pair the fastest path (Dijkstra); its demand is added to every link on it
- **Assumes:** positive link times; every trip pair connected
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



flow on each link in the order given (veh) (flows) returned by transportation-ai999x for the worked example, one value for each position in the list (10 values, first 0, last 1000, largest 1000). Inputs: network = A-B 5, B-A 5, A-C 3, C-A 3, B-D 4, D-B 4, C-D…, od = A-D 800, C-B 200, D-A 300.



The network network (one-way links as 'A-B time' with travel times in min) of the worked example (4 nodes, 10 edges, laid out on a circle); on each edge the flows returned by transportation-ai999x. Returned: VHT = 150 h. Other inputs: od = A-D 800, C-B 200, D-A 300.

```
transportation_ai999x.solve('transport_all_or_nothing_assignment', network='A-B 5, B-A 5, A-C 3, C-A 3, B-D 4, D-B 4, C-D 7, D-C 7, B-C 1, C-B 1', od='A-D 800, C-B 200, D-A 300')
# -> flows = [0.0, 0.0, 800.0, 300.0, 800.0, 300.0, 0.0, 0.0, 300.0, 1000.0], VHT = 150 h   (SOLVED)
```

```
transportation_ai999x.solve('transport_all_or_nothing_assignment', network='1-2 4, 2-3 6, 1-3 12, 3-4 3, 2-4 10', od='1-4 1000, 2-4 400')
# -> flows = [1000.0, 1400.0, 0.0, 1400.0, 0.0], VHT = 276.667 h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
All or nothing assignment with network = [A-B 5, B-A 5, A-C 3, C-A 3, B-D 4, D-B 4, C-D 7, D-C 7, B-C 1, C-B 1] and od = [A-D 800, C-B 200, D-A 300].	SOLVED	flows = [0.0, 0.0, 800.0, 300.0, 800.0, 300.0, 0.0, 0.0, 300.0, 1000.0], VHT = 150 h
All or nothing assignment with network = [1-2 4, 2-3 6, 1-3 12, 3-4 3, 2-4 10] and od = [1-4 1000, 2-4 400].	SOLVED	flows = [1000.0, 1400.0, 0.0, 1400.0, 0.0], VHT = 276.667 h
All or nothing assignment with network = [A-B 5, B-A 5, A-C 3, C-A 3, B-D 4, D-B 4, C-D 7, D-C 7, B-C 1, C-B 1].	NEEDS_INPUT	transport_all_or_nothing_assignment needs od (trip demands as 'origin-destination trips' (veh)). Example: solve('transpo
All or nothing assignment with network = [A-B 5, B-A 5, A-C 3, C-A 3, B-D 4, D-B 4, C-D 7, D-C 7, B-C 1, C-B 1] and od = [A-D 800, C-B 200, D-A 300] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
All or nothing assignment with network = [A-B 5, B-C 4] and od = [C-A 100].	INVALID_INPUT	All-or-nothing traffic assignment: A cannot be reached from C.

Doubly constrained gravity model of trip distribution, `transport_gravity_model`

Trips between zones $Tij = A_i$ O_i B_j D_j f(c_ij) that match both the productions O_i and the attractions D_j, with deterrence f(c) = exp(-beta c) or $c^{-\beta}$, balanced by Furness iterations until every total matches to $1e-13$.

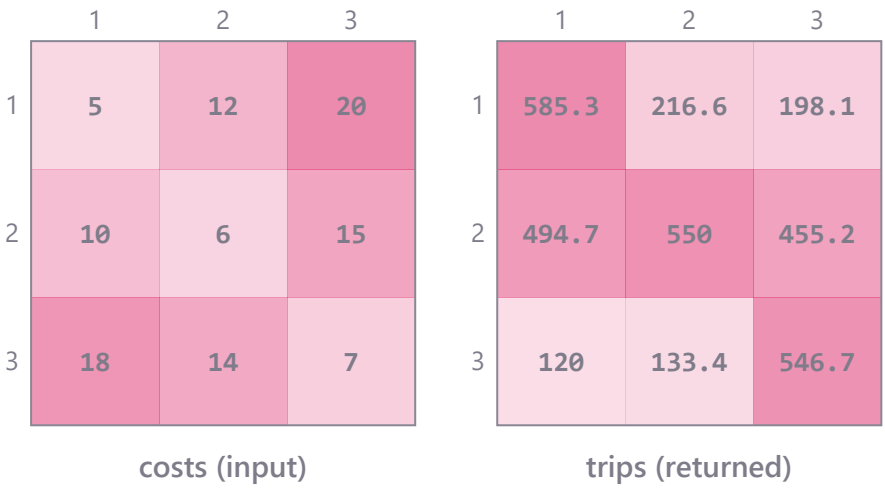
$$T_{ij} = A_i O_i B_j D_j f(c_{ij}), \quad \sum_j T_{ij} = O_i, \ \sum_i T_{ij} = D_j$$

input	meaning	condition
`O`	trip productions of the origin zones	
`D`	trip attractions of the destination zones	
`costs`	travel cost (time) matrix, rows origins, in min	
`beta`	deterrence parameter	
`deterrence`	exponential (default) or power	

output	meaning
`trips`	trip matrix (rows origins, columns destinations)

- **Method:** Furness (iterative proportional fitting) of O_i D_j f(c_ij) to the row and column totals

- **Assumes:** sum of productions = sum of attractions; $\beta \geq 0$; power deterrence: positive costs
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): costs (input); trips (returned). Other inputs: O = [1000, 1500, 800], D = [1200, 900, 1200], beta = 0.1.

```
transportation_ai999x.solve('transport_gravity_model', O=[1000, 1500, 800], D=[1200, 900, 1200],
costs=[[5, 12, 20], [10, 6, 15], [18, 14, 7]], beta=0.1)
# -> trips = [[585.291609215, 216.598390059, 198.110000726], [494.749778142, 550.037708887,
455.212512971], [119.958612642, 133.363901055, 546.677486303]]   (SOLVED)
```

```
transportation_ai999x.solve('transport_gravity_model', O=[500, 700], D=[600, 600], costs=[[4, 9],
[8, 3]], beta=2.0, deterrence='power')
# -> trips = [[452.776813945, 47.223186055], [147.223186055, 552.776813945]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gravity model with productions = [1000, 1500, 800], attractions = [1200, 900, 1200], costs = [5 12 20; 10 6 15; 18 14 7] and beta = 0.1.	SOLVED	trips = [[585.291609215, 216.598390059, 198.110000726], [494.749778142, 550.037708887, 455.212512971], [119.958612642, 133.363901055, 546.677486303]]
Gravity model with productions = [500, 700], attractions = [600, 600], costs = [4 9; 8 3], beta = 2 and deterrence = power.	SOLVED	trips = [[452.776813945, 47.223186055], [147.223186055, 552.776813945]]
Gravity model with productions = [1000, 1500, 800], attractions = [1200, 900, 1200] and costs = [5 12 20; 10 6 15; 18 14 7].	NEEDS_INPUT	transport_gravity_model needs beta (deterrence parameter). Example: solve('transport_gravity_model', O=[1000, 1500, 800]
Gravity model with productions = [1000, 1500, 800], attractions = [1200, 900, 1200], costs = [5 12 20; 10 6 15; 18 14 7] and beta = 0.1 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Gravity model with productions = [1000, 1500], attractions = [1200, 900], costs = [5 12; 10 6] and beta = 0.1.	INVALID_INPUT	Doubly constrained gravity model of trip distribution: total productions must equal total attractions.

Growth-factor (Furness) balancing of a trip matrix, `transport_furness_balancing`

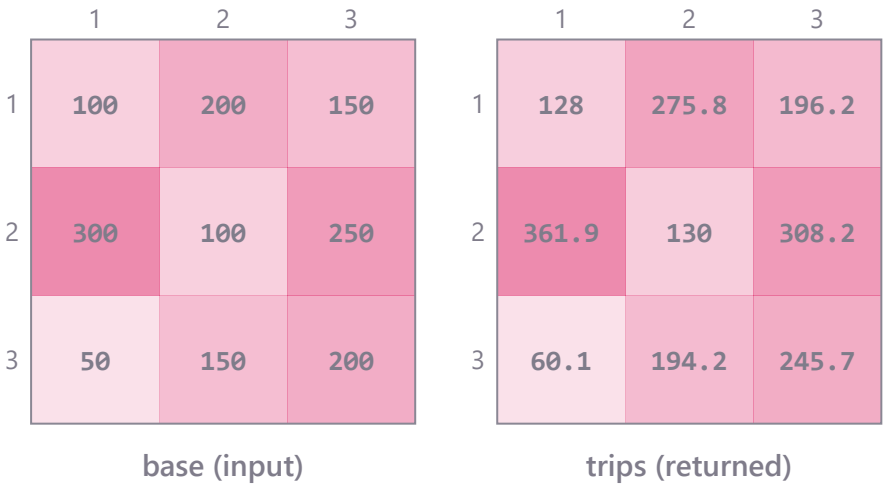
Scales a base-year trip matrix to future row totals (productions) and column totals (attractions) by the Furness (biproportional) method, iterated until every total matches to $1e-13$.

$$T_{ij} = t_{ij}\, a_i\, b_j$$

input	meaning	condition
`base`	base-year trip matrix (rows origins)	
`O`	target row totals (productions)	
`D`	target column totals (attractions)	

output	meaning
`trips`	balanced trip matrix

- **Method:** alternately scale rows to O_i and columns to D_j until both match
- **Assumes:** sum of row totals = sum of column totals; non-negative base matrix with a positive entry in every row and column
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): base (input); trips (returned). Other inputs: O = [600, 800, 500], D = [550, 600, 750].

```
transportation_ai999x.solve('transport_furness_balancing', base=[[100, 200, 150], [300, 100, 250],
[50, 150, 200]], O=[600, 800, 500], D=[550, 600, 750])
# -> trips = [[128.005362822, 275.799789296, 196.194847882], [361.892088736, 129.955182633,
308.15272863], [60.1025484415, 194.24502807, 245.652423488]]   (SOLVED)
```

```
transportation_ai999x.solve('transport_furness_balancing', base=[[40, 60], [80, 20]], O=[130, 110],
D=[140, 100])
# -> trips = [[52.0, 78.0], [88.0, 22.0]]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Furness balancing with base matrix = [100 200 150; 300 100 250; 50 150 200], productions = [600, 800, 500] and attractions = [550, 600, 750].	SOLVED	trips = [[128.005362822, 275.799789296, 196.194847882], [361.892088736, 129.955182633, 308.15272863], [60.1025484415, 194.24502807, 245.652423488]]
Furness balancing with base matrix = [40 60; 80 20], productions = [130, 110] and attractions = [140, 100].	SOLVED	trips = [[52.0, 78.0], [88.0, 22.0]]
Furness balancing with base matrix = [100 200 150; 300 100 250; 50 150 200] and productions = [600, 800, 500].	NEEDS_INPUT	transport_furness_balancing needs D (target column totals (attractions)). Example: solve('transport_furness_balancing',
Furness balancing with base matrix = [100 200 150; 300 100 250; 50 150 200], productions = [600, 800, 500] and attractions = [550, 600, 750] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Furness balancing with base matrix = [100 200; 300 100], productions = [600, 800] and attractions = [550, 600].	INVALID_INPUT	Growth-factor (Furness) balancing of a trip matrix: total productions must equal total attractions.

Multinomial logit mode shares from utilities, `transport_logit_mode_shares`

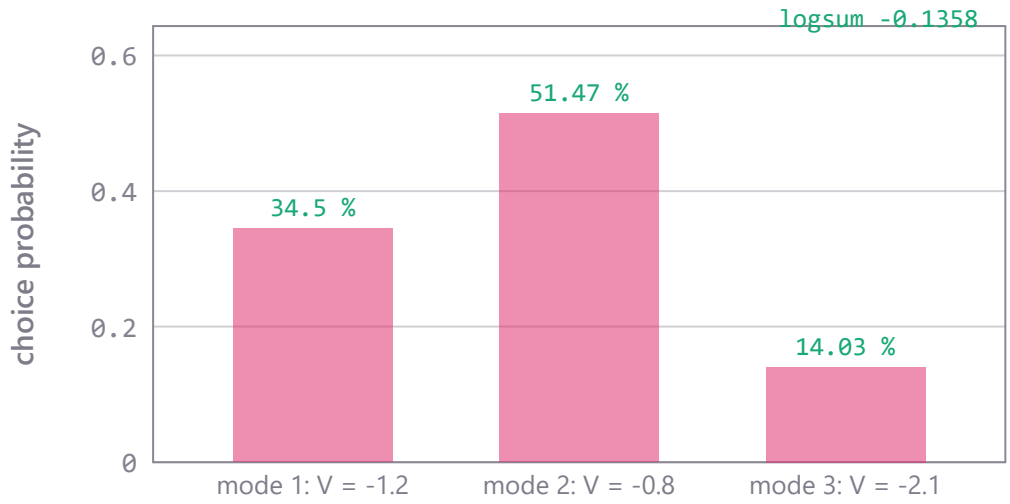
Choice probabilities (mode shares) of a multinomial logit model from the systematic utilities of the alternatives, and the logsum (expected maximum utility).

$$P_i = \frac{e^{V_i}}{\sum_j e^{V_j}}$$

input	meaning	condition
`V`	utility of each alternative	

output	meaning
`P`	probability of each alternative
`logsum`	logsum ln(sum exp V)

- **Method:** $P_i = exp$(V_i) / sum_j exp(V_j)
- **Assumes:** at least two alternatives
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Multinomial logit shares of 3 modes with utilities V = [-1.2, -0.8, -2.1]: the probabilities the package returned (0.345019, 0.514707, 0.140274) and the logsum -0.135843.

```
transportation_ai999x.solve('transport_logit_mode_shares', V=[-1.2, -0.8, -2.1])
# -> P = [0.345018612617, 0.514707287448, 0.140274099936], logsum = -0.135843   (SOLVED)
```

```
transportation_ai999x.solve('transport_logit_mode_shares', V=[0.5, -0.3])
# -> P = [0.689974481128, 0.310025518872], logsum = 0.871101   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logit mode shares with utilities = [-1.2, -0.8, -2.1].	SOLVED	P = [0.345018612617, 0.514707287448, 0.140274099936], logsum = -0.135843
Logit mode shares with utilities = [0.5, -0.3].	SOLVED	P = [0.689974481128, 0.310025518872], logsum = 0.871101
Logit mode shares with utilities = [-1.2, -0.8, -2.1] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Logit mode shares with utilities = [0.5].	INVALID_INPUT	V: needs at least 2 entries, got 1.

Logit mode choice from attributes and coefficients, `transport_logit_from_attributes`

Utilities V_i = ASC_i + sum_k beta_k x_ik of each mode from its attributes (rows of X: modes; columns: attributes such as time in min and cost) and the coefficients, and the resulting logit choice probabilities.

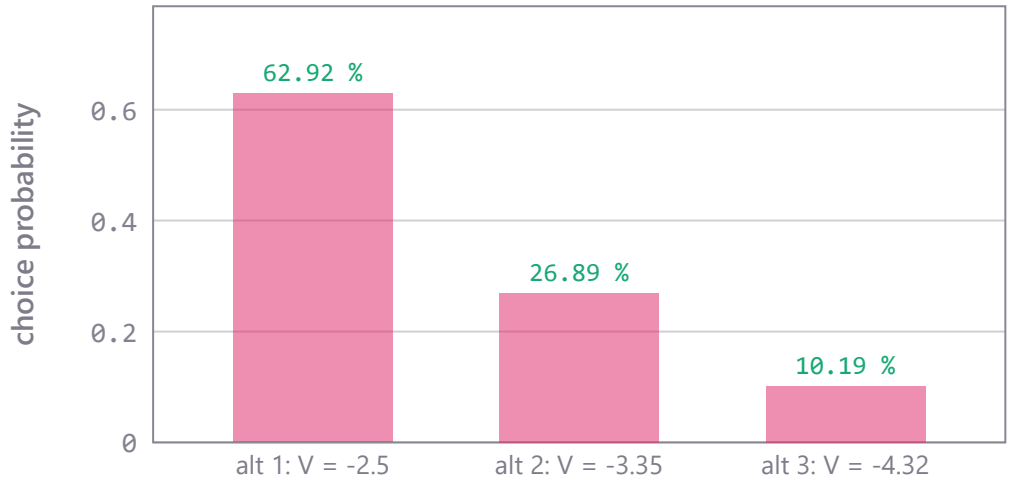
$$V_i = ASC_i + \sum_k \beta_k x_{ik}, \quad P_i = \frac{e^{V_i}}{\sum_j e^{V_j}}$$

input	meaning	condition
`X`	attributes: one row per mode, one column per attribute	
`beta`	coefficient of each attribute	

input	meaning	condition
`asc`	alternative-specific constant of each mode (default 0)	

output	meaning
`V`	utility of each mode
`P`	probability of each mode

- **Method:** $V = X$ beta + ASC, P = softmax(V)
- **Assumes:** one coefficient per attribute column; at least two modes
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Logit choice among 3 alternatives with attributes X = [[30, 2.5], [45, 1.5], [60, 0.8]], coefficients [-0.05, -0.4] and constants [0, -0.5, -1.0]: the returned utilities (-2.5, -3.35, -4.32) under each bar and the returned probabilities (0.629152, 0.268909, 0.101939).

```
transportation_ai999x.solve('transport_logit_from_attributes', X=[[30, 2.5], [45, 1.5], [60, 0.8]],
beta=[-0.05, -0.4], asc=[0, -0.5, -1.0])
# -> V = [-2.5, -3.35, -4.32], P = [0.629152135572, 0.268909017211, 0.101938847218]   (SOLVED)
```

```
transportation_ai999x.solve('transport_logit_from_attributes', X=[[25, 4], [40, 2]], beta=[-0.03,
-0.25])
# -> V = [-1.75, -1.7], P = [0.487502603516, 0.512497396484]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logit from attributes with attributes = [30 2.5; 45 1.5; 60 0.8], coefficients = [-0.05, -0.4] and constants = [0, -0.5, -1].	SOLVED	V = [-2.5, -3.35, -4.32], P = [0.629152135572, 0.268909017211, 0.101938847218]
Logit from attributes with attributes = [25 4; 40 2] and coefficients = [-0.03, -0.25].	SOLVED	V = [-1.75, -1.7], P = [0.487502603516, 0.512497396484]
Logit from attributes with attributes = [30 2.5; 45 1.5; 60 0.8] and constants =	NEEDS_INPUT	transport_logit_from_attributes needs beta (coefficient of each attribute). Example:

question	outcome	result
[0, -0.5, -1].		solve('transport_logit_from_attribu
Logit from attributes with attributes = [30 2.5; 45 1.5; 60 0.8], coefficients = [-0.05, -0.4] and constants = [0, -0.5, -1] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Logit from attributes with attributes = [30 2.5; 45 1.5] and coefficients = [-0.05].	INVALID_INPUT	Logit mode choice from attributes and coefficients: beta must have 2 entries.

Direct and cross elasticities of logit choice probabilities, `transport_logit_elasticities`

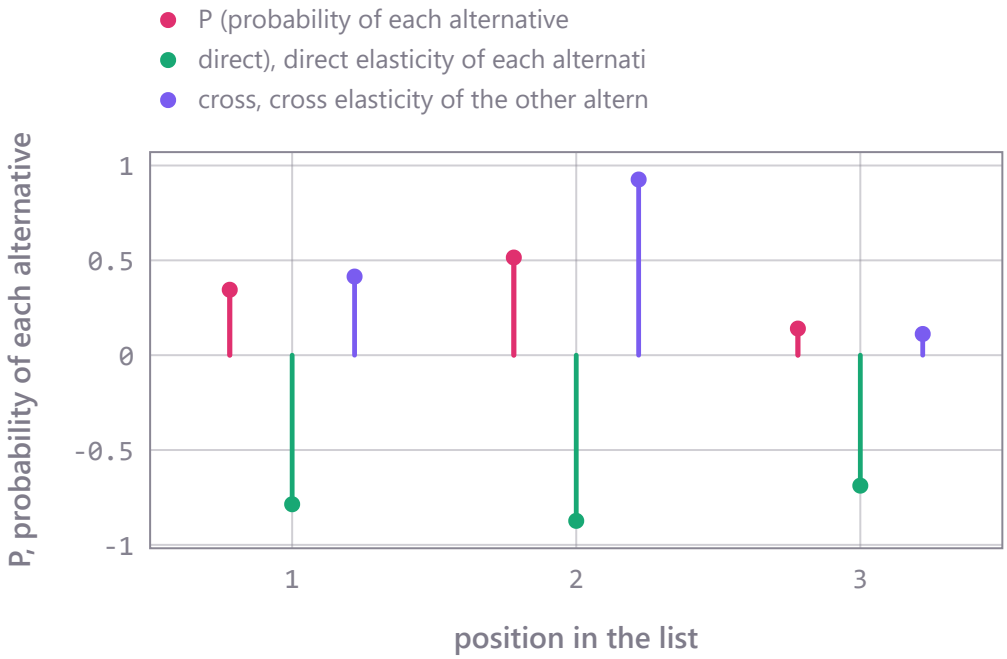
Point elasticities of logit probabilities with respect to an attribute x (such as cost or time) that enters the utility linearly with coefficient beta: direct $E_i i = \beta$ x_i $(1 - P_i)$ and cross $E_j i = -\beta$ x_i P_i (the same for every other alternative j, the IIA property).

$$E_{x_i}^{P_i} = \beta x_i (1 - P_i), \quad E_{x_i}^{P_j} = -\beta x_i P_i$$

input	meaning	condition
`V`	utility of each alternative	
`x`	value of the attribute for each alternative	
`beta`	coefficient of the attribute	

output	meaning
`P`	probability of each alternative
`direct`	direct elasticity of each alternative
`cross`	cross elasticity of the other alternatives with respect to each alternative's attribute

- **Method:** $E_i i = \beta$ x_i $(1 - P_i)$, $E_j i = -\beta$ x_i P_i
- **Assumes:** one attribute value per alternative
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



probability of each alternative (P) returned by transportation-ai999x for the worked example, one value for each position in the list (3 values, first 0.345019, last 0.140274, largest 0.514707); with direct (direct elasticity of each alternative), cross (cross elasticity of the other alternatives with respect to each alternative's attribute). Inputs: V = [-1.2, -0.8, -2.1], x = [30, 45, 20], beta = -0.04.

```
transportation_ai999x.solve('transport_logit_elasticities', V=[-1.2, -0.8, -2.1], x=[30, 45, 20], beta=-0.04)
# -> P = [0.345018612617, 0.514707287448, 0.140274099936], direct = [-0.78597766486, -0.873526882594, -0.687780720051], cross = [0.41402233514, 0.926473117406, 0.112219279949]  (SOLVED)
```

```
transportation_ai999x.solve('transport_logit_elasticities', V=[0.2, -0.4], x=[3.5, 2.0], beta=-0.3)
# -> P = [0.645656306226, 0.354343693774], direct = [-0.372060878463, -0.387393783735], cross = [0.677939121537, 0.212606216265]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logit elasticities with utilities = [-1.2, -0.8, -2.1], attribute = [30, 45, 20] and beta = -0.04.	SOLVED	P = [0.345018612617, 0.514707287448, 0.140274099936], direct = [-0.78597766486, -0.873526882594, -0.687780720051], cross = [0.41402233514, 0.926473117406, 0.112219279949]
Logit elasticities with utilities = [0.2, -0.4], attribute = [3.5, 2] and beta = -0.3.	SOLVED	P = [0.645656306226, 0.354343693774], direct = [-0.372060878463, -0.387393783735], cross = [0.677939121537, 0.212606216265]
Logit elasticities with utilities = [-1.2, -0.8, -2.1] and attribute = [30, 45, 20].	NEEDS_INPUT	transport_logit_elasticities needs beta (coefficient of the attribute). Example: solve('transport_logit_elasticities', V
Logit elasticities with utilities = [-1.2, -0.8, -2.1], attribute = [30, 45, 20] and beta = -0.04 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Logit elasticities with utilities = [-1.2, -0.8, -2.1], attribute = [30, 45] and beta = -0.04.	INVALID_INPUT	Direct and cross elasticities of logit choice probabilities: x must have 3 entries.

Nested logit choice probabilities, `transport_nested_logit`

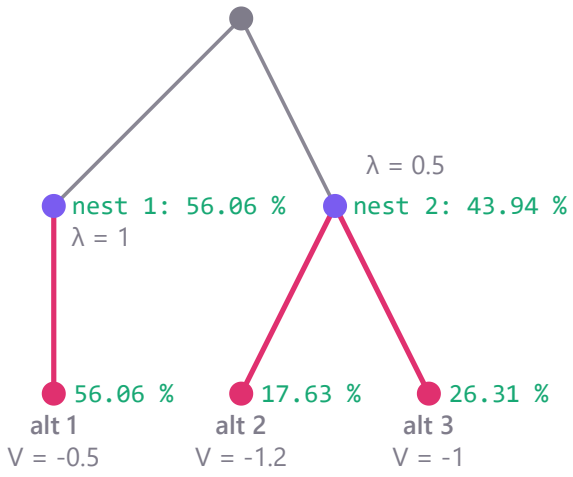
Choice probabilities of a two-level nested logit model (for example car / public transport nests holding bus and rail): alternatives grouped into nests 1..K with dissimilarity (logsum) parameters lam_k in (0, 1]; also the probability of each nest.

$$P_i = \frac{e^{V_i/\lambda_k}}{e^{I_k}} \cdot \frac{e^{\lambda_k I_k}}{\sum_m e^{\lambda_m I_m}}, \quad I_k = \ln \sum_{j \in k} e^{V_j/\lambda_k}$$

input	meaning	condition
`V`	utility of each alternative	
`nest`	nest number (1, 2, ...) of each alternative	
`lam`	dissimilarity parameter of each nest, in (0, 1]	

output	meaning
`P`	probability of each alternative
`P_nest`	probability of each nest

- **Method:** $I_k = ln$ sum_{j in k} exp(V_j / lam_k); P(k) = exp(lam_k I_k) / sum_m exp(lam_m I_m); P(i|k) = exp(V_i/lam_k - I_k)
- **Assumes:** nests numbered 1..K, each used; 0 < lam_k <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A nested logit tree: alternatives with utilities V = [-0.5, -1.2, -1.0] grouped into nests [1, 2, 2] with nest parameters λ = [1.0, 0.5]. Marked: the returned nest probabilities 0.560574, 0.439426 and alternative probabilities 0.560574, 0.176347, 0.263079.

```
transportation_ai999x.solve('transport_nested_logit', V=[-0.5, -1.2, -1.0], nest=[1, 2, 2], lam=
[1.0, 0.5])
# -> P = [0.560574109573, 0.176347032294, 0.263078858132], P_nest = [0.560574109573,
0.439425890427]   (SOLVED)
```

```
transportation_ai999x.solve('transport_nested_logit', V=[0.2, -0.6, -0.4, -1.5], nest=[1, 1, 2, 2],
lam=[0.7, 0.4])
# -> P = [0.518080024916, 0.165219117164, 0.297671364261, 0.0190294936597], P_nest =
[0.68329914208, 0.31670085792]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Nested logit with utilities = [-0.5, -1.2, -1], nests = [1, 2, 2] and nest parameters = [1, 0.5].	SOLVED	P = [0.560574109573, 0.176347032294, 0.263078858132], P_nest = [0.560574109573, 0.439425890427]
Nested logit with utilities = [0.2, -0.6, -0.4, -1.5], nests = [1, 1, 2, 2] and nest parameters = [0.7, 0.4].	SOLVED	P = [0.518080024916, 0.165219117164, 0.297671364261, 0.0190294936597], P_nest = [0.68329914208, 0.31670085792]
Nested logit with utilities = [-0.5, -1.2, -1] and nests = [1, 2, 2].	NEEDS_INPUT	transport_nested_logit needs lam (dissimilarity parameter of each nest, in (0, 1]). Example: solve('transport_nested_log
Nested logit with utilities = [-0.5, -1.2, -1], nests = [1, 2, 2] and nest parameters = [1, 0.5] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Nested logit with utilities = [-0.5, -1.2, -1], nests = [1, 2, 2] and nest parameters = [1, 1.5].	INVALID_INPUT	Nested logit choice probabilities: nest parameters must lie in (0, 1].

User benefit of a change from the logit logsum, `transport_logsum_user_benefit`

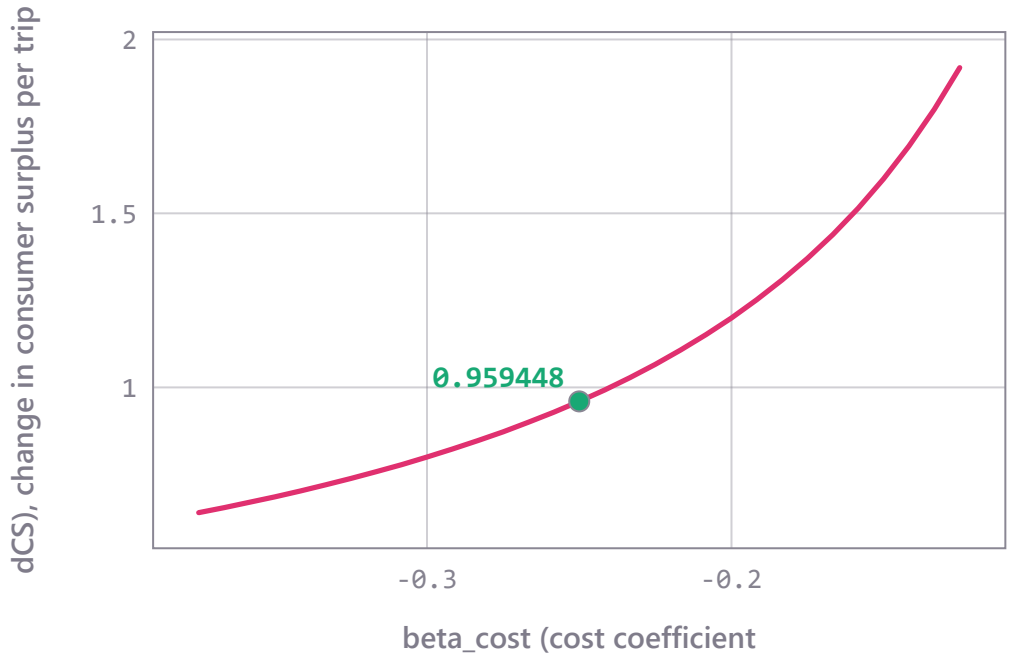
Change in consumer surplus per trip (in money units) when the utilities of the alternatives change from V0 to V1, from the difference of the logsums divided by the (negative) cost coefficient.

$$\Delta CS = \frac{1}{-\beta_c}\left[\ln\sum_j e^{V_j^1} - \ln\sum_j e^{V_j^0}\right]$$

input	meaning	condition
`V0`	utilities before the change	
`V1`	utilities after the change	
`beta_cost`	cost coefficient (utility per money unit, negative)	

output	meaning
`dCS`	change in consumer surplus per trip (money units)
`logsum0`	logsum before
`logsum1`	logsum after

- **Method:** dCS = (ln sum exp V_1 $-$ ln sum exp V0) / (-beta_cost)
- **Assumes:** same alternatives before and after; beta_cost < 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dCS (change in consumer surplus per trip (money units)) against beta_cost (cost coefficient (utility per money unit, negative)) from -0.375 to -0.125. Other inputs as in the worked example: V0 = [-1.2, -0.8, -2.1], V1 = [-1.2, -0.5, -1.6]. 31 points computed by transportation-ai999x; marked: the worked example, beta_cost = -0.25, dCS = 0.959448.

```
transportation_ai999x.solve('transport_logsum_user_benefit', V0=[-1.2, -0.8, -2.1], V1=[-1.2, -0.5, -1.6], beta_cost=-0.25)
# -> dCS = 0.959448, logsum0 = -0.135843, logsum1 = 0.104019   (SOLVED)
```

```
transportation_ai999x.solve('transport_logsum_user_benefit', V0=[0.3, -0.7], V1=[0.1, -0.2], beta_cost=-0.1)
# -> dCS = 0.410936, logsum0 = 0.613262, logsum1 = 0.654355   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Logsum user benefit with utilities before = [-1.2, -0.8, -2.1], utilities after = [-1.2, -0.5, -1.6] and cost coefficient = -0.25.	SOLVED	dCS = 0.959448, logsum0 = -0.135843, logsum1 = 0.104019
Logsum user benefit with utilities before = [0.3, -0.7], utilities after = [0.1, -0.2] and cost coefficient = -0.1.	SOLVED	dCS = 0.410936, logsum0 = 0.613262, logsum1 = 0.654355
Logsum user benefit with utilities before = [-1.2, -0.8, -2.1] and utilities after = [-1.2, -0.5, -1.6].	NEEDS_INPUT	transport_logsum_user_benefit needs beta_cost (cost coefficient (utility per money unit, negative)). Example: solve('tra
Logsum user benefit with utilities before = [-1.2, -0.8, -2.1], utilities after = [-1.2, -0.5, -1.6] and cost coefficient = -0.25 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Logsum user benefit with utilities before = [-1.2, -0.8, -2.1], utilities after = [-1.2, -0.5, -1.6] and cost coefficient = 0.25.	INVALID_INPUT	User benefit of a change from the logit logsum: the cost coefficient must be negative (utility falls as cost rises).

Trip generation by multiple linear regression, `transport_trip_generation_regression`

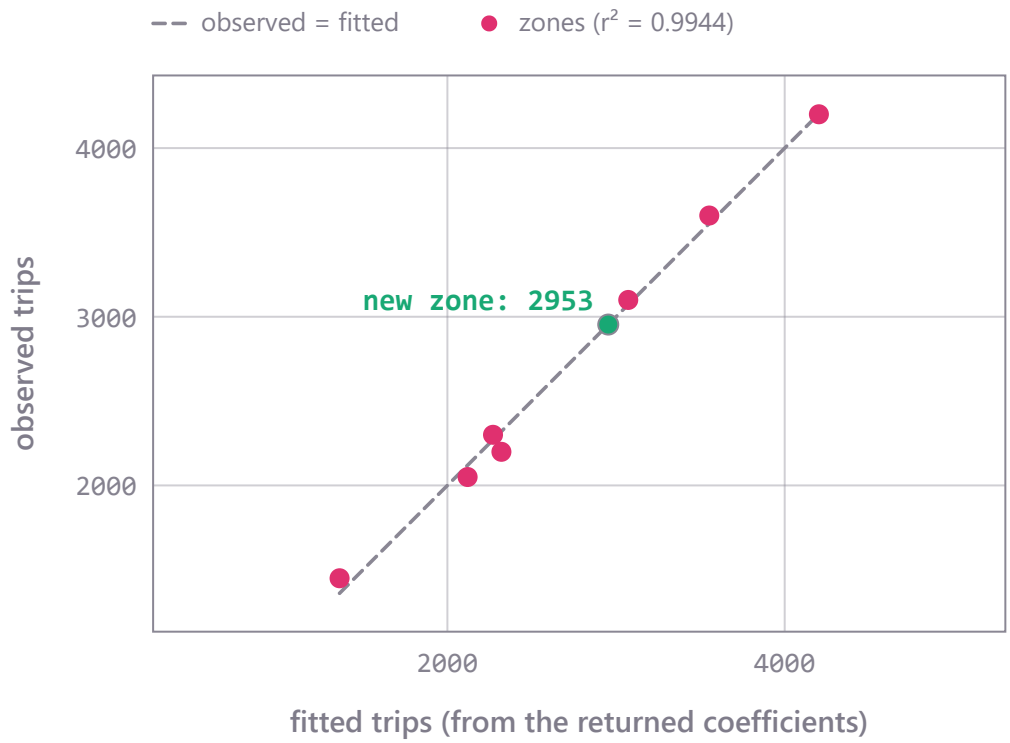
Least-squares trip-generation model trips = b0 + b1 $x_1 + \ldots$ fitted to zonal data (rows of X: zones; columns: households, employment, cars, ...), its coefficient of determination, and optionally the trips predicted for a new zone.

$$\hat{\mathbf{b}} = \arg\min \|\mathbf{y} - [\mathbf{1}\ X]\mathbf{b}\|^2$$

input	meaning	condition
`X`	zonal predictors: one row per zone	
`y`	observed trips of each zone	
`x_new`	predictors of a zone to forecast (optional)	

output	meaning
`coef`	coefficients b0 (intercept), b1, ...
`r2`	coefficient of determination
`y_new`	forecast trips of the new zone

- **Method:** ordinary least squares with an intercept (Householder QR)
- **Assumes:** more zones than coefficients; predictors not collinear
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Trip-generation regression on 7 zones (explanatory values [[120, 300], [250, 420], [400, 380], [180, 600], [520, 700], [300, 250], [450, 520]], observed trips [1450, 2300, 3100, 2050, 4200, 2200, 3600]): each zone's observed trips against the trips given by the returned coefficients b = 258.228, 5.7182, 1.38738 (derived, with the 1:1 line); r^2 = 0.994434. Marked: the returned prediction 2953.29 trips for the new zone [350, 500].

```
transportation_ai999x.solve('transport_trip_generation_regression', X=[[120, 300], [250, 420],
[400, 380], [180, 600], [520, 700], [300, 250], [450, 520]], y=[1450, 2300, 3100, 2050, 4200, 2200,
3600], x_new=[350, 500])
# -> coef = [258.227672282, 5.71819751176, 1.38738416969], r2 = 0.994434, y_new = 2953.29
(SOLVED)
```

```
transportation_ai999x.solve('transport_trip_generation_regression', X=[[800], [1200], [1500],
[2000], [2600]], y=[2100, 3000, 3900, 5050, 6400])
# -> coef = [187.347560976, 2.40904471545], r2 = 0.998189   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Trip generation regression with predictors = [120 300; 250 420; 400 380; 180 600; 520 700; 300 250; 450 520], trips = [1450, 2300, 3100, 2050, 4200, 2200, 3600] and new zone = [350, 500].	SOLVED	coef = [258.227672282, 5.71819751176, 1.38738416969], r2 = 0.994434, y_new = 2953.29
Trip generation regression with predictors = [800; 1200; 1500; 2000; 2600] and trips = [2100, 3000, 3900, 5050, 6400].	SOLVED	coef = [187.347560976, 2.40904471545], r2 = 0.998189
Trip generation regression with predictors = [120 300; 250 420; 400 380; 180 600; 520 700; 300 250; 450 520] and new zone = [350, 500].	NEEDS_INPUT	transport_trip_generation_regression needs y (observed trips of each zone). Example: solve('transport_trip_generation_re
Trip generation regression with predictors = [120 300; 250 420; 400 380; 180 600; 520 700; 300 250; 450 520], trips = [1450, 2300, 3100, 2050, 4200, 2200, 3600] and new zone = [350, 500] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Trip generation regression with predictors = [120 300; 250 420] and trips = [1450, 2300].	INVALID_INPUT	Trip generation by multiple linear regression: need more zones than coefficients.

Understeer gradient and characteristic or critical speed (bicycle model),

`transport_understeer_gradient`

Understeer gradient K of the linear single-track model from axle cornering stiffnesses and the centre-of-gravity position, in rad/(m/s^2) and deg/g, whether the vehicle understeers or oversteers, and its characteristic speed (understeer: the steer angle is twice the Ackermann angle) or critical speed (oversteer: it becomes unstable).

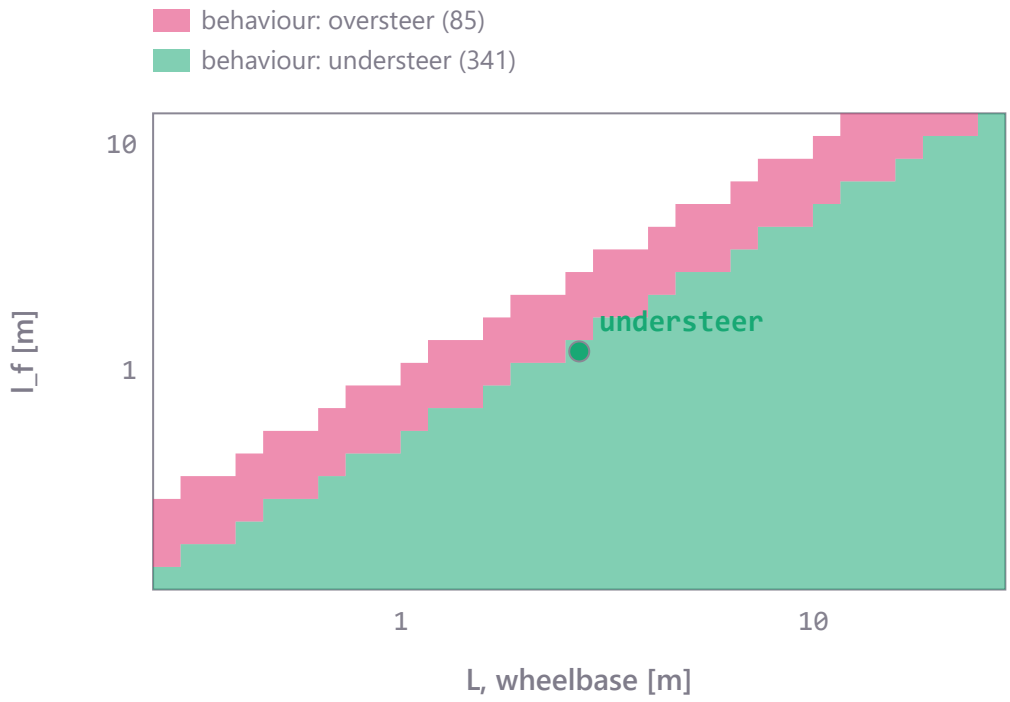
$$K = \frac{m}{L}\left(\frac{l_r}{C_f} - \frac{l_f}{C_r}\right), \quad v_{ch} = \sqrt{L/K}$$

input	meaning	condition
`m`	vehicle mass	
`L`	wheelbase	
`l_f`	distance from the front axle to the centre of gravity	

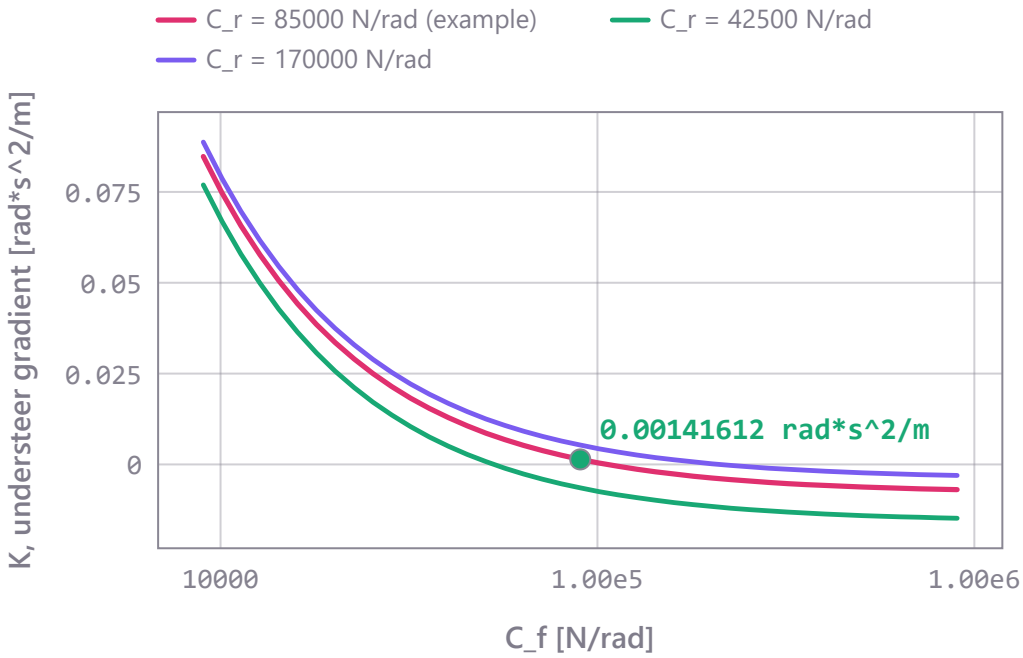
input	meaning	condition
`C_f`	cornering stiffness of the front axle	
`C_r`	cornering stiffness of the rear axle	

output	meaning
`K`	understeer gradient (rad s^2/m)
`K_deg_per_g`	understeer gradient (deg/g)
`behaviour`	understeer or oversteer
`v_c`	characteristic or critical speed (m/s)
`v_c_kmh`	characteristic or critical speed (km/h)

- **Method:** $K = m$ (l_r / (L C_f) - l_f / (L C_r)); $v_c h = sqrt$(L / K) $(K > 0)$, v_crit = sqrt(-L / K) $(K < 0)$
- **Assumes:** $0 < l_f < L$; positive cornering stiffnesses; not exactly neutral steer
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



understeer or oversteer (behaviour) over L (wheelbase) from 0.27 m to 27 m and l_f (distance from the front axle to the centre of gravity) from 0.12 m to 12 m (logarithmic axes): each of the 31 × 21 cells is one run of transportation-ai999x, coloured by the verdict it returned, oversteer: 85 cells, understeer: 341 cells; blank cells: inputs the capability refuses. Other inputs as in the worked example: m = 1500.0 kg, C_f = 90000.0 N/rad, C_r = 85000.0 N/rad. Marked: the worked example, L = 2.7 m, l_f = 1.2 m → understeer.



K (understeer gradient (rad s^2/m)) against C_f (cornering stiffness of the front axle) from 9000 N/rad to 900000 N/rad on a logarithmic axis. One curve for each value of C_r (cornering stiffness of the rear axle): 85000 N/rad, 42500 N/rad, 170000 N/rad. Other inputs as in the worked example: m = 1500.0 kg, L = 2.7 m, l_f = 1.2 m. 41 points per curve computed by transportation-ai999x; marked: the worked example, C_f = 90000 N/rad, K = 0.00141612 rad*s^2/m.

```
transportation_ai999x.solve('transport_understeer_gradient', m=1500.0, L=2.7, l_f=1.2, C_f=90000.0,
C_r=85000.0)
# -> K = 0.00141612 rad*s^2/m, K_deg_per_g = 0.795962 deg, behaviour = understeer, v_c = 43.6648
m/s, v_c_kmh = 157.193 km/h   (SOLVED)
```

```
transportation_ai999x.solve('transport_understeer_gradient', m=1300.0, L=2.5, l_f=1.4, C_f=90000.0,
C_r=60000.0)
# -> K = -0.00577778 rad*s^2/m, K_deg_per_g = -3.24752 deg, behaviour = oversteer, v_c = 20.8013
m/s, v_c_kmh = 74.8845 km/h   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Understeer gradient with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad and rear cornering stiffness = 85000 N/rad.	SOLVED	K = 0.00141612 rad*s^2/m, K_deg_per_g = 0.795962 deg, behaviour = understeer, v_c = 43.6648 m/s, v_c_kmh = 157.193 km/h
Understeer gradient with mass = 1300 kg, wheelbase = 2.5 m, cg to front axle = 1.4 m, front cornering stiffness = 90000 N/rad and rear cornering stiffness = 60000 N/rad.	SOLVED	K = -0.00577778 rad*s^2/m, K_deg_per_g = -3.24752 deg, behaviour = oversteer, v_c = 20.8013 m/s, v_c_kmh = 74.8845 km/h
Understeer gradient with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m and front cornering stiffness = 90000 N/rad.	NEEDS_INPUT	transport_understeer_gradient needs C_r (cornering stiffness of the rear axle). Example: solve('transport_understeer_gra
Understeer gradient with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
and rear cornering stiffness = 85000 N/rad and colour = 3.		
Understeer gradient with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 3 m, front cornering stiffness = 90000 N/rad and rear cornering stiffness = 85000 N/rad.	INVALID_INPUT	Understeer gradient and characteristic or critical speed (bicycle model): the distance from the front axle to the centre

Steady-state cornering: yaw rate, lateral acceleration, radius, sideslip,

`transport_steady_state_cornering`

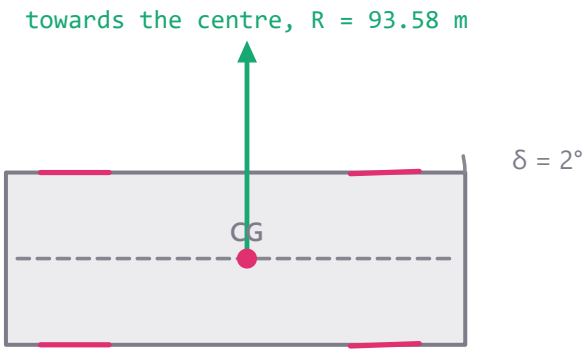
Steady turning response of the linear bicycle model at speed u to a front-wheel steer angle delta: yaw rate, lateral acceleration, turn radius and the sideslip angle at the centre of gravity (small angles).

$$\frac{r}{\delta} = \frac{u}{L + Ku^2}$$

input	meaning	condition
`m`	vehicle mass	
`L`	wheelbase	
`l_f`	distance from the front axle to the centre of gravity	
`C_f`	cornering stiffness of the front axle	
`C_r`	cornering stiffness of the rear axle	
`u`	forward speed	
`delta`	front-wheel steer angle (degrees in questions; rad in solve(), or '2 deg')	

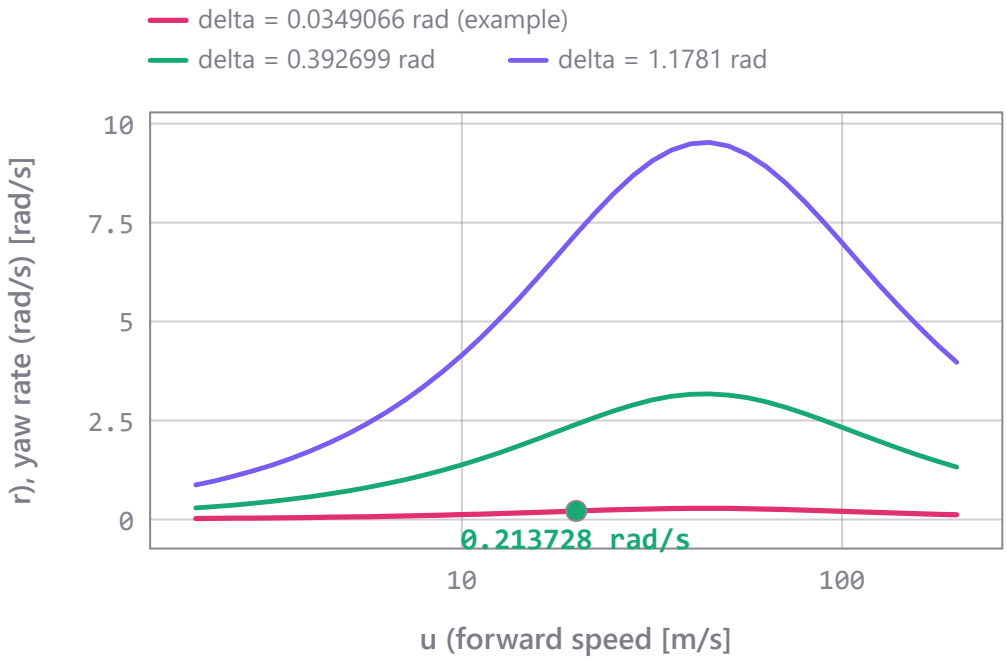
output	meaning
`r`	yaw rate (rad/s)
`a_y`	lateral acceleration (m/s^2)
`R`	turn radius (m)
`beta`	sideslip angle at the centre of gravity (degrees)

- **Method:** $r = u$ delta / (L + K u^2), $a_y = u$ r, $R = u/r$, $\beta = \delta$ (l_r - m l_f u^2 / (L C_r)) / (L + K u^2)
- **Assumes:** below the critical speed; $\delta \neq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



yaw rate 0.2137 rad/s, lateral acceleration 4.275 m/s²
sideslip at the CG -1.002°, speed 72 km/h

Plan view of the bicycle model of a vehicle (m = 1500.0 kg, wheelbase 2.7 m, CG 1.2 m behind the front axle, cornering stiffnesses 90000.0 and 85000.0 N/rad) at 72 km/h with a front steer angle δ = 2° (the wheel angle and its arc exaggerated ×6). Marked: the returned turn radius 93.5769 m, yaw rate 0.213728 rad/s, lateral acceleration 4.27456 m/s² and sideslip -1.00247°.



r (yaw rate (rad/s)) against u (forward speed) from 2 m/s to 200 m/s on a logarithmic axis. One curve for each value of delta (front-wheel steer angle (degrees in questions; rad in solve(), or '2 deg')): 0.0349066 rad, 0.392699 rad, 1.1781 rad. Other inputs as in the worked example: m = 1500.0 kg, L = 2.7 m, l_f = 1.2 m, C_f = 90000.0 N/rad, C_r = 85000.0 N/rad. 41 points per curve computed by transportation-ai999x; marked: the worked example, u = 20 m/s, r = 0.213728 rad/s.

```
transportation_ai999x.solve('transport_steady_state_cornering', m=1500.0, L=2.7, l_f=1.2,
C_f=90000.0, C_r=85000.0, u=20.0, delta=0.03490658503988659)
# -> r = 0.213728 rad/s, a_y = 4.27456 m/s^2, R = 93.5769 m, beta = -1.00247 deg   (SOLVED)
```

```
transportation_ai999x.solve('transport_steady_state_cornering', m=1300.0, L=2.5, l_f=1.4,
C_f=60000.0, C_r=95000.0, u=15.0, delta=0.05235987755982989)
# -> r = 0.268899 rad/s, a_y = 4.03349 m/s^2, R = 55.783 m, beta = -0.641139 deg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Steady state cornering with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad, speed = 72 km/h and steer angle = 2 deg.	SOLVED	r = 0.213728 rad/s, a_y = 4.27456 m/s^2, R = 93.5769 m, beta = -1.00247 deg
Steady state cornering with mass = 1300 kg, wheelbase = 2.5 m, cg to front axle = 1.4 m, front cornering stiffness = 60000 N/rad, rear cornering stiffness = 95000 N/rad, speed = 54 km/h and steer angle = 3 deg.	SOLVED	r = 0.268899 rad/s, a_y = 4.03349 m/s^2, R = 55.783 m, beta = -0.641139 deg
Steady state cornering with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad and speed = 72 km/h.	NEEDS_INPUT	transport_steady_state_cornering needs delta (front-wheel steer angle (degrees in questions; rad in solve(), or '2 deg')
Steady state cornering with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad, speed = 72 km/h and steer angle = 2 deg and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Steady state cornering with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad, speed = -72 km/h and steer angle = 2 deg.	INVALID_INPUT	Steady-state cornering: yaw rate, lateral acceleration, radius, sideslip: u must be positive.

Steer angle needed for a turn of given radius (bicycle model), `transport_steer_angle_for_radius`

Front-wheel steer angle the linear bicycle model needs to hold a turn of radius R at speed u (Ackermann angle L/R plus the understeer term K u^2 / R).

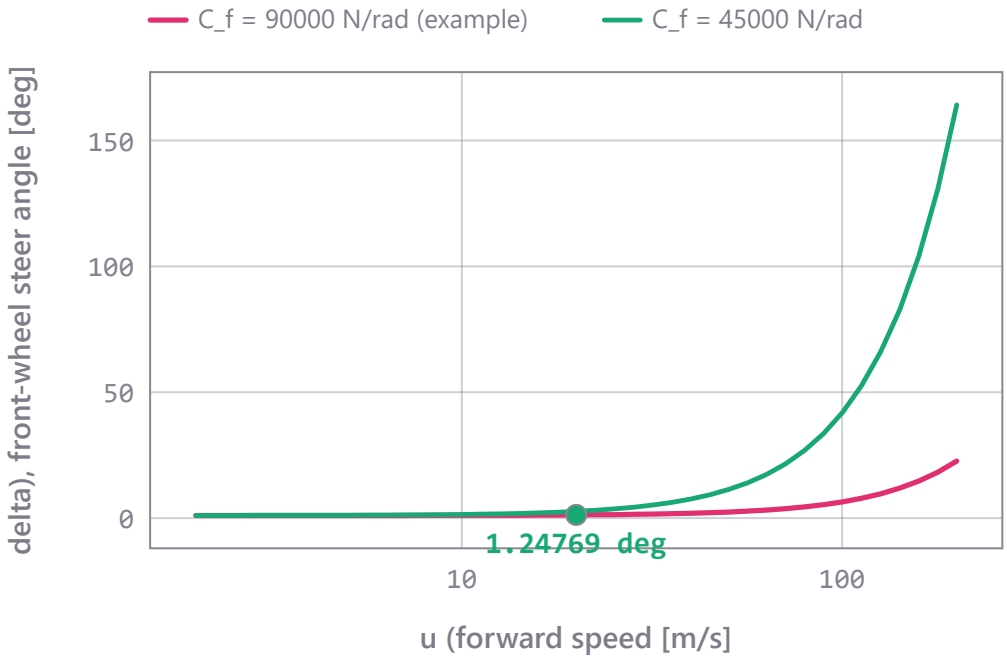
$$\delta = \frac{L}{R} + K\frac{u^2}{R}$$

input	meaning	condition
`m`	vehicle mass	
`L`	wheelbase	
`l_f`	distance from the front axle to the centre of gravity	
`C_f`	cornering stiffness of the front axle	
`C_r`	cornering stiffness of the rear axle	
`u`	forward speed	
`R`	turn radius	

output	meaning
`delta`	front-wheel steer angle (degrees)

- **Method:** $\delta = L/R + K\,u^2/R$
- **Assumes:** below the critical speed

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



delta (front-wheel steer angle (degrees)) against u (forward speed) from 2 m/s to 200 m/s on a logarithmic axis. One curve for each value of C_f (cornering stiffness of the front axle): 90000 N/rad, 45000 N/rad. Other inputs as in the worked example: m = 1500.0 kg, L = 2.7 m, l_f = 1.2 m, C_r = 85000.0 N/rad, R = 150.0 m. 41 points per curve computed by transportation-ai999x; marked: the worked example, u = 20 m/s, delta = 1.24769 deg.

```
transportation_ai999x.solve('transport_steer_angle_for_radius', m=1500.0, L=2.7, l_f=1.2,
C_f=90000.0, C_r=85000.0, u=20.0, R=150.0)
# -> delta = 1.24769 deg   (SOLVED)
```

```
transportation_ai999x.solve('transport_steer_angle_for_radius', m=1300.0, L=2.5, l_f=1.4,
C_f=60000.0, C_r=95000.0, u=15.0, R=80.0)
# -> delta = 2.09186 deg   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Steer angle for a turn with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad, speed = 72 km/h and radius = 150 m.	SOLVED	delta = 1.24769 deg
Steer angle for a turn with mass = 1300 kg, wheelbase = 2.5 m, cg to front axle = 1.4 m, front cornering stiffness = 60000 N/rad, rear cornering stiffness = 95000 N/rad, speed = 54 km/h and radius = 80 m.	SOLVED	delta = 2.09186 deg
Steer angle for a turn with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000	NEEDS_INPUT	transport_steer_angle_for_radius needs R (turn radius). Example:

question	outcome	result
N/rad, rear cornering stiffness = 85000 N/rad and speed = 72 km/h.		solve('transport_steer_angle_for_radius', m='1500 kg',
Steer angle for a turn with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad, speed = 72 km/h and radius = 150 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Steer angle for a turn with mass = 1500 kg, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad, speed = 72 km/h and radius = -150 m.	INVALID_INPUT	Steer angle needed for a turn of given radius (bicycle model): R must be positive.

Yaw natural frequency and damping of the bicycle model, `transport_yaw_natural_frequency`

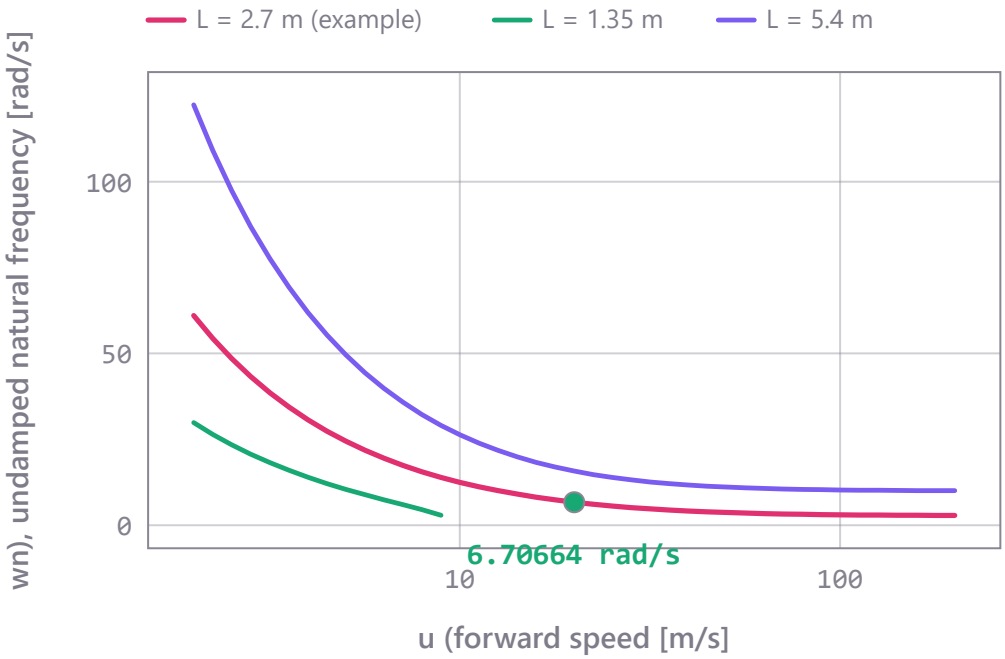
Undamped natural frequency and damping ratio of the lateral-yaw motion of the linear bicycle model at speed u (from the characteristic equation of its two states), below the critical speed.

$$\omega_n^2 = \frac{C_f C_r L^2}{m I_z u^2}\left(1 + \frac{K u^2}{L}\right)$$

input	meaning	condition
`m`	vehicle mass	
`Iz`	yaw moment of inertia	
`L`	wheelbase	
`l_f`	distance from the front axle to the centre of gravity	
`C_f`	cornering stiffness of the front axle	
`C_r`	cornering stiffness of the rear axle	
`u`	forward speed	

output	meaning
`wn`	undamped natural frequency (rad/s)
`zeta`	damping ratio
`f_n`	undamped natural frequency (Hz)

- **Method:** $wn^2 = C_f$ C_r L^2 / (m Iz u^2) (1 + K u^2 / L), 2 zeta wn = (C_f + C_r)/(m u) + (C_f $l_f^2 + C_r$ l_r^2)/(Iz u)
- **Assumes:** below the critical speed
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



wn (undamped natural frequency (rad/s)) against u (forward speed) from 2 m/s to 200 m/s on a logarithmic axis. One curve for each value of L (wheelbase): 2.7 m, 1.35 m, 5.4 m. Other inputs as in the worked example: m = 1500.0 kg, Iz = 2500.0 kg*m^2, l_f = 1.2 m, C_f = 90000.0 N/rad, C_r = 85000.0 N/rad. 41 points per curve computed by transportation-ai999x; marked: the worked example, u = 20 m/s, wn = 6.70664 rad/s.

```
transportation_ai999x.solve('transport_yaw_natural_frequency', m=1500.0, Iz=2500.0, L=2.7, l_f=1.2, C_f=90000.0, C_r=85000.0, u=20.0)
# -> wn = 6.70664 rad/s, zeta = 0.913299 1, f_n = 1.06739 Hz   (SOLVED)
```

```
transportation_ai999x.solve('transport_yaw_natural_frequency', m=1300.0, Iz=1900.0, L=2.5, l_f=1.4, C_f=60000.0, C_r=95000.0, u=15.0)
# -> wn = 8.65402 rad/s, zeta = 0.930687 1, f_n = 1.37733 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Yaw natural frequency with mass = 1500 kg, yaw inertia = 2500 kg*m^2, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad and speed = 72 km/h.	SOLVED	wn = 6.70664 rad/s, zeta = 0.913299 1, f_n = 1.06739 Hz
Yaw natural frequency with mass = 1300 kg, yaw inertia = 1900 kg*m^2, wheelbase = 2.5 m, cg to front axle = 1.4 m, front cornering stiffness = 60000 N/rad, rear cornering stiffness = 95000 N/rad and speed = 54 km/h.	SOLVED	wn = 8.65402 rad/s, zeta = 0.930687 1, f_n = 1.37733 Hz
Yaw natural frequency with mass = 1500 kg, yaw inertia = 2500 kg*m^2, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad and rear cornering stiffness = 85000 N/rad.	NEEDS_INPUT	transport_yaw_natural_frequency needs u (forward speed). Example: solve('transport_yaw_natural_frequency', m='1500 kg',

question	outcome	result
Yaw natural frequency with mass = 1500 kg, yaw inertia = 2500 kg*m^2, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad and speed = 72 km/h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Yaw natural frequency with mass = 1500 kg, yaw inertia = -2500 kg*m^2, wheelbase = 2.7 m, cg to front axle = 1.2 m, front cornering stiffness = 90000 N/rad, rear cornering stiffness = 85000 N/rad and speed = 72 km/h.	INVALID_INPUT	Yaw natural frequency and damping of the bicycle model: Iz must be positive.

Quarter-car model: body and wheel-hop natural frequencies,

`transport_quarter_car_frequencies`

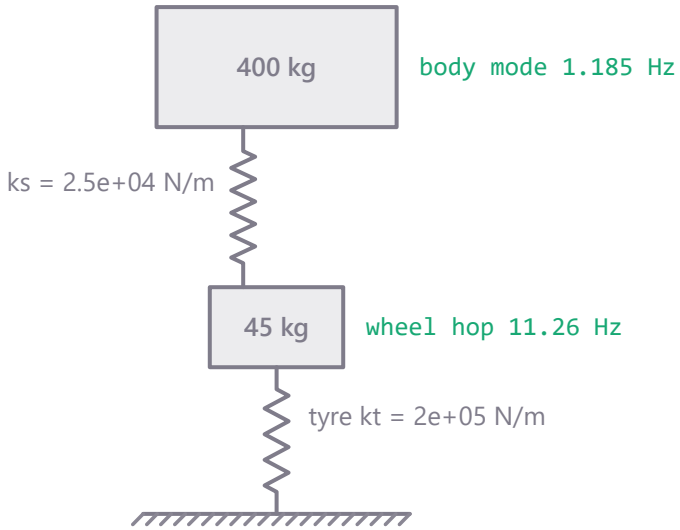
Undamped natural frequencies of the two-degree-of-freedom quarter-car ride model: the body (bounce) mode and the wheel-hop mode, from the sprung and unsprung masses, suspension and tyre stiffnesses.

$$m_s m_u \omega^4 - [k_s m_u + (k_s + k_t) m_s] \omega^2 + k_s k_t = 0$$

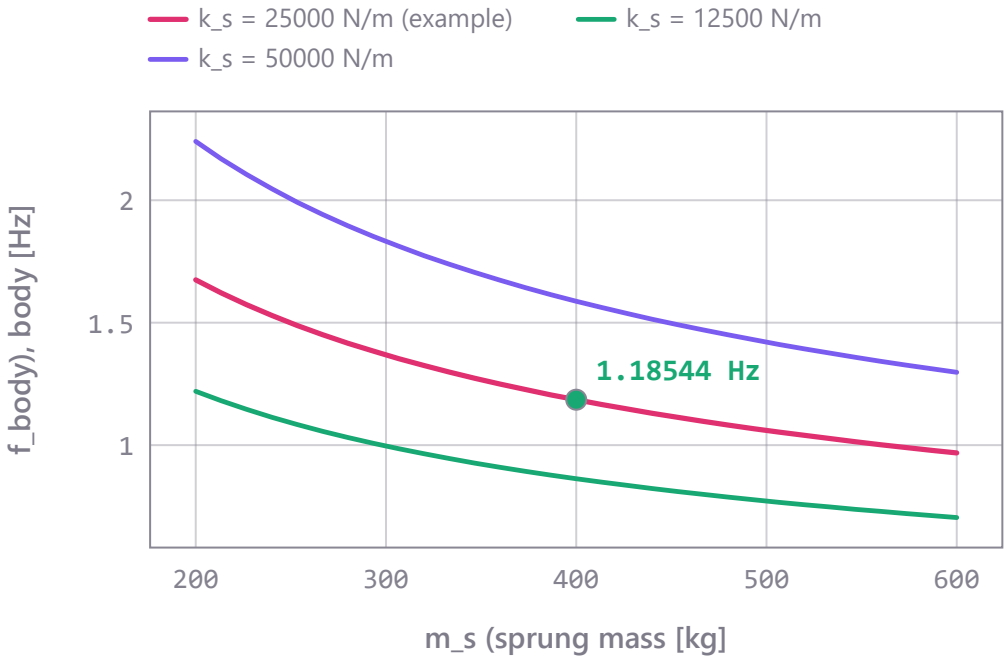
input	meaning	condition
`m_s`	sprung mass (a quarter of the body)	
`m_u`	unsprung mass (wheel)	
`k_s`	suspension spring stiffness	
`k_t`	tyre vertical stiffness	

output	meaning
`f_body`	body (bounce) natural frequency (Hz)
`f_wheel`	wheel-hop natural frequency (Hz)

- **Method:** roots of m_s m_u w^4 - (k_s m_u + (k_s + k_t) m_s) $w^2 + k_s\ k_t = 0$
- **Assumes:** all inputs positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A quarter-car model: sprung mass 400.0 kg on a suspension spring ks = 25000.0 N/m, unsprung mass 45.0 kg on a tyre spring kt = 200000.0 N/m. Marked: the returned natural frequencies, body mode 1.18544 Hz and wheel hop 11.2619 Hz.



f_body (body (bounce) natural frequency (Hz)) against m_s (sprung mass (a quarter of the body)) from 200 kg to 600 kg. One curve for each value of k_s (suspension spring stiffness): 25000 N/m, 12500 N/m, 50000 N/m. Other inputs as in the worked example: m_u = 45.0 kg, k_t = 200000.0 N/m. 31 points per curve computed by transportation-ai999x; marked: the worked example, m_s = 400 kg, f_body = 1.18544 Hz.

```
transportation_ai999x.solve('transport_quarter_car_frequencies', m_s=400.0, m_u=45.0, k_s=25000.0, k_t=200000.0)
# -> f_body = 1.18544 Hz, f_wheel = 11.2619 Hz   (SOLVED)
```

```
transportation_ai999x.solve('transport_quarter_car_frequencies', m_s=300.0, m_u=35.0, k_s=18000.0, k_t=180000.0)
# -> f_body = 1.17487 Hz, f_wheel = 11.9765 Hz   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Quarter car natural frequencies with sprung mass = 400 kg, unsprung mass = 45 kg, spring stiffness = 25 kN/m and tyre stiffness = 200 kN/m.	SOLVED	f_body = 1.18544 Hz, f_wheel = 11.2619 Hz
Quarter car natural frequencies with sprung mass = 300 kg, unsprung mass = 35 kg, spring stiffness = 18 kN/m and tyre stiffness = 180 kN/m.	SOLVED	f_body = 1.17487 Hz, f_wheel = 11.9765 Hz
Quarter car natural frequencies with sprung mass = 400 kg, unsprung mass = 45 kg and spring stiffness = 25 kN/m.	NEEDS_INPUT	transport_quarter_car_frequencies needs k_t (tyre vertical stiffness). Example: solve('transport_quarter_car_frequencies
Quarter car natural frequencies with sprung mass = 400 kg, unsprung mass = 45 kg, spring stiffness = 25 kN/m and tyre stiffness = 200 kN/m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Quarter car natural frequencies with sprung mass = 400 kg, unsprung mass = -45 kg, spring stiffness = 25 kN/m and tyre stiffness = 200 kN/m.	INVALID_INPUT	Quarter-car model: body and wheel-hop natural frequencies: m_u must be positive.

Quarter-car model: road-to-body transmissibility at a frequency,

`transport_quarter_car_transmissibility`

Amplitude ratio between body (sprung-mass) motion and road displacement, and between wheel motion and road displacement, of the damped quarter-car model for a sinusoidal road input of frequency f.

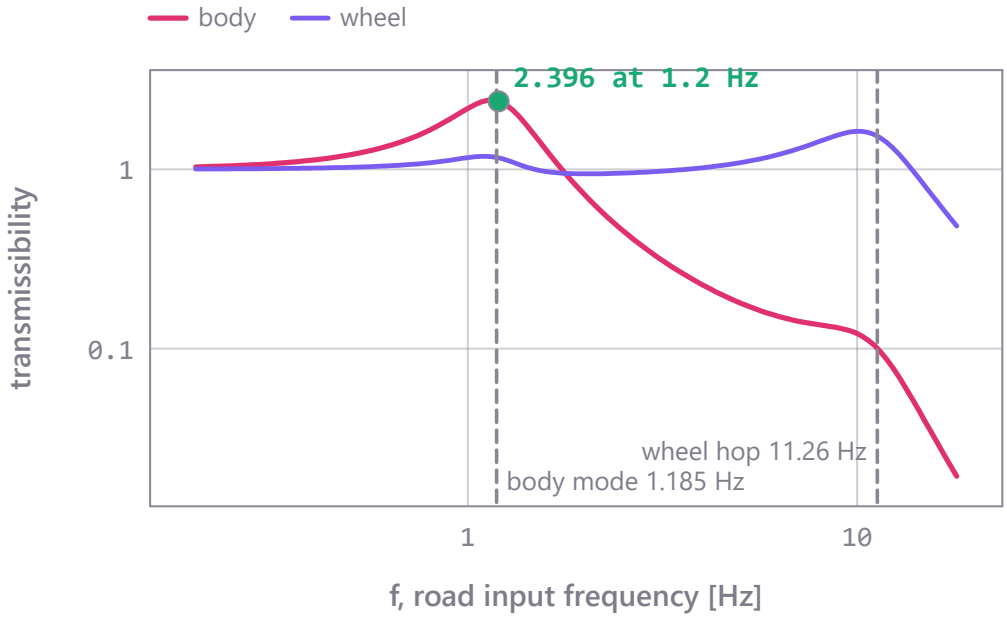
$$\left(K + i\omega C - \omega^2 M\right)\mathbf{Z} = \begin{bmatrix} 0 \\ k_t \end{bmatrix} Z_r$$

input	meaning	condition
`m_s`	sprung mass (a quarter of the body)	
`m_u`	unsprung mass (wheel)	
`k_s`	suspension spring stiffness	
`c_s`	suspension damping coefficient	
`k_t`	tyre vertical stiffness	
`f`	road input frequency	

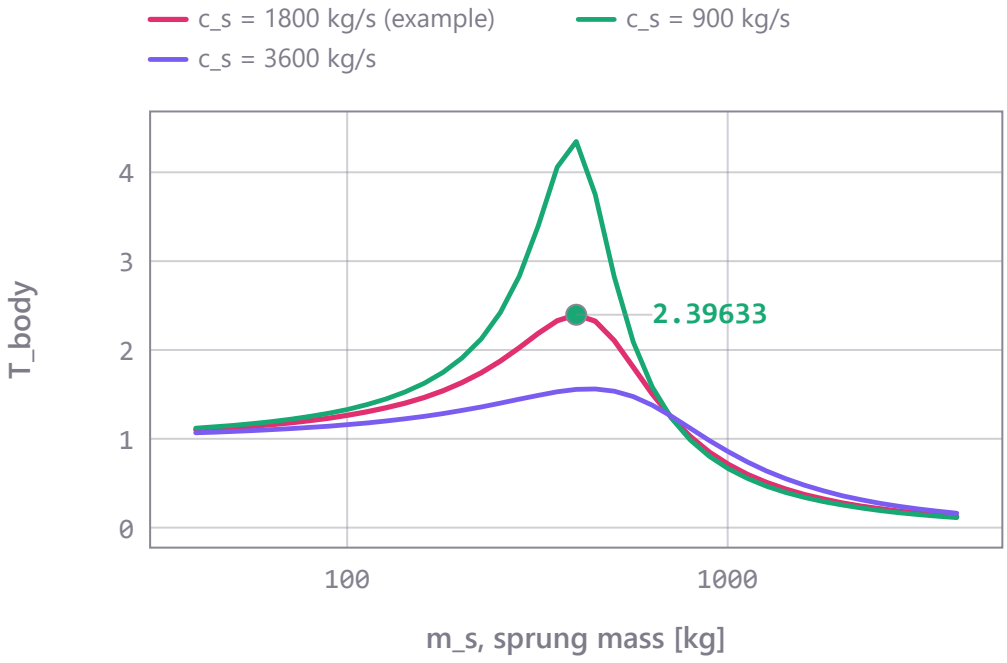
output	meaning
`T_body`	body displacement / road displacement
`T_wheel`	wheel displacement / road displacement

- **Method:** solve the 2x2 complex dynamic-stiffness system [K + i w $C - w^2$ M] Z = [0, k_t] for the road input

- **Assumes:** masses and stiffnesses positive; $c_s \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Body and wheel transmissibility of a quarter car (m_s = 400.0 kg, m_u = 45.0 kg, ks = 25000.0 N/m, cs = 1800.0 N·s/m, kt = 200000.0 N/m) from road input to motion, 121 frequencies each computed by the package (log–log); dashed: the natural frequencies from the package's quarter-car frequency capability. Marked: at f = 1.2 Hz the returned body transmissibility 2.39633 (wheel 1.15904).



T_body (body displacement / road displacement) against m_s (sprung mass (a quarter of the body)) from 40 kg to 4000 kg on a logarithmic axis. One curve for each value of c_s (suspension damping coefficient): 1800 kg/s, 900 kg/s, 3600 kg/s. Other inputs as in the worked example: m_u = 45.0 kg, k_s = 25000.0 N/m, k_t = 200000.0 N/m, f = 1.2 Hz. 41 points per curve computed by transportation-ai999x; marked: the worked example, m_s = 400 kg, T_body = 2.39633.

```
transportation_ai999x.solve('transport_quarter_car_transmissibility', m_s=400.0, m_u=45.0,
k_s=25000.0, c_s=1800.0, k_t=200000.0, f=1.2)
# -> T_body = 2.39633, T_wheel = 1.15904   (SOLVED)
```

```
transportation_ai999x.solve('transport_quarter_car_transmissibility', m_s=300.0, m_u=35.0,
k_s=18000.0, c_s=1200.0, k_t=180000.0, f=10.0)
# -> T_body = 0.125631, T_wheel = 1.89425   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Quarter car transmissibility with sprung mass = 400 kg, unsprung mass = 45 kg, spring stiffness = 25 kN/m, damping = 1800 N*s/m, tyre stiffness = 200 kN/m and frequency = 1.2 Hz.	SOLVED	T_body = 2.39633, T_wheel = 1.15904
Quarter car transmissibility with sprung mass = 300 kg, unsprung mass = 35 kg, spring stiffness = 18 kN/m, damping = 1200 N*s/m, tyre stiffness = 180 kN/m and frequency = 10 Hz.	SOLVED	T_body = 0.125631, T_wheel = 1.89425
Quarter car transmissibility with sprung mass = 400 kg, unsprung mass = 45 kg, spring stiffness = 25 kN/m, damping = 1800 N*s/m and tyre stiffness = 200 kN/m.	NEEDS_INPUT	transport_quarter_car_transmissibility needs f (road input frequency). Example: solve('transport_quarter_car_transmissib
Quarter car transmissibility with sprung mass = 400 kg, unsprung mass = 45 kg, spring stiffness = 25 kN/m, damping = 1800 N*s/m, tyre stiffness = 200 kN/m and frequency = 1.2 Hz and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Quarter car transmissibility with sprung mass = 400 kg, unsprung mass = 45 kg, spring stiffness = 25 kN/m, damping = -1800 N*s/m, tyre stiffness = 200 kN/m and frequency = 1.2 Hz.	INVALID_INPUT	Quarter-car model: road-to-body transmissibility at a frequency: c_s must not be negative.

Optimum cycle length by Webster's formula, `transport_webster_optimum_cycle`

Webster's optimum cycle length C0 = (1.5 L + 5)/(1 - Y) of a fixed-time signal from the total lost time L (s) and the critical flow ratios $y_i = v_i/s_i$ (Y = sum y_i), with the minimum cycle $L/(1-Y)$ and the total effective green. Times in s.

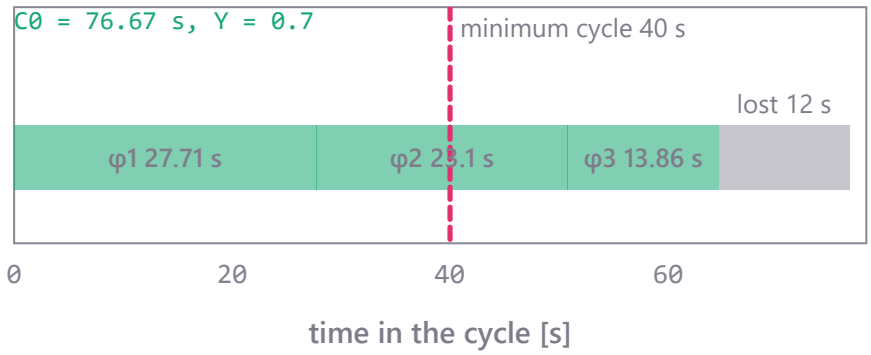
$$C_0 = \frac{1.5L + 5}{1 - Y}$$

input	meaning	condition
`L`	total lost time per cycle	
`y`	critical flow ratio v/s of each phase	

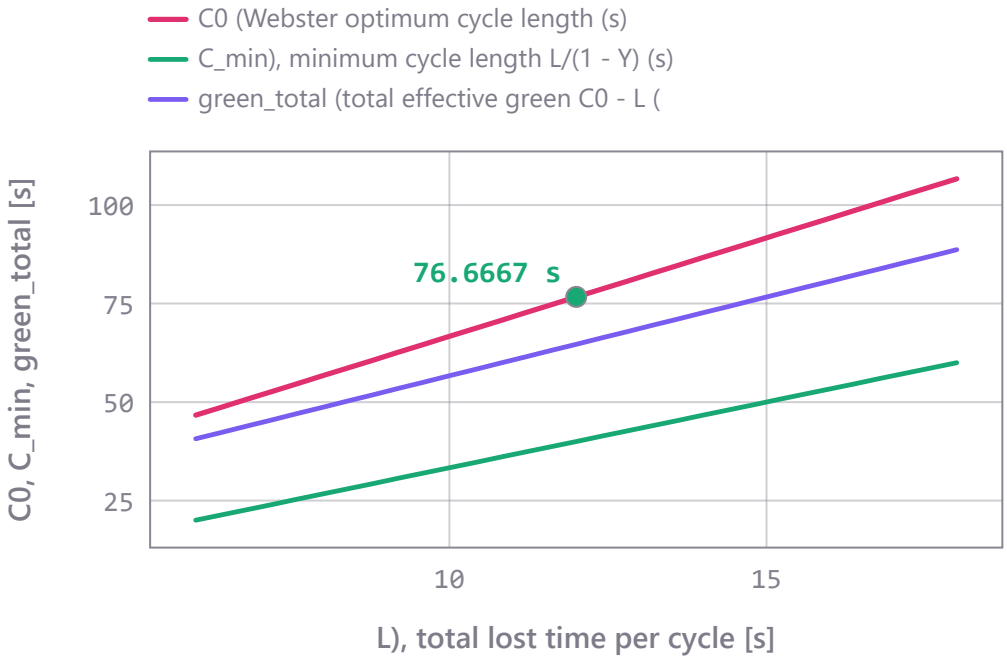
output	meaning
`C0`	Webster optimum cycle length (s)
`Y`	sum of the critical flow ratios
`C_min`	minimum cycle length $L/(1-Y)$ (s)

output	meaning
`green_total`	total effective green $C_0 - L$ (s)

- **Method:** C0 = (1.5 L + 5)/(1 - Y), Y = sum of critical y_i
- **Assumes:** $0 < y_i < 1$; $Y < 0.95$; fixed-time signal, Webster's delay model
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Webster's optimum cycle for lost time L = 12 s and critical flow ratios [0.3, 0.25, 0.15]: the returned C0 = 76.6667 s (Y = 0.7), total effective green 64.6667 s shared in proportion to the flow ratios (derived) and the lost time; marked: the returned minimum cycle 40 s.



C0 (Webster optimum cycle length (s)) against L (total lost time per cycle) from 6 s to 18 s. Outputs with the same unit drawn together: C_min (minimum cycle length L/(1 - Y) (s)), green_total (total effective green C0 - L (s)). Other inputs as in the worked example: y = [0.3, 0.25, 0.15]. 31 points computed by transportation-ai999x; marked: the worked example, L = 12 s, C0 = 76.6667 s.

```
transportation_ai999x.solve('transport_webster_optimum_cycle', L=12.0, y=[0.3, 0.25, 0.15])
# -> C0 = 76.6667 s, Y = 0.7, C_min = 40 s, green_total = 64.6667 s   (SOLVED)
```

```
transportation_ai999x.solve('transport_webster_optimum_cycle', L=8.0, y=[0.4, 0.3])
# -> C0 = 56.6667 s, Y = 0.7, C_min = 26.6667 s, green_total = 48.6667 s   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Webster optimum cycle length with lost time = 12 s and flow ratios = [0.3, 0.25, 0.15].	SOLVED	C0 = 76.6667 s, Y = 0.7, C_min = 40 s, green_total = 64.6667 s
Webster optimum cycle length with lost time = 8 s and flow ratios = [0.4, 0.3].	SOLVED	C0 = 56.6667 s, Y = 0.7, C_min = 26.6667 s, green_total = 48.6667 s
Webster optimum cycle length with lost time = 12 s.	NEEDS_INPUT	transport_webster_optimum_cycle needs y (critical flow ratio v/s of each phase). Example: solve('transport_webster_optim
Webster optimum cycle length with lost time = 12 s and flow ratios = [0.3, 0.25, 0.15] and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Webster optimum cycle length with lost time = 12 s and flow ratios = [0.5, 0.5].	INVALID_INPUT	Optimum cycle length by Webster's formula: the sum of the critical flow ratios must be below 0.95 (the intersection is n

Curve radius or speed by the simplified e + f formula, `transport_simplified_curve_formula`

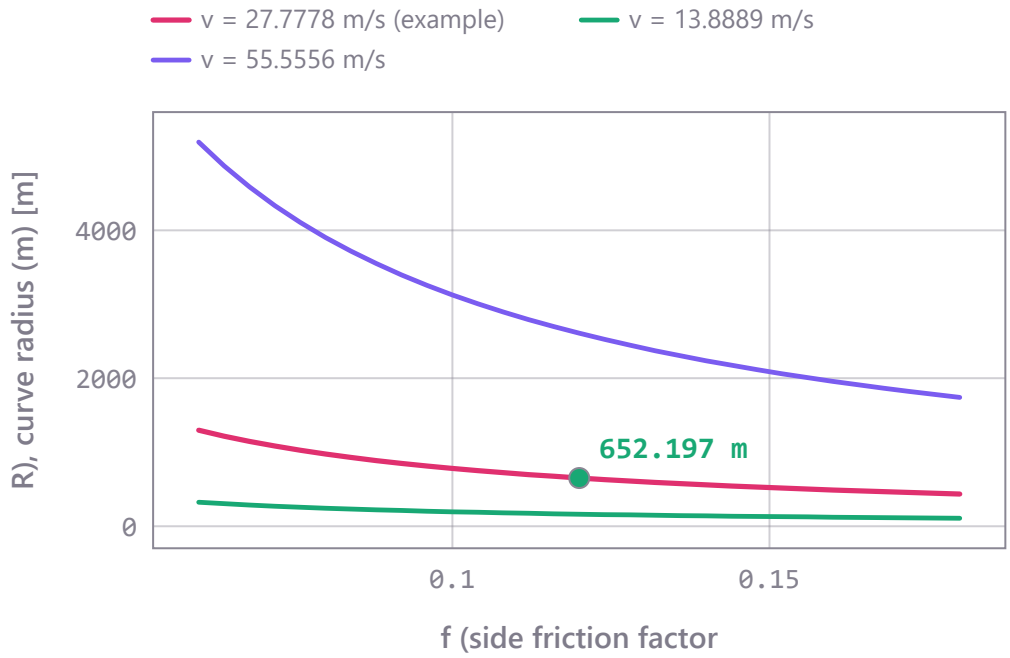
The minimum radius R = v^2/(g (e + f)) for a speed, or the maximum speed sqrt(g R (e + f)) on a radius, by the simplified (small-angle) point-mass formula of horizontal curves (the exact form is minimum curve radius).

$$R = \frac{v^2}{g\,(e + f)}$$

input	meaning	condition
`e`	superelevation rate, decimal	
`f`	side friction factor	
`v`	design speed (give v or R)	
`R`	curve radius (give v or R)	

output	meaning
`R`	curve radius (m)
`v`	speed (m/s)
`v_kmh`	speed (km/h)

- **Method:** R = v^2/(g (e + f)); $v = sqrt$(g R (e + f))
- **Assumes:** $-0.1 \le e \le 0.2$; $0 < f \le 0.5$; the speed or the radius (one of them); small-angle (simplified) form
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



R (curve radius (m)) against f (side friction factor) from 0.06 to 0.18. One curve for each value of v (design speed (give v or R)): 27.7778 m/s, 13.8889 m/s, 55.5556 m/s. Other inputs as in the worked example: e = 0.0006. 31 points per curve computed by transportation-ai999x; marked: the worked example, f = 0.12, R = 652.197 m.

```
transportation_ai999x.solve('transport_simplified_curve_formula', e=0.0006, f=0.12,
v=27.77777777777778)
# -> R = 652.197 m, v = 27.7778 m/s, v_kmh = 100   (SOLVED)
```

```
transportation_ai999x.solve('transport_simplified_curve_formula', e=0.0008, f=0.15, R=250.0)
# -> R = 250 m, v = 19.2311 m/s, v_kmh = 69.2321   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Simplified curve formula with superelevation = 0.06 %, side friction = 0.12 and speed = 100 km/h.	SOLVED	R = 652.197 m, v = 27.7778 m/s, v_kmh = 100
Simplified curve formula with superelevation = 0.08 %, side friction = 0.15 and radius = 250 m.	SOLVED	R = 250 m, v = 19.2311 m/s, v_kmh = 69.2321
Simplified curve formula with superelevation = 0.06 % and speed = 100 km/h.	NEEDS_INPUT	transport_simplified_curve_formula needs f (side friction factor). Example: solve('transport_simplified_curve_formula',
Simplified curve formula with superelevation = 0.06 %, side friction = 0.12 and speed = 100 km/h and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Simplified curve formula with superelevation = 0.06 %, side friction = 0.9 and speed = 100 km/h.	INVALID_INPUT	Curve radius or speed by the simplified e + f formula: the side friction factor must lie in (0, 0.5].

Sag vertical curve length for headlight sight distance, `transport_sag_curve_headlight`

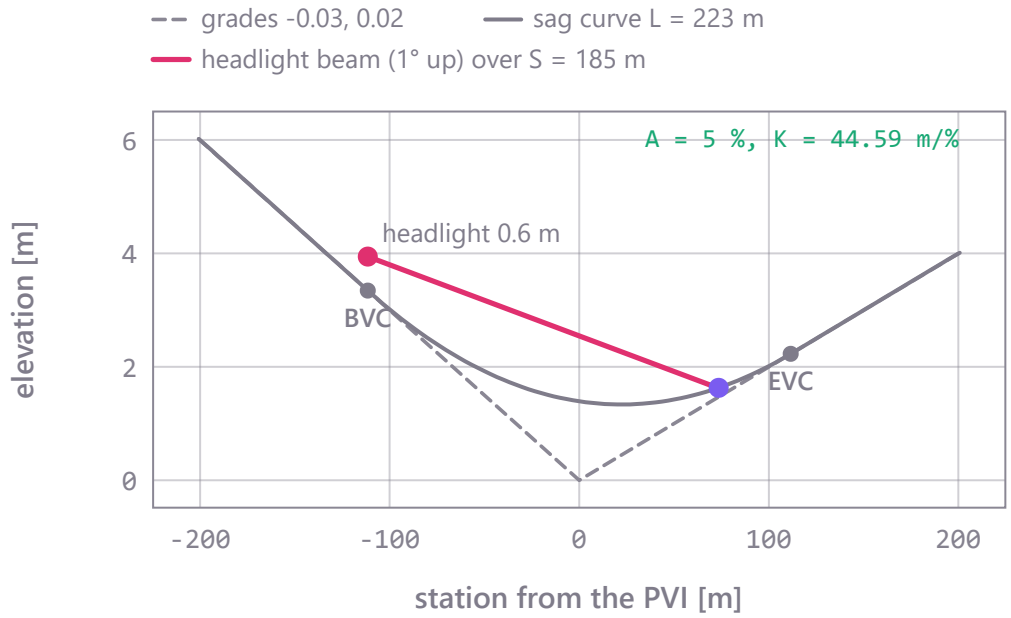
The minimum length of a sag vertical curve for the headlight sight distance S by the AASHTO formulas (headlight 0.6 m high, beam rising 1 degree): $L = A$ S^2/(120 + 3.5 S) when $S < L$, L = 2S - (120 + 3.5 S)/A when $S > L$, $A = g_2 - g_1$ in percent.

$$L = \frac{AS^2}{120 + 3.5S} \ (S < L), \quad L = 2S - \frac{120 + 3.5S}{A} \ (S > L)$$

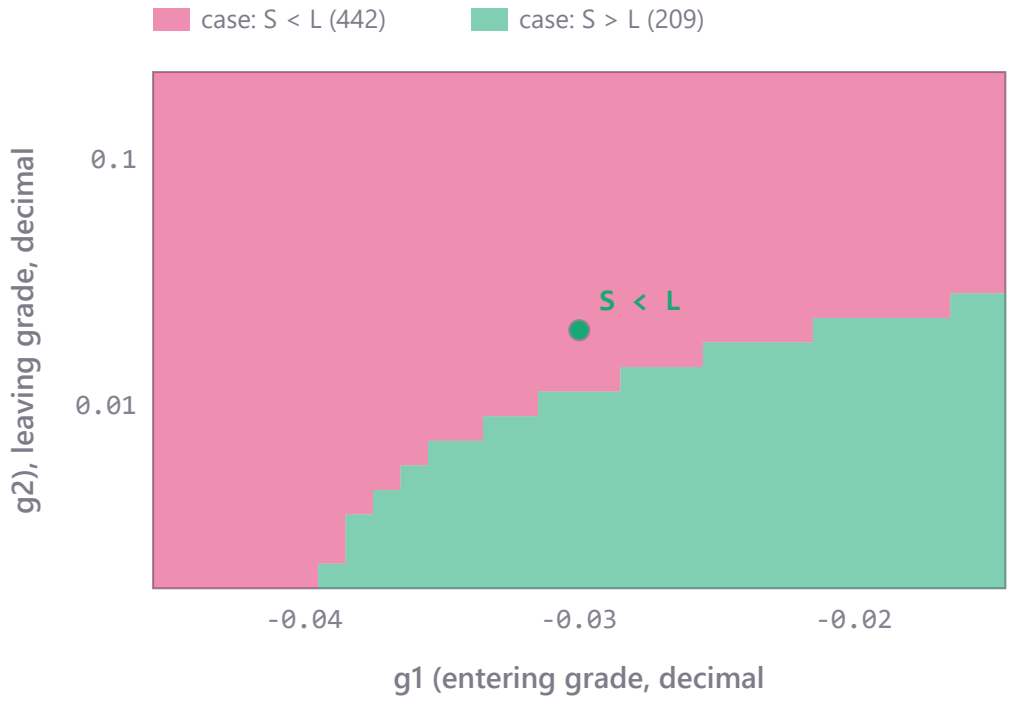
input	meaning	condition
`g1`	entering grade, decimal	
`g2`	leaving grade, decimal	
`S`	headlight sight distance	

output	meaning
`L`	minimum curve length (m)
`A`	algebraic grade difference (%)
`K`	rate of vertical curvature L/A (m per %)
`case`	$S < L$ or $S > L$

- **Method:** $S < L$: $L = A$ S^2/(120 + 3.5 S); $S > L$: L = 2S - (120 + 3.5 S)/A
- **Assumes:** $g_2 > g_1$ (a sag); AASHTO headlight height 0.6 m and 1 degree upward beam (coefficients 120 and 3.5)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A sag vertical curve between grades g1 = -0.03 and g2 = 0.02 sized for a headlight sight distance S = 185 m: the returned length L = 222.964 m (A = 5 %, K = 44.5928 m per %, case S < L). The profile is drawn from the stated grades and the returned length; the beam from a headlight 0.6 m high at the BVC rising 1° above the entering grade is drawn over the stated S. Heights are exaggerated by the axes.



S < L or S > L (case) over g1 (entering grade, decimal) from -0.045 to -0.015 and g2 (leaving grade, decimal) from 0.002 to 0.2 (logarithmic vertical axis): each of the 31 × 21 cells is one run of transportation-ai999x, coloured by the verdict it returned, S < L: 442 cells, S > L: 209 cells. Other inputs as in the worked example: S = 185.0 m. Marked: the worked example, g1 = -0.03, g2 = 0.02 → S < L.

```
transportation_ai999x.solve('transport_sag_curve_headlight', g1=-0.03, g2=0.02, S=185.0)
# -> L = 222.964 m, A = 5, K = 44.5928, case = S < L   (SOLVED)
```

```
transportation_ai999x.solve('transport_sag_curve_headlight', g1=-0.02, g2=0.02, S=120.0)
# -> L = 105 m, A = 4, K = 26.25, case = S > L   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sag vertical curve length with g1 = -3 %, g2 = 2 % and sight distance = 185 m.	SOLVED	L = 222.964 m, A = 5, K = 44.5928, case = S < L
Sag vertical curve length with g1 = -2 %, g2 = 2 % and sight distance = 120 m.	SOLVED	L = 105 m, A = 4, K = 26.25, case = S > L
Sag vertical curve length with g1 = -3 % and g2 = 2 %.	NEEDS_INPUT	transport_sag_curve_headlight needs S (headlight sight distance). Example: solve('transport_sag_curve_headlight', g1='-0
Sag vertical curve length with g1 = -3 %, g2 = 2 % and sight distance = 185 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Sag vertical curve length with g1 = 2 %, g2 = -3 % and sight distance = 185 m.	INVALID_INPUT	Sag vertical curve length for headlight sight distance: a sag curve needs g2 > g1 (the grade increases).

Vehicle resistance forces and power at a speed, `transport_vehicle_resistance`

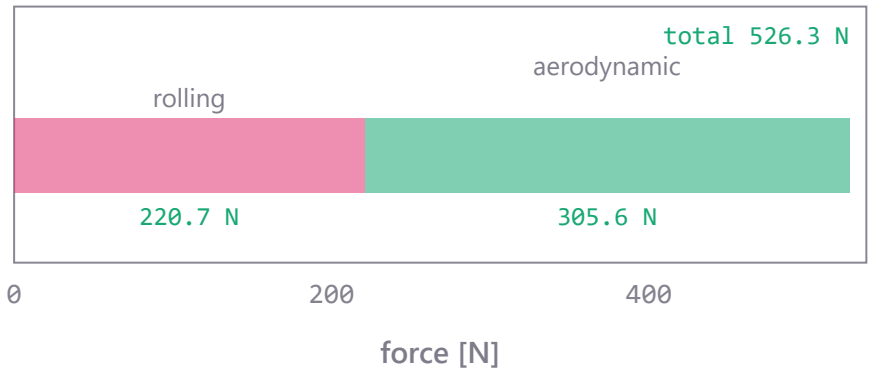
Rolling resistance f_r m g cos(theta), aerodynamic drag rho C_D A $v^2/2$ and grade resistance m g sin(theta) (tan theta = G) of a road vehicle at a speed, their sum and the power needed to overcome them.

$$F = f_r m g \cos\theta + \tfrac{1}{2}\rho C_D A v^2 + m g \sin\theta, \quad P = F v$$

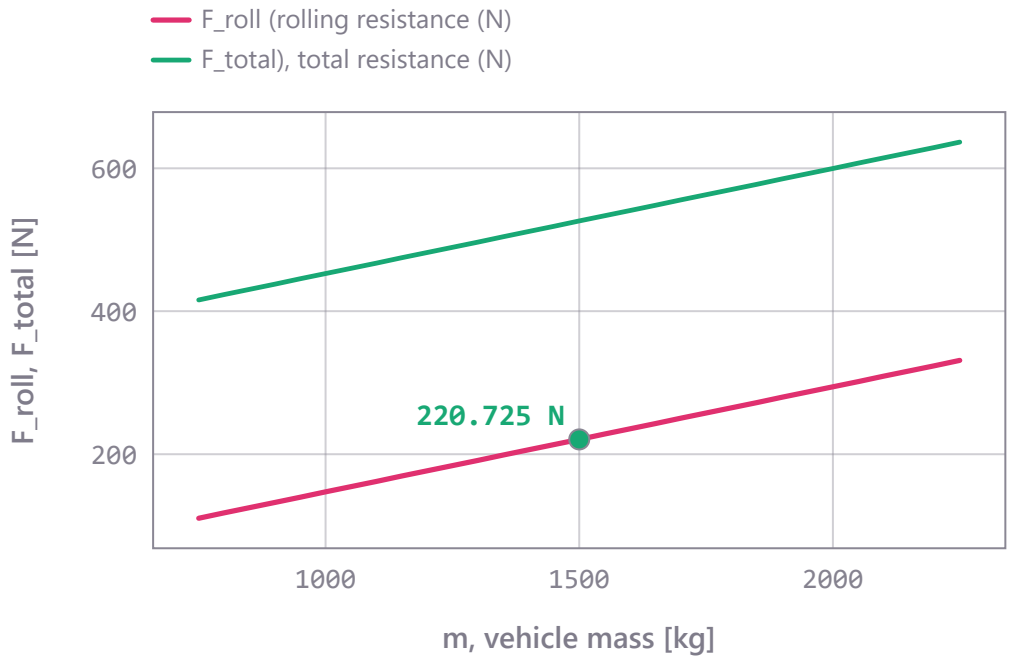
input	meaning	condition
`m`	vehicle mass	
`v`	speed	
`C_D`	drag coefficient	
`A`	frontal area	
`rho`	air density (default 1.2 kg/m^3)	
`f_r`	rolling resistance coefficient (default 0.015)	
`G`	grade, decimal (positive uphill)	

output	meaning
`F_roll`	rolling resistance (N)
`F_aero`	aerodynamic drag (N)
`F_grade`	grade resistance (N)
`F_total`	total resistance (N)
`P`	power to overcome the resistance (W)
`P_kW`	power (kW)

- **Method:** $F = f_r$ m g cos(theta) + rho C_D A $v^2/2 + m$ g sin(theta), tan(theta) = G; $P = F$ v
- **Assumes:** m, C_D, A, $\rho > 0$; $v \geq 0$; $0 \leq f_r \leq 0.1$; |G| <= 0.3; still air
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Road-load forces of a vehicle (m = 1500 kg, CD = 0.3, A = 2.2 m^2) at 100 km/h, as returned and added end to end: rolling 220.725 N, aerodynamic 305.556 N; total 526.281 N, power needed 14.6189 kW.



F_roll (rolling resistance (N)) against m (vehicle mass) from 750 kg to 2250 kg. Outputs with the same unit drawn together: F_total (total resistance (N)). Other inputs as in the worked example: v = 27.77777777777778 m/s, C_D = 0.3, A = 2.2 m^2. 31 points computed by transportation-ai999x; marked: the worked example, m = 1500 kg, F_roll = 220.725 N.

```
transportation_ai999x.solve('transport_vehicle_resistance', m=1500.0, v=27.77777777777778, C_D=0.3, A=2.2)
# -> F_roll = 220.725 N, F_aero = 305.556 N, F_grade = 0 N, F_total = 526.281 N, P = 14618.9 W, P_kW = 14.6189   (SOLVED)
```

```
transportation_ai999x.solve('transport_vehicle_resistance', m=20000.0, v=22.22222222222222, C_D=0.6, A=9.0, f_r=0.008, G=0.0003)
# -> F_roll = 1569.6 N, F_aero = 1600 N, F_grade = 58.86 N, F_total = 3228.46 N, P = 71743.6 W, P_kW = 71.7436   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Vehicle resistance forces with mass = 1500 kg, speed = 100 km/h, drag coefficient = 0.3 and frontal area = 2.2 m^2.	SOLVED	F_roll = 220.725 N, F_aero = 305.556 N, F_grade = 0 N, F_total = 526.281 N, P = 14618.9 W, P_kW = 14.6189
Vehicle resistance forces with mass = 20000 kg, speed = 80 km/h, drag coefficient = 0.6, frontal area = 9 m^2, rolling resistance = 0.008 and grade = 0.03 %.	SOLVED	F_roll = 1569.6 N, F_aero = 1600 N, F_grade = 58.86 N, F_total = 3228.46 N, P = 71743.6 W, P_kW = 71.7436
Vehicle resistance forces with mass = 1500 kg, speed = 100 km/h and drag coefficient = 0.3.	NEEDS_INPUT	transport_vehicle_resistance needs A (frontal area). Example: solve('transport_vehicle_resistance', m='1500 kg', v='27.7
Vehicle resistance forces with mass = 1500 kg, speed = 100 km/h, drag coefficient = 0.3 and frontal area = 2.2 m^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Vehicle resistance forces with mass = 1500 kg, speed = 100 km/h, drag coefficient = -0.3 and frontal area = 2.2 m^2.	INVALID_INPUT	Vehicle resistance forces and power at a speed: C_D must be positive.

Top speed of a road vehicle, `transport_top_speed`

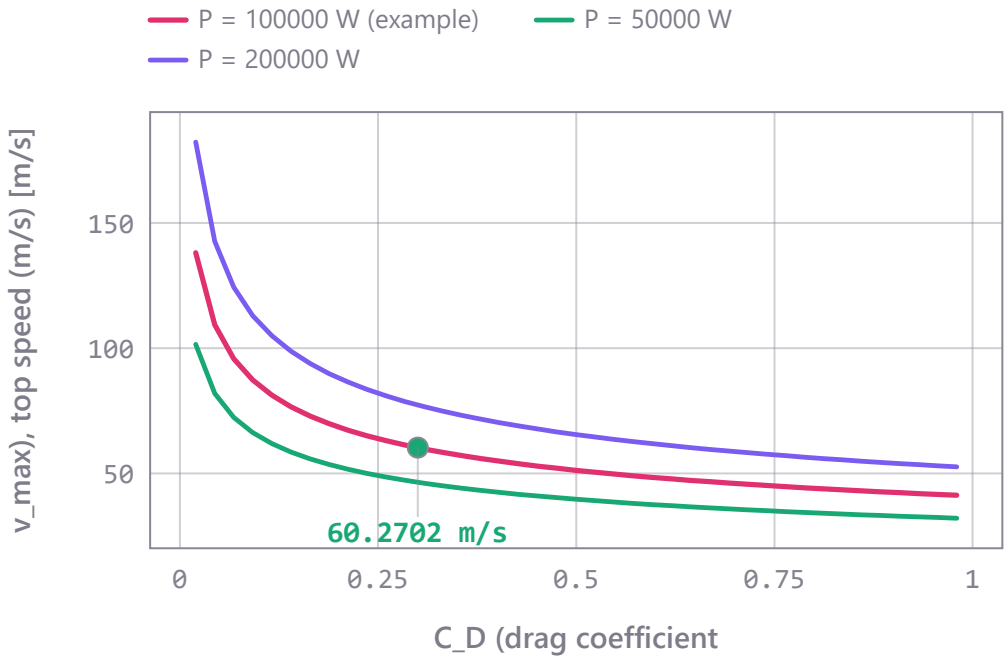
The steady top speed at which the power at the wheels eta P equals the power absorbed by rolling, aerodynamic and grade resistance: rho C_D A $v^3/2 + m$ g (f_r cos(theta) + sin(theta)) $v = \eta$ P.

$$\tfrac{1}{2}\rho C_D A v^3 + mg\,(f_r \cos\theta + \sin\theta) v = \eta P$$

input	meaning	condition
`m`	vehicle mass	
`P`	engine power	
`C_D`	drag coefficient	
`A`	frontal area	
`rho`	air density (default 1.2 kg/m^3)	
`f_r`	rolling resistance coefficient (default 0.015)	
`G`	grade, decimal (positive uphill)	
`eta`	drivetrain efficiency (default 1)	

output	meaning
`v_max`	top speed (m/s)
`v_max_kmh`	top speed (km/h)

- **Method:** solve rho C_D A $v^3/2 + m$ g (f_r cos(theta) + sin(theta)) $v = \eta$ P by bisection
- **Assumes:** m, P, C_D, A, $\rho > 0$; $0 \le f_r \le 0.1$; |G| <= 0.3; $0 < \eta \le 1$; power-limited (no gearing or traction limit)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



v_max (top speed (m/s)) against C_D (drag coefficient) from 0.02 to 0.98. One curve for each value of P (engine power): 100000 W, 50000 W, 200000 W. Other inputs as in the worked example: m = 1500.0 kg, A = 2.2 m^2. 41 points per curve computed by transportation-ai999x; marked: the worked example, C_D = 0.3, v_max = 60.2702 m/s.

```
transportation_ai999x.solve('transport_top_speed', m=1500.0, P=100000.0, C_D=0.3, A=2.2)
# -> v_max = 60.2702 m/s, v_max_kmh = 216.973   (SOLVED)
```

```
transportation_ai999x.solve('transport_top_speed', m=20000.0, P=300000.0, C_D=0.6, A=9.0,
f_r=0.008, G=0.0002, eta=0.9)
# -> v_max = 39.8999 m/s, v_max_kmh = 143.64   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Top speed of a road vehicle with mass = 1500 kg, power = 100 kW, drag coefficient = 0.3 and frontal area = 2.2 m^2.	SOLVED	v_max = 60.2702 m/s, v_max_kmh = 216.973
Top speed of a road vehicle with mass = 20000 kg, power = 300 kW, drag coefficient = 0.6, frontal area = 9 m^2, rolling resistance = 0.008, grade = 0.02 % and efficiency = 0.9.	SOLVED	v_max = 39.8999 m/s, v_max_kmh = 143.64
Top speed of a road vehicle with mass = 1500 kg, power = 100 kW and drag coefficient = 0.3.	NEEDS_INPUT	transport_top_speed needs A (frontal area). Example: solve('transport_top_speed', m='1500 kg', P='100000 W', C_D='0.3 1'
Top speed of a road vehicle with mass = 1500 kg, power = 100 kW, drag coefficient = 0.3 and frontal area = 2.2 m^2 and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Top speed of a road vehicle with mass = 1500 kg, power = 100 kW, drag coefficient = 0.3, frontal area = 2.2 m^2 and efficiency = 1.5.	INVALID_INPUT	Top speed of a road vehicle: the drivetrain efficiency must lie in (0, 1].

Passing sight distance (four-component method), `transport_passing_sight_distance`

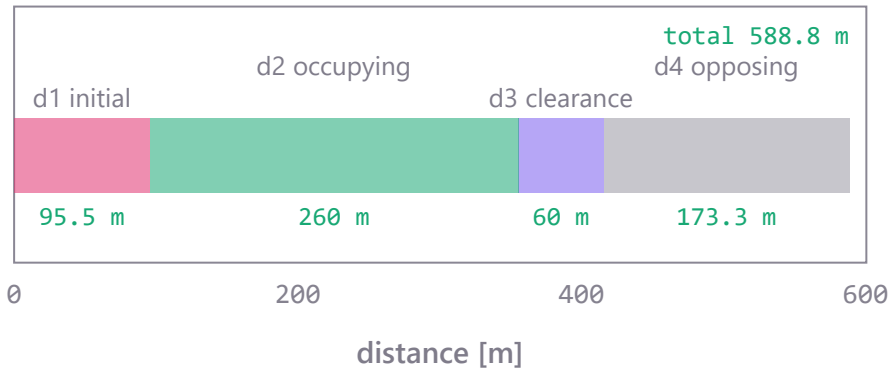
The passing sight distance on a two-lane road as $d_1 + d_2 + d_3 + d_4$: the initial manoeuvre $d_1 = t_1$ (v - m + a t1/2), the time in the left lane $d_2 = v$ t2, the clearance d3 and the opposing vehicle's travel $d_4 = 2\,d_2/3$ (AASHTO 2001).

$$PSD = t_1\left(v - m + \frac{a t_1}{2}\right) + v t_2 + d_3 + \frac{2}{3} v t_2$$

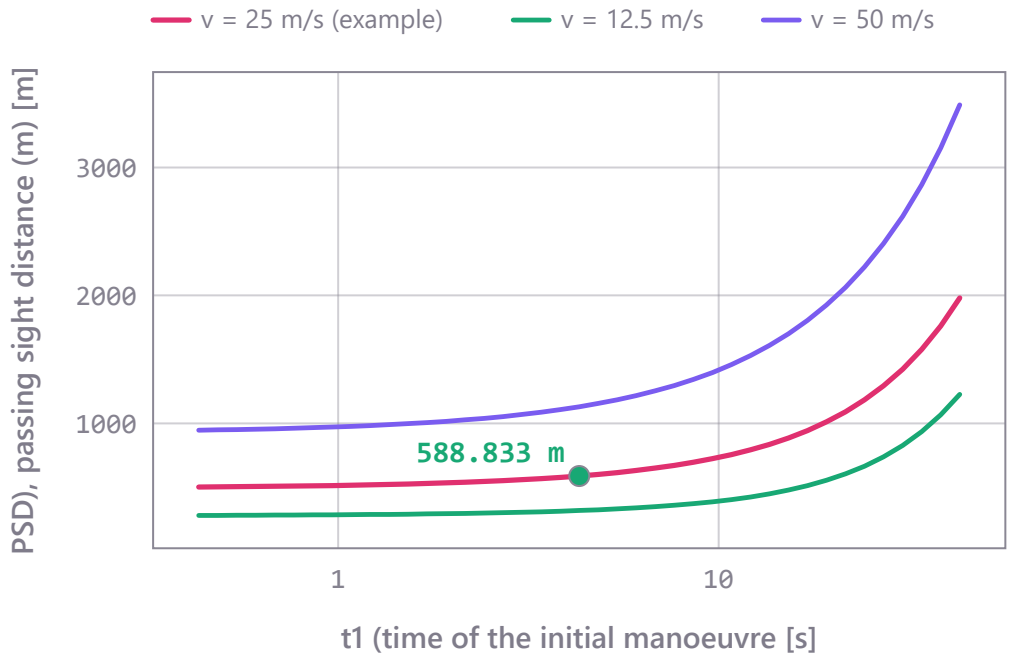
input	meaning	condition
`v`	average speed of the passing vehicle	
`m_diff`	speed difference between passing and passed vehicles	
`a`	average acceleration during the initial manoeuvre	
`t1`	time of the initial manoeuvre	
`t2`	time in the left lane	
`d3`	clearance between the passing and opposing vehicles	

output	meaning
`PSD`	passing sight distance (m)
`d1`	initial manoeuvre distance (m)
`d2`	distance in the left lane (m)
`d3`	clearance (m)
`d4`	opposing vehicle distance (m)

- **Method:** $d_1 = t_1$ (v - m + a t1/2), $d_2 = v$ t2, $d_4 = 2\,d_2/3$, PSD = d1 + d2 + d3 + d4
- **Assumes:** $0 < m < v$; a, t1, $t_2 > 0$; $d_3 \geq 0$; AASHTO 2001 four-component model
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Passing sight distance at v = 90 km/h (speed difference 15 km/h, acceleration 0.64 m/s², times t1 = 4.3 s and t2 = 10.4 s): its four parts as returned, laid end to end to scale. Marked: d1 = 95.5001 m, d2 = 260 m, d3 = 60 m, d4 = 173.333 m, PSD = 588.833 m.



PSD (passing sight distance (m)) against t1 (time of the initial manoeuvre) from 0.43 s to 43 s on a logarithmic axis. One curve for each value of v (average speed of the passing vehicle): 25 m/s, 12.5 m/s, 50 m/s. Other inputs as in the worked example: m_diff = 4.166666666666667 m/s, a = 0.64 m/s^2, t2 = 10.4 s, d3 = 60.0 m. 41 points per curve computed by transportation-ai999x; marked: the worked example, t1 = 4.3 s, PSD = 588.833 m.

```
transportation_ai999x.solve('transport_passing_sight_distance', v=25.0, m_diff=4.166666666666667,
a=0.64, t1=4.3, t2=10.4, d3=60.0)
# -> PSD = 588.833 m, d1 = 95.5001 m, d2 = 260 m, d3 = 60 m, d4 = 173.333 m   (SOLVED)
```

```
transportation_ai999x.solve('transport_passing_sight_distance', v=16.666666666666668,
m_diff=4.444444444444445, a=0.63, t1=3.6, t2=9.3, d3=30.0)
# -> PSD = 336.416 m, d1 = 48.0824 m, d2 = 155 m, d3 = 30 m, d4 = 103.333 m   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Passing sight distance with speed = 90 km/h, speed difference = 15 km/h, acceleration = 0.64 m/s^2, initial manoeuvre time = 4.3 s, left lane time = 10.4 s and clearance = 60 m.	SOLVED	PSD = 588.833 m, d1 = 95.5001 m, d2 = 260 m, d3 = 60 m, d4 = 173.333 m
Passing sight distance with speed = 60 km/h, speed difference = 16 km/h, acceleration = 0.63 m/s^2, initial manoeuvre time = 3.6 s, left lane time = 9.3 s and clearance = 30 m.	SOLVED	PSD = 336.416 m, d1 = 48.0824 m, d2 = 155 m, d3 = 30 m, d4 = 103.333 m
Passing sight distance with speed = 90 km/h, speed difference = 15 km/h, acceleration = 0.64 m/s^2, initial manoeuvre time = 4.3 s and left lane time = 10.4 s.	NEEDS_INPUT	transport_passing_sight_distance needs d3 (clearance between the passing and opposing vehicles). Example: solve('transpo
Passing sight distance with speed = 90 km/h, speed difference = 15 km/h, acceleration = 0.64 m/s^2, initial manoeuvre time = 4.3 s, left lane time = 10.4 s and clearance = 60 m and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()

question	outcome	result
Passing sight distance with speed = 90 km/h, speed difference = 100 km/h, acceleration = 0.64 m/s^2, initial manoeuvre time = 4.3 s, left lane time = 10.4 s and clearance = 60 m.	INVALID_INPUT	Passing sight distance (four-component method): the speed difference must be positive and below the passing speed.

Travel-time reliability indices (buffer, planning time, travel time index),

`transport_travel_time_reliability`

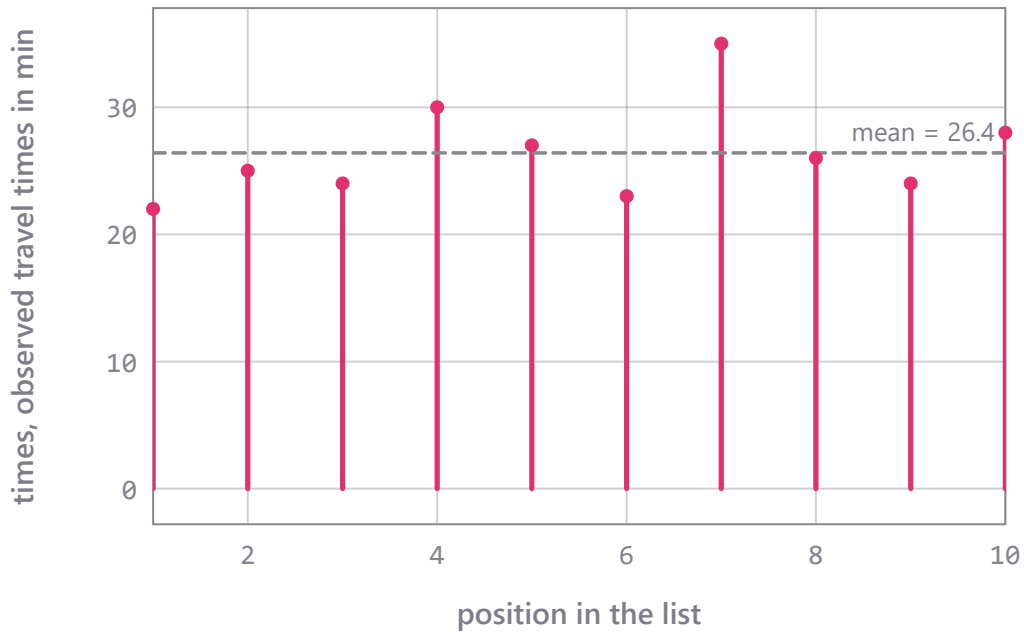
The buffer index (T95 - mean)/mean, the planning time index T95/T_free and the travel time index mean/T_free of observed travel times on a route (T95 the 95th percentile by linear interpolation). Times in minutes.

$$BI = \frac{T_{95} - \bar{T}}{\bar{T}}, \quad PTI = \frac{T_{95}}{T_{free}}, \quad TTI = \frac{\bar{T}}{T_{free}}$$

input	meaning	condition
`times`	observed travel times in min	
`t_free`	free-flow travel time in min	

output	meaning
`mean`	mean travel time (min)
`t95`	95th-percentile travel time (min)
`buffer_index`	buffer index (T95 - mean)/mean
`planning_time_index`	planning time index T95/T_free
`travel_time_index`	travel time index mean/T_free

- **Method:** BI = (T95 - mean)/mean; PTI = T95/T_free; TTI = mean/T_free
- **Assumes:** at least five positive travel times; T95 by linear interpolation (type 7)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The input times (observed travel times in min) as given in the worked example, one value per position (10 values); marked from what transportation-ai999x returned: mean = 26.4. Also returned: t95 = 32.75, buffer_index = 0.24053, planning_time_index = 1.81944, travel_time_index = 1.46667. Other inputs: t_free = 18.0.

```
transportation_ai999x.solve('transport_travel_time_reliability', times=[22, 25, 24, 30, 27, 23, 35, 26, 24, 28], t_free=18.0)
# -> mean = 26.4, t95 = 32.75, buffer_index = 0.24053, planning_time_index = 1.81944, travel_time_index = 1.46667   (SOLVED)
```

```
transportation_ai999x.solve('transport_travel_time_reliability', times=[12, 14, 13, 15, 20, 12], t_free=10.0)
# -> mean = 14.3333, t95 = 18.75, buffer_index = 0.30814, planning_time_index = 1.875, travel_time_index = 1.43333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Travel time reliability indices with travel times = [22, 25, 24, 30, 27, 23, 35, 26, 24, 28] and free flow time = 18 min.	SOLVED	mean = 26.4, t95 = 32.75, buffer_index = 0.24053, planning_time_index = 1.81944, travel_time_index = 1.46667
Travel time reliability indices with travel times = [12, 14, 13, 15, 20, 12] and free flow time = 10 min.	SOLVED	mean = 14.3333, t95 = 18.75, buffer_index = 0.30814, planning_time_index = 1.875, travel_time_index = 1.43333
Travel time reliability indices with travel times = [22, 25, 24, 30, 27, 23, 35, 26, 24, 28].	NEEDS_INPUT	transport_travel_time_reliability needs t_free (free-flow travel time in min). Example: solve('transport_travel_time_rel
Travel time reliability indices with travel times = [22, 25, 24, 30, 27, 23, 35, 26, 24, 28] and free flow time = 18 min and colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: transportation_ai999x.templates()
Travel time reliability indices with travel times = [22, 25, -24, 30, 27] and free flow time = 18 min.	INVALID_INPUT	Travel-time reliability indices (buffer, planning time, travel time index): every travel time must be positive.